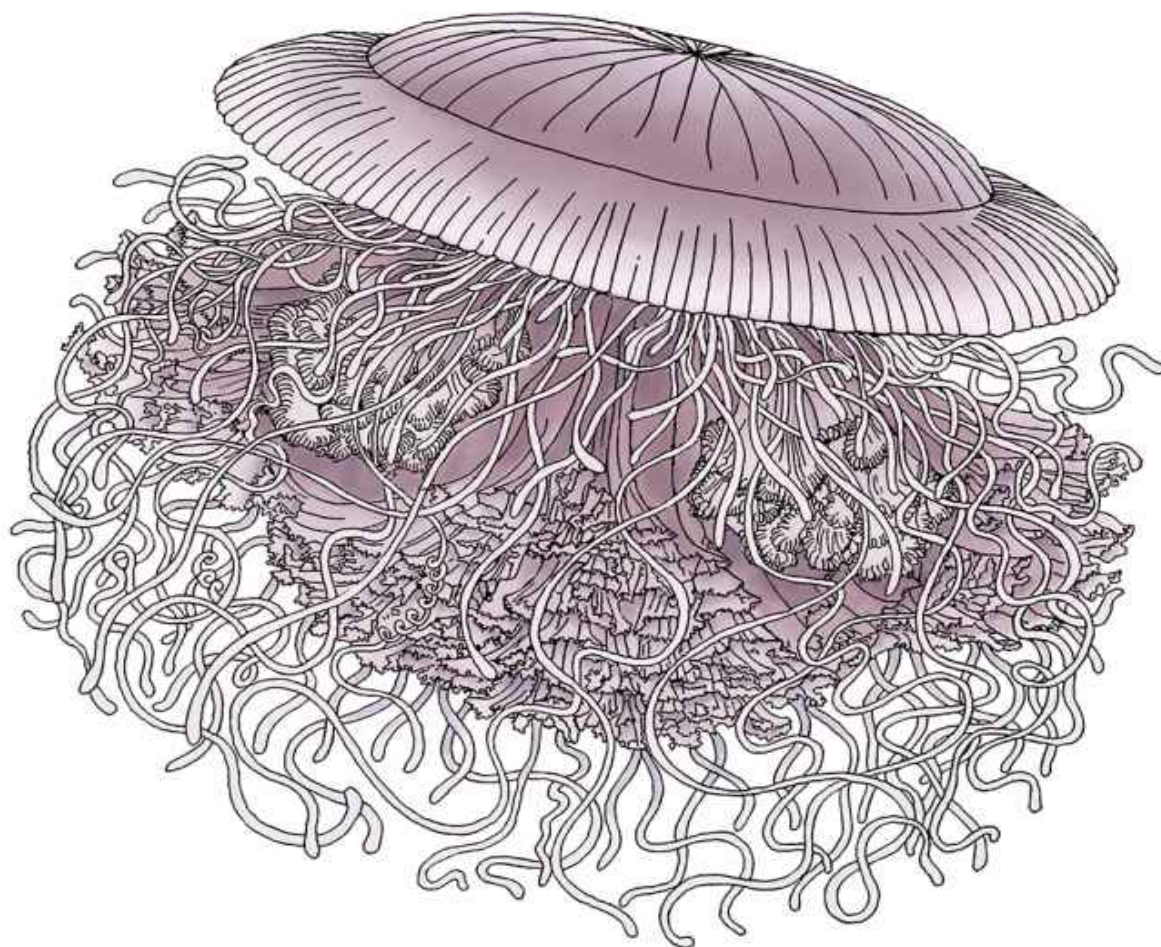


O'REILLY®

Migrando sistemas monolíticos para microsserviços

Padrões evolutivos para transformar seu sistema monolítico



novatec

Sam Newman

Migrando sistemas monolíticos para microserviços

Sam Newman

O'REILLY®
Novatec

Authorized portuguese translation of the english edition of Monolith to Microservices, ISBN 9781492047841 © 2020 Sam Newman. This translation is published and sold by permission of O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

Tradução em português autorizada da edição em inglês da obra Monolith to Microservices, ISBN 9781492047841 © 2020 Sam Newman. Esta tradução é publicada e vendida com a permissão da O'Reilly Media, Inc., detentora de todos os direitos para publicação e venda desta obra.

© Novatec Editora Ltda. [2020].

Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998. É proibida a reprodução desta obra, mesmo parcial, por qualquer processo, sem prévia autorização, por escrito, do autor e da Editora.

Editor: Rubens Prates

Tradução: Lúcia A. Kinoshita

Revisão gramatical: Tássia Carvalho

Editoração eletrônica: Carolina Kuwabata

ISBN: 978-65-86057-05-8

Histórico de edições impressas:

Março/2020 Primeira edição

Novatec Editora Ltda.

Rua Luís Antônio dos Santos 110

02460-000 – São Paulo, SP – Brasil

Tel.: +55 11 2959-6529

Email: novatec@novatec.com.br

Site: www.novatec.com.br

Twitter: twitter.com/novateceditora

Facebook: facebook.com/novatec

LinkedIn: linkedin.com/in/novatec

Sumário

Prefácio

Capítulo 1 ■ Apenas o suficiente sobre os microserviços

O que são microserviços?

Implantações independentes

Modelagem em torno de um domínio de negócio

Seja responsável pelos seus próprios dados

Quais as vantagens que os microserviços podem trazer?

Quais os problemas criados pelos microserviços?

Interfaces de usuário

Tecnologia

Tamanho

Responsabilidade

Arquitetura monolítica

Sistema monolítico de um só processo

Sistema monolítico distribuído

Sistemas caixa-preta de terceiros

Desafios dos sistemas monolíticos

Vantagens dos sistemas monolíticos

Sobre o acoplamento e a coesão

Coesão

Acoplamento

Apenas o suficiente sobre design orientado a domínios

Agregado

Contexto delimitado

Mapeando agregados e contextos delimitados aos microserviços

Leituras complementares

Resumo

Capítulo 2 ■ Planejando uma migração

Compreendendo o objetivo

Três perguntas básicas

Por que você optaria pelos microserviços?

Aumentar a autonomia das equipes

Reduzir o tempo para chegar ao mercado

Escalar para lidar com a carga, com um custo viável
Aumentar a robustez
Escalar o número de desenvolvedores
Incorporar novas tecnologias
Quando os microsserviços não seriam uma boa ideia?
Domínio nebuloso
Startups
Software instalado e administrado pelos clientes
Não ter um bom motivo!
Custo-benefício
Incluindo pessoas na jornada
Mudanças organizacionais
Estabelecimento de um senso de urgência
Criação da coalizão para direcionar a mudança
Desenvolvimento de uma visão e de uma estratégia
Comunicação da visão sobre as mudanças
Empoderamento dos funcionários para ações mais amplas
Conquista de vitórias rápidas
Consolidação dos ganhos e promoção de novas mudanças
Inclusão das novas abordagens na cultura
Importância da migração gradual
É o ambiente de produção que conta
Custo da mudança
Decisões reversíveis e irreversíveis
Pontos mais apropriados para um experimento
Então, por onde devemos começar?
Design orientado a domínios
Até que ponto devemos ir?
Event Storming
Usando um modelo de domínios para atribuição de prioridades
Um modelo combinado
Reorganizando as equipes
Modificando as estruturas
Não há uma solução única para todos
Fazendo uma mudança
Outras habilidades
Como você saberá que a transição está funcionando?
Ter pontos de verificação regulares
Critérios de avaliação quantitativos
Critérios de avaliação qualitativos
Evitando a falácia do custo irrecuperável
Esteja aberto a novas abordagens
Resumo

Capítulo 3 ■ Dividindo o sistema monolítico

Mudar ou não mudar o sistema monolítico?

Cortar, copiar ou reimplementar?

Refatorando o sistema monolítico

Padrões de migração

Padrão: aplicação strangler fig

Como o padrão funciona

Quando usar o padrão

Exemplo: proxy HTTP reverso

Dados?

Opções de proxy

Mudança de protocolos

Exemplo: FTP

Exemplo: interceptação de mensagens

Outros protocolos

Outros exemplos do padrão strangler fig

Mudança de comportamentos concomitantes à migração de funcionalidades

Padrão: composição de UI

Exemplo: composição de páginas

Exemplo: composição de widgets

Exemplo: Micro Frontends

Quando usar o padrão

Padrão: branch por abstração

Como o padrão funciona

Como método de fallback

Quando usar o padrão

Padrão: execução em paralelo

Exemplo: comparando os cálculos de preços de derivativos de crédito

Exemplo: geração de listas na Homepage

Técnicas de verificação

Usando Spies

Scientist do GitHub

Lançamento no escuro e versão canário

Quando usar o padrão

Padrão: colaborador decorador

Exemplo: programa de fidelidade

Quando usar o padrão

Padrão: captura de dados modificados

Exemplo: gerando cartões de fidelidade.

Implementando a captura de dados modificados

Quando usar o padrão

Resumo

Capítulo 4 ■ Decompondo o banco de dados

Padrão: banco de dados compartilhado

Padrões para lidar com o problema

Quando usar o padrão

Mas isso não é possível!

Padrão: visão de banco de dados

Banco de dados como um contrato público

Visões a serem apresentadas

Limitações

Responsabilidade

Quando usar o padrão

Padrão: serviço de encapsulamento de banco de dados

Quando usar o padrão

Padrão: interface de banco de dados como serviço

Implementando uma engine de mapeamento

Comparação com as visões

Quando usar o padrão

Transferência de responsabilidades

Padrão: sistema monolítico expondo agregados

Padrão: mudança do responsável pelos dados

Sincronização de dados

Padrão: sincronização de dados na aplicação

Passo 1: Sincronizar grandes volumes de dados

Passo 2: Sincronizar na escrita, ler do esquema antigo

Passo 3: Sincronizar na escrita, ler do novo esquema

Quando usar esse padrão

Padrão: tracer write

Sincronização de dados

Exemplo: pedidos na Square

Quando usar o padrão

Separando o banco de dados

Separação física versus separação lógica de um banco de dados

Separar o banco de dados ou o código antes?

Separar o banco de dados antes

Separar o código antes

Separar o banco de dados e o código ao mesmo tempo

Então, o que devo separar antes?

Exemplos de separação de esquemas

Padrão: separação de tabelas

Quando usar o padrão

Padrão: transferência de relacionamentos de chave estrangeira para o código

Transferindo a junção

Consistência dos dados

Quando usar o padrão

[Exemplo: dados estáticos compartilhados](#)
[Transações](#)
[Transações ACID](#)
[Continuar sendo ACID, mas sem atomicidade?](#)
[Commits de duas fases](#)
[Transações distribuídas – basta dizer não](#)
[Sagas](#)
[Modos de falha das sagas](#)
[Implementando as sagas](#)
[Sagas versus transações distribuídas](#)
[Resumo](#)

Capítulo 5 ■ Dificuldades crescentes

[Mais serviços, mais dificuldades](#)
[Responsabilidade em escala](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Mudanças que causam incompatibilidade](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Relatórios](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Monitoração e resolução de problemas](#)
[Quando esses problemas poderão ocorrer?](#)
[Como esses problemas poderão ocorrer?](#)
[Possíveis soluções](#)
[Experiência do desenvolvedor local](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Administração de muitas tarefas](#)
[Como esse problema poderia se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Testes fim a fim](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Otimização global versus local](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)

[Possíveis soluções](#)
[Robustez e resiliência](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Serviços órfãos](#)
[Como esse problema pode se manifestar?](#)
[Quando esse problema poderia ocorrer?](#)
[Possíveis soluções](#)
[Resumo](#)

Capítulo 6 ■ Palavras finais

Apêndice A ■ Bibliografia

Apêndice B ■ Índice de padrões

Prefácio

Alguns anos atrás, alguns de nós falavam sobre os microsserviços serem uma área interessante. E eis que eles passaram a ser a arquitetura padrão para centenas de empresas no mundo (muitas delas, provavelmente, começaram como startups com o intuito de resolver os problemas causados pelos microsserviços), e isso fez com que todos corressem para pegar um trem que temiam que fosse desaparecer no horizonte.

Devo admitir que sou parcialmente culpado por isso. Desde que escrevi meu livro sobre o assunto, *Building Microservices* (<https://oreil.ly/building-microservices-2e>), em 2015, ganho a vida trabalhando para ajudar as pessoas a compreender esse tipo de arquitetura. Sempre procurei deixar de lado o modismo e ajudar as empresas a decidir se os microsserviços são a opção certa para elas. Para muitos de meus clientes que já possuem sistemas (não orientados a microsserviços), o desafio tem sido como adotar as arquiteturas de microsserviços. Como partir de um sistema existente e refazer sua arquitetura sem ter de interromper todo o trabalho restante? É nesse cenário que este livro entra em cena. Além disso, meu objetivo é fazer uma apreciação franca dos desafios associados à arquitetura de microsserviços e ajudar você a compreender se iniciar essa jornada é, inclusive, a opção correta para você.

O que você aprenderá

Este livro foi concebido para explorar com detalhes a maneira como você deve pensar ao dividir sistemas existentes em uma arquitetura de microsserviços – e como colocar isso em prática. Abordaremos diversos assuntos relacionados à arquitetura de microsserviços,

porém o foco estará no aspecto da decomposição do sistema. Para um guia mais genérico sobre arquiteturas de microsserviços, meu livro anterior, *Building Microservices*, seria um bom ponto de partida. Com efeito, recomendo que você considere esse livro como um parceiro desta obra.

O Capítulo 1 apresenta uma visão geral sobre o que são os microsserviços e explora melhor as ideias que nos levaram a esse tipo de arquitetura. Deve ser apropriado às pessoas a quem os microsserviços são uma novidade, mas também peço encarecidamente aos leitores mais experientes que não pulem esse capítulo. Sinto que, em meio a tanta tecnologia, algumas das ideias básicas importantes sobre os microsserviços muitas vezes passam despercebidas: são conceitos que o livro retomará continuamente.

Compreender melhor os microsserviços é bom, mas entender se são uma opção correta para você já é outra história. No Capítulo 2, descreverei como avaliar se os microsserviços são ou não uma opção correta para você, além de apresentar algumas diretrizes realmente importantes sobre como administrar uma transição de uma arquitetura monolítica para uma arquitetura de microsserviços. Nesse capítulo, abordaremos de tudo: de design orientado a domínio (domain-driven design) a modelos de mudanças organizacionais – pontos essenciais que deixarão você em uma boa posição, mesmo que decida não adotar uma arquitetura de microsserviços.

Nos capítulos 3 e 4, exploraremos melhor os aspectos técnicos associados à decomposição de um sistema monolítico, explorando exemplos do mundo real e extraíndo padrões para migração. O Capítulo 3 tem como foco os aspectos acerca da decomposição de uma aplicação, enquanto o Capítulo 4 explora profundamente os problemas com os dados. Se quiser realmente passar de um sistema monolítico para uma arquitetura de microsserviços, será necessário separar alguns bancos de dados!

Por fim, o Capítulo 5 analisa os tipos de desafios que você

enfrentará à medida que sua arquitetura de microsserviços aumentar. Esses sistemas podem proporcionar enormes vantagens, mas vêm acompanhados de muita complexidade e de problemas que não precisavam ser enfrentados antes. Esse capítulo é minha tentativa de ajudar você a identificar esses problemas à medida que começarem a surgir, e apresentar maneiras de lidar com as dores de crescimento associadas aos microsserviços.

Convenções usadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

Itálico

Indica termos novos, URLs, endereços de email, nomes e extensões de arquivos.

Largura constante

Usada para listagens de programas, assim como em parágrafos para se referir a elementos de programas, como nomes de variáveis ou de funções, bancos de dados, tipos de dados, variáveis de ambiente, comandos e palavras-chave.

Largura constante em negrito

Exibe comandos ou outros textos que devem ser digitados literalmente pelo usuário.

Largura constante em itálico

Exibe texto que deve ser substituído por valores fornecidos pelo usuário ou valores determinados pelo contexto.



Este elemento significa uma dica ou sugestão.



Este elemento significa uma observação geral.



Este elemento significa um aviso ou uma precaução.

Como entrar em contato conosco

Envie comentários e dúvidas sobre este livro para: novatec@novatec.com.br.

Temos uma página da web para este livro, na qual incluímos a lista de erratas, exemplos e qualquer outra informação adicional.

- Página da edição em português

<http://www.novatec.com.br/livros/migrando-monoliticos-para-microservicos>

- Página da edição original, em inglês

<https://oreil.ly/monolith-to-microservices>

Para obter mais informações sobre livros da Novatec, acesse nosso site em: <http://www.novatec.com.br>

Agradecimentos

Sem a ajuda e a compreensão de minha esposa maravilhosa Lindy Stephens, este livro não teria sido possível. Este livro é dedicado a ela. Lindy, me perdoe por ter sido tão mal-humorado quando vários prazos se esgotaram e se perderam. Também gostaria de agradecer ao amável clã Gillman Staynes por todo seu apoio – sou uma pessoa de sorte por ter uma família tão boa.

Este livro se beneficiou bastante com as pessoas que gentilmente cederam seu tempo e energia para ler diversas versões preliminares e forneceram insights valiosos. Em especial, gostaria de destacar Chris O'Dell, Daniel Bryant, Pete Hodgson, Martin Fowler, Stefan Schrass e Derek Hammer por seus esforços nessa área. Houve também pessoas que contribuíram diretamente de diversas maneiras, portanto gostaria de agradecer a Graham Tackley, Erik Doernenberg, Marcin Zasepa, Michael Feathers, Randy Shoup, Kief Morris, Peter Gillard-Moss, Matt Heath, Steve Freeman, Rene Lengwinat, Sarah Wells, Rhys Evans e Berke Sokhan. Se você encontrar algum erro neste livro, será meu, e não deles.

A equipe da O'Reilly também ofereceu um apoio incrível, e gostaria de dar destaque às minhas editoras Eleanor Bru e Alicia Young, e igualmente a Christopher Guzikowski, Mary Treseler e Rachel Roumeliotis. Além disso, gostaria de agradecer muito a Helen Codling e seus colegas em outros lugares do mundo por levarem continuamente meus livros a diversas conferências, a Susan Conant por manter minha sanidade enquanto navegava pelo mundo sempre em mudança das publicações, e a Mike Loukides pelo meu envolvimento inicial com a O'Reilly. Sei que há muitas outras pessoas que me ajudaram nos bastidores, portanto agradeço igualmente a todos vocês.

Além daqueles que contribuíram diretamente com este livro, gostaria também de destacar outros que, independentemente de terem percebido ou não, ajudaram este livro a se tornar realidade. Assim, gostaria de agradecer (em nenhuma ordem em particular) a Martin Kelpmann, Ben Stopford, Charity Majors, Alistair Cockburn, Gregor Hohpe, Bobby Woolf, Eric Evans, Larry Constantine, Leslie Lamport, Edward Yourdon, David Parnas, Mike Bland, David Woods, John Allspaw, Alberto Brandolini, Frederick Brooks, Cindy Sridharan, Dave Farley, Jez Humble, Gene Kim, James Lewis, Nicole Forsgren, Hector Garcia-Molina, Sheep & Cheese, Kenneth Salem, Adrian Colyer, Pat Helland, Kresten Thorup, Henrik Kniberg, Anders Ivarsson, Manuel Pais, Steve Smith, Bernd Rucker, Matthew Skelton, Alexis Richardson, James Governor e Kane Stephens.

Como sempre ocorre nesses casos, é bem provável que eu tenha me esquecido de alguém que tenha contribuído substancialmente com este livro. Para essas pessoas, tudo que posso dizer é que sinto muito por ter me esquecido de agradecer-lhes citando seu nome e espero que possam me perdoar.

Por fim, algumas pessoas ocasionalmente me perguntam sobre as ferramentas que usei para escrever o livro. Eu o escrevi em AsciiDoc usando o Visual Studio Code com o plug-in de João Pinto para AsciiDoc. O livro estava no sistema de controle de versões Git

e fez uso do sistema Atlas da O'Reilly. Escrevi a maior parte dele em meu notebook usando um teclado mecânico Razer externo, mas, próximo ao final, também utilizei intensamente um iPad Pro executando o Working Copy para dar os toques finais. Isso permitiu que eu escrevesse enquanto viajava, proporcionando uma ocasião memorável em que escrevi sobre refatoração de banco de dados em uma balsa para as Órcades. O consequente enjoo valeu totalmente a pena.

CAPÍTULO 1

Apenas o suficiente sobre os microserviços

Bem, aquilo escalou de forma rápida e realmente escapou de nosso controle!

– RON BURGUNDY, *ANCHORMAN*

Antes de explorar o modo de trabalhar com microserviços, é importante que tenhamos um entendimento comum e compartilhado sobre o que são as arquiteturas de microserviços. Gostaria de abordar alguns conceitos equivocados que vejo regularmente, assim como as nuances que muitas vezes passam despercebidas. Você precisará desse conhecimento básico sólido para tirar o máximo proveito do que está na parte restante do livro. Desse modo, este capítulo apresentará uma explicação sobre as arquiteturas de microserviços, descreverá rapidamente como os microserviços se desenvolveram (o que significa, naturalmente, rever os sistemas monolíticos) e analisará algumas das vantagens e desafios de trabalhar com os microserviços.

O que são microserviços?

Microserviços são serviços que podem ser implantados de forma independente, e são modelados em torno de um domínio de negócio. Eles se comunicam entre si por meio de redes e, como uma opção de arquitetura, oferecem diversos modos de resolver os problemas que você poderá enfrentar. Isso implica que uma arquitetura de microserviços tem como base vários microserviços que colaboram entre si.

É um *tipo* de SOA (Service-Oriented Architecture, ou Arquitetura Orientada a Serviços), ainda que seja um com uma forte opinião sobre como as fronteiras dos serviços devem ser traçadas, e para o qual a possibilidade de uma implantação independente é essencial. Os microsserviços também têm a vantagem de ser independentes de tecnologia.

Do ponto de vista da tecnologia, os microsserviços expõem as funcionalidades de negócio que encapsulam por meio de um ou mais endpoints de rede. Os microsserviços se comunicam entre si por meio dessas redes – fazendo com que sejam uma espécie de sistema distribuído. Também encapsulam armazenagem e acesso aos dados, expondo-os por meio de interfaces bem definidas. Desse modo, os bancos de dados permanecem ocultos dentro das fronteiras de um serviço. Há muito a ser detalhado no meio de tudo isso, portanto, vamos explorar melhor algumas dessas ideias.

Implantações independentes

A possibilidade de *implantações independentes* é a ideia de que podemos fazer uma alteração em um microsserviço e implantá-lo em um ambiente de produção sem ter de utilizar outros serviços. Acima de tudo, não é apenas o fato de que *podemos* fazer isso; é o fato de que esse é *realmente* o modo de gerenciar as implantações em seu sistema. É uma disciplina que deve ser colocada em prática na maior parte das versões que você lançar. É uma ideia que, apesar de simples, é complexa quanto à execução.



Se houver apenas um ponto que você deva reter neste livro, deve ser o seguinte: certifique-se de que vai incorporar o conceito de implantações independentes em seus microsserviços. Adquira o hábito de disponibilizar mudanças em um único microsserviço no ambiente de produção sem ter de fazer outras implantações. A partir daí, muitas coisas boas virão.

Para garantir a possibilidade de implantações independentes, devemos garantir que nossos serviços tenham *baixo acoplamento* – em outras palavras, devemos ser capazes de modificar um serviço sem ter de mudar outras partes. Isso significa que precisamos ter

contratos explícitos, bem definidos e estáveis entre os serviços. Algumas opções de implementação dificultam essa tarefa – o compartilhamento de bancos de dados, por exemplo, é particularmente problemático. O desejo de ter um serviço com baixo acoplamento, com interfaces estáveis, deve direcionar o nosso raciocínio sobre como determinar as fronteiras entre os serviços, antes de tudo.

Modelagem em torno de um domínio de negócio

Fazer uma mudança que ultrapasse a fronteira de um processo é uma tarefa custosa. Se você tiver de fazer uma alteração em dois serviços para disponibilizar uma funcionalidade, e for necessário coordenar a implantação dessas duas alterações, isso exigirá mais trabalho do que fazer a mesma alteração em um único serviço (ou, no caso, em um sistema monolítico). Segue-se, então, que queremos encontrar meios de garantir que faremos alterações que envolvam mais de um serviço com a menor frequência possível.

Seguindo a mesma abordagem que usei no livro *Building Microservices*, este livro utiliza um domínio e uma empresa fictícios para ilustrar certos conceitos quando não for possível compartilhar histórias reais. A empresa em questão é a Music Corp, uma grande empresa multinacional que, de alguma maneira, permanece no mercado, apesar de manter o foco quase totalmente na venda de CDs.

Decidimos manter a Music Corp totalmente ativa no século 21 e, como parte disso, avaliaremos a arquitetura do sistema existente. Na Figura 1.1, vemos uma arquitetura simples, de três camadas. Temos uma interface de usuário web, uma camada de lógica de negócios na forma de um backend monolítico, e a armazenagem de dados é feita com um banco de dados tradicional. Essas camadas, como é usual, estão sob a responsabilidade de diferentes equipes.

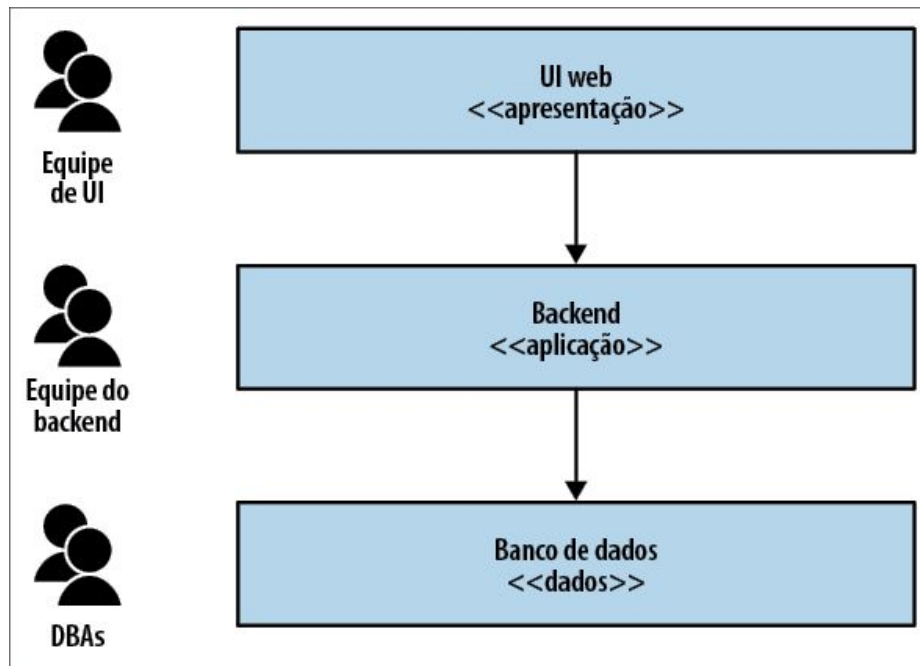


Figura 1.1 – Sistemas da Music Corp conforme uma arquitetura tradicional de três camadas.

Queremos fazer uma alteração simples em nossa funcionalidade: queremos permitir que nossos clientes especifiquem seu gênero musical favorito. Essa mudança exige que modifiquemos a interface do usuário a fim de exibir a UI do gênero escolhido, o serviço de backend para permitir que o gênero seja apresentado na UI e o valor possa ser alterado, e o banco de dados para aceitar essa mudança. Essas alterações terão de ser gerenciadas por equipe, conforme representado na Figura 1.2, e essas alterações deverão ser implantadas na ordem correta.

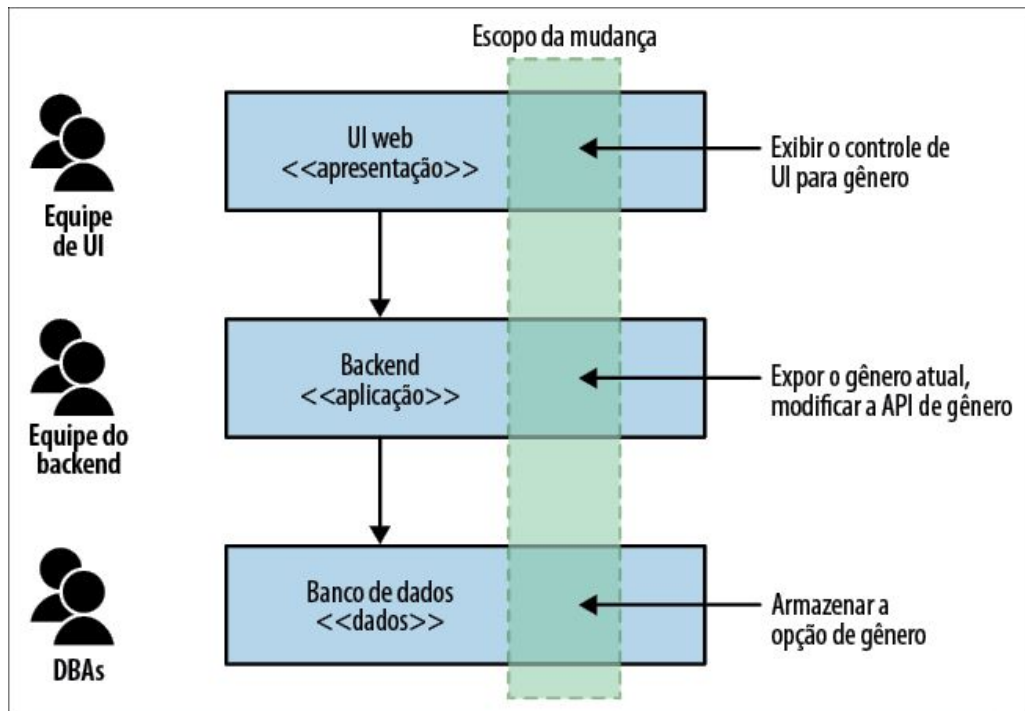


Figura 1.2 – Fazer uma alteração em todas as três camadas é mais complicado.

Essa arquitetura não é ruim. Toda arquitetura acaba sendo otimizada em torno de um conjunto de objetivos. A arquitetura de três camadas é muito comum, em parte por ser universal – todo mundo já ouviu falar dela. Assim, escolher uma arquitetura comum, que você talvez já tenha visto em outros lugares, muitas vezes é um dos motivos pelos quais continuamos a ver esse padrão. Contudo, acho que o principal motivo pelo qual vemos essa arquitetura repetidamente é porque ela se baseia no modo como organizamos nossas equipes.

A atualmente famosa lei de Conway estabelece o seguinte:

Qualquer empresa que projete um sistema ... inevitavelmente produzirá um design cuja estrutura é uma cópia da estrutura de comunicação da empresa.

– MELVIN CONWAY, *How Do COMMITTEES INVENT?*

A arquitetura de três camadas é um bom exemplo disso em ação. No passado, o principal modo como as empresas de TI agrupavam as pessoas era com base em sua competência principal: administradores de bancos de dados formavam uma equipe com outros administradores de bancos de dados, desenvolvedores Java formavam uma equipe com outros desenvolvedores Java, enquanto desenvolvedores de frontend (que, atualmente, têm conhecimento de tópicos exóticos como JavaScript e desenvolvimento de aplicações nativas móveis) estavam em outra equipe. Agrupamos as pessoas com base em sua competência principal, e assim criamos componentes de TI que estejam alinhados com essas equipes.

Portanto, isso explica por que essa arquitetura é tão comum. Ela não é ruim; só está otimizada em torno de um conjunto de forças – tradicionalmente, era assim que agrupávamos as pessoas, com base na familiaridade. No entanto, as forças mudaram. Nossas aspirações em torno de nossos softwares mudaram. Hoje em dia, agrupamos pessoas em equipes com múltiplas habilidades, de modo a reduzir os silos e as passagens de responsabilidades. Queremos lançar softwares de modo muito mais rápido do que jamais fizemos. Isso tem feito com que façamos diferentes escolhas sobre como organizamos nossas equipes e, desse modo, sobre como dividimos nossos sistemas.

Mudanças em funcionalidades dizem respeito principalmente a mudanças nas funcionalidades de negócio. Contudo, na Figura 1.1, a funcionalidade de nosso negócio está, com efeito, distribuída pelas três camadas, aumentando as chances de uma mudança em uma funcionalidade cruzar a fronteira entre as camadas. Essa é uma arquitetura na qual temos um alto nível de coesão entre tecnologias relacionadas, porém uma baixa coesão nas

funcionalidades de negócio. Se quisermos facilitar as modificações, devemos mudar o modo como agrupamos o código – devemos optar pela coesão nas funcionalidades de negócio, em vez da coesão nas tecnologias. Assim, cada serviço poderá ou não acabar contendo uma mistura dessas três camadas, mas essa será uma preocupação local da implementação do serviço.

Vamos comparar essa arquitetura com uma possível arquitetura alternativa, exibida na Figura 1.3. Temos um serviço Cliente dedicado, que expõe uma UI que permite aos clientes atualizar suas informações, e o estado do cliente também é armazenado nesse serviço. A opção por um gênero favorito está associada a um dado cliente, portanto essa alteração será muito mais localizada. Na Figura 1.3, também mostramos a lista dos gêneros disponíveis sendo acessada em um serviço Catálogo, algo que provavelmente já deve existir. Também vemos um novo serviço Recomendação acessando informações sobre nosso gênero favorito – algo que poderia ser facilmente incluído em uma versão subsequente.

Em uma situação como essa, nosso serviço Cliente encapsulará uma pequena parcela de cada uma das três camadas – conterá uma parte da UI, uma parte da lógica da aplicação e uma parte da armazenagem de dados – porém, essas camadas estarão todas encapsuladas em um único serviço.

Nosso domínio de negócio passa a ser a força principal a orientar a arquitetura de nosso sistema, e esperamos que isso facilite fazer alterações e simplifique a organização das equipes em torno do domínio de negócio. Esse ponto é tão importante que, antes de encerrarmos este capítulo, retomaremos o conceito de modelagem de software em torno de um domínio, de modo que eu possa compartilhar algumas ideias sobre design orientado a domínios que modelam o modo como pensamos em nossa arquitetura de microsserviços.

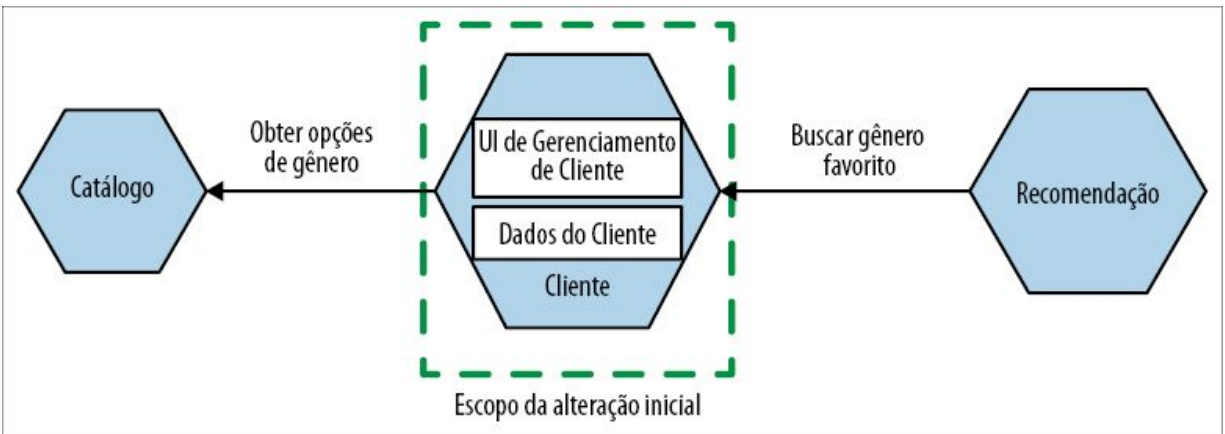


Figura 1.3 – Um serviço Cliente dedicado pode facilitar bastante guardar o gênero musical favorito de um cliente.

Seja responsável pelos seus próprios dados

Um dos pontos em que vejo as pessoas terem mais dificuldade é com a ideia de que os microsserviços não devem compartilhar bancos de dados. Se um serviço quiser acessar dados mantidos por outro serviço, ele deverá pedir os dados de que precisa para esse serviço. Isso permite que o serviço decida o que será compartilhado e o que será oculto. Também permite que o serviço mapeie os detalhes de implementação internos, que podem mudar por diversos motivos arbitrários, para um contrato público mais estável, garantindo interfaces estáveis para o serviço. Ter interfaces estáveis entre os serviços é essencial se quisermos ter implantações independentes – se a interface exposta por um serviço mudar continuamente, haverá um efeito em cascata, fazendo com que outros serviços precisem mudar também.



Não compartilhe bancos de dados, a menos que seja realmente necessário. Mesmo nesse caso, faça tudo que for possível para evitar isso. Em minha opinião, é uma das piores escolhas que você poderia fazer se estiver tentando possibilitar implantações independentes.

Conforme discutimos na seção anterior, queremos pensar em nossos serviços como partes completas das funcionalidades de negócio, os quais encapsulem devidamente a UI, a lógica da aplicação e a armazenagem dos dados. Isso porque queremos

reduzir os esforços necessários para modificar as funcionalidades relacionadas ao negócio. O encapsulamento dos dados e do comportamento dessa forma nos possibilitar ter um alto nível de coesão nas funcionalidades de negócio. Ao ocultar o banco de dados que dá suporte ao nosso serviço, também garantimos uma redução no acoplamento. Retomaremos os tópicos de acoplamento e coesão em breve.

Pode ser difícil pensar dessa forma, particularmente se você já tiver um sistema monolítico com um banco de dados gigantesco com o qual tenha de lidar. Felizmente, o Capítulo 4 é totalmente dedicado a se distanciar dos bancos de dados monolíticos.

Quais as vantagens que os microsserviços podem trazer?

As vantagens proporcionadas pelos microsserviços são muitas e variadas. A natureza independente das implantações dá abertura a novos modelos para aumentar a escala e a robustez dos sistemas, além de permitir que você misture e combine diferentes tecnologias. Como podemos trabalhar nos serviços em paralelo, é possível incluir mais desenvolvedores para resolver um problema sem que atrapalhem uns aos outros. Também pode ser mais fácil para esses desenvolvedores compreender suas partes no sistema, pois poderão manter o foco de sua atenção somente em uma única parte. O isolamento de processos também torna possível diversificar as opções de tecnologia que escolhemos, talvez misturando diferentes linguagens e estilos de programação, plataformas de implantação ou bancos de dados para encontrar a combinação correta.

Acima de tudo, as arquiteturas de microsserviços talvez proporcionem mais flexibilidade. Elas dão abertura para muito mais opções no que diz respeito ao modo de resolver problemas no futuro.

No entanto, é importante observar que nenhuma dessas vantagens

vem gratuitamente. Há muitas maneiras de abordar uma decomposição de sistemas e, basicamente, aquilo que você estiver tentando alcançar levará essa decomposição para direções diferentes. Portanto, compreender aonde você está tentando chegar com a sua arquitetura de microsserviços é muito importante.

Quais os problemas criados pelos microsserviços?

A arquitetura orientada a serviços tornou-se realidade, parcialmente porque os computadores ficaram mais baratos, de modo que podíamos ter mais deles. Em vez de implantar sistemas em um único mainframe gigante, fazia mais sentido utilizar várias máquinas mais baratas. A arquitetura orientada a serviços foi uma tentativa de determinar a melhor maneira de construir aplicações que estivessem distribuídas em diversas máquinas. Um dos principais desafios em tudo isso é o modo como esses computadores conversam entre si: as redes.

A comunicação entre os computadores por meio de redes não é instantânea (aparentemente, tem algo a ver com a física). Isso significa que temos de nos preocupar com latências – e, especificamente, com latências que superam de longe as latências que vemos em operações locais, internas aos processos. A situação piora se considerarmos que essas latências podem variar, deixando imprevisível o comportamento dos sistemas. Além disso, temos de lidar com o fato de que as redes às vezes falham – pacotes se perdem, cabos de rede são desconectados.

Esses desafios fazem com que atividades que são relativamente simples em um sistema monolítico com um só processo, como as transações, sejam muito mais difíceis. Com efeito, tão difíceis que, à medida que a complexidade de nossos sistemas aumentar, é provável que você tenha de abrir mão das transações, e da segurança que elas proporcionam, em troca de outros tipos de técnicas (que, infelizmente, têm um custo-benefício muito diferente).

Lidar com o fato de que qualquer chamada de rede pode e vai falhar passa a ser um problema, assim como o fato de que os serviços com os quais você está conversando poderiam ficar offline por qualquer que seja o motivo, ou poderiam começar a se comportar de maneira estranha. Somando-se a tudo isso, você também terá de começar a pensar em uma solução para poder ter uma visão consistente dos dados em várias máquinas distintas.

Além do mais, é claro, temos uma diversidade enorme de novas tecnologias apropriadas aos microsserviços, que devem ser levadas em consideração – novas tecnologias que, se usadas de forma indevida, poderão contribuir para que você cometa erros mais rapidamente, de formas mais interessantes e custosas. Honestamente, os microsserviços parecem ser uma péssima ideia, exceto por todos os pontos positivos.

Vale a pena observar que, virtualmente, todos os sistemas que classificamos como “monolíticos” também são sistemas distribuídos. Uma aplicação com um só processo provavelmente lê dados de um banco de dados que executa em uma máquina diferente e apresentará esses dados em um navegador web. Há pelo menos três computadores nesse conjunto, e a comunicação entre eles se dá por meio de redes. A diferença está na extensão com que sistemas monolíticos estão distribuídos, em comparação com as arquiteturas de microsserviços. À medida que tiver mais computadores no conjunto, comunicando-se por meio de mais redes, é mais provável que você vá deparar com os terríveis problemas associados aos sistemas distribuídos. Esses problemas que discuti rapidamente talvez não apareçam no início, mas, com o tempo, quando seu sistema aumentar, é provável que você verá a maioria deles, se não todos.

Como diria James Lewis, meu velho amigo e companheiro especialista em microsserviços: “Os microsserviços compram opções”. James escolheu deliberadamente suas palavras – eles *compram opções* para você. Essas opções têm um custo, e você

deverá decidir se vale a pena pagar pelas opções que você quer ter. Exploraremos esse assunto com mais detalhes no Capítulo 2.

Interfaces de usuário

Com muita frequência, vejo as pessoas concentrarem seus esforços em incorporar os microsserviços exclusivamente do lado do servidor – deixando a interface de usuário como uma camada única e monolítica. Se quisermos uma arquitetura que nos facilite implantar novas funcionalidades com mais rapidez, deixar a UI como uma porção monolítica pode ser um grande erro. Podemos – e devemos – separar nossas interfaces de usuário também; é um assunto que será explorado no Capítulo 3.

Tecnologia

Pode ser bastante tentador lançar mão de diversas tecnologias novas para usar com sua novíssima arquitetura de microsserviços, mas peço que você não caia nessa tentação. Adotar qualquer tecnologia nova tem um custo – sempre haverá perturbações. Espera-se que ela vá compensar (se você escolheu a tecnologia correta, é claro!), mas, ao adotar inicialmente uma arquitetura de microsserviços, haverá muita coisa acontecendo.

Determinar a maneira apropriada de evoluir e gerenciar uma arquitetura de microsserviços envolve enfrentar diversos desafios relacionados aos sistemas distribuídos – desafios que talvez você não tenha enfrentado antes. Acho muito mais conveniente que você pense nesses problemas à medida que deparar com eles, fazendo uso de uma pilha de tecnologias com as quais você tenha familiaridade e, então, considere se mudar a tecnologia existente pode ajudar a resolver esses problemas quando os encontrar.

Conforme já mencionamos, os microsserviços são, basicamente, independentes de tecnologia. Desde que seus serviços possam se comunicar uns com os outros por meio de uma rede, tudo mais estará à disposição. Essa pode ser uma grande vantagem – poder

misturar e combinar diferentes pilhas de tecnologias, caso você queira.

Não é necessário usar Kubernetes, Docker, contêineres ou a nuvem pública. Você não precisa escrever código em Go ou em Rust, ou em qualquer outra linguagem. Com efeito, sua escolha da linguagem de programação não será muito importante quando se trata de arquiteturas de microsserviços, exceto pelo fato de que algumas linguagens poderão ter um ecossistema mais rico de bibliotecas e frameworks para suporte. Se você tem mais conhecimento de PHP, comece a desenvolver serviços com PHP!¹ Há muito esnobismo técnico em relação a algumas pilhas de tecnologia por aí, que, infelizmente, beira a um desdém pelas pessoas que trabalham com determinadas ferramentas.² Não seja parte do problema! Escolha a abordagem mais apropriada para você e faça alterações para solucionar os problemas à medida que, e quando, você os vir.

Tamanho

“Qual deverá ser o tamanho de um microsserviço?” provavelmente é a pergunta que ouço com mais frequência. Considerando que o prefixo “micro” está presente no nome, isso não é de se surpreender. No entanto, quando procuramos saber os motivos pelos quais os microsserviços funcionam como um tipo de arquitetura, o conceito de tamanho, na verdade, é um dos pontos menos interessantes.

Como avaliamos o tamanho? Em linhas de código? Isso não faz muito sentido para mim. Algo que exigiria 25 linhas de código em Java poderia, possivelmente, ser escrito com 10 linhas de Clojure. Isso não quer dizer que Clojure seja melhor ou pior do que Java, mas que algumas linguagens são mais expressivas que outras.

O mais próximo que consigo pensar sobre o fato de o “tamanho” ter algum sentido no que diz respeito aos microsserviços é em algo que Chris Richardson, especialista em microsserviços, disse uma vez:

que o objetivo dos microsserviços é ter “a menor interface possível”. Isso está alinhado com o conceito de ocultação de informações (que será discutido em breve), mas representa uma tentativa de encontrar um sentido após o fato consumado – quando estávamos discutindo inicialmente esses assuntos, nosso foco principal, ao menos no princípio, era que esses componentes deveriam ser realmente fáceis de ser substituídos.

Em última instância, o conceito de “tamanho” é extremamente contextual. Fale com uma pessoa que tenha trabalhado em um sistema por 15 anos, e ela achará que seu sistema com 100K linhas de código é realmente fácil de entender. Peça a opinião de alguém que seja novo no projeto, e ela dirá que o sistema é grande demais. Do mesmo modo, pergunte a uma empresa que tenha acabado de iniciar sua transição para os microsserviços, talvez com dez ou menos microsserviços, e você obterá uma resposta diferente daquela que teria de uma empresa de porte semelhante, na qual os microsserviços sejam a norma há vários anos, e que tenha atualmente centenas deles.

Peço às pessoas que não se preocupem com o tamanho. No início, será muito mais importante que você se concentre em dois pontos essenciais. Em primeiro lugar, com quantos microsserviços você é capaz de lidar? À medida que houver mais serviços, a complexidade de seu sistema aumentará e você terá de adquirir novas habilidades (e, talvez, adotar novas tecnologias) para lidar com a situação. É por esse motivo que sou um forte defensor de uma migração gradual para uma arquitetura de microsserviços. Em segundo lugar, como definimos as fronteiras dos microsserviços a fim de tirar o máximo proveito deles, sem que tudo se transforme em uma terrível confusão, com alto grau de acoplamento? Esses assuntos serão discutidos na parte restante deste capítulo.

História do termo “microsserviços”

Em 2011, quando ainda trabalhava em uma empresa de consultoria chamada ThoughtWorks, meu amigo e então colega

James Lewis ficou realmente interessado em algo que ele estava chamando de “microapps”. Ele havia percebido que esse padrão vinha sendo utilizado por algumas empresas que usavam arquiteturas orientadas a serviços – elas estavam otimizando essa arquitetura para facilitar a substituição de serviços. As empresas em questão estavam interessadas em fazer com que funcionalidades específicas fossem implantadas rapidamente, mas com vistas a permitir que fossem reescritas com outras pilhas de tecnologia se, e quando, tudo tivesse de ser escalado.

Na época, o fato que mais se destacava era quão pequeno era o escopo desses serviços. Alguns desses serviços podiam ser escritos (ou reescritos) em alguns dias. James passou a dizer que “os serviços não deveriam ser maiores que a minha cabeça”. A ideia era que o escopo da funcionalidade deveria ser fácil de compreender e, desse modo, fácil de mudar.

Posteriormente, em 2012, James estava compartilhando essas ideias em uma conferência de arquiteturas, na qual alguns de nós estavam presentes. Naquela sessão, discutimos o fato de que, na verdade, essas não eram aplicações autocontidas, portanto “microapps” não era um termo muito correto. Em seu lugar, “microserviços” parecia ser um nome mais adequado.³

Responsabilidade

Com os microserviços modelados com base no domínio de negócio, vemos um alinhamento entre nossos artefatos de TI (nossos microserviços que podem ser implantados de modo independente) e nosso domínio de negócio. Essa ideia tem boa repercussão quando consideramos a mudança das empresas de tecnologia que estão acabando com as divisões entre “O Negócio” e o “TI”. Em empresas tradicionais de TI, o ato de desenvolver software muitas vezes é gerenciado por uma parte totalmente separada dos negócios, em relação àquela que realmente define os requisitos e tem uma ligação com o cliente, como mostra a Figura 1.4. As disfunções desses tipos de organizações são tantas e tão

variadas que, provavelmente, não precisam ser mais bem detalhadas neste livro.

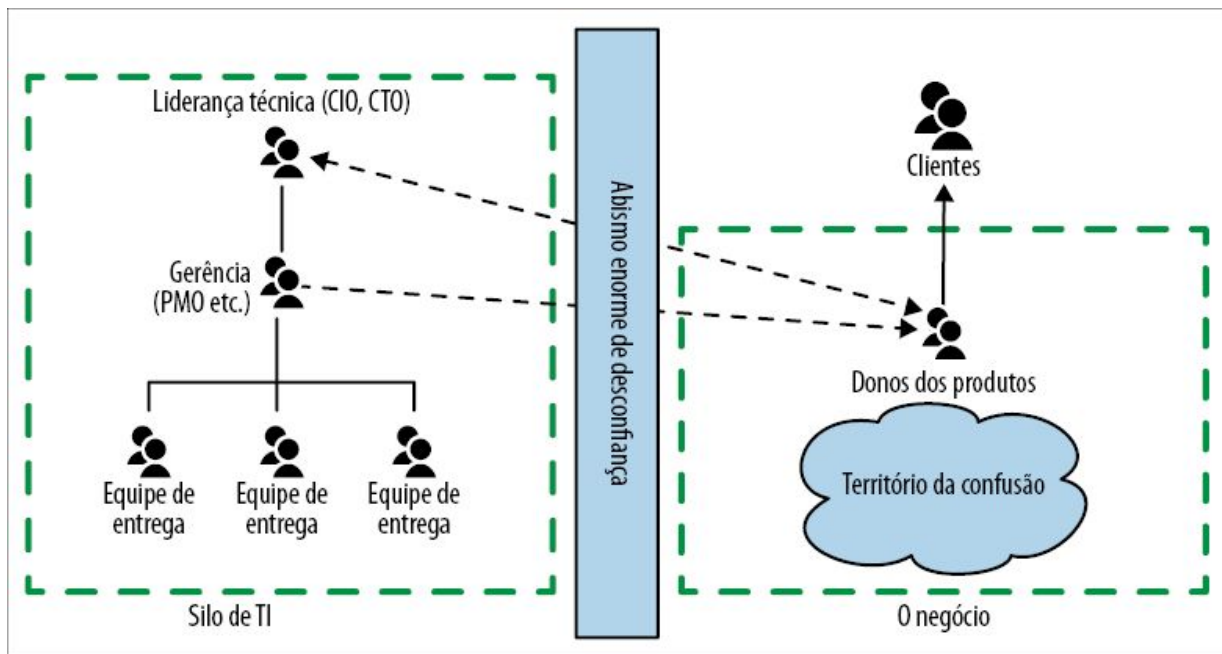


Figura 1.4 – Uma visão organizacional da tradicional divisão entre TI/negócios.

Hoje em dia, vemos empresas que são realmente de tecnologia combinarem totalmente esses silos organizacionais díspares de antes, conforme mostra a Figura 1.5. Os donos dos produtos agora trabalham diretamente como parte das equipes de entrega, com essas equipes alinhadas em torno das linhas de produto voltadas aos clientes, e não em torno de agrupamentos técnicos arbitrários. Em vez de funções de TI centralizadas serem a norma, a existência de qualquer função central de TI serve para oferecer suporte a essas equipes de entrega com foco nos clientes.

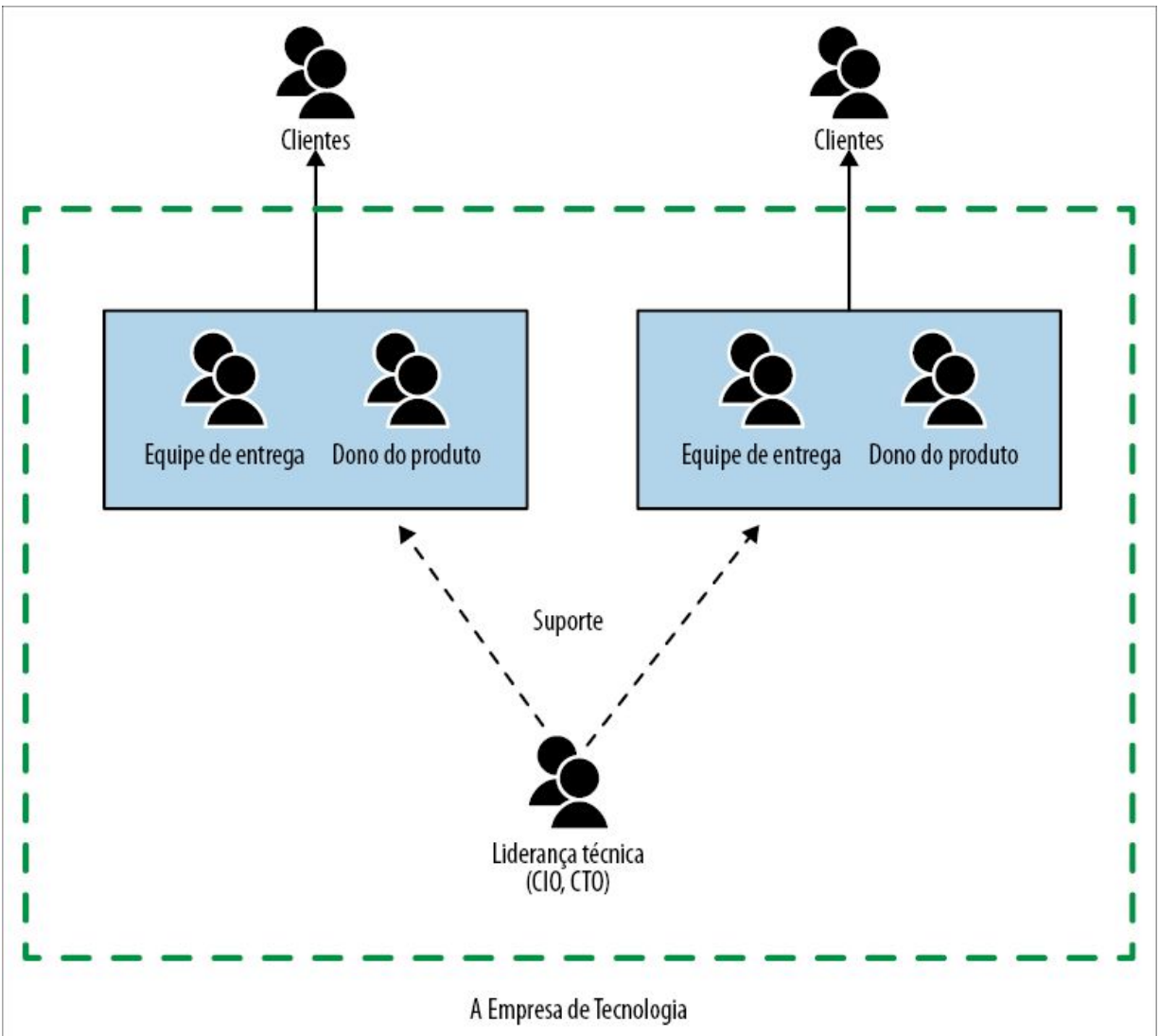


Figura 1.5 – Um exemplo de como empresas que são realmente de tecnologia estão integrando a entrega de softwares.

Embora nem todas as empresas tenham feito essa transição, as arquiteturas de microsserviços facilitam muito essa mudança. Se você quiser que as equipes de entrega estejam alinhadas em torno das linhas de produto, e os serviços estejam alinhados ao domínio de negócio, será mais fácil atribuir claramente as responsabilidades a essas equipes de entrega orientadas a produtos. Reduzir os serviços que são compartilhados entre várias equipes é essencial para minimizar a desavença nas entregas – arquiteturas de microsserviços orientadas a domínio de negócio fazem com que essa mudança nas estruturas organizacionais seja muito mais

simples.

Arquitetura monolítica

Falamos sobre microsserviços, mas este livro diz respeito a como passar *de* uma arquitetura monolítica *para* os microsserviços, portanto precisamos definir também o que queremos dizer com o termo *monolítico*.

Quando falo de sistemas monolíticos neste livro, refiro-me principalmente a uma unidade de implantação. Quando todas as funcionalidades de um sistema tiverem de ser implantadas em conjunto, esse sistema será considerado monolítico. Há pelo menos três tipos de sistemas monolíticos que se enquadram nesse perfil: o sistema com um único processo, o sistema monolítico distribuído e sistemas caixa-preta de terceiros.

Sistema monolítico de um só processo

O exemplo mais comum que vêm à mente quando discutimos sistemas monolíticos é um sistema no qual todo o código é implantado como um *único processo*, como mostra a Figura 1.6. Podemos ter várias instâncias desse processo por questões de robustez ou escala, mas, basicamente, todo o código estará contido em um único processo. Na verdade, esses sistemas com um único processo podem ser considerados como sistemas distribuídos simples, pois quase sempre acabam lendo ou armazenando dados em um banco de dados.

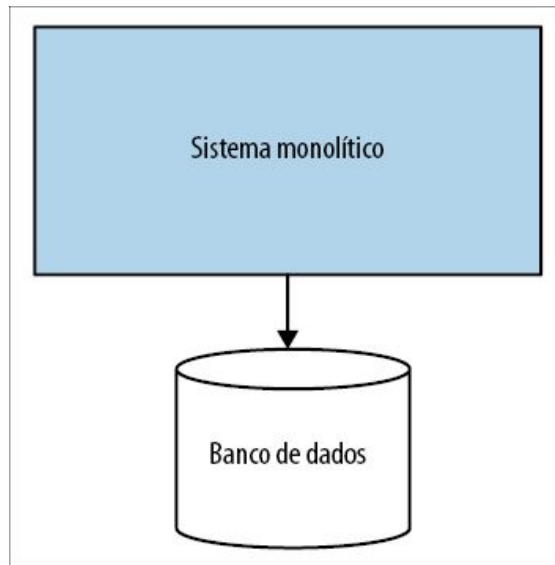


Figura 1.6 – Um sistema monolítico com um só processo: todo o código está contido em um único processo.

Esses sistemas monolíticos com um único processo provavelmente representam a grande maioria dos sistemas monolíticos com os quais vejo as pessoas lutando e, desse modo, são o tipo de sistemas monolíticos nos quais manteremos o foco na maior parte de nosso tempo. Quando utilizar o termo “sistema monolítico” a partir de agora, estarei falando desses tipos de sistemas, a menos que eu diga o contrário.

Sistema monolítico modular

Como um subconjunto dos sistemas monolíticos com um só processo, o *sistema monolítico modular* é uma variação: o processo único é composto de módulos separados e é possível trabalhar em cada módulo de forma independente, mas esses ainda devem ser combinados na implantação, como mostra a Figura 1.7. O conceito de separar o software em módulos não é nenhuma novidade; veremos parte da história relacionada a esse assunto mais adiante neste capítulo.

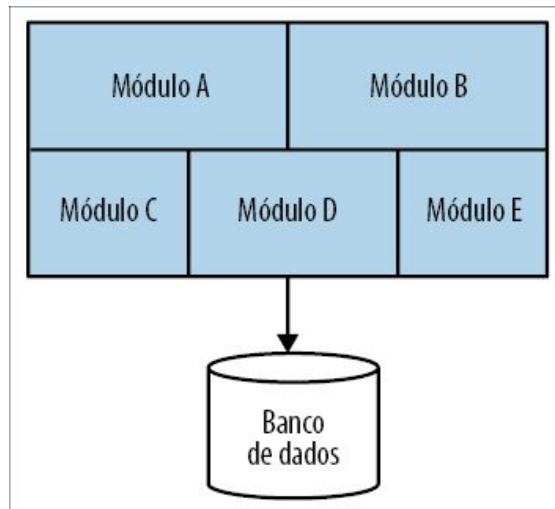


Figura 1.7 – Um sistema monolítico modular: o código do processo é dividido em módulos.

Para muitas empresas, o sistema monolítico modular pode ser uma ótima opção. Se as fronteiras dos módulos estiverem bem definidas, isso pode possibilitar um elevado grau de paralelismo nos trabalhos, tirando do caminho os desafios da arquitetura de microsserviços mais distribuída, havendo preocupações mais simples com a implantação. O Shopify é um ótimo exemplo de uma empresa que utilizou essa técnica como uma alternativa à decomposição em microsserviços, e ela parece funcionar muito bem para essa empresa.⁴

Um dos desafios de um sistema monolítico modular é que o banco de dados tende a não ter uma decomposição similar à que vemos no nível do código, resultando em desafios significativos que poderão ter de ser enfrentados caso você queira acabar com o sistema monolítico no futuro. Já vi algumas equipes tentarem levar adiante a ideia de um sistema monolítico modular, fazendo o banco de dados ser decomposto do mesmo modo que os módulos, como mostra a Figura 1.8. Basicamente, fazer uma alteração como essa em um sistema monolítico existente ainda pode ser bastante desafiador, mesmo que você não modifique o código – muitos dos padrões que exploraremos no Capítulo 4 poderão ajudar, se você quiser tentar fazer algo parecido.

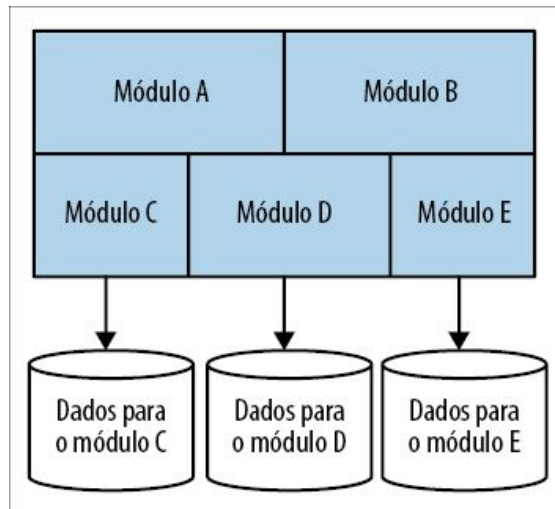


Figura 1.8 – Um sistema monolítico modular com um banco de dados decomposto.

Sistema monolítico distribuído

*Um sistema distribuído é um sistema no qual a falha em um computador que você nem sequer sabia que existia pode deixar o seu próprio computador inutilizável.*⁵

– LESLIE LAMPORT

Um *sistema monolítico distribuído* é um sistema composto de vários serviços, mas, por algum motivo, o sistema como um todo deve ser implantado em conjunto. Um sistema monolítico distribuído pode muito bem corresponder à definição de uma arquitetura orientada a serviços, porém, com muita frequência, não é capaz de cumprir as promessas de uma SOA. De acordo com minha experiência, os sistemas monolíticos distribuídos têm todas as desvantagens de um sistema distribuído e as desvantagens de um sistema monolítico de um único processo, sem ter vantagens suficientes de nenhum dos dois tipos de sistema. Ter deparado com sistemas monolíticos distribuídos em meu trabalho em boa parte teve influência em meu próprio interesse pela arquitetura de microsserviços.

Em geral, os sistemas monolíticos distribuídos surgem em um ambiente em que não houve foco suficiente em conceitos como ocultação de informações e coesão nas funcionalidades de negócio, resultando em arquiteturas extremamente acopladas, nas quais as mudanças se propagam pelas fronteiras dos serviços, e alterações aparentemente inocentes, que parecem ser locais quanto ao escopo, causam falhas em outras partes do sistema.

Sistemas caixa-preta de terceiros

Também podemos considerar alguns *softwares de terceiros* como sistemas monolíticos que podemos querer “decompor” como parte de um esforço de migração. Esses poderiam incluir componentes como sistemas de folha de pagamento, sistemas CRM e sistemas de RH. O fator comum, nesse caso, é que são softwares desenvolvidos por outras pessoas, e você não será capaz de modificar o código. Poderia ser um software de prateleira que você tenha implantado em sua infraestrutura, ou um produto SaaS

(Software-as-a-Service, ou Software como Serviço) que você esteja usando. Muitas das técnicas de decomposição que exploraremos neste livro podem ser usadas mesmo com sistemas para os quais não seja possível modificar o código subjacente.

Desafios dos sistemas monolíticos

Um sistema monolítico, seja um sistema com um único processo ou um sistema distribuído, com frequência é mais vulnerável aos perigos causados pelo acoplamento – particularmente, acoplamentos de implementação e de implantação, assuntos que exploraremos melhor em breve.

À medida que você tiver mais pessoas trabalhando no mesmo lugar, elas começarão a atrapalhar umas às outras. Desenvolvedores diferentes querendo alterar a mesma parte do código, diferentes equipes querendo implantar funcionalidades em diferentes horários (ou atrasar as implantações). Haverá confusão sobre quem é dono de quê, e quem toma as decisões. Diversos estudos mostram os desafios das linhas confusas de responsabilidade.⁶ Chamo a esse problema de *desavença nas entregas*.

Ter um sistema monolítico não significa que você vai definitivamente deparar com os desafios da desavença nas entregas, não mais do que ter uma arquitetura de microsserviços significa que você jamais vai ter de encarar o problema. No entanto, uma arquitetura de microsserviços fornece fronteiras mais concretas em um sistema quanto às linhas que marcam quem são os responsáveis, proporcionando mais flexibilidade para reduzir esse problema.

Vantagens dos sistemas monolíticos

O sistema monolítico com um só processo, porém, também tem uma grande gama de vantagens. Sua topologia de implantação muito mais simples é capaz de evitar muitas das armadilhas associadas aos sistemas distribuídos. Esse tipo de sistema pode resultar em fluxos de trabalho muito mais simples para os

desenvolvedores; além disso, atividades como monitoração, resolução de problemas e testes fim a fim podem ser bastante simplificados.

Os sistemas monolíticos também podem simplificar a reutilização de código dentro do próprio sistema monolítico. Se quisermos reutilizar código em um sistema distribuído, devemos decidir se queremos copiar o código, separar as bibliotecas ou passar a funcionalidade compartilhada para um serviço. Em um sistema monolítico, nossas opções serão mais simples, e muitas pessoas gostam dessa simplicidade – todo o código estará ali, portanto basta usá-lo!

Infelizmente, as pessoas passaram a ver os sistemas monolíticos como algo a ser evitado – algo que é inerentemente problemático. Já conheci várias pessoas a quem o termo sistema *monolítico* é sinônimo de *legado*. Isso é um problema. Uma arquitetura monolítica é uma opção – e é uma opção válida. Pode não ser a opção correta para todas as circunstâncias, não mais do que os microsserviços seriam – mas, apesar de tudo, ainda é uma opção. Se cairmos na armadilha de denegrir sistematicamente o sistema monolítico como uma opção viável para entregar nosso software, correremos o risco de não agirmos corretamente, para nós mesmos ou para os usuários de nosso software. Exploraremos melhor o custo-benefício associado aos sistemas monolíticos e aos microsserviços no Capítulo 3, e discutiremos algumas ferramentas que ajudarão a avaliar melhor o que é apropriado para o seu contexto.

Sobre o acoplamento e a coesão

Compreender as forças de equilíbrio entre acoplamento e coesão é importante para definir as fronteiras dos microsserviços. O *acoplamento* diz respeito a uma alteração em um local exigir uma mudança em outro; a *coesão* concerne ao modo como agrupamos códigos relacionados. Esses conceitos estão diretamente ligados. A lei de Constantine articula muito bem essa relação:

Uma estrutura será estável se a coesão for alta e o acoplamento for baixo.

– LARRY CONSTANTINE

Parece ser uma observação sensata e conveniente. Se tivermos duas porções de código altamente relacionadas, a coesão será baixa, pois a funcionalidade associada estará distribuída entre elas. Também teremos um alto acoplamento, pois, quando esse código relacionado mudar, as duas partes terão de ser alteradas.

Se a estrutura de nosso sistema de códigos mudar, será custoso lidar com isso, pois o custo de mudanças que cruzem as fronteiras dos serviços em sistemas distribuídos é muito alto. Ter de fazer mudanças em um ou mais serviços que possam ser implantados de modo independente, talvez lidando com o impacto de mudanças que causem ruptura nos contratos dos serviços, provavelmente será uma dor de cabeça enorme.

O problema com o sistema monolítico é que, com muita frequência, ele é o oposto de ambos. Em vez de apresentar uma tendência à coesão, e manter junto aquilo que tende a mudar junto, nós colocamos todo tipo de código não relacionado juntos. Do mesmo modo, um baixo acoplamento não existe realmente: se eu quiser modificar uma linha de código, pode ser que seja possível fazer isso facilmente, mas não poderei implantar essa modificação sem um possível impacto na maior parte do código monolítico restante, e, certamente, terei de reimplantar todo o sistema.

Também queremos que o sistema tenha estabilidade, pois nosso objetivo, quando possível, é incorporar o conceito de implantações independentes – isto é, gostaríamos de poder fazer uma alteração em nosso serviço e implantá-lo no ambiente de produção *sem ter de fazer outras alterações*. Para que isso funcione, é necessário que os serviços que consumimos tenham estabilidade, e devemos fornecer um contrato estável para os serviços que sejam nossos consumidores.

Considerando a infinidade de informações que há por aí acerca desses termos, seria tolo de minha parte apresentar explicações longas demais neste livro, mas acho que um resumo se faz necessário, particularmente, para colocar essas ideias no contexto das arquiteturas de microsserviços. Em última instância, esses conceitos de coesão e de acoplamento influenciam bastante o modo como pensamos na arquitetura de microsserviços. E isso não é nenhuma surpresa – a coesão e o acoplamento são preocupações associadas a um software modular, e o que é a arquitetura de microsserviços além de módulos que se comunicam por meio de redes e que podem ser implantados de forma independente?

Uma breve história do acoplamento e da coesão

Os conceitos de coesão e de acoplamento estão presentes na computação há muito tempo, e foram inicialmente apresentados por Larry Constantine em 1968. Essas ideias gêmeas de acoplamento e coesão evoluíram para formar a base de boa parte de como pensamos na escrita de programas de computador. Livros como *Structured Design* de Larry Constantine & Edward Yourdon (Prentice Hall, 1979)⁷, posteriormente, influenciaram gerações de programadores (era uma leitura obrigatória em minha própria graduação, quase vinte anos depois da primeira publicação).

Larry apresentou pela primeira vez o seu conceito de coesão e de acoplamento em 1968 (um ano particularmente auspicioso para a computação) no National Symposium on Modular Programming (Simpósio Nacional de Programação Modular) – a mesma conferência em que a lei de Conway recebeu seu nome. Naquele ano, também houve duas conferências patrocinadas pela atualmente infame OTAN, durante as quais a engenharia de software como um conceito também adquiriu relevância (um termo cunhado antes por Margaret H. Hamilton).

Coesão

Uma das definições mais sucintas que já ouvi para descrever a

coesão é esta: “códigos que mudam juntos devem permanecer juntos”. Para o nosso contexto, é uma definição muito boa. Conforme já discutimos, estamos otimizando nossa arquitetura de microsserviços visando à facilidade de alterações nas funcionalidades de negócio – portanto, queremos que as funcionalidades estejam agrupadas de modo que possamos fazer alterações no menor número possível de lugares.

Se eu quiser mudar o modo como a aprovação de uma fatura é gerenciada, não quero ter de procurar a funcionalidade que precisa ser modificada em vários serviços, e então coordenar o lançamento desses serviços recém-modificados a fim de fazer o rollout da nova funcionalidade. Em vez disso, gostaria de garantir que a alteração envolva modificações no menor número possível de serviços, de modo que o custo da modificação permaneça baixo.

Acoplamento

Ocultação de informações, assim como uma dieta, é algo mais fácil de descrever do que de fazer.

– DAVID PARNAS, **THE SECRET HISTORY OF INFORMATION HIDING**

Gostamos da coesão, é claro, mas devemos tomar cuidado com o acoplamento. Quanto mais partes estiverem “acopladas”, mais elas terão de mudar juntas. No entanto, há diferentes tipos de acoplamento, e cada tipo pode exigir soluções distintas.

Houve muitas tentativas anteriores no que diz respeito a classificar os tipos de acoplamento, com destaque ao trabalho feito por Meyer, Yourdan e Constantine. Apresentarei o meu próprio trabalho, não para dizer que o trabalho feito anteriormente está incorreto, mas porque acho minha classificação mais conveniente para ajudar as pessoas a compreender os aspectos associados ao acoplamento em sistemas distribuídos. Desse modo, não tenho a intenção de apresentar uma classificação exaustiva das diferentes formas de acoplamento.

Ocultação de informações

Um conceito que sempre retorna quando se trata de discussões sobre acoplamento é a técnica chamada de *ocultação de informações*. Esse conceito, apresentado inicialmente por David Parnas em 1971, teve origem em seu trabalho que analisava o modo de definir fronteiras entre os módulos.⁸

A ideia principal na ocultação de informações é separar as partes do código que mudam frequentemente daquelas que são estáticas. Queremos que a fronteira do módulo seja estável, e ele deve ocultar as partes da implementação que esperamos mudar com mais frequência. A ideia é que alterações internas possam ser feitas com segurança, desde que a compatibilidade com o módulo seja mantida.

Pessoalmente, adoto a abordagem de expor o mínimo possível na fronteira de um módulo (ou de um microserviço). Assim que algo passa a fazer parte da interface de um módulo, será difícil voltar atrás. Contudo, se você ocultar um aspecto agora, sempre

será possível decidir compartilhá-lo depois.

O encapsulamento como um conceito de software orientado a objetos (OO) está relacionado com o que foi descrito, mas, dependendo da definição que você adotar, talvez não seja exatamente a mesma coisa. O encapsulamento na programação OO passou a significar o empacotamento de um ou mais itens em um contêiner – pense em uma classe contendo tanto os campos como os métodos que atuam nesses campos. Você poderia então usar a visibilidade na definição da classe para ocultar partes da implementação.

Para uma exploração mais detalhada da história da ocultação de informações, recomendo o artigo “The Secret History of Information Hiding” de Parnass.⁹

Acoplamento de implementação

Em geral, o *acoplamento de implementação* é a forma mais prejudicial de acoplamento que costumo ver, mas, felizmente para nós, com frequência, é a mais fácil de ser reduzida. No acoplamento de implementação, A está acoplado a B no que diz respeito ao modo como B está implementado – quando a implementação de B mudar, A também mudará.

O problema, nesse caso, é que os detalhes de implementação geralmente são uma escolha arbitrária dos desenvolvedores. Há várias maneiras de resolver um problema; escolhemos uma, mas podemos mudar de ideia. Quando decidimos mudar de ideia, não queremos que isso cause problemas para os consumidores (lembre-se das implantações independentes).

Um exemplo clássico e comum de acoplamento de implementação se apresenta na forma de um compartilhamento de banco de dados. Na Figura 1.9, nosso serviço Pedido contém um registro de todos os pedidos feitos em nosso sistema. O serviço Recomendação sugere discos que nossos clientes talvez queiram comprar com base em compras anteriores. Atualmente, o serviço Recomendação acessa diretamente esses dados no banco de dados.

As recomendações exigem informações sobre os pedidos que foram feitos. Até certo ponto, esse é um acoplamento de *domínio* inevitável, sobre o qual falaremos em breve. Contudo, nessa situação em particular, estamos acoplados a uma estrutura específica de esquema, ao dialeto SQL e, talvez, até mesmo ao conteúdo das linhas. Se o serviço Pedido mudar o nome de uma coluna, dividir a tabela Pedido de Cliente ou fazer qualquer outra mudança, conceitualmente, ela ainda conterá as informações de pedidos, mas haverá um problema no modo como o serviço Recomendação busca essas informações. Uma opção melhor seria ocultar esses detalhes de implementação, como mostra a Figura 1.10 – agora, o serviço Recomendação acessa as informações de que precisa por meio de uma chamada de API.

Também poderíamos fazer o serviço Pedido publicar um conjunto de dados, na forma de um banco de dados, cujo propósito seria ser usado para um acesso geral dos consumidores – como vemos na Figura 1.11. Desde que o serviço Pedido possa publicar os dados apropriados, qualquer mudança feita no serviço Pedido será invisível aos consumidores, pois ele manterá o contrato público. Essa solução também cria uma oportunidade para melhorar o modelo de dados exposto aos consumidores, ajustando-os de acordo com suas necessidades. Exploraremos padrões como esse com mais detalhes nos capítulos 3 e 4.

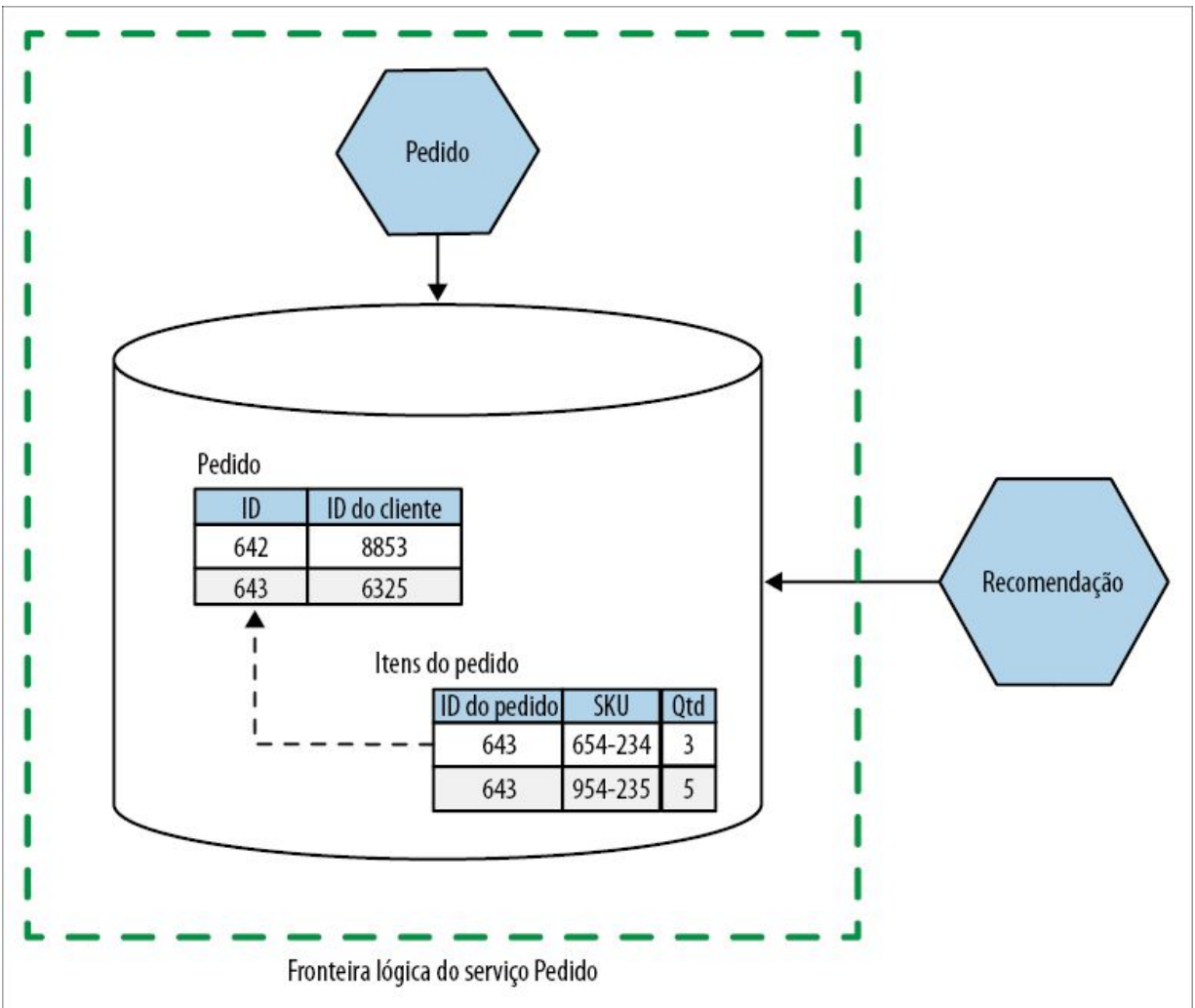


Figura 1.9 – O serviço Recomendação acessa diretamente os dados armazenados no serviço Pedido.

Com efeito, nos dois exemplos anteriores, estamos fazendo uso da ocultação de informações. O ato de ocultar um banco de dados atrás de uma interface de serviço bem definida permite que o serviço limite o escopo do que é exposto, e pode permitir que façamos modificações no modo como esses dados são representados.

Outro truque interessante é usar a ideia de trabalhar “de fora para dentro” quando se trata de definir a interface de um serviço – determine a interface do serviço pensando na situação inicialmente do ponto de vista dos consumidores do serviço e, em seguida, defina como implementar o contrato desse serviço. A abordagem

alternativa (que, infelizmente, vejo ser bastante comum) é fazer o inverso. A equipe que trabalha com o serviço adota um modelo de dados, ou outro detalhe de implementação interno, e então pensa em expô-lo ao mundo.

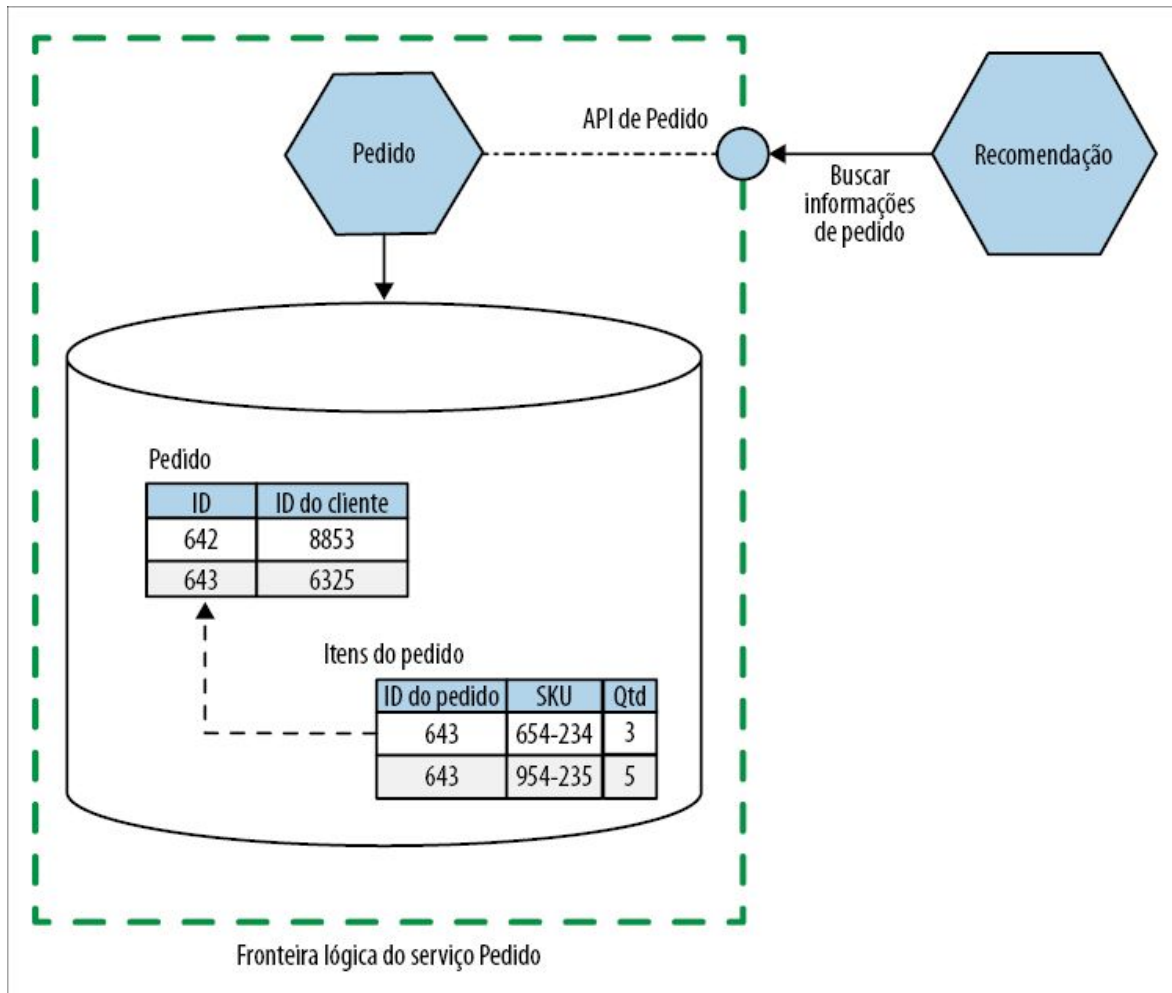


Figura 1.10 – O serviço Recomendação agora acessa as informações de pedido por meio de uma API, ocultando os detalhes de implementação internos.

Ao pensar “de fora para dentro”, você inicialmente pergunta: “Do que é que os consumidores de meu serviço precisam?”. Com isso, não quero dizer que você perguntará *a si mesmo* o que seus consumidores precisam; quero dizer que você realmente deve perguntar às pessoas que chamarão o seu serviço!



Trate as interfaces de serviço expostas pelo seu microserviço como uma interface de usuário. Utilize o pensamento “de fora para dentro” para modelar o design da interface, em parceria com as pessoas que chamarão o seu serviço.

Pense no contrato de seu serviço com o mundo externo como uma interface de usuário. Ao fazer o design de uma interface de usuário, você pergunta aos seus usuários o que eles querem e faz iterações no design junto a eles. Você deve modelar o contrato de seu serviço da mesma maneira. Além do fato de significar que você terá um serviço mais fácil de ser usado pelos seus consumidores, isso também ajudará a manter um certo nível de separação entre o contrato externo e a implementação interna.

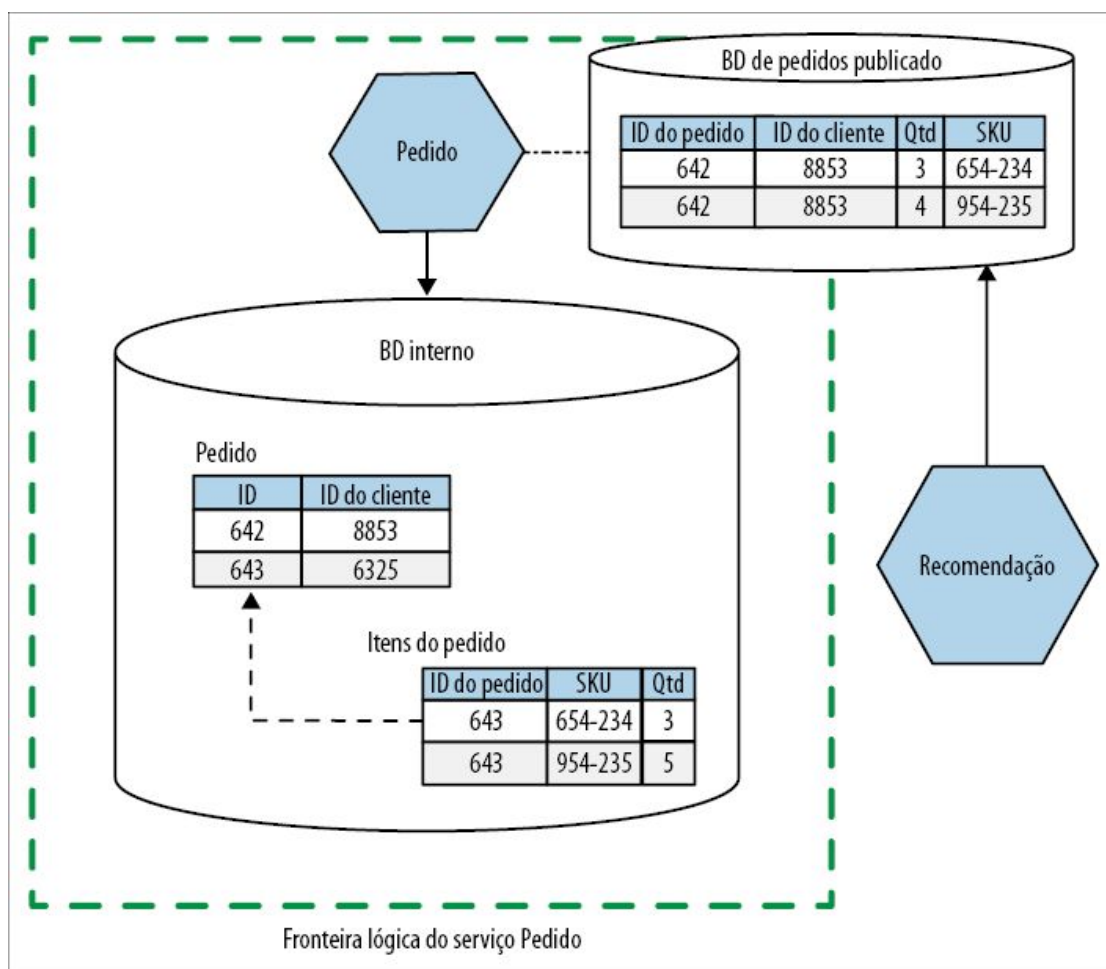


Figura 1.11 – O serviço Recomendação agora acessa as informações de pedido em um banco de dados exposto, que é

estruturado de modo diferente do banco de dados interno.

Acoplamento temporal

Um *acoplamento temporal* é uma preocupação essencialmente durante a execução, e que, em geral, está relacionado a um dos principais desafios das chamadas síncronas em um ambiente distribuído. Quando uma mensagem é enviada, e o modo como essa mensagem é tratada está conectada no tempo, dizemos que há um acoplamento temporal. Isso pode soar um pouco estranho, portanto, vamos analisar um exemplo explícito na Figura 1.12.

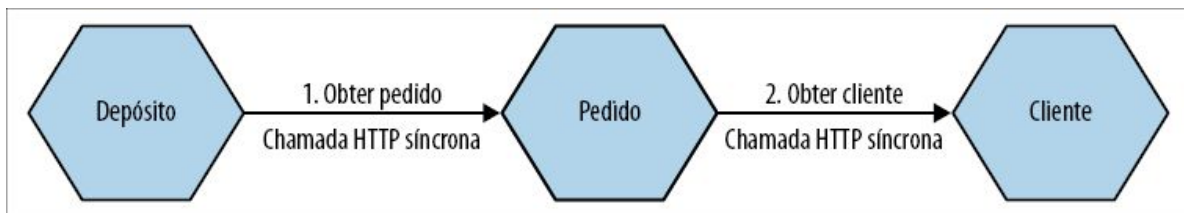


Figura 1.12 – Podemos dizer que três serviços que fazem uso de chamadas síncronas para realizar uma operação apresentam um acoplamento temporal.

Nesse caso, vemos uma chamada HTTP síncrona feita a partir do serviço Depósito para um serviço Pedido subsequente a fim de buscar informações necessárias sobre um pedido. Para satisfazer à requisição, o serviço Pedido, por sua vez, deve buscar informações do serviço Cliente, novamente, por meio de uma chamada HTTP síncrona. Para que essa operação como um todo seja concluída, os serviços Depósito, Pedido e Cliente devem estar ativos e acessíveis. Eles apresentam um acoplamento temporal.

É possível reduzir esse problema de várias maneiras. Poderíamos considerar o uso de caching – se o serviço Pedido fizesse caching das informações de que precisa do serviço Cliente, o serviço Pedido poderia evitar um acoplamento temporal com o serviço subsequente, em alguns casos. Também poderíamos considerar o uso de um meio de transporte assíncrono para enviar as requisições, talvez usando algo como um broker de mensagens. Isso permitiria que uma mensagem fosse enviada para um serviço

subsequente, e que essa mensagem fosse tratada depois que o serviço subsequente estiver disponível.

Uma exploração completa dos tipos de comunicação de serviço a serviço está fora do escopo deste livro, porém há uma discussão com mais detalhes no Capítulo 4 do livro *Building Microservices*.

Acoplamento de implantação

Considere um único processo, composto de vários módulos ligados estaticamente. Uma mudança é feita em uma única linha de código em um dos módulos, e queremos fazer a implantação dessa mudança. Para isso, é necessário fazer a implantação do sistema monolítico completo – incluindo até mesmo os módulos que não sofreram alterações. Tudo deverá ser implantado em conjunto, portanto, temos um *acoplamento de implantação*.

O acoplamento de implantação pode ser forçado, como no exemplo de nosso processo estaticamente ligado, mas também pode ser uma questão de opção, determinada por práticas como um release train. Em um release train, agendas planejadas para lançamento de versões são definidas com antecedência, em geral com uma agenda que se repete. Quando a data para o lançamento de uma versão chegar, todas as mudanças feitas desde o último release train são implantadas. Para algumas pessoas, o release train pode ser uma técnica conveniente, mas prefiro vê-la muito mais como um passo de transição em direção a técnicas apropriadas de release-on-demand, em vez de vê-la como um objetivo definitivo. Já trabalhei até mesmo em empresas que implantavam todos os serviços de um sistema de uma só vez como parte desses processos de release train, sem pensar se esses serviços precisavam ser alterados.

Fazer uma implantação implica riscos. Há muitas maneiras de reduzir os riscos de uma implantação, e uma delas é modificar apenas o que precisa ser modificado. Se pudermos reduzir o acoplamento de implantação, talvez decompondo processos maiores em microsserviços que possam ser implantados de modo independente, poderemos reduzir o risco de cada implantação

reduzindo o seu escopo.

Implantações menores representam menos riscos. Há menos partes para dar errado. Se algo der errado, descobrir o que foi e corrigir será mais fácil, pois teremos feito menos mudanças. Encontrar meios de reduzir o tamanho da implantação está no coração da entrega contínua, que enfatiza a importância de um feedback rápido e de métodos de release-on-demand.¹⁰ Quanto menor o escopo da implantação, mais fácil e mais seguro será o rollout, e mais rápido será receber um feedback. Meu próprio interesse nos microsserviços nasceu de um foco anterior na entrega contínua – eu estava à procura de arquiteturas que facilitassem a adoção da entrega contínua.

Reduzir o acoplamento de implantação não exige microsserviços, é claro. Runtimes como Erlang permitem um hot-deployment de novas versões de módulos em um processo em execução. Futuramente, talvez mais pessoas poderão ter acesso a recursos desse tipo nas pilhas de tecnologia usadas no dia a dia.¹¹

Acoplamento de domínio

Basicamente, em um sistema composto de vários serviços independentes, deve haver algum tipo de interação entre os participantes. Em uma arquitetura de microsserviços, o *acoplamento de domínio* é o resultado – as interações entre os serviços modelam as interações em nosso domínio real. Se quiser criar um pedido, você deve saber quais itens estavam no carrinho de compras de um cliente. Se quiser fazer a remessa de um produto, deve saber para onde deverá enviá-lo. Em nossa arquitetura de microsserviços, por definição, essas informações poderão estar contidas em serviços diferentes.

Para dar um exemplo concreto, considere a Music Corp. Temos um depósito que armazena os produtos. Quando os clientes fazem pedidos de CDs, os funcionários que trabalham no depósito devem saber quais itens devem ser selecionados e empacotados, e para onde o pacote deverá ser enviado. Assim, informações sobre o

pedido devem ser compartilhadas com as pessoas que trabalham no depósito.

A Figura 1.13 mostra um exemplo desse caso: um serviço Processamento de Pedido envia todos os detalhes do pedido para o serviço Depósito, que então dá início ao empacotamento do pedido. Como parte dessa operação, o serviço Depósito utiliza o ID do cliente para buscar informações sobre ele no serviço Cliente separado, para que saibamos como notificar esse cliente quando o pedido for enviado.

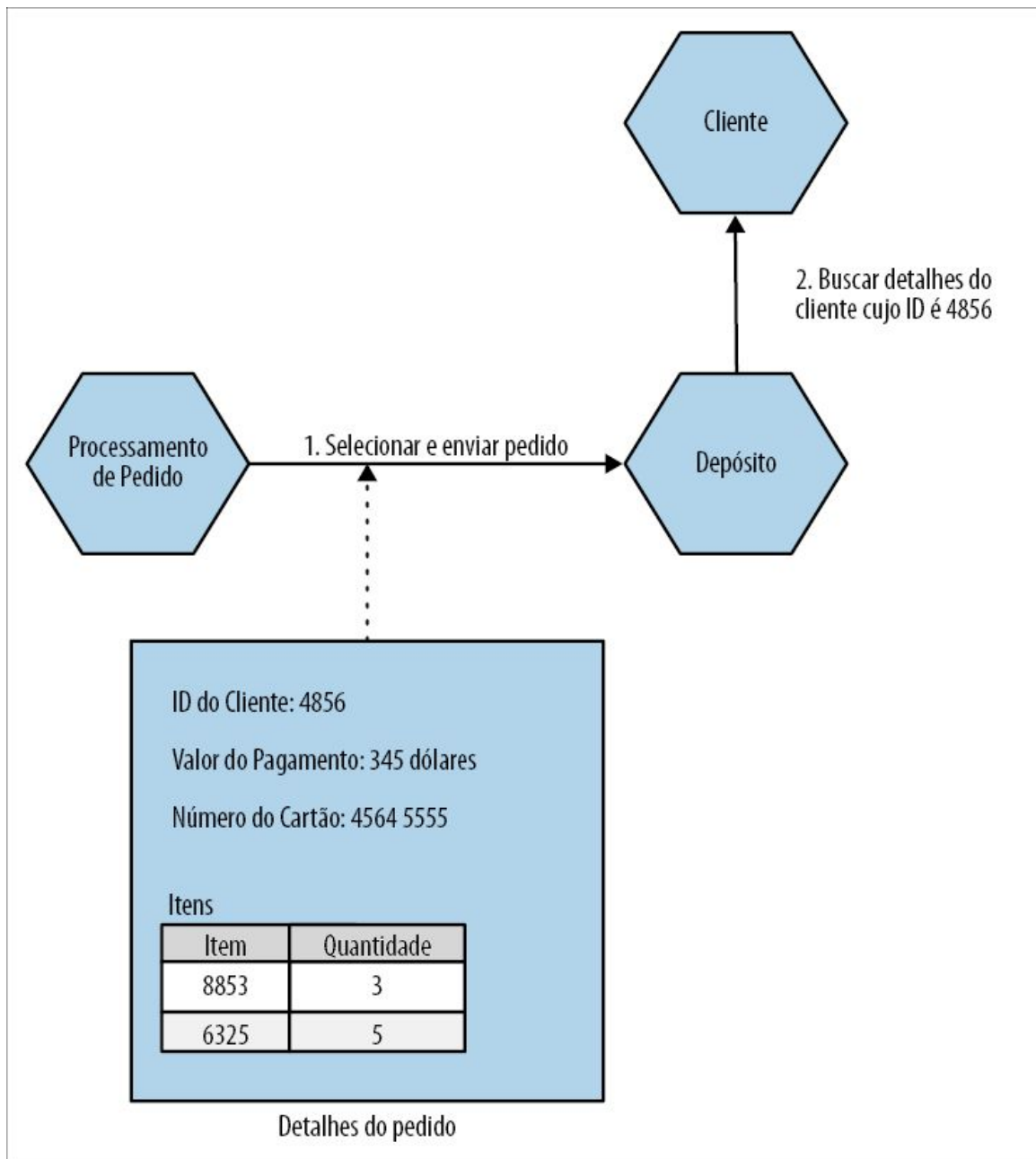


Figura 1.13 – Um pedido é enviado para o depósito para que o processo de embalagem possa ter início.

Nessa situação, estamos compartilhando o pedido completo com o depósito, o que pode não fazer sentido – o depósito só precisa das informações sobre o que deve ser embalado e para onde o pacote deve ser enviado. Eles não precisam saber quanto custa o item (se precisarem incluir uma nota fiscal com o pacote, ela poderia ser passada como um PDF previamente formatado). Também teríamos

problemas com o fato de informações cujo acesso deveríamos controlar estarem sendo amplamente compartilhadas – se compartilharmos o pedido completo, poderíamos acabar expondo detalhes sobre cartões de crédito para os serviços que não precisam desses dados, por exemplo.

Desse modo, poderíamos criar um conceito de Instrução de Seleção no domínio, contendo apenas as informações necessárias ao serviço Depósito, como vemos na Figura 1.14. Esse é outro exemplo de ocultação de informações.

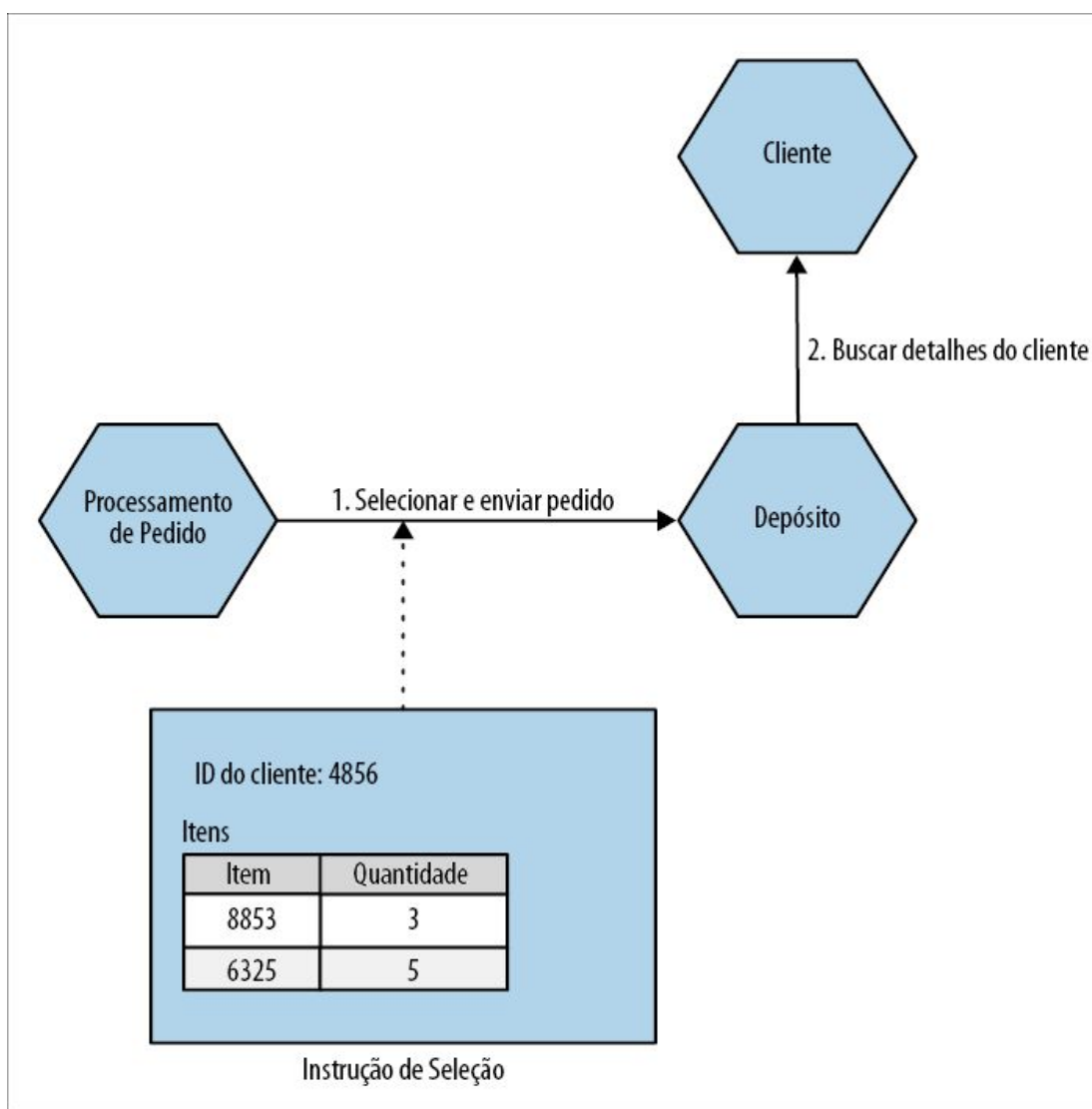


Figura 1.14 – Usando uma Instrução de Seleção para reduzir a quantidade de informações que enviamos para o serviço Depósito.

Poderíamos reduzir mais ainda o acoplamento eliminando até mesmo a necessidade de o serviço Depósito ter de saber sobre um cliente, se quisermos – poderíamos fornecer todos os detalhes apropriados por meio da Instrução de Seleção, como mostra a Figura 1.15.

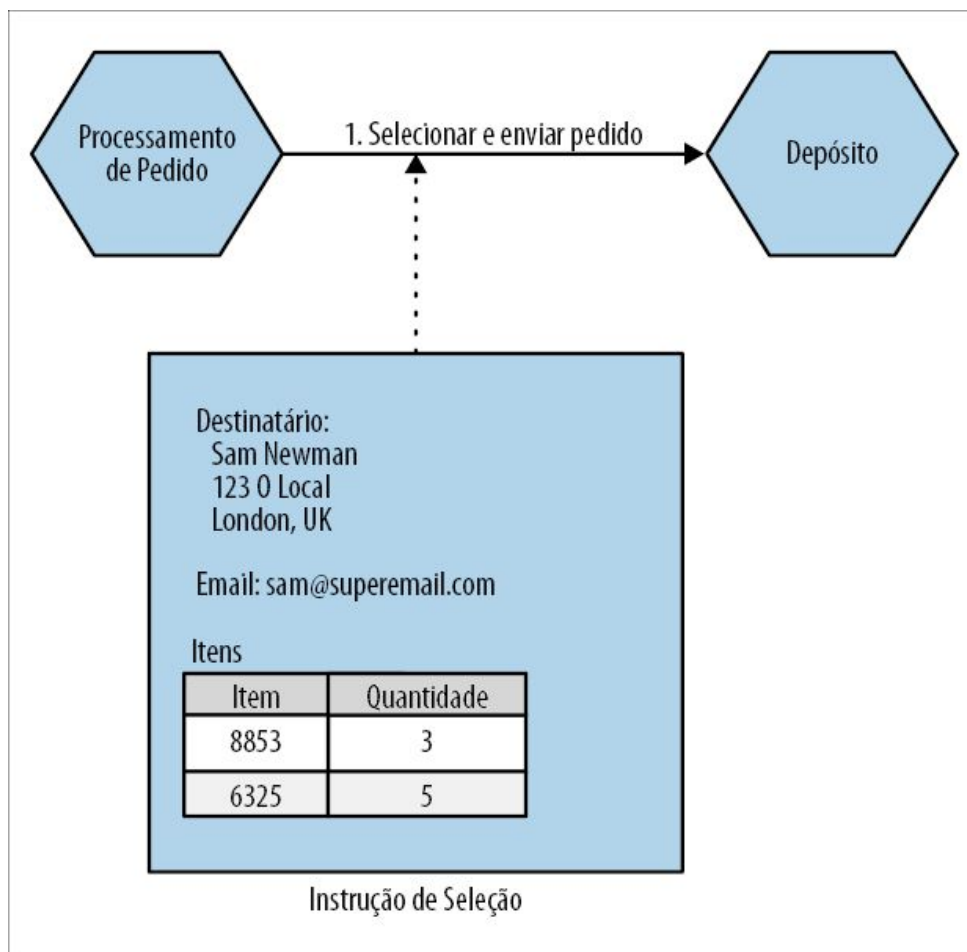


Figura 1.15 – Colocar mais informações na Instrução de Seleção pode evitar a necessidade de uma chamada ao serviço Cliente.

Para que essa abordagem funcione, é provável que, em algum momento, o Processamento de Pedido tenha de acessar o serviço Cliente para que possa, antes de tudo, gerar essa Instrução de Seleção; no entanto, de qualquer modo, esse Processamento de Pedido provavelmente teria de acessar informações do cliente por outros motivos, portanto, é improvável que isso seja um grande problema. Esse processo de “enviar” uma Instrução de Seleção

implica que uma chamada de API será feita de Processamento de Pedido para o serviço Depósito.

Uma alternativa seria fazer o Processamento de Pedido gerar algum tipo de evento que fosse consumido pelo Depósito, como mostra a Figura 1.16. Ao gerar um evento para que seja consumido por Depósito, invertemos efetivamente as dependências. Passamos de Processamento de Pedido dependendo do serviço Depósito para poder garantir que um pedido seja enviado, para Depósito ouvindo eventos do serviço Processamento de Pedido. As duas abordagens têm seus méritos, e a abordagem escolhida provavelmente dependeria de uma compreensão mais ampla das interações entre a lógica do Processamento de Pedido e a funcionalidade encapsulada pelo serviço Depósito – é algo com que algumas técnicas de modelagem de domínio poderão ajudar, e um assunto que será explorado a seguir.

Basicamente, algumas informações sobre um pedido são necessárias para que o serviço Depósito faça qualquer tarefa. Não podemos evitar esse nível de acoplamento de domínio. Contudo, ao pensar com cuidado sobre os conceitos que compartilhamos e como os compartilhamos, podemos visar à redução do nível de acoplamento.

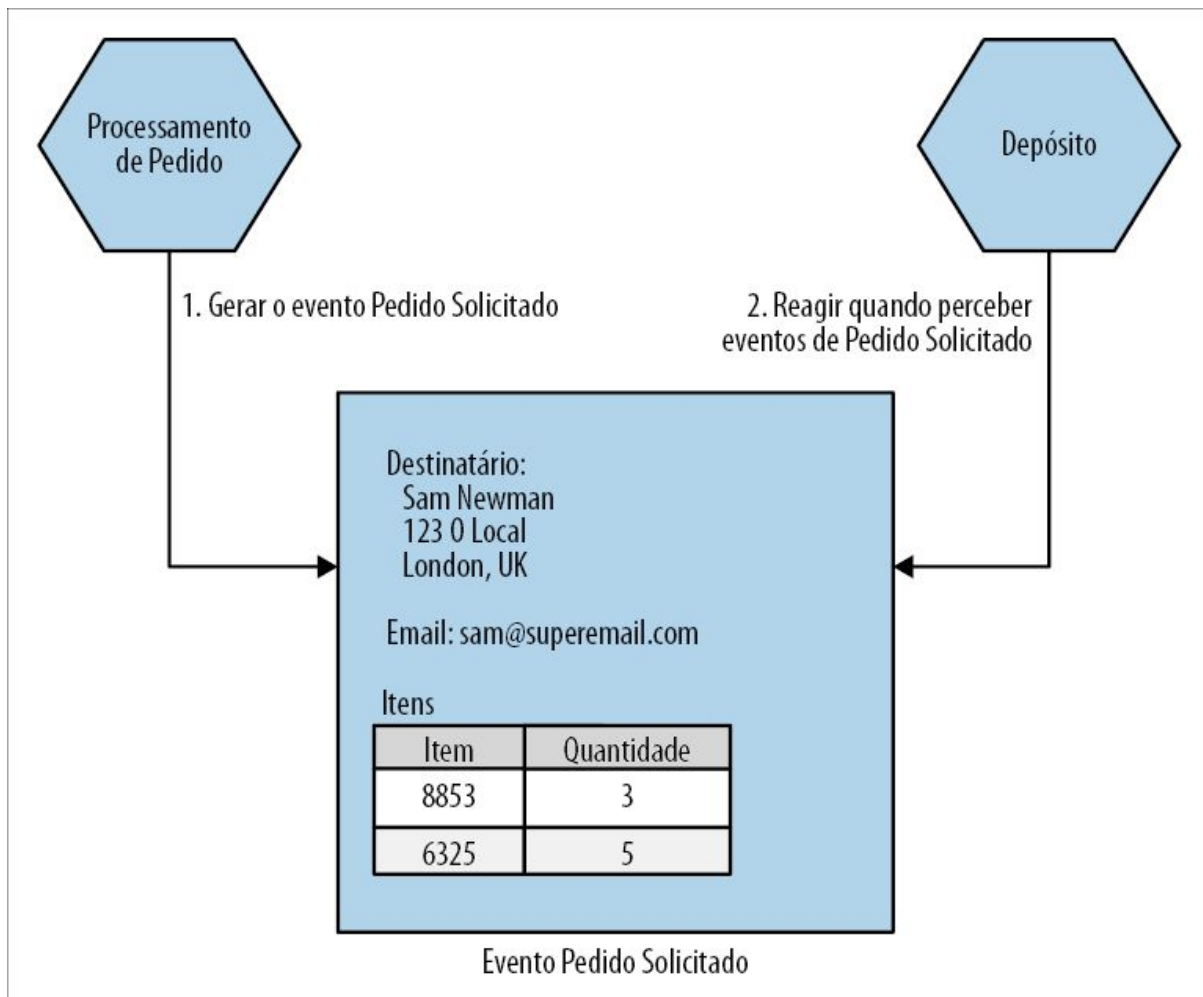


Figura 1.16 – Gerando um evento que o serviço Depósito possa receber, contendo informações suficientes apenas para que o pedido seja embalado e enviado.

Apenas o suficiente sobre design orientado a domínios

Conforme já discutimos, modelar nossos serviços em torno de um domínio de negócio proporciona vantagens significativas para a nossa arquitetura de microsserviços. A questão é como chegar a esse modelo – e é nesse cenário que o DDD (Domain-Driven Design, ou Design Orientado a Domínio) entra em cena.

O desejo de ver nossos programas representarem melhor o mundo real no qual os próprios programas atuarão não é uma ideia nova.

As linguagens de programação orientadas a objetos como o Simula foram desenvolvidas para nos permitir modelar domínios reais. No entanto, é necessário mais do que os recursos de uma linguagem de programação para que essa ideia realmente tome forma.

O livro *Domain-Driven Design* de Eric Evans¹² apresenta uma série de ideias importantes que nos ajudam a representar melhor o domínio do problema em nossos programas. Uma exploração completa dessas ideias está fora do escopo deste livro, mas apresentarei uma visão geral rápida das ideias mais importantes envolvidas ao considerar as arquiteturas de microsserviços.

Agregado

No DDD, um *agregado* é um conceito de certa forma confuso, com muitas definições diferentes por aí. É apenas um conjunto arbitrário de objetos? É a menor unidade que devo obter de um banco de dados? O modelo que sempre funcionou para mim é primeiro considerar um agregado como uma representação de um conceito real do domínio – pense em algo como Pedido, Fatura, Item do Estoque etc. Em geral, os agregados têm um ciclo de vida associado a eles, o que dá abertura para que sejam implementados como uma máquina de estados. Queremos tratar os agregados como unidades autocontidas; queremos garantir que o código que trata as transições de estado de um agregado esteja agrupado, junto com o próprio estado.

Ao pensar em agregados e microsserviços, veremos que um único microsserviço cuidará do ciclo de vida e da armazenagem de dados de um ou mais diferentes tipos de agregados. Se uma funcionalidade em outro serviço quiser modificar um desses agregados, ele deverá requisitar diretamente uma mudança nesse agregado, ou terá de fazer com que o próprio agregado reaja a outros eventos do sistema para iniciar suas próprias transições de estado – vemos exemplos na Figura 1.17.

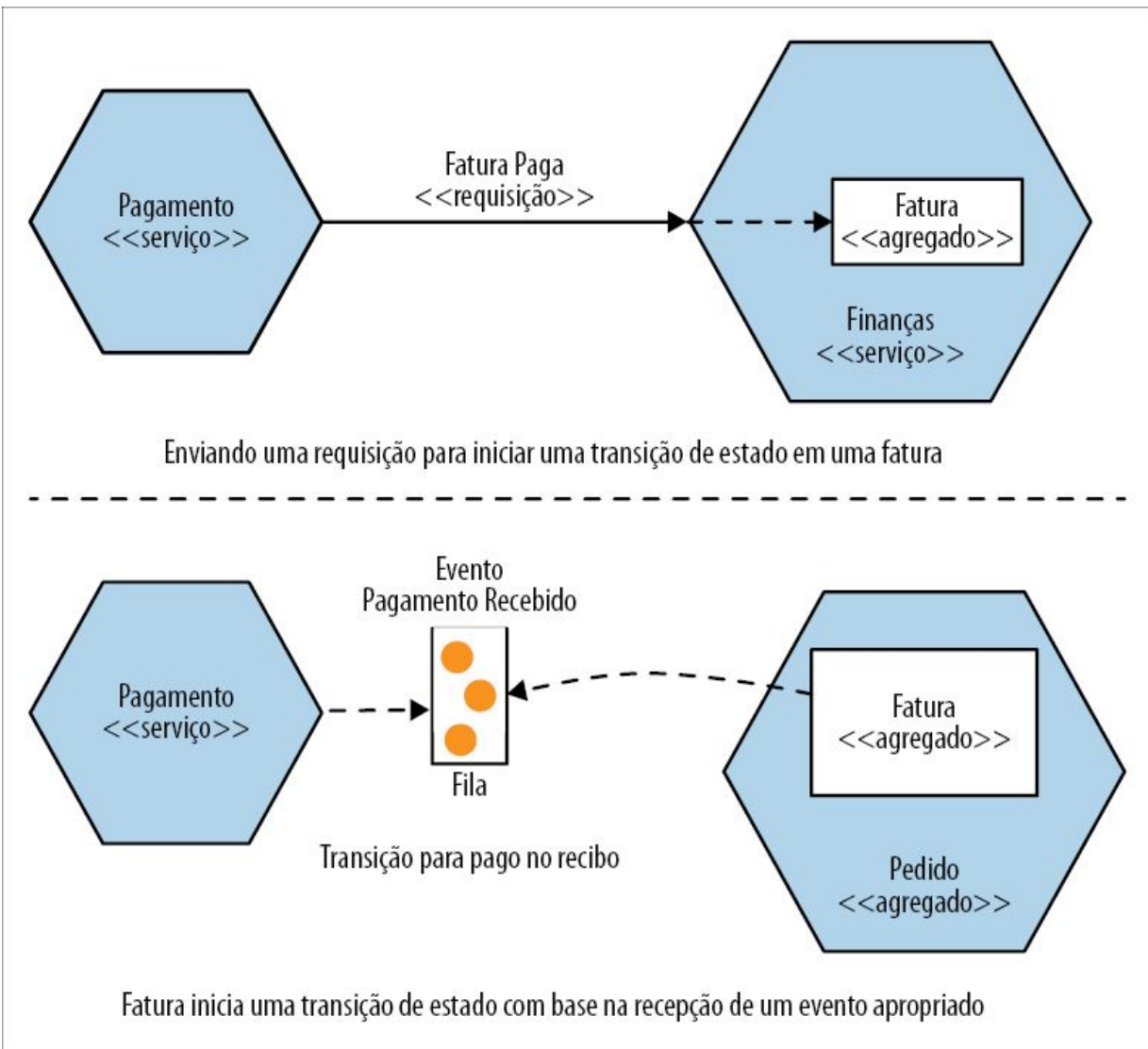


Figura 1.17 – Modos diferentes pelos quais o nosso serviço Pagamento poderia disparar uma transição para Pago em nosso agregado Fatura.

O ponto principal a ser compreendido nesse caso é que, se um código externo requisitar uma transição de estado a um agregado, o agregado poderá dizer não. O ideal é que você queira implementar seus agregados de modo que transições de estado ilegais sejam impossíveis.

Os agregados podem ter relacionamentos com outros agregados. Na Figura 1.18, temos um agregado Cliente, que está associado a um ou mais Pedidos. Decidimos modelar Cliente e Pedido como

agregados distintos, possíveis de serem tratados por serviços diferentes.

Há muitas maneiras de dividir um sistema em agregados, e algumas opções são extremamente subjetivas. Você poderia, por razões de desempenho ou facilidade de implementação, decidir remodelar os agregados com o tempo. Para começar, porém, considero que as preocupações com a implementação sejam secundárias, deixando que, no princípio, o modelo mental dos usuários do sistema seja minha estrela-guia no design inicial, até que outros fatores entrem em cena. No Capítulo 2, apresentarei o Event Storming (Tempestade de Eventos) como um exercício colaborativo para ajudar a dar forma a esses modelos de domínio, com a ajuda de seus colegas que não são desenvolvedores.

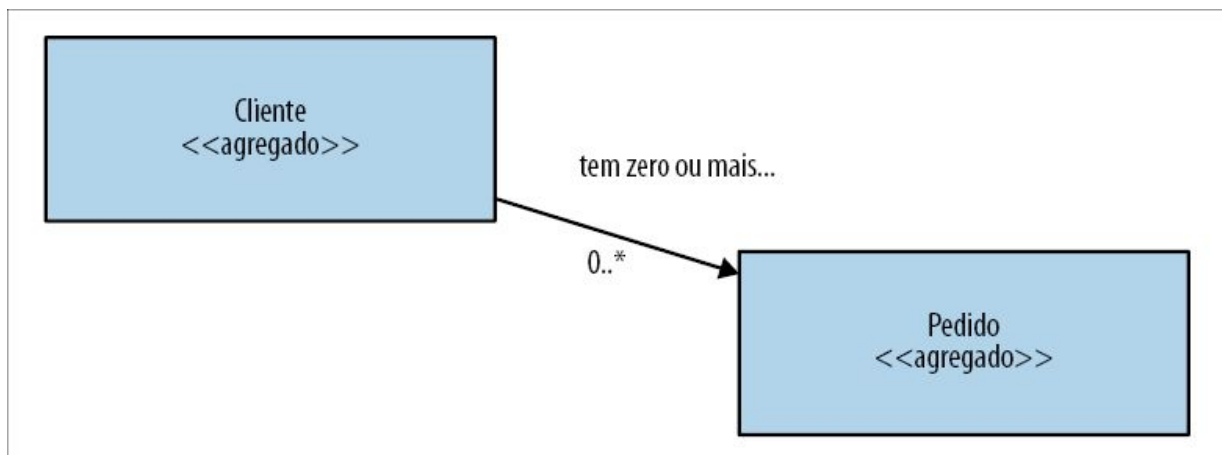


Figura 1.18 – Um agregado Cliente pode estar associado a um ou mais agregados Pedido.

Contexto delimitado

Um *contexto delimitado* (bounded context) em geral representa uma fronteira organizacional maior dentro de uma empresa. No escopo dessa fronteira, responsabilidades explícitas devem ser atribuídas. Tudo isso é muito vago, portanto, vamos analisar outro exemplo específico.

Na Music Corp, nosso depósito está repleto de atividades –

gerenciar pedidos a serem enviados (e as devoluções ocasionais), receber novos estoques, fazer corridas de empilhadeiras e assim por diante. Em outro lugar, o departamento financeiro talvez seja menos amante de diversão, mas ainda cumpre uma função importante em nossa organização, cuidando da folha de pagamentos, pagando os envios de remessas, e tarefas desse tipo.

Os contextos delimitados ocultam os detalhes de implementação. Há preocupações internas – por exemplo, os tipos de empilhadeira usados são de pouco interesse para qualquer pessoa que não seja funcionário do depósito. Essas preocupações internas devem permanecer ocultas ao mundo externo – eles não precisam saber, nem deveriam se importar.

Do ponto de vista da implementação, os contextos delimitados contêm um ou mais agregados. Alguns agregados podem ser expostos para fora do contexto delimitado; outros poderão estar ocultos internamente. Como no caso dos agregados, os contextos delimitados podem ter relacionamentos com outros contextos delimitados – quando são mapeados para os serviços, essas dependências se tornam dependências entre serviços.

Mapeando agregados e contextos delimitados aos microsserviços

Tanto o agregado como o contexto delimitado nos fornecem unidades de coesão com interfaces bem definidas, dentro do sistema mais amplo. O agregado é uma máquina de estados autocontida com foco no conceito de um único domínio em nosso sistema, enquanto o contexto delimitado representa um conjunto de agregados associados, novamente, com uma interface explícita para o mundo mais amplo.

Desse modo, ambos podem funcionar bem como fronteiras de serviços. Ao iniciar os trabalhos, conforme já mencionamos, sugiro que você reduza o número de serviços com os quais vai trabalhar. Como resultado, acho que você deve visar aos serviços que

englobem contextos delimitados inteiros. À medida que tiver uma base sólida e decidir separar esses serviços em serviços menores, procure separá-los de acordo com as fronteiras dos agregados.

Um truque, nesse caso, é que, mesmo que você decida separar um serviço que modele um contexto delimitado completo em serviços menores depois, será possível ocultar essa decisão do mundo externo – talvez apresentando uma API mais genérica aos consumidores. A decisão de decompor um serviço em partes menores é, sem dúvida, uma decisão de implementação, portanto, podemos ocultá-la da mesma forma, se for possível!

Leituras complementares

Uma exploração completa do design orientado a domínio é uma tarefa que vale a pena fazer, porém fora do escopo deste livro. Se quiser ler mais sobre o assunto, sugiro ler o livro original de Eric Evans, *Domain Driven Design*, ou o livro *Domain-Driven Design Distilled* de Vaughn Vernon.¹³

Resumo

Conforme discutimos neste capítulo, os microsserviços são serviços que podem ser implantados de forma independente, modelados em torno de um domínio de negócio. Eles se comunicam entre si por meio de redes. Usamos os princípios de ocultação de informações junto com o design orientado a domínio para criar serviços com fronteiras estáveis, com os quais seja mais fácil trabalhar de modo independente, e devemos fazer o que for possível para reduzir as diversas formas de acoplamento.

Também vimos uma breve história da origem dos microsserviços, e tivemos tempo inclusive de ver uma pequena fração do enorme volume de trabalhos anteriores nos quais eles se baseiam. Também vimos rapidamente alguns dos desafios associados às arquiteturas de microsserviços. Esse é um assunto que exploraremos com mais detalhes no próximo capítulo, no qual discutirei também como

planejar uma transição para uma arquitetura de microsserviços – além disso, apresentarei diretrizes que o ajudarão a decidir se, antes de tudo, os microsserviços são a escolha certa para você.

- 1 Para mais informações sobre esse assunto, recomendo o livro *PHP Web Services* de Lorna Jane Mitchell (O'Reilly) (N.T.: Edição disponível em português: *Web Services em PHP* [Novatec, 2013]).
- 2 Depois de ter lido a postagem de blog “Contempt Culture” (Cultura do desdém) (<http://bit.ly/2oeICgL>) de Aurynn Shaw, reconheci que, no passado, fui culpado de demonstrar certo grau de desdém em relação a diferentes tecnologias e, por extensão, às comunidades em torno delas.
- 3 Não consigo me lembrar exatamente da primeira vez que escrevemos o termo, mas me lembro claramente de minha insistência, apesar de toda a lógica em torno da gramática, de que o termo *não* deveria ter um hífen. Olhando retrospectivamente, era uma posição difícil de justificar, à qual, apesar de tudo, me ative. Permaneço fiel à minha opção desarrazoada, porém, em última instância, vitoriosa.
- 4 Para uma visão geral do raciocínio da Shopify para justificar o uso de um sistema monolítico modular no lugar de microsserviços, a apresentação de Kirsten Westeinde no YouTube (<http://bit.ly/2oauZ29>) contém insights interessantes.
- 5 Mensagem de email enviada para um bulletin board DEC SRC às 12:23:29 PDT em 28 de maio de 1987 (acesse <https://www.microsoft.com/en-us/research/publication/distribution/> para mais informações).
- 6 A Microsoft Research tem conduzido estudos nessa área, e eu recomendo todos eles. Como ponto de partida, minha sugestão é o artigo “Don’t Touch My Code! Examining the Effects of Ownership on Software Quality” (Não toque no meu código! Analisando os efeitos da responsabilidade na qualidade do software, <http://bit.ly/2p5RIT1>) de Christian Bird et al.
- 7 N.T.: Edição publicada no Brasil: *Projeto Estruturado de Sistemas* (Ed. Campus, 1990).
- 8 Embora o famoso artigo de Parnas de 1972, “On the Criteria to be Used in Decomposing Systems into Modules” (Sobre os critérios a serem usados na decomposição de sistemas em módulos), seja citado com frequência como a fonte, ele compartilhou inicialmente esse conceito em “Information Distributions Aspects of Design Methodology” (Aspectos da distribuição de informações na metodologia de design), Anais do Congresso IFIP de 1971.
- 9 Veja Parnas, David, “The Secret History of Information Hiding” (A história secreta da ocultação de informações). Publicado em *Software Pioneers*, eds. M. Broy e E. Denert (Berlin Heidelberg: Springer, 2002).
- 10 Veja Jez Humble e David Farley, *Continuous Delivery: Reliable Software*

Releases through Build, Test, and Deployment Automation (Upper Saddle River: Addison Wesley, 2010) para mais detalhes (N.T.: Edição publicada no Brasil: *Entrega Contínua: como entregar software de forma rápida e confiável* [Bookman, 2014]).

11 A décima lei de Greenspun determina que “Qualquer programa C ou Fortran suficientemente complicado contém uma implementação *ad hoc* lenta, informalmente especificada e repleta de bugs, de metade de Common Lisp”. Essa lei se transformou na nova piada: “Toda arquitetura de microserviços contém uma reimplementação parcialmente quebrada de Erlang”. Acho que há uma boa dose de verdade aí.

12 Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software* (Boston: Addison-Wesley, 2004) (N.T.: Edição publicada no Brasil: *Domain-Driven Design: atacando as complexidades no coração do software* [Alta Books, 2016])

13 Veja Vaughn Vernon, *Domain-Driven Design Distilled* (Boston: Addison-Wesley, 2014).

CAPÍTULO 2

Planejando uma migração

É muito fácil mergulhar diretamente nos aspectos técnicos mais importantes da decomposição monolítica – e esse será o foco da parte restante do livro! Antes, porém, precisamos explorar diversos problemas menos técnicos. Por onde devemos iniciar a migração? Como gerenciar a mudança? Como envolver outras pessoas na jornada? E a pergunta importante que deve ser feita logo no início: você deve, antes de tudo, usar microsserviços?

Compreendendo o objetivo

Os microsserviços não são o objetivo. Você não “vence” por ter microsserviços. Adotar uma arquitetura de microsserviços deve ser uma decisão consciente, tomada de forma racional. Você deve pensar em migrar para uma arquitetura de microsserviços a fim de fazer algo que não consegue fazer atualmente com sua arquitetura de sistema atual.

Sem saber ao certo o que você está tentando alcançar, de que modo o seu processo de tomada de decisão quanto às opções que você deve escolher poderia ter uma base sólida? Aquilo que você está tentando alcançar ao adotar os microsserviços mudará bastante o ponto no qual você concentrará o seu tempo e de que forma definirá as prioridades para os seus esforços.

Também ajudará você a evitar ser uma vítima da paralisia da análise – isto é, ficar sobrecarregado de opções. Você também correrá o risco de cair na mentalidade do “culto à carga”¹ se simplesmente supor que “Se os microsserviços são bons para a Netflix, serão bons

para nós!”.

Uma falha comum

Vários anos atrás, eu estava ministrando um workshop sobre microsserviços em uma conferência. Como faço em todas as minhas aulas, gosto de ter uma ideia do motivo pelo qual as pessoas estão ali e o que elas esperam do workshop. Nessa turma em particular, várias pessoas eram da mesma empresa, e fiquei curioso em saber por que a empresa as havia mandado. “Por que vocês estão no workshop? Por que estão interessados em utilizar microsserviços?” Perguntei a um deles. Sua resposta? “Não sabemos; nosso chefe nos mandou vir!” Intrigado, procurei saber mais. “Então, alguma ideia do motivo pelo qual seu chefe quis que vocês estivessem aqui?” “Bem, você poderia perguntar para ele – ele está sentado lá atrás”, respondeu o participante. Voltei-me para o chefe, fazendo-lhe a mesma pergunta: “Então, por que vocês estão querendo usar microsserviços?”. A resposta do chefe? “Nosso CTO disse que estamos adotando microsserviços, portanto achei que deveríamos descobrir o que são!”

Essa história real, embora seja divertida até certo ponto, infelizmente é muito comum. Conheci muitas equipes que tomaram a decisão de adotar uma arquitetura de microsserviços sem realmente entender o motivo, ou o que esperam alcançar com isso.

Os problemas de não ter uma visão clara sobre o motivo para usar microsserviços são variados. A adoção de microsserviços pode exigir um investimento significativo, seja diretamente com o acréscimo de mais pessoas ou mais dinheiro, ou para dar mais prioridade ao trabalho de transição em detrimento ao acréscimo de novas funcionalidades. A situação se complica mais ainda pelo fato de que pode demorar para que as vantagens de uma transição sejam percebidas. Ocasionalmente, as pessoas poderão se ver em uma situação em que estarão há um ano ou mais trabalhando com uma transição, mas não se lembrarão mais do motivo pelo qual,

antes de tudo, a iniciaram. Não é apenas uma questão da falácia do custo irrecuperável (sunk cost fallacy); elas realmente não sabem por que estão fazendo o trabalho.

Ao mesmo tempo, as pessoas me perguntam sobre o ROI (Return on Investment, ou Retorno sobre o Investimento) de passar para uma arquitetura de microsserviços. Algumas pessoas querem fatos e números concretos para justificar por que deveriam sequer considerar essa abordagem. A verdade é que, além do fato de estudos detalhados sobre esses assuntos serem raros e escassos, mesmo quando esses estudos existem, as observações feitas com base neles raramente são diretamente aplicáveis por causa da diferença de contextos nos quais as pessoas podem se encontrar.

Então, em que situação isso nos deixa? À mercê de palpites? Bem, não. Estou convencido de que podemos, e devemos, ter melhores estudos sobre a eficácia de nossas opções de desenvolvimento, tecnologia e arquitetura. Alguns trabalhos já estão sendo feitos com essa finalidade na forma de estudos como o “The State of DevOps Report” (Relatório sobre a situação do DevOps, <http://bit.ly/2ojVq5o>), porém a arquitetura é considerada apenas superficialmente. Em vez de depender desses estudos rigorosos, devemos ao menos nos esforçar para aplicar um raciocínio mais crítico à nossa tomada de decisão e, ao mesmo tempo, adotar uma mentalidade mais experimental.

Você deve ter um entendimento claro sobre o que espera alcançar. Nenhum cálculo de ROI poderá ser feito sem avaliar devidamente o retorno que você está buscando. Devemos manter o foco nos resultados que esperamos alcançar, em vez de ficarmos presos a uma única abordagem, de forma subserviente e dogmática. Devemos pensar na melhor maneira de obter o que precisamos, de forma clara e sensata, mesmo que isso signifique jogar muito trabalho fora ou voltar à boa e velha abordagem maçante.

Três perguntas básicas

Ao trabalhar com uma empresa a fim de ajudá-la a entender se ela deve considerar a adoção de uma arquitetura de microsserviços, tenho a tendência de lhe fazer o mesmo conjunto de perguntas:

O que vocês esperam alcançar?

A resposta deve ser um conjunto de resultados alinhado ao objetivo de negócios, e pode ser articulada de modo a descrever as vantagens que os usuários finais do sistema terão.

Vocês consideraram alternativas ao uso de microsserviços?

Conforme exploraremos mais tarde, muitas vezes, há outras maneiras de obter as mesmas vantagens proporcionadas pelos microsserviços. Vocês consideraram essas opções? Se não, por quê? Com muita frequência, é possível conseguir o que se quer usando uma técnica muito mais simples e maçante.

Como vocês saberão que a transição está funcionando?

Se vocês decidirem embarcar nessa transição, como saberão que estão indo na direção correta? Retomaremos esse assunto no final do capítulo.

Em mais de uma ocasião, percebi que fazer essas perguntas é suficiente para que as empresas pensem duas vezes antes de seguir em frente com uma arquitetura de microsserviços.

Por que você optaria pelos microsserviços?

Não posso definir as metas que você tem para a sua empresa. Você conhece melhor as aspirações de sua empresa e os desafios que enfrenta. O que posso descrever são as razões que, em geral, são apresentadas para justificar por que os microsserviços estão sendo adotados pelas empresas no mundo todo. Para ser justo, também apresentarei outras maneiras pelas quais você poderia, possivelmente, chegar aos mesmos resultados usando abordagens diferentes.

Aumentar a autonomia das equipes

Não importa o mercado no qual você atue, o mais importante são as pessoas, e fazer com que elas façam o que é certo, proporcionando-lhes a confiança, a motivação, a liberdade e o desejo de atingir o seu verdadeiro potencial.

– JOHN TIMPSON

Muitas empresas já mostraram as vantagens de criar equipes autônomas. Manter grupos organizacionais pequenos, permitindo que construam relacionamentos fortes e trabalhem juntos de modo eficaz, sem muita burocracia, tem ajudado muitas empresas a crescer e a escalar com mais eficiência em comparação com algumas empresas similares. A Gore tem tido bastante sucesso ao garantir que nenhuma de suas unidades de negócio tenha mais que 150 pessoas para ter certeza de que todos se conheçam. Para que essas unidades de negócio pequenas funcionem, elas devem ter capacidade e responsabilidade para trabalhar como unidades autônomas.

A Timpsons – uma empresa de varejo britânica extremamente bem-sucedida – atingiu uma escala enorme ao capacitar sua força de trabalho, reduzindo a necessidade de funções centralizadas e permitindo que as lojas locais tomassem decisões por conta própria, por exemplo, dando-lhes autonomia para decidir o valor de reembolso aos consumidores insatisfeitos. O presidente atual da empresa, John Timpson, ficou famoso por acabar com todas as regras internas e substituí-las por apenas duas:

- Olhe o produto e coloque o dinheiro na caixa registradora.
- Você pode fazer tudo mais para melhor servir aos clientes.

A autonomia também funciona em escalas menores, e as empresas mais modernas com as quais trabalho estão procurando criar equipes mais autônomas, muitas vezes tentando copiar modelos de outras empresas como o modelo de equipe duas pizzas da Amazon ou o modelo do Spotify.²

Se a mudança for feita de forma correta, dar autonomia às equipes

pode capacitar as pessoas, ajudá-las a assumir mais responsabilidades e a crescer, e agilizar o trabalho. Quando as equipes se tornam responsáveis pelos microsserviços e têm controle total sobre esses serviços, elas terão mais autonomia na organização mais ampla.

De que outra maneira você poderia fazer isso?

A autonomia – isto é, a distribuição de responsabilidades – pode estar presente de várias maneiras. Descobrir maneiras de atribuir mais responsabilidades a uma equipe não exige uma mudança de arquitetura. Basicamente, porém, é um processo de identificar quais responsabilidades poderão ser atribuídas às equipes, e isso pode ser feito de várias maneiras. Fazer com que diferentes equipes sejam responsáveis por diferentes partes da base de código poderia ser uma resposta (um sistema monolítico modular ainda poderia ser vantajoso para você, nesse caso): o mesmo também poderia ser feito identificando as pessoas com poder de tomar decisões sobre partes da base de código do ponto de vista funcional (por exemplo, Ryan conhece melhor a exibição de anúncios, portanto será responsável por essa parte; Jane é a que mais sabe sobre como melhorar o desempenho de nossas consultas, portanto faça com que tudo que estiver nessa área passe por ela antes).

Um aumento de autonomia também pode se dar simplesmente se não houver necessidade de esperar que outras pessoas façam as tarefas por você; desse modo, adotar abordagens self-service para o provisionamento de máquinas ou de ambientes pode ser um importante fator viabilizador, evitando que equipes de operações centralizadas tenham de responder a tickets de solicitações para atividades do cotidiano.

Reduzir o tempo para chegar ao mercado

Ao ser capaz de fazer e implantar mudanças em microsserviços individuais, e implantar essas mudanças sem ter de esperar lançamentos coordenados de versões, temos o potencial de lançar

funcionalidades aos nossos clientes com maior rapidez. Ser capaz de trazer mais pessoas para resolver um problema também é um fator – discutiremos esse assunto em breve.

De que outra maneira você poderia fazer isso?

Bem, por onde podemos começar? Há muitas variáveis em cena quando consideramos maneiras de lançar um software mais rapidamente. Sempre sugiro que você faça algum tipo de exercício de modelagem sobre o caminho para chegar à produção, pois pode ajudar a mostrar que o maior obstáculo talvez não seja o que você imagina.

Lembro-me de um projeto muitos anos atrás em um grande banco de investimentos, para o qual fomos chamados para ajudar a agilizar a entrega do software. “Os desenvolvedores demoram muito para levar o software até o ambiente de produção!”, disseram para nós. Um de meus colegas, Kief Morris, resolveu mapear todas as etapas envolvidas na entrega do software, observando o processo, do momento em que o dono do produto surgia com uma ideia até o ponto em que essa ideia realmente chegava ao ambiente de produção.

Ele identificou rapidamente que demorava cerca de seis semanas, em média, entre o instante em que um desenvolvedor começava um trabalho até este ser implantado em um ambiente de produção. Achamos que podíamos eliminar umas duas semanas desse processo usando uma automação apropriada, pois havia processos manuais envolvidos. No entanto, Kief identificou um problema muito pior no caminho até o ambiente de produção – com frequência, demorava mais de 40 semanas para que as ideias passassem do dono do produto até o ponto em que os desenvolvedores pudessem começar a trabalhar. Ao manter o foco de nossos esforços em melhorar essa parte do processo, ajudamos muito mais o cliente a reduzir o tempo para alcançar o mercado (time to market) com uma nova funcionalidade.

Portanto, pense em todos os passos envolvidos no lançamento de

um software. Observe quanto tempo demora, as durações (tanto o tempo decorrido como o tempo útil) de cada passo e destaque os pontos mais complicados do caminho. Depois de tudo isso, você pode até mesmo chegar à conclusão de que os microsserviços devem fazer parte da solução, mas, provavelmente, identificará várias outras atividades que poderiam ser tentadas em paralelo.

Escalar para lidar com a carga, com um custo viável

Ao separar nosso processamento em microsserviços individuais, esses processos poderão ser escalados de forma independente. Isso significa que poderemos também escalar com um custo viável – será necessário escalar somente as partes do nosso processamento que estão atualmente limitando a nossa capacidade de lidar com a carga. Segue-se daí que também será possível reduzir os microsserviços que estejam com menos carga, talvez até mesmo desativando-os quando não forem necessários. Este é, em parte, o motivo pelo qual muitas empresas que criam produtos SaaS adotam uma arquitetura de microsserviços – ela lhes dá mais controle sobre os custos operacionais.

De que outra maneira você poderia fazer isso?

Nesse caso, há um número enorme de alternativas que podem ser consideradas, a maioria sendo fácil de ser testada, antes de se comprometer com uma abordagem orientada a microsserviços. Poderíamos simplesmente adquirir uma máquina maior, para começar – se você estiver em uma nuvem pública ou em outro tipo de plataforma virtualizada, poderia simplesmente provisionar máquinas maiores nas quais o seu processo executará. Sem dúvida, essa escalabilidade “vertical” tem suas limitações, mas, para uma melhoria rápida no curto prazo, não deveria ser prontamente menosprezada.

A escalabilidade horizontal tradicional para um sistema monolítico existente – basicamente, executando várias cópias – poderia se

mostrar extremamente eficaz. Executar várias cópias de seu sistema monolítico atrás de um sistema de distribuição de carga, por exemplo, um balanceador de carga ou uma fila, poderia possibilitar um tratamento simples de mais carga, embora não vá ajudar se o gargalo estiver no banco de dados, o qual, dependendo da tecnologia, pode não aceitar um sistema como esse para escalar. A escalabilidade horizontal é fácil de ser testada, e você realmente deveria lhe dar uma chance antes de considerar os microsserviços, pois, provavelmente, poderá avaliar com rapidez se essa é uma solução adequada, e ela apresenta muito menos pontos negativos do que uma arquitetura de microsserviços completa.

Você também poderia substituir a tecnologia em uso por alternativas que lidem melhor com a carga. Em geral, porém, essa não é uma tarefa trivial – considere o trabalho de portar um programa atual para um novo tipo de banco de dados ou outra linguagem de programação. Uma mudança para os microsserviços pode fazer com que uma mudança de tecnologia seja mais fácil, pois você poderá mudar a tecnologia em uso no momento somente dentro dos microsserviços que precisarem dela, deixando a parte restante inalterada e, desse modo, reduzindo o impacto da mudança.

Aumentar a robustez

A passagem de um software de um só tenant para aplicações SaaS multitenant significa que o impacto das interrupções de serviço do sistema pode estar significativamente distribuído. As expectativas de nossos clientes para a disponibilidade de seus softwares, bem como a importância do software em suas vidas, aumentaram. Quando separamos nossa aplicação em processos individuais, possíveis de serem implantados de modo independente, abrimos as portas para uma série de métodos que aumentam a robustez de nossas aplicações.

Ao usar microsserviços, podemos implementar uma arquitetura mais robusta, pois as funcionalidades estarão decompostas – isto é, um

impacto em uma área de funcionalidade não derrubará o sistema como um todo. Também conseguimos manter o foco de nosso tempo e energia nas partes da aplicação que exijam mais robustez – garantindo que partes cruciais de nosso sistema permaneçam operacionais.

Resiliência *versus* robustez

Em geral, quando queremos melhorar a capacidade de um sistema para evitar interrupções de serviço, tratar falhas de forma elegante quando ocorrerem e recuperar-se rapidamente em caso de problemas, estaremos falando de *resiliência*. Muito trabalho tem sido feito na área que atualmente é conhecida como *engenharia de resiliência*, encarando o assunto como um todo, pois ele se aplica a todas as áreas, e não apenas à computação. O modelo para resiliência, que teve David Woods como pioneiro, observa o conceito de resiliência de modo mais amplo e enfatiza o fato de que *ser resiliente* não é tão simples quanto possamos pensar à primeira vista; o modelo faz uma separação entre a nossa habilidade de lidar com causas conhecidas e com causas desconhecidas de falhas, entre outros aspectos.³

John Allspaw, um colega de David Woods, ajudou a fazer uma distinção entre os conceitos de robustez e de resiliência. A *robustez* é a capacidade de ter um sistema que possa reagir a variações esperadas. A *resiliência* diz respeito a ter uma organização capaz de se adaptar a situações que não foram planejadas, e isso poderia incluir a criação de uma cultura de experimentação por meio de abordagens como a engenharia de caos. Por exemplo, estamos cientes de que uma máquina específica poderia falhar, portanto, podemos prover redundância ao nosso sistema fazendo o balanceamento de carga de uma instância. Esse é um exemplo de como tratar a robustez. A resiliência é o processo de uma empresa se preparar para o fato de que não será capaz de se antecipar a todos os problemas em potencial.

Uma consideração importante, nesse caso, é que os microsserviços

não proporcionam necessariamente uma robustez de forma gratuita. Elas criam oportunidades para fazer o design de um sistema de modo que ele possa tolerar melhor as partições de rede, as interrupções de serviço e problemas desse tipo. Simplesmente distribuir suas funcionalidades em vários processos e máquinas separados não garante mais robustez; poderia significar exatamente o contrário, pois a área de superfície para falhas seria maior.

De que outra maneira você poderia fazer isso?

Ao executar várias cópias de seu sistema monolítico, talvez atrás de um balanceador de carga ou outro método de distribuição de carga, por exemplo, uma fila⁴, teremos redundância em nosso sistema. Podemos aumentar mais a robustez de nossas aplicações se distribuirmos instâncias de nosso sistema monolítico em vários planos de falha (por exemplo, não deixando todas as máquinas no mesmo rack ou no mesmo data center).

Investimentos em hardware e software mais confiáveis poderiam, do mesmo modo, resultar em benefícios, assim como uma investigação completa das causas atuais das interrupções de serviços do sistema. Já vi vários problemas de produção serem causados por uma confiança excessiva em processos manuais, por exemplo, ou por pessoas que “não seguem o protocolo”, o que, com frequência, significa que um erro inocente cometido por um indivíduo pode causar impactos significativos. A British Airways sofreu uma grande interrupção de serviço em 2017, fazendo com que todos os seus voos de partida e de chegada aos aeroportos de Heathrow e Gatwick em Londres fossem cancelados. Esse problema aparentemente foi causado de forma inadvertida por uma sobrecarga de energia como consequência das ações de um indivíduo. Se a robustez de sua aplicação depende de seres humanos jamais cometerem erros, você estará sujeito a viver grandes emoções.

Escalar o número de desenvolvedores

Todos nós provavelmente já vimos o problema de colocar desenvolvedores em um projeto a fim de fazê-lo andar mais depressa – com muita frequência, é um tiro no pé. Contudo, alguns problemas realmente exigem mais pessoas para que sejam resolvidos. Como descreve Frederick Brooks em seu livro atualmente consagrado *The Mythical Man Month*⁵, acrescentar mais pessoas continuará a aumentar a rapidez com que você consegue entregar o seu trabalho somente se esse trabalho puder ser dividido em partes, com interações limitadas entre elas. Ele dá o exemplo de fazer colheitas em um campo – ter várias pessoas trabalhando em paralelo é uma tarefa simples, pois o trabalho feito por cada uma delas não exige interações com outras pessoas. Um software raramente funciona dessa maneira, pois o trabalho feito não é sempre igual e, com frequência, o resultado de uma parte do trabalho será necessário como entrada para outra parte.

Com fronteiras claramente definidas, e uma arquitetura que tenha como foco garantir que nossos microsserviços limitem os acoplamentos, teremos porções de código nas quais será possível trabalhar de modo independente. Dessa forma, esperamos poder escalar o número de desenvolvedores reduzindo a desavença nas entregas.

Escalar o número de desenvolvedores trazidos para resolver um problema de forma bem-sucedida exige um bom grau de autonomia entre as equipes. Apenas ter microsserviços não será suficiente. Você terá de pensar em como as equipes estão alinhadas quanto às responsabilidades pelos serviços, e no tipo de coordenação que será necessário entre as equipes. Você também terá de dividir as tarefas de modo que uma coordenação envolvendo vários serviços não seja necessária para fazer as mudanças.

De que outra maneira você poderia fazer isso?

Os microsserviços funcionam bem para equipes maiores, pois os próprios microsserviços se tornam partes desacopladas de funcionalidades, com as quais será possível trabalhar de modo

independente. Uma abordagem alternativa poderia ser considerar a implementação de um sistema monolítico modular: equipes diferentes são responsáveis por cada módulo e, desde que a interface com os outros módulos permaneça estável, elas poderão continuar a fazer alterações isoladamente.

Contudo, essa abordagem é, de certo modo, limitada. Ainda temos uma espécie de contenda entre as diferentes equipes, pois o software continuará sendo formado por um único pacote, de modo que o ato de fazer uma implantação ainda exigirá uma coordenação entre as partes envolvidas.

Incorporar novas tecnologias

Em geral, os sistemas monolíticos limitam nossas opções de tecnologia. Geralmente temos uma linguagem de programação no backend, que faz uso de um padrão de programação. Estamos amarrados a uma plataforma de implantação, um sistema operacional e um tipo de banco de dados. Com uma arquitetura de microsserviços, temos a possibilidade de variar essas opções para cada serviço.

Ao limitar a mudança de tecnologia dentro de um serviço, podemos compreender as vantagens da nova tecnologia de forma isolada e restringir o impacto caso a tecnologia acabe apresentando algum problema.

De acordo com minha experiência, embora empresas experientes em microsserviços muitas vezes limitem a quantidade de pilhas de tecnologia com que podem trabalhar, raramente elas são homogêneas quanto às tecnologias que usam. A flexibilidade para experimentar uma nova tecnologia de forma segura pode lhes dar uma vantagem competitiva, tanto no que diz respeito a entregar melhores resultados para os clientes como para ajudar a manter seus desenvolvedores satisfeitos, pois eles poderão adquirir novas habilidades.

De que outra maneira você poderia fazer isso?

Se continuarmos lançando o nosso software como um único processo, estaremos limitados quanto às tecnologias que poderemos introduzir. Sem dúvida, poderíamos adotar novas linguagens de forma segura no mesmo ambiente de execução – a JVM, como um exemplo, é capaz de hospedar um código escrito em várias linguagens no mesmo processo em execução. Novos tipos de bancos de dados, porém, se tornam mais problemáticos, pois isso implica algum tipo de decomposição de um modelo de dados anteriormente monolítico a fim de possibilitar uma migração gradual, a menos que você opte por uma mudança imediata e completa para uma nova tecnologia de banco de dados, o que é complicado e arriscado.

Se a pilha atual de tecnologias for considerada uma “plataforma em chamadas”, você talvez não tenha outras opções além de substituí-la por uma pilha mais nova, com mais suporte.⁶ É claro que não há nada que impeça você de substituir gradualmente o seu sistema monolítico atual por um novo sistema monolítico – padrões como a abordagem strangler fig descrita no Capítulo 3 podem funcionar muito bem nesse caso.

Reutilização?

A reutilização é um dos objetivos mais citados na migração para microsserviços e, em minha opinião, não é um bom objetivo, para começar. Basicamente, a reutilização *não* é um resultado direto desejado pelas pessoas. É algo que as pessoas esperam que vá levar a outros benefícios. Esperamos que, por meio da reutilização, seja possível lançar funcionalidades com mais rapidez ou, quem sabe, reduzir os custos, mas, se esses forem os seus objetivos, monitore esses aspectos, ou você poderá acabar otimizando as partes erradas.

Para explicar o que quero dizer, vamos observar detalhadamente um dos motivos usuais pelos quais a reutilização é escolhida como um objetivo. Queremos lançar funcionalidades com mais rapidez. Achemos que, ao otimizar nosso processo de desenvolvimento em torno da reutilização de códigos existentes,

teremos de escrever menos código – e, com menos trabalho a ser feito, poderemos disponibilizar nosso software mais rapidamente, certo? Mas vamos considerar um exemplo simples. A equipe de Atendimento ao Cliente da Music Corp precisa formatar um PDF a fim de disponibilizar notas fiscais aos clientes. Outra parte do sistema já lida com a geração de PDFs: geramos PDFs para serem impressos no depósito a fim de gerar etiquetas para as embalagens enviadas aos clientes e para enviar requisições de pedidos aos fornecedores.

Tendo a reutilização como objetivo, nossa equipe poderia ser levada a utilizar a funcionalidade existente de geração de PDF. Todavia, essa funcionalidade atualmente é gerenciada por uma equipe diferente, em outra parte da empresa. Desse modo, é necessário agora um trabalho de coordenação com eles a fim de fazer as alterações necessárias para dar suporte às nossas funcionalidades. Isso pode significar que teremos de pedir a eles que façam o trabalho para nós ou, talvez, teremos de fazer as modificações por conta própria e submeter um pull request (supondo que nossa empresa trabalhe dessa forma). Qualquer que seja a maneira, deverá haver um trabalho de coordenação com outra parte da empresa para que a mudança seja feita.

Poderíamos dispor de tempo para fazer o trabalho de coordenação com outras pessoas e ter as alterações feitas para que pudéssemos fazer o rollout de nossa mudança. Contudo, percebemos que poderíamos simplesmente escrever nossa própria implementação de modo muito mais rápido e disponibilizar a funcionalidade para o cliente com mais agilidade do que faríamos se gastássemos o tempo para adaptar o código atual. Se seu objetivo for reduzir o tempo para atingir o mercado, essa poderia ser a opção correta. Porém, se você otimizar visando à reutilização, esperando alcançar mais rapidamente o mercado, poderá acabar tomando decisões que deixarão seu processo mais lento.

Avaliar a reutilização em sistemas complexos é difícil e, conforme já descrevemos, em geral é algo que fazemos para alcançar

outros objetivos. Invista seu tempo com vistas ao verdadeiro objetivo, e reconheça que a reutilização talvez nem sempre seja a resposta correta.

Quando os microsserviços não seriam uma boa ideia?

Passamos bastante tempo explorando as possíveis vantagens das arquiteturas de microsserviços. Todavia, em algumas situações, recomendo que você não utilize os microsserviços. Vamos analisar agora algumas dessas situações.

Domínio nebuloso

Definir fronteiras incorretas para os serviços pode ser custoso. Pode resultar em um número maior de mudanças envolvendo mais de um serviço, componentes demasiadamente acoplados e, em geral, poderia ser pior do que ter apenas um único sistema monolítico. No livro *Building Microservices*, compartilhei as experiências da equipe do produto SnapCI na ThoughtWorks. Apesar de conhecer muito bem o domínio da integração contínua, a tentativa inicial dessa equipe em definir as fronteiras dos serviços para sua solução de CI hospedado não foi muito correta. Essa decisão resultou em um custo alto para as mudanças e em responsabilidades confusas. Após muitos meses lutando com esse problema, a equipe decidiu combinar os serviços de volta em uma única grande aplicação. Mais tarde, quando o conjunto de funcionalidades da aplicação havia, até certo ponto, estabilizado, e a equipe tinha uma compreensão mais sólida do domínio, tornou-se mais fácil determinar essas fronteiras estáveis.

O SnapCI era uma ferramenta hospedada de integração e entrega contínuas. A equipe havia trabalhado antes em outra ferramenta semelhante, o Go-CD – atualmente, é uma ferramenta de entrega contínua de código aberto, que pode ser implantada localmente, em vez de estar hospedada na nuvem. Embora tivesse havido um

pouco de reutilização de código no início entre os projetos SnapCI e Go-CD, no final, o SnapCI acabou com uma base de código totalmente nova. Apesar disso, a experiência anterior da equipe no domínio de ferramentas de CD fez com que eles se sentissem audaciosos e começassem a fazer rapidamente a identificação das fronteiras e a construir o sistema como um conjunto de microsserviços.

Depois de alguns meses, porém, ficou claro que os casos de uso do SnapCI eram sutilmente diferentes, de modo que a primeira tentativa de definir as fronteiras dos serviços não estava muito correta. Isso resultou em muitas mudanças que tiveram de ser feitas envolvendo vários serviços, e em um alto custo associado às mudanças. Posteriormente, a equipe combinou os serviços de volta em um sistema monolítico, dando-lhes tempo para entender melhor os locais em que as fronteiras deveriam estar. Um ano depois, a equipe pôde então separar o sistema monolítico em microsserviços, cujas fronteiras se mostraram ser muito mais estáveis. Esse não é, nem de longe, o único exemplo que já vi dessa situação. Decompor um sistema em microsserviços de modo prematuro pode ser custoso, particularmente se o domínio for uma novidade para você. De acordo com vários aspectos, ter uma base de código existente que você queira decompor em microsserviços é muito mais fácil do que tentar adotar os microsserviços desde o princípio.

Se você achar que ainda não tem um entendimento total de seu domínio, resolver isso antes de se comprometer com uma decomposição do sistema pode ser uma boa ideia. (Esse é outro motivo para fazer uma modelagem de domínios! Discutiremos melhor esse assunto em breve.)

Startups

Pode parecer um pouco controverso, pois muitas das empresas famosas pelo uso de microsserviços são consideradas startups, mas, na verdade, várias delas, incluindo Netflix, Airbnb e outras

empresas do tipo, passaram a usar uma arquitetura de microsserviços somente em um ponto mais adiantado de sua evolução. Os microsserviços podem ser uma ótima opção para as “scale-ups” – empresas startups que definiram pelo menos as bases para o seu produto/mercado, e agora estão escalando para aumentar os lucros (ou, provavelmente, para apenas obtê-los).

As startups, distintas das scale-ups, em geral fazem experimentos com várias ideias, em uma tentativa de encontrar algo adequado aos seus clientes. Isso pode resultar em grandes mudanças na visão original do produto à medida que o espaço é explorado, levando a grandes mudanças no domínio do produto.

Uma verdadeira startup provavelmente é uma empresa pequena, com financiamento limitado, que precisa manter toda a sua atenção concentrada em tornar seu produto adequado. Os microsserviços resolvem principalmente os tipos de problemas que as startups terão assim que encontrarem o produto adequado para a sua base de clientes. Falando de outra maneira, os microsserviços são ótimos para solucionar os tipos de problemas que você terá assim que tiver um sucesso inicial como uma startup. Portanto, mantenha o foco inicialmente em ter sucesso! Se sua ideia inicial for ruim, não importa se você vai implementá-la com microsserviços ou não.

É muito mais fácil dividir um sistema “brownfield”^Z existente do que fazer isso diretamente com um sistema greenfield novo, que seria criado por uma startup. Você terá mais com que trabalhar. Haverá códigos que poderão ser examinados, e você poderá conversar com pessoas que utilizam e mantêm o sistema. Você também saberá qual é o significado de *bom* – terá um sistema em funcionamento para modificar, fazendo com que seja mais fácil saber quando poderia ter cometido um erro ou estaria sendo agressivo demais em seu processo de tomada de decisões.

Você também teria um sistema que estaria realmente executando. Saberá como ele funciona e como se comporta no ambiente de produção. A decomposição em microsserviços pode provocar

alguns problemas desagradáveis de desempenho, por exemplo, mas, com um sistema brownfield, você terá a chance de definir uma linha de base saudável, antes de fazer mudanças que possam causar impactos no desempenho.

Não estou absolutamente dizendo para *jámais usar microsserviços em startups*, mas que esses fatores mostram que você deve ser cauteloso. Separe somente as fronteiras que estejam claras no início e mantenha a parte restante no lado mais monolítico. Com isso, você também terá tempo para avaliar a sua maturidade do ponto de vista operacional – se tiver dificuldades para gerenciar dois serviços, gerenciar dez será muito mais complicado.

Software instalado e administrado pelos clientes

Se você cria softwares que são empacotados e enviados aos clientes que, por sua vez, administram esses softwares, os microsserviços talvez não sejam uma boa ideia. Ao migrar para uma arquitetura de microsserviços, você incluirá muita complexidade no domínio operacional. Técnicas anteriores que você usava para monitorar e resolver problemas de sua implantação monolítica poderão não funcionar com seu novo sistema distribuído. As equipes que passam por uma migração para microsserviços enfrentam esses desafios adquirindo novas habilidades ou, talvez, adotando novas tecnologias – não são atitudes que, em geral, você poderia esperar de seus clientes finais.

Geralmente, com um software instalado pelo cliente, você terá uma plataforma específica como alvo. Por exemplo, você poderá dizer que “exige Windows 2016 Server” ou “é necessário ter macOS 10.12 ou uma versão mais recente”. Esses são ambientes de implantação alvos muito bem definidos, e é bem provável que você empacote o seu software monolítico usando métodos muito bem conhecidos das pessoas que administram esses sistemas (por exemplo, disponibilizando serviços Windows, empacotados para Windows Installer). É provável que seus clientes tenham

familiaridade com a compra e a execução de softwares dessa maneira.

Pense no problema que haveria se você passasse de um processo, disponibilizado para eles executarem e administrarem, para 10 ou 20? Ou, talvez, de modo mais agressivo ainda, esperasse que eles executassem seu software em um cluster Kubernetes ou algo semelhante?

A verdade é que você não pode esperar que seus clientes tenham as habilidades ou as plataformas disponíveis para gerenciar arquiteturas de microsserviços. Mesmo que tivessem, eles talvez não tenham as mesmas habilidades ou plataformas exigidas por você. Há uma grande variação entre as instalações de Kubernetes, por exemplo.

Não ter um bom motivo!

Por fim, temos o principal motivo para não adotar os microsserviços: é o fato de você não ter uma ideia clara do que, exatamente, está tentando alcançar. Conforme exploraremos, o resultado que você está buscando com a adoção dos microsserviços definirá *onde* você iniciará essa migração e *como* vai decompor o sistema. Sem uma visão clara de seus objetivos, você estará tateando no escuro. Adotar microsserviços somente porque todos estão fazendo isso é uma péssima ideia.

Custo-benefício

Até agora, apresentei os motivos pelos quais as pessoas poderiam querer adotar os microsserviços isoladamente, e expliquei (embora de forma rápida) por que elas deveriam considerar também outras opções. No entanto, no mundo real, é comum que as pessoas tentem mudar não só um, mas vários pontos, tudo ao mesmo tempo. Isso pode resultar em prioridades confusas, que podem rapidamente aumentar o volume de alterações necessárias e fazer com que seja mais demorado perceber qualquer vantagem.

Tudo começa de modo muito inocente. Precisamos refazer a arquitetura de nossa aplicação para lidar com um aumento significativo no tráfego e decidirmos que os microsserviços são a solução. Alguém vem e diz: “Bem, já que vamos trabalhar com microsserviços, poderemos deixar nossas equipes mais autônomas ao mesmo tempo!”. Outra pessoa sugere que “Isso nos dará uma ótima oportunidade para experimentar o Kotlin como linguagem de programação!”. Antes que você perceba, haverá uma quantidade enorme de iniciativas de mudanças, com tentativas de dar mais autonomia às equipes, escalar a aplicação e introduzir uma nova tecnologia, tudo de uma só vez, junto com outras tarefas que, certamente, as pessoas também incluirão no plano de trabalho.

Além do mais, nessa situação, os microsserviços passam a ser enquadrados como a abordagem. Se você mantiver o foco somente no aspecto da escalabilidade, durante a sua migração, você poderá perceber que será melhor escalar a sua aplicação monolítica atual apenas horizontalmente. Porém, fazer isso não ajudará a atingir os novos objetivos secundários de aumentar a autonomia das equipes ou introduzir Kotlin como uma linguagem de programação.

Desse modo, é importante separar o motivo principal da mudança de qualquer vantagem secundária que você também queira obter. Nesse caso, lidar com uma escala maior da aplicação é o aspecto mais importante – um trabalho feito para ter progressos nos demais objetivos secundários (como aumentar a autonomia das equipes) pode ser útil, mas, se atrapalhar ou causar desvios em relação ao objetivo principal, ele deverá ser colocado em segundo plano.

O ponto importante, nessa situação, é reconhecer que alguns pontos *são* mais importantes que outros. Caso contrário, não será possível atribuir prioridades de forma adequada. Um exercício de que gosto, nesse cenário, é pensar em cada um dos seus resultados desejados como se fosse um slider (controle deslizante). Todos os sliders começam no meio. À medida que considerar uma tarefa mais importante, você deverá diminuir a prioridade de outra –

podemos ver um exemplo disso na Figura 2.1. Essa figura articula claramente o fato de que, por exemplo, embora você quisesse facilitar uma programação poliglota, essa tarefa não é *tão* importante quanto garantir que a aplicação tenha mais resiliência. Quando se trata de descobrir como devemos seguir em frente, ter esses resultados claramente articulados e classificados pode facilitar muito uma tomada de decisão.

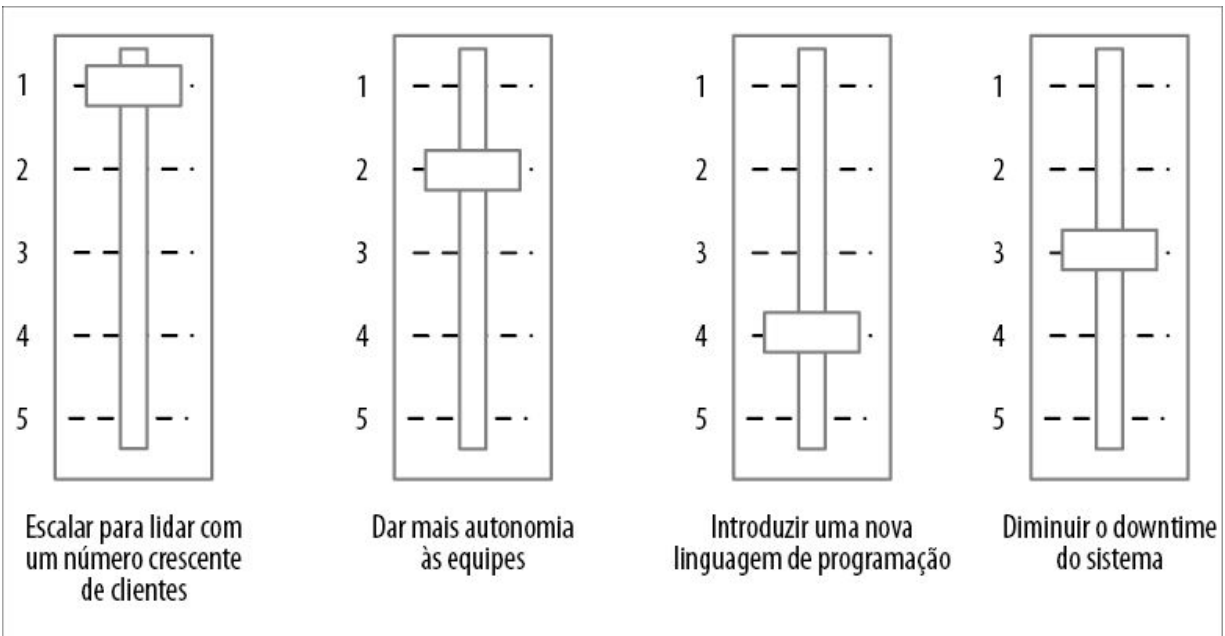


Figura 2.1 – Usando sliders para balancear as prioridades concorrentes que você possa ter.

Essas prioridades relativas podem mudar (e devem, à medida que você aprender mais). No entanto, elas podem ajudar a orientar uma tomada de decisão. Se quiser distribuir as responsabilidades, dando mais poder de decisão às equipes agora autônomas, modelos simples como esse poderão ajudar a dar uma base melhor para uma tomada de decisão local e ajudar essas equipes a fazer escolhas melhores, que estejam alinhadas com os objetivos que você estiver tentando alcançar na empresa como um todo.

Incluindo pessoas na jornada

Com frequência, me perguntam: “Como posso vender a ideia dos

microsserviços para o meu chefe?”. Em geral, essa pergunta é feita por um desenvolvedor – alguém que percebeu o potencial da arquitetura de microsserviços e está convencido de que esse é o caminho a ser seguido.

Muitas vezes, quando as pessoas discordam sobre uma abordagem, é porque elas podem ter visões diferentes sobre o que estão tentando alcançar. É importante que você e as outras pessoas que precisam ser incluídas na jornada tenham uma compreensão comum sobre *o que* estão tentando alcançar. Se vocês estiverem em sintonia quanto a isso, então, no mínimo, saberão que discordam apenas quanto ao modo de chegar lá. Portanto, tudo se reduz ao objetivo novamente – se as outras pessoas na empresa tiverem o mesmo objetivo, é muita mais provável que elas se comprometam em fazer uma mudança.

Com efeito, vale a pena explorar com mais detalhes a maneira de vender a ideia bem como de fazê-la se concretizar, buscando inspiração em um dos modelos mais conhecidos para ajudar a fazer mudanças organizacionais. Vamos ver esse assunto a seguir.

Mudanças organizacionais

O processo de oito passos de John Kotter para implementar mudanças organizacionais é uma referência para os gerentes do mundo todo, em parte porque ele faz um bom trabalho de reduzir as atividades necessárias em passos discretos e compreensíveis. Está longe de ser o único modelo existente, mas é o modelo que acho mais conveniente.

Há muito conteúdo por aí que descreve o processo, o qual está representado na Figura 2.2, portanto não vou me deter muito nesse assunto.⁸ No entanto, vale a pena apresentar rapidamente os passos e pensar em como poderiam nos ajudar se estivermos considerando a adoção de uma arquitetura de microsserviços.

Antes de descrever o processo, devo mencionar que esse modelo para mudanças em geral é usado para instituir mudanças

organizacionais de larga escala no comportamento. Desse modo, poderá ser um exagero se tudo que você estiver tentando fazer for introduzir os microsserviços em uma equipe de dez pessoas. Mesmo nesses ambientes de escopo reduzido, porém, percebi que esse modelo pode ser útil, particularmente no que concerne aos primeiros passos.

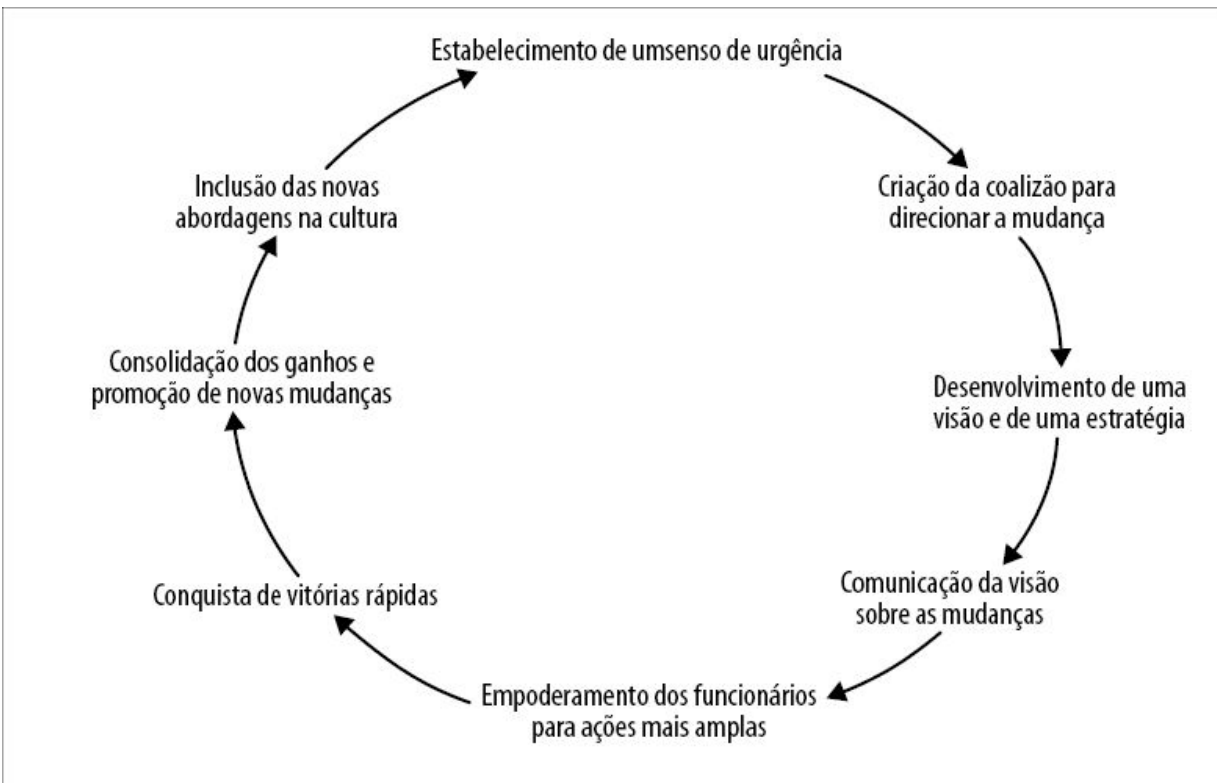


Figura 2.2 – Processo de oito passos de Kotter para mudanças organizacionais.

Estabelecimento de um senso de urgência

As pessoas podem pensar que sua ideia de passar para microsserviços é uma boa ideia. O problema é que sua ideia é apenas uma de várias ideias boas que provavelmente circulam pela empresa. O truque é ajudar as pessoas a entender que *agora* é a hora de fazer essa mudança em particular.

Buscar momentos “que ensinam”, nesse caso, pode ajudar. Às vezes, o momento certo para fechar a porta do estábulo é *depois*

que o cavalo fugiu porque as pessoas de repente percebem que cavalos que fogem é algo em que devem pensar e, nesse instante, se dão conta de que têm uma porta, e “Ah, olha, ela pode ser fechada e tudo mais!”. Após ter lidado com uma crise, você terá um breve momento na consciência das pessoas no qual pressionar por mudanças pode funcionar. Espere demais, e o sofrimento – e as causas desse sofrimento – terão diminuído.

Lembre-se de que o que você está tentando fazer não é dizer: “Devemos adotar os microsserviços agora!”. Você está tentando compartilhar um senso de urgência quanto ao que quer alcançar – e, conforme já mencionei, os microsserviços não são o objetivo!

Criação da coalizão para direcionar a mudança

Não é preciso que todos estejam envolvidos, mas é necessário ter pessoas suficientes para fazer a ideia se concretizar. É preciso identificar em sua empresa as pessoas que possam ajudar você a levar adiante essa mudança. Se você é um desenvolvedor, essa tarefa provavelmente terá início com os colegas imediatos de sua equipe e, talvez, com alguém mais experiente – pode ser um líder técnico, um arquiteto ou um gerente de entregas. Conforme o escopo da mudança, talvez não seja necessário haver muitas pessoas envolvidas. Se você está apenas mudando o modo como sua equipe faz uma tarefa, garantir que haja um apoio local talvez baste. Se estiver tentando transformar o modo como o software é desenvolvido em sua empresa, pode ser que precise de alguém no nível executivo para patrocinar essa mudança (talvez o CIO ou o CTO).

Persuadir as pessoas a ajudar nessa mudança talvez não seja fácil. Não importa quão boa você ache que é a sua ideia, se a pessoa jamais ouviu falar de você, ou se nunca trabalhou com você, por que ela deveria apoiá-la? A confiança é conquistada. É muito mais provável que alguém vá apoiar a sua grande ideia se essa pessoa já tiver trabalhado com você em tarefas menores, que tenham sido

bem-sucedidas.

Nesse caso, é importante que você tenha o envolvimento de pessoas externas à entrega de software. Se você já trabalha em uma empresa na qual as barreiras entre “TI” e “O Negócio” foram eliminadas, provavelmente não haverá problemas. Por outro lado, se esses silos continuam existindo, talvez seja necessário recorrer a pessoas de outras áreas para encontrar alguém que dê apoio. É claro que, se sua adoção da arquitetura de microsserviços tiver como foco a resolução de problemas enfrentados pelo negócio, será muito mais fácil vender a ideia.

O motivo pelo qual você precisa do envolvimento de pessoas externas ao silo de TI é porque muitas das mudanças que você fizer poderão, possivelmente, ter impactos significativos no modo como o software funciona e se comporta. Será necessário tomar diferentes decisões sobre o custo-benefício relacionado ao modo como seu sistema se comporta em modos de falha, por exemplo, ou sobre a forma de lidar com a latência. Por exemplo, fazer caching de dados a fim de evitar uma chamada de serviço é uma boa maneira de garantir que você reduzirá a latência de operações básicas em um sistema distribuído, mas, por outro lado, os usuários de seu sistema poderão ver dados desatualizados como consequência. Essa é a decisão certa a ser tomada? Provavelmente, você terá de discutir o assunto com seus usuários – e essa será uma discussão difícil de ser feita se as pessoas que defendem a causa de seus usuários na empresa não compreenderem o raciocínio que justifique a mudança.

Desenvolvimento de uma visão e de uma estratégia

É nesse ponto que você reúne seu pessoal e chega a um acordo sobre as mudanças que espera fazer (a *visão*) e como vai chegar lá (a *estratégia*). Visões são complicadas. Elas precisam ser realistas e, ao mesmo tempo, inspiradoras, e encontrar um equilíbrio entre os dois pontos é fundamental. Quanto mais amplamente compartilhada

for a visão, mais trabalho será necessário para sintetizá-la e persuadir as pessoas. Contudo, uma visão pode ser vaga e, apesar disso, funcionar para equipes menores (“Temos de reduzir o número de bugs!”).

Na maioria das vezes, a visão diz respeito ao objetivo – *o que* é que você está buscando. A estratégia trata do *como*. Os microsserviços farão esse objetivo ser alcançado (é o que se espera; eles farão parte de sua estratégia). Lembre-se de que sua estratégia pode mudar. Comprometer-se com uma visão é importante, mas comprometer-se demais com uma estratégia específica diante de evidências contrárias é perigoso e pode levar à significativa falácia do custo irrecuperável (sunk cost fallacy).

Comunicação da visão sobre as mudanças

Ter uma visão grandiosa pode ser ótimo, mas não a torne grandiosa demais a ponto de as pessoas não acreditarem que seja possível. Vi uma declaração feita recentemente pelo CEO de uma empresa de grande porte que afirmava o seguinte (parafraseando, de certo modo):

Nos próximos doze meses, reduziremos os custos e faremos entregas mais rápidas, pois passaremos a usar microsserviços e adotaremos tecnologias nativas de nuvem.

– CEO NÃO IDENTIFICADO

Nenhum dos funcionários da empresa com quem falei acreditava que algo desse tipo seria possível. Parte do problema na afirmação anterior são os objetivos potencialmente contraditórios apresentados – uma mudança completa no modo como o software é entregue pode ajudar você a fazer entregas mais rápidas, mas fazer isso em doze meses não reduzirá os custos, pois, provavelmente, será necessário desenvolver novas habilidades e haverá impactos negativos na produtividade até que essas novas habilidades estejam consolidadas. O outro problema é a janela de tempo mencionada. Nessa empresa em particular, a velocidade da mudança era tão lenta que um objetivo de doze meses era considerado risível. Portanto, qualquer que seja a visão que você compartilhar, ela deve ser verossímil.

Você pode começar com algo pequeno quando se trata de compartilhar uma visão. Eu fiz parte de um programa chamado “Test Mercenaries” (Mercenários de testes) para ajudar na implantação de práticas de automação de testes no Google muitos anos atrás. Esse programa teve início, antes de tudo, por causa de empreendimentos anteriores relacionados àquilo que chamaríamos hoje de comunidade de práticas (um “Grouplet” no jargão do Google) para ajudar a compartilhar a importância dos testes automatizados. Um dos primeiros esforços do programa para compartilhar informações sobre testes foi uma iniciativa chamada “Testing on the Toilet” (Testando no banheiro). Artigos curtos, de uma página, eram pregados na porta dos banheiros, de modo que as pessoas pudessem lê-los “em seu tempo livre”! Não estou sugerindo que essa técnica funcionaria em qualquer lugar, mas funcionou bem no Google – e era uma maneira realmente eficaz de compartilhar pequenos conselhos práticos.⁹

Eis uma última observação nesse caso. Há uma tendência cada vez maior de se afastar da comunicação face a face em favor de sistemas como o Slack. Quando se trata de compartilhar mensagens importantes desse tipo, uma comunicação face a face (o ideal é que seja pessoalmente, mas talvez possa ser por vídeo) será significativamente mais eficaz. Isso faz com que seja mais fácil compreender a reação das pessoas ao ouvir essas ideias e ajudará a dar os toques finais à sua mensagem, além de evitar entendimentos equivocados. Mesmo que você precise de outras formas de comunicação para divulgar sua visão em uma empresa maior, faça isso pessoalmente no início, se for possível. Isso ajudará você a aperfeiçoar a sua mensagem de modo muito mais eficaz.

Empoderamento dos funcionários para ações mais amplas

“Empoderamento dos funcionários” é um discurso dos consultores de gerentes para ajudar esses gerentes a fazerem o seu trabalho. Com muita frequência, o significado dessa expressão é muito simples – eliminar os obstáculos.

Você compartilhou a sua visão e gerou empolgação – e então, o que acontece? Surgem obstáculos no caminho. Um dos problemas mais comuns é o fato de as pessoas estarem muito ocupadas fazendo seus trabalhos agora para que tenham espaço para mudanças – em geral, é por isso que as empresas trazem pessoas novas (talvez contratando outras pessoas ou como consultores) para dar mais espaço de manobra e trazer expertise às equipes a fim de possibilitar uma mudança.

Como um exemplo concreto, quando se trata da adoção de microsserviços, os processos atuais relacionados ao provisionamento de infraestrutura podem ser um problema real. Se o modo como sua empresa cuida da implantação de um novo serviço em produção envolve fazer um pedido de hardware com seis meses de antecedência, adotar uma tecnologia que permita um

provisionamento por demanda de ambientes de execução virtualizados (como máquinas virtuais ou contêineres) poderia ser um avanço enorme, assim como uma mudança para um provedor de nuvem pública.

Não quero, porém, repetir os conselhos dados no capítulo anterior. Não inclua uma nova tecnologia no processo, somente para tê-la. Introduza-a para solucionar os problemas concretos que você tiver. À medida que identificar os obstáculos, introduza a nova tecnologia para corrigir esses problemas. Não caia na armadilha de gastar um ano definindo a Plataforma Perfeita para os Microsserviços, somente para descobrir que ela, na verdade, não resolve os problemas que você tem.

Como parte do programa Test Mercenaries do Google, acabamos criando frameworks para facilitar a criação e o gerenciamento de suítes de testes, promovemos a visibilidade dos testes como parte do sistema de revisão de código e acabamos, inclusive, influenciando a criação de uma nova ferramenta de CI utilizada em toda a empresa para facilitar a execução dos testes. No entanto, não fizemos tudo isso de uma só vez. Trabalhamos com algumas equipes, vimos os pontos que apresentavam dificuldades, aprendemos com eles e, então, investimos tempo para introduzir novas ferramentas. Também começamos com pouco – facilitar a criação de suítes de testes foi um processo bem simples, mas mudar o sistema de revisão de código da empresa como um todo foi uma tarefa muito maior. Não tentamos isso até termos tido certo nível de sucesso em outros lugares.

Conquista de vitórias rápidas

Se demorar muito para as pessoas virem algum progresso sendo feito, elas perderão a fé na visão. Portanto, procure conquistar algumas vitórias rápidas. Manter o foco inicialmente em pequenos objetivos simples e acessíveis ajudará a dar impulso. Quando se trata da decomposição em microsserviços, as funcionalidades que

podem ser facilmente extraídas de seu sistema monolítico deverão ter prioridade em sua lista. Porém, conforme já mencionamos, os microsserviços, por si só, não são o objetivo – portanto, é necessário encontrar um equilíbrio entre a facilidade de extrair alguma funcionalidade *versus* as vantagens que isso trará. Retomaremos essa ideia mais adiante neste capítulo.

É claro que, se você escolher algo que ache que seja simples e acabar tendo problemas enormes para concretizar a tarefa, esse poderia ser um insight valioso acerca de sua estratégia, e você poderia reconsiderar o que estiver fazendo. Não há problema algum nisso! O ponto principal é que, se você se concentrar em algo mais simples antes, é provável que vá ter esse insight *mais cedo*. Cometer erros é natural – tudo que podemos fazer é estruturar o processo a fim de garantir que aprenderemos com esses erros o mais rápido possível.

Consolidação dos ganhos e promoção de novas mudanças

Depois de ter conquistado um pouco de sucesso, é importante não ficar preso às glórias. Vitórias rápidas poderão ser as únicas vitórias se você não continuar a pressionar. É importante que faça uma pausa e reflita após os sucessos (e as falhas) para pensar em como deve continuar direcionando as mudanças. Talvez seja necessário mudar sua abordagem à medida que atingir diferentes partes da empresa.

Em uma transição para microsserviços, à medida que se aprofundar mais, talvez você ache mais difícil avançar. Lidar com a decomposição de um banco de dados pode ser uma tarefa que você prefira deixar de lado em um primeiro momento, mas não será possível adiá-la para sempre. Conforme exploraremos no Capítulo 4, há muitas técnicas à sua disposição, mas determinar qual é a abordagem correta exigirá considerações minuciosas. Lembre-se de que a técnica de decomposição que funcionou para você em uma

área de seu sistema monolítico talvez não funcione em outros lugares – você deverá testar continuamente novas formas de fazer progressos.

Inclusão das novas abordagens na cultura

Ao continuar iterando, fazendo a implantação das mudanças e compartilhando as histórias de sucesso (e de falhas), a nova forma de trabalhar começará a se transformar em um trabalho rotineiro. Boa parte dela diz respeito a compartilhar histórias com seus colegas, com outras equipes e outras pessoas da empresa. É muito comum que, uma vez que tenhamos resolvido um problema complicado, passemos simplesmente para o próximo problema. Para que as mudanças escalem – e permaneçam –, encontrar continuamente novos modos de compartilhar informações em sua empresa é essencial.

Com o tempo, o novo modo de fazer algo passará a ser o modo como o trabalho é feito. Se observar as empresas que estão bem adiantadas na adoção de arquiteturas de microserviços, o fato de ser uma abordagem correta ou não deixa de ser uma questão. Esse é o modo como as tarefas são feitas, e a empresa sabe muito bem como fazê-las.

Por sua vez, isso pode dar origem a outro problema. Assim que a Grande Nova Ideia passar a ser o Modo Consagrado de Trabalhar, como será possível garantir que abordagens melhores no futuro tenham espaço para emergir e, talvez, substituir o modo como o trabalho é feito?

Importância da migração gradual

Se você fizer uma reescrita Big Bang, a única coisa que poderá garantir é que terá um Big Bang.

– MARTIN FOWLER

Se você chegar ao ponto de decidir que separar seu sistema monolítico atual é a atitude correta a ser tomada, recomendo enfaticamente que quebre esses sistemas monolíticos extraíndo pequenas porções de cada vez. Uma abordagem gradual o ajudará a conhecer os microsserviços enquanto avançar, além de limitar o impacto de fazer algo errado (e você fará!). Pense em seu sistema monolítico como se fosse um bloco de mármore. Poderíamos explodir o bloco todo, mas isso raramente acabaria bem. Faz muito mais sentido simplesmente o talhar de maneira gradual.

O problema é que o custo de um experimento para passar de um sistema monolítico não trivial para uma arquitetura de microsserviços pode ser alto; se você fizer tudo de uma só vez, poderá ser difícil ter um bom feedback sobre o que está (e o que não está) funcionando bem. É muito mais fácil dividir uma jornada como essa em etapas menores; cada etapa poderá ser analisada, e será possível aprender com elas. É por esse motivo que sou um grande fã da entrega de software iterativa, até mesmo antes do surgimento do Agile – aceitando que cometerei erros e que, portanto, precisarei de um modo de reduzir a dimensão desses erros.

Qualquer transição para uma arquitetura de microsserviços deve ser feita com esses princípios em mente. Divida a grande jornada em vários passos pequenos. Podemos dar um passo de cada vez e aprender com eles. Se for um passo a ser dado para trás, será apenas um passo pequeno. De qualquer maneira, aprenderemos com ele, e o próximo passo a ser dado terá uma base mais sólida por causa dos passos que foram dados antes.

Conforme já discutimos, uma divisão em partes menores também permite identificar vitórias rápidas e aprender com elas. Isso pode contribuir para que o próximo passo seja mais fácil e ajudará a dar

impulso. Ao separar os microsserviços, um de cada vez, também poderemos perceber gradualmente a importância deles, em vez de ter de esperar por alguma implantação Big Bang.

Tudo isso resultou naquilo que passou a ser quase um conselho rotineiro às pessoas que pensam em adotar os microsserviços. Se você acha que essa é uma boa ideia, comece em algum ponto pequeno. Escolha uma ou duas áreas de funcionalidade, implemente-as como microsserviços, faça com que sejam implantadas no ambiente de produção e pense se funcionaram. Descreverei um modelo para identificar com quais microsserviços você deve começar, mais adiante neste capítulo.

É o ambiente de produção que conta

É *realmente* importante observar que a extração de um microsserviço não poderá ser considerada completa até que esse serviço esteja no ambiente de produção e seja efetivamente usado. Parte do objetivo da extração gradual é nos dar oportunidades para conhecer e compreender o impacto da própria decomposição. A maioria das lições importantes não será aprendida até que seu serviço chegue ao ambiente de produção.

A decomposição em microsserviços pode complicar a resolução de problemas, a monitoração dos fluxos de trabalho, pode causar problemas de latência, de integridade referencial, falhas em cascata e uma série de outros problemas. A maioria desses problemas você só perceberá depois que o sistema estiver no ambiente de produção. Nos próximos capítulos, veremos técnicas que lhe permitirão fazer uma implantação em um ambiente de produção, mas que limitarão o impacto dos problemas à medida que ocorrerem. Se você fizer uma alteração pequena, será muito mais fácil identificar (e corrigir) um problema que você vier a causar.

Custo da mudança

Há muitos motivos pelos quais, ao longo do livro, defenderei a

necessidade de fazer mudanças pequenas e graduais, mas um dos pontos principais é compreender o impacto de cada alteração que fizermos e mudar o curso de ação se for preciso. Isso nos permite atenuar mais facilmente o custo dos erros, porém não elimina totalmente as chances de cometer erros. Podemos – e vamos – cometer erros, e devemos aceitar isso. Contudo, devemos também saber qual é a melhor forma de reduzir os custos desses erros.

Decisões reversíveis e irreversíveis

Em suas cartas anuais aos acionistas, Jeff Bezos, CEO da Amazon, apresenta esclarecimentos interessantes sobre como a Amazon trabalha. A carta de 2015 continha a seguinte preciosidade:

Algumas decisões apresentam consequências e são irreversíveis ou quase irreversíveis – são portas unidirecionais – e essas decisões devem ser tomadas metodicamente, de forma lenta e criteriosa, envolvendo muita deliberação e consultas. Se você passar por essa porta e não gostar do que vê do outro lado, não será possível voltar ao ponto em que você estava antes. Podemos chamar essas decisões de decisões do Tipo 1. No entanto, a maioria das decisões não é desse tipo – elas são mutáveis, reversíveis – são portas que permitem ir e vir. Se você tomar uma decisão do Tipo 2 que não tenha sido a melhor decisão possível, não será necessário viver com as consequências por muito tempo. Você poderá abrir a porta novamente e retornar. As decisões do Tipo 2 podem e devem ser tomadas rapidamente por indivíduos com elevado grau de discernimento ou por grupos pequenos.

– JEFF BEZOS, CARTA AOS ACIONISTAS DA AMAZON (2015)

Bezos prossegue dizendo que as pessoas que não tomam decisões com frequência podem cair na armadilha de tratar as decisões do Tipo 2 como se fossem decisões do Tipo 1. Tudo se torna uma questão de vida ou morte; tudo passa a ser uma tarefa de dimensões enormes. O problema é que adotar uma arquitetura de microsserviços traz consigo uma série de opções quanto ao modo de realizar as tarefas – e isso significa que você talvez tenha de tomar muito mais decisões do que costumava tomar. Se você – ou a sua empresa – não estiver acostumado com isso, poderá se ver caindo nessa armadilha, e o progresso será totalmente interrompido. Os termos não são muito descritivos, e pode ser difícil se lembrar do que Tipo 1 e Tipo 2 realmente significam, portanto prefiro os nomes *Irreversíveis* (para o Tipo 1) ou *Reversíveis* (para o Tipo 2).¹⁰

Ainda que goste desse conceito, não acho que as decisões sempre se enquadrem muito bem em uma dessas categorias; parece haver um pouco mais de nuances envolvidas. Prefiro pensar em *Irreversíveis* e *Reversíveis* como duas extremidades de um

espectro, como mostra a Figura 2.3.

Avaliar em que ponto do espectro você se encontra pode ser desafiador no início, mas, basicamente, tudo se reduz a compreender o impacto caso você decida mudar de ideia depois. Quanto maior o impacto causado por uma mudança de curso feita posteriormente, mais próxima do tipo Irreversível será a decisão.

A verdade é que a maioria das decisões que você tomará como parte de uma transição para microsserviços estará mais perto da extremidade Reversíveis do espectro. O software tem como propriedade possibilitar rollbacks e desfazer mudanças em geral; é possível fazer o rollback de uma mudança de software ou de uma implantação. O que você deve levar em consideração é o custo de mudar de ideia depois.



Figura 2.3 – As diferenças entre decisões Irreversíveis e Reversíveis, com exemplos ao longo do espectro.

As decisões Irreversíveis exigirão mais engajamento, planejamento e consideração minuciosa, e você deverá lhes dedicar (devidamente) mais tempo. Quanto mais nos aproximamos do lado direito desse espectro, em direção às decisões Reversíveis, mais poderemos simplesmente contar com nossos colegas que estejam próximos do problema em questão para que tomem a decisão certa, sabendo que, se tomarem a decisão errada, corrigir depois será mais fácil.

Pontos mais apropriados para um experimento

O custo envolvido em mover um código de lugar em uma base de código é bem baixo. Há diversas ferramentas que podem ajudar e, se causarmos algum problema, a correção em geral será rápida. Separar um banco de dados, porém, implica muito mais trabalho, e fazer o rollback de uma mudança em um banco de dados é igualmente complexo. Do mesmo modo, desembaraçar serviços altamente acoplados, ou ter de reescrever totalmente uma API usada por vários clientes, pode ser um empreendimento considerável. O alto custo das mudanças significa que essas operações são muito arriscadas. Como podemos administrar esse risco? Minha abordagem é tentar cometer erros nos pontos em que o impacto será menor.

Tenho a tendência de colocar meu raciocínio no local em que o custo das mudanças e o custo dos erros será o menor possível: o quadro branco. Faça um esboço do design que você está propondo. Veja o que acontecerá ao executar casos de uso nos pontos em que você imagina que estarão as fronteiras de seus serviços. Em nossa loja musical, por exemplo, pense no que acontecerá quando um cliente procurar um disco, registrar-se no site ou comprar um álbum. Quais chamadas serão feitas? Você começa a notar referências circulares estranhas? Vê dois serviços conversando demais, o que poderia sinalizar que deveriam ser um único serviço?

Então, por onde devemos começar?

Tudo bem, já falamos da importância de articular claramente os nossos objetivos e compreender as possíveis relações de custo-benefício que possa haver. O que vem a seguir? Bem, precisamos ter uma visão das funcionalidades que queremos extrair e migrar para os serviços para que possamos começar a pensar racionalmente nos microsserviços que criaremos em seguida. Quando se trata de decompor um sistema monolítico existente, devemos ter alguma espécie de lógica de decomposição com a qual

trabalharemos, e é aí que o design orientado a domínios pode ser conveniente.

Design orientado a domínios

No Capítulo 1, apresentamos o design orientado a domínios como um conceito importante para ajudar a definir as fronteiras de nossos serviços. Desenvolver um modelo de domínios também nos ajudará quando for necessário atribuir prioridades em nossa decomposição. Na Figura 2.4, temos um exemplo de um modelo de domínios de alto nível para a Music Corp. O que vemos é um conjunto de contextos delimitados, identificados como resultado de um exercício de modelagem de domínios. Podemos ver claramente os relacionamentos entre esses contextos delimitados, que achamos que podem representar as interações na própria empresa.

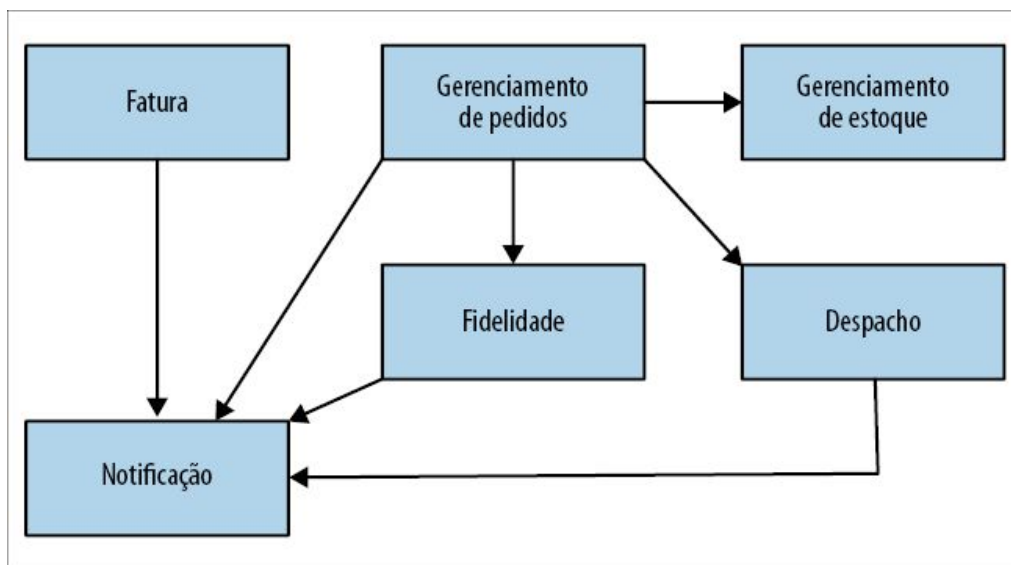


Figura 2.4 – Os contextos delimitados e os relacionamentos entre eles na Music Corp.

Cada um desses contextos delimitados representa uma possível unidade de decomposição. Conforme já discutimos, os contextos delimitados são um ótimo ponto de partida para definir fronteiras para os microsserviços. Desse modo, já temos nossa lista de itens para uma atribuição de prioridades. Contudo, também temos

informações úteis na forma de relacionamentos entre esses contextos delimitados – o que poderá nos ajudar a avaliar a dificuldade relativa na extração de diferentes funcionalidades. Retomaremos essa ideia em breve.

Considero a definição de um modelo de domínios como um passo praticamente imprescindível na estruturação de uma transição para os microsserviços. O que pode em geral ser assustador é o fato de muitas pessoas não terem experiência direta na definição de uma visão como essa. Elas também se preocupam muito com o volume de trabalho envolvido. A verdade é que, embora ter experiência possa ajudar bastante na definição de um modelo lógico como esse, dedicar mesmo que seja um pequeno volume de esforços pode resultar em algumas vantagens realmente proveitosas.

Até que ponto devemos ir?

Abordar a decomposição de um sistema existente é uma perspectiva assustadora. Muitas pessoas provavelmente desenvolvem e continuam desenvolvendo o sistema, e é muito provável que um grupo muito maior de pessoas o utilize diariamente. Considerando o escopo, tentar definir um modelo de domínios detalhado para todo o sistema pode ser assustador.

É importante entender que o que precisamos de um modelo de domínios são apenas informações *suficientes* para tomarmos uma decisão sensata sobre o ponto de partida de nossa decomposição. Você provavelmente já deve ter algumas ideias das partes do sistema que precisam de mais atenção e, desse modo, talvez seja suficiente definir um modelo genérico para o sistema monolítico com base em agrupamentos de alto nível das funcionalidades, e selecionar as partes que você queira detalhar mais. Se você observar apenas algumas partes do sistema, sempre haverá o risco de que problemas sistêmicos maiores, que precisam ser resolvidos, sejam deixados de lado. No entanto, eu não ficaria obcecado com isso – você não tem a obrigação de acertar na primeira vez; só

precisa de informações suficientes para que os próximos passos sejam dados com segurança. Você pode – e deve – aperfeiçoar continuamente o seu modelo de domínios à medida que aprender mais, e deve mantê-lo atualizado quando novas funcionalidades forem incluídas.

Event Storming

O *Event Storming* (Tempestade de Eventos), criado por Alberto Brandolini, é um exercício de colaboração que envolve pessoas-chaves (stakeholders) técnicas e não técnicas que, juntas, definem um modelo de domínios compartilhado. Um Event Storming funciona de baixo para cima. Os participantes começam definindo os “Eventos do Domínio” (Domain Events) – eventos que ocorrem no sistema. Esses eventos são então agrupados em agregados (aggregates), e os agregados, por sua vez, são agrupados em contextos delimitados (bounded contexts).

É importante observar que um Event Storming não implica que você deva desenvolver um sistema orientado a eventos. Em vez disso, seu foco é compreender os eventos (lógicos) que ocorrem no sistema – identificar os fatos com os quais você deve se preocupar no papel de uma pessoa-chave para o sistema. Esses eventos do domínio podem ser mapeados a eventos disparados como parte de um sistema orientado a eventos, mas poderiam ser representados de outras maneiras.

Um dos pontos no qual Alberto realmente mantém o foco com essa técnica é a ideia de definir coletivamente o modelo. O resultado desse exercício não é apenas o modelo em si, mas a *compreensão compartilhada* do modelo. Para que esse processo funcione, é necessário ter as pessoas-chaves apropriadas na sala – e, com frequência, esse é o maior desafio.

Explorar o Event Storming com mais detalhes está fora do escopo deste livro, mas é uma técnica que venho usando e da qual gosto muito. Se quiser explorar mais o assunto, leia o livro *Introducing*

EventStorming (https://leanpub.com/introducing_eventstorming) de Alberto Brandolini (que está atualmente em andamento).

Usando um modelo de domínios para atribuição de prioridades

Podemos ter insights interessantes a partir de diagramas como aquele mostrado na Figura 2.4. Com base no número de dependências upstream e downstream, podemos extrapolar e ter uma visão das funcionalidades que provavelmente serão as mais fáceis – ou as mais difíceis – de extrair. Por exemplo, se considerarmos a extração da funcionalidade de Notificação, podemos ver claramente uma série de dependências de entrada, como mostra a Figura 2.5 – muitas partes de nosso sistema exigem o uso desse comportamento. Se quisermos extrair nosso novo serviço de Notificação, teremos, portanto, muito trabalho a ser feito no código atual, mudando as chamadas à funcionalidade de notificação, que atualmente são locais, e fazendo com que passem a ser chamadas para um serviço. Veremos diversas técnicas relacionadas a esse tipo de mudança no Capítulo 3.

Desse modo, a Notificação talvez não seja um bom ponto de partida. Por outro lado, conforme em destaque na Figura 2.6, a Fatura pode representar uma parte do comportamento do sistema que seja mais fácil de extrair; ela não apresenta dependências de entrada, o que reduziria as mudanças necessárias que teríamos de fazer no sistema monolítico atual. Um padrão como o strangler fig poderia ser eficaz nesses casos, pois poderemos facilmente fazer um proxy dessas chamadas de entrada antes que alcancem o sistema monolítico. Exploraremos esse padrão, além de vários outros, no próximo capítulo.

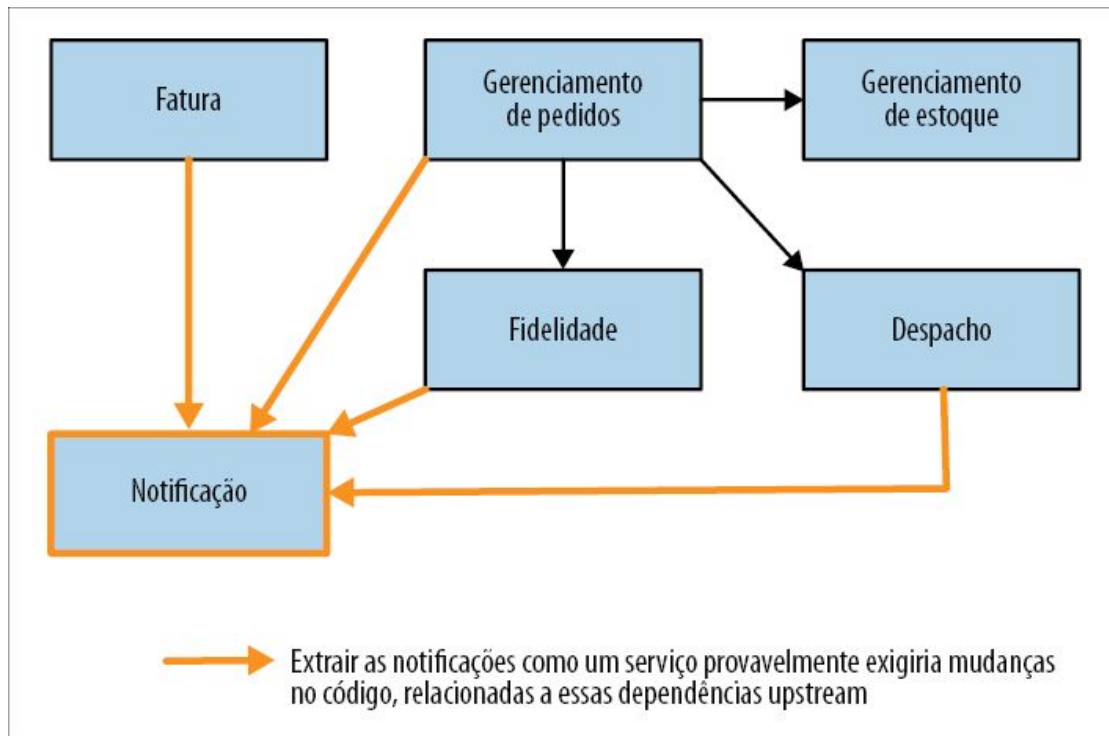


Figura 2.5 – A funcionalidade de Notificação, com base em nosso modelo de domínios, parece estar acoplada do ponto de vista lógico, portanto poderá ser mais difícil extraí-la.

Ao avaliar a dificuldade de extração, esses relacionamentos são um bom ponto de partida, mas devemos compreender que esse modelo de domínios representa uma visão *lógica* de um sistema existente. Não há garantias de que a estrutura de código subjacente de nosso sistema monolítico esteja organizada dessa forma. Isso significa que nosso modelo lógico pode ajudar a nos guiar no que concerne às funcionalidades que estejam provavelmente mais (ou menos) acopladas; contudo, talvez ainda seja necessário verificar o código para fazer uma melhor avaliação do grau de acoplamento da funcionalidade em questão. Um modelo de domínios como esse não nos mostrará quais contextos delimitados armazenam dados em um banco de dados. Poderíamos constatar que Fatura gerencia muitos dados, o que significaria que deveríamos considerar o impacto do trabalho da decomposição do banco de dados. Conforme será discutido no Capítulo 4, também podemos, e devemos, tentar separar os bancos de dados monolíticos, mas essa talvez não seja

uma tarefa com a qual devemos começar em nossos primeiros microsserviços.

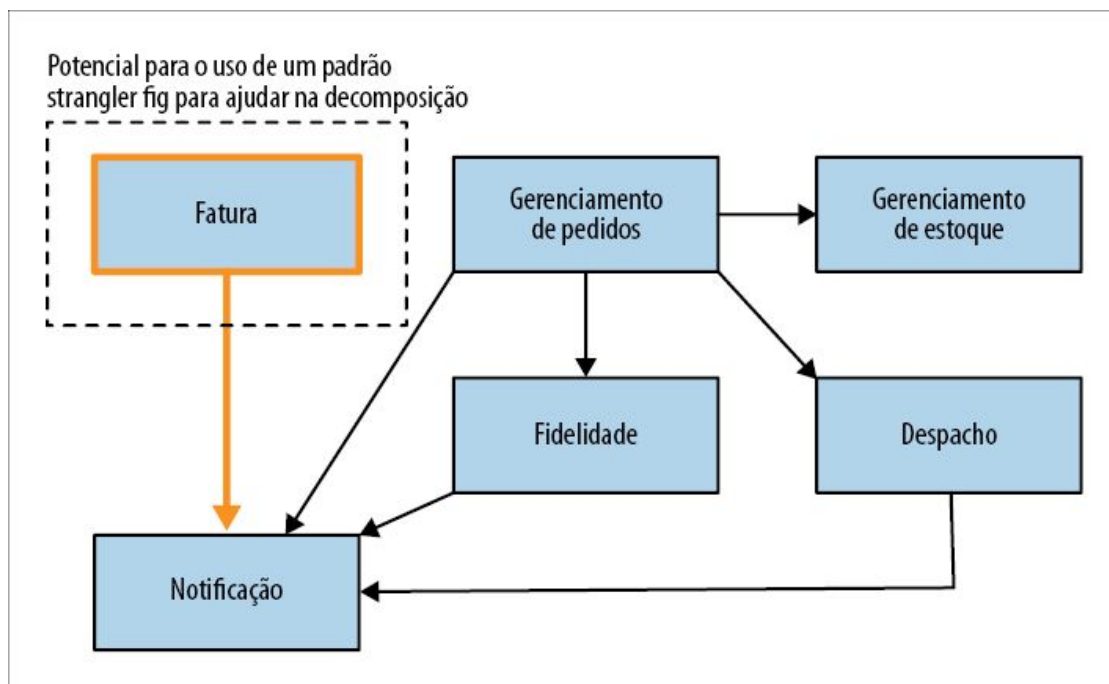


Figura 2.6 – A Fatura parece ser um serviço mais fácil de ser extraído.

Desse modo, podemos olhar para a situação através das lentes do que parece ser mais fácil ou mais difícil, e essa é uma atividade que vale a pena fazer – afinal de contas, queremos conquistar algumas vitórias rápidas! No entanto, devemos nos lembrar de que estamos encarando os microsserviços como uma forma de conseguir algo específico. Poderíamos determinar que a Fatura, na verdade, representa um passo inicial simples, mas, se o nosso objetivo for ajudar a reduzir o tempo para alcançar o mercado (time to market), e a funcionalidade Fatura raramente sofre alterações, então esse talvez não seja um bom uso do nosso tempo.

Precisamos, portanto, combinar nossa visão do que é fácil e do que é difícil com nossa visão das vantagens que a decomposição em microsserviços trará.

Um modelo combinado

Queremos algumas vitórias rápidas para fazer progressos iniciais, dar impulso e obter feedbacks rápidos sobre a eficácia de nossa abordagem. Isso nos levará a querer escolher os itens mais fáceis de extrair. Porém, também é necessário que tenhamos algumas vantagens com a decomposição – portanto, como incorporamos isso em nosso raciocínio?

Basicamente, as duas formas de definir prioridades fazem sentido, mas precisamos ter um método para visualizar ambas e analisar o custo-benefício de forma apropriada. Gosto de utilizar uma estrutura simples para isso, como mostra a Figura 2.7. Para cada serviço candidato a ser extraído, coloque-o nos dois eixos exibidos. O eixo x representa a vantagem que você acha que a decomposição trará. No eixo y, ordene os itens de acordo com a dificuldade.

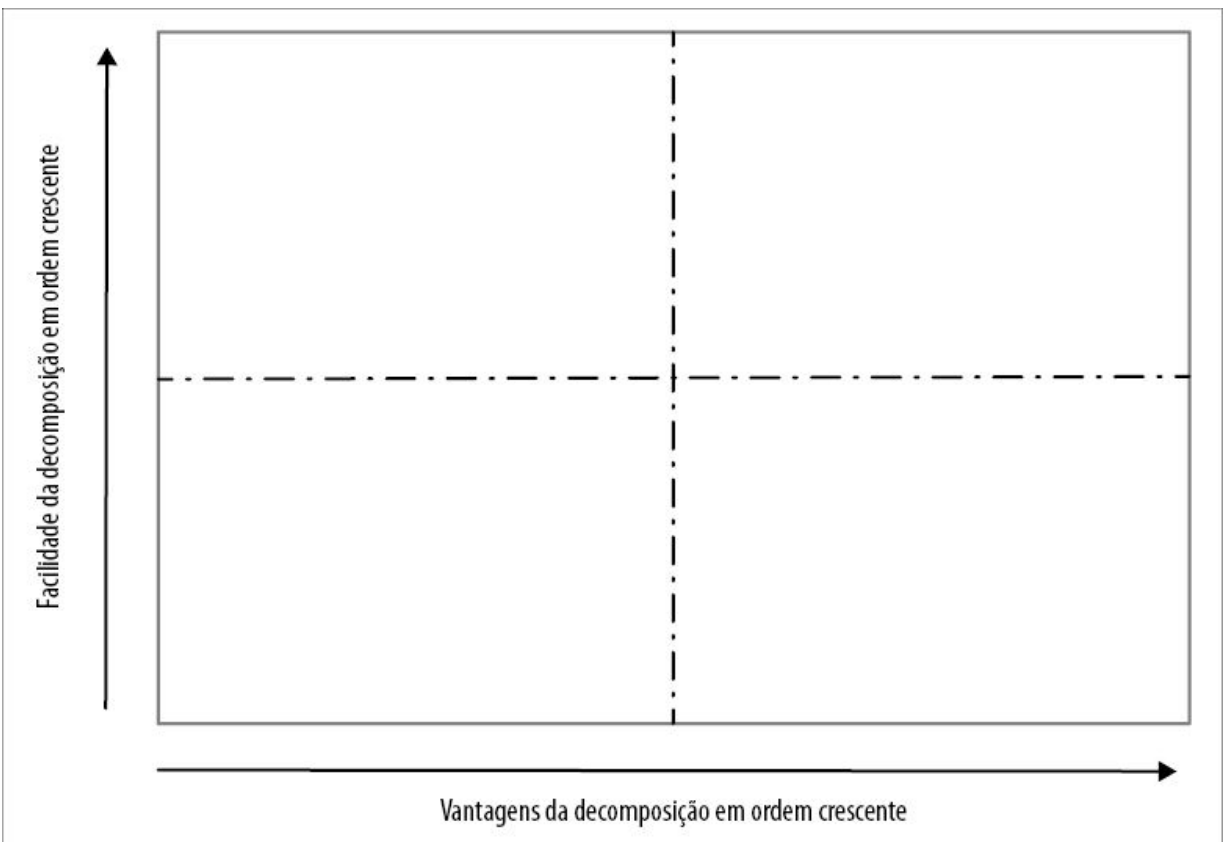


Figura 2.7 – Um modelo simples de dois eixos para atribuição de prioridades na decomposição dos serviços.

Ao trabalhar nesse processo como uma equipe, você terá uma visão

de quais seriam os bons candidatos para a extração – e, como todo bom modelo de quadrantes, é aquilo que estiver na parte superior à direita que queremos, como mostra a Figura 2.8. As funcionalidades que estiverem ali, incluindo a Fatura, representarão as funcionalidades que achamos que devem ser fáceis de extrair e que também nos trarão benefícios. Portanto, escolha um serviço (ou talvez dois) desse grupo para que seja o primeiro a ser extraído.

À medida que começar a fazer as mudanças, você aprenderá mais. Alguns pontos que você achava que seriam fáceis acabarão se revelando difíceis. Outros que você achava que seriam difíceis acabarão sendo fáceis. É natural! Contudo, isso significa que é importante rever esse exercício de atribuição de prioridades e replanejar à medida que você aprender mais. Talvez, ao dividir o sistema, você perceba que extrair o serviço de Notificação seja mais fácil do que você imaginava.

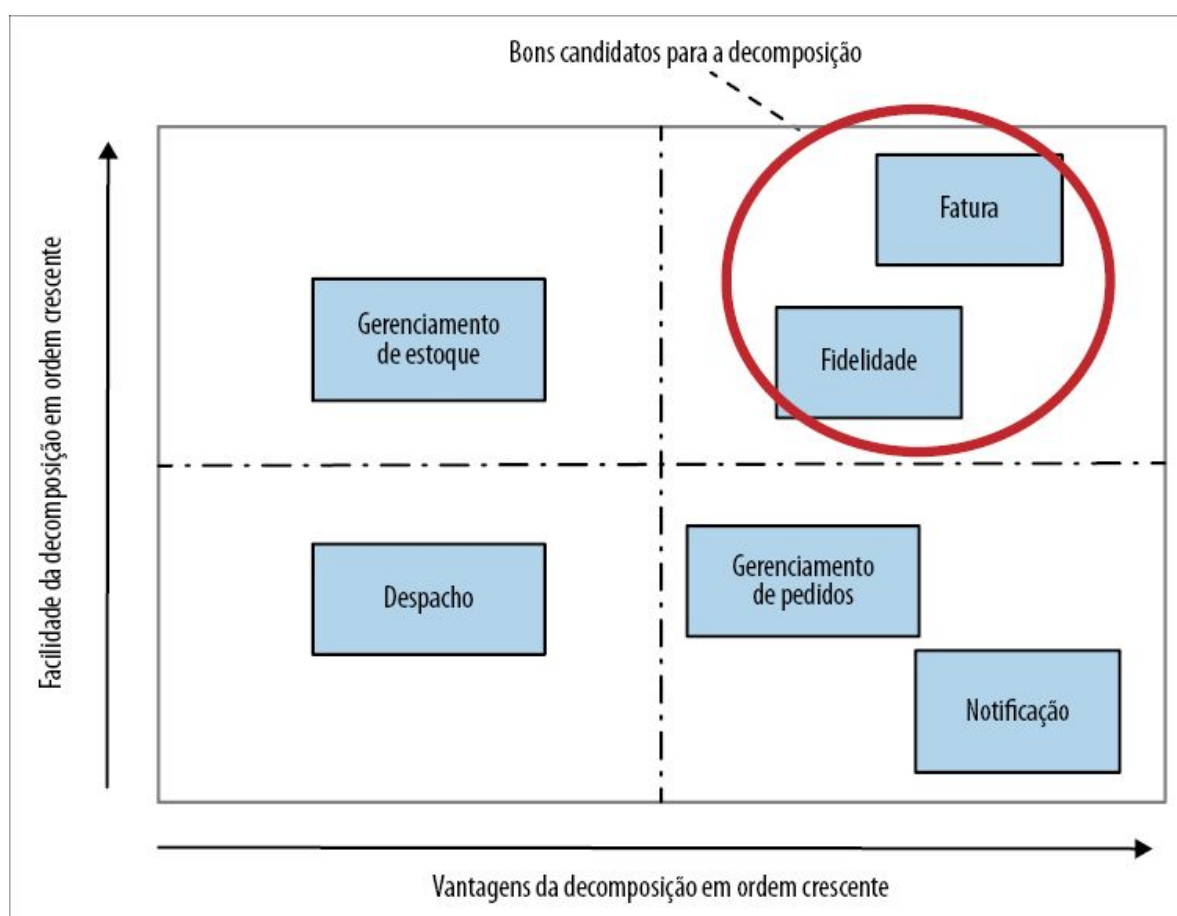


Figura 2.8 – Um exemplo do quadrante de prioridades em uso.

Reorganizando as equipes

Depois deste capítulo, mantereí o foco principalmente nas mudanças que você terá de fazer em sua arquitetura e no código a fim de fazer uma transição bem-sucedida para os microsserviços. Contudo, como já explicamos, alinhar a arquitetura e a empresa pode ser essencial para tirar o máximo proveito de uma arquitetura de microsserviços.

Entretanto, você poderia se ver em uma situação em que sua empresa precisa fazer mudanças para se beneficiar com essas novas ideias. Embora um estudo detalhado das mudanças organizacionais esteja fora do escopo deste livro, quero apresentar algumas ideias antes de explorarmos mais profundamente o lado mais técnico da situação.

Modificando as estruturas

Historicamente, as empresas de TI estavam estruturadas em torno de competências básicas. Desenvolvedores Java formavam uma equipe com outros desenvolvedores Java. Profissionais de teste formavam uma equipe com outros profissionais de teste. Os DBAs permaneciam todos em uma equipe própria. Ao criar um software, as pessoas pertencentes a essas equipes eram alocadas para trabalhar nessas iniciativas muitas vezes de curta duração.

O ato de criar um software, portanto, exigia várias passagens de responsabilidade entre as equipes. O analista de negócios conversava com os clientes para descobrir o que eles queriam. Em seguida, o analista redigia um requisito, passando-o para a equipe de desenvolvimento para que ela pudesse trabalhar nele. Um desenvolvedor concluía o trabalho e o passava para a equipe de testes. Se a equipe de testes encontrasse um problema, o trabalho era enviado de volta. Se não houvesse problemas, poderia ser enviado para a equipe de operações para que fosse implantado.

Essa divisão em silos parece bastante familiar. Considere as arquiteturas em camadas que discutimos no capítulo anterior, as quais poderiam exigir que vários serviços fossem alterados para o rollout de mudanças simples. O mesmo se aplica aos silos organizacionais: quanto mais equipes tiverem de estar envolvidas na criação ou modificação de um software, mais tempo será necessário.

Esses silos vêm sendo eliminados. Equipes de teste dedicadas atualmente fazem parte do passado em muitas empresas. Em seu lugar, especialistas em testes estão se tornando parte integrante das equipes de entrega, permitindo que os desenvolvedores e os responsáveis pelos testes trabalhem de modo muito mais próximo. O movimento DevOps também fez com que, em parte, muitas empresas se afastassem das equipes centralizadas de operações, atribuindo mais responsabilidade pelos aspectos operacionais às equipes de entrega.

Nos cenários em que as funções dessas equipes dedicadas foram passadas para as equipes de entrega, as funções dessas equipes sofreram mudanças. Essas equipes, que antes faziam, elas mesmas, o trabalho, passaram a ajudar as equipes de entrega a fazê-lo. Isso pode envolver a inclusão de especialistas nas equipes, a criação de ferramentas self-service, o oferecimento de treinamento ou toda uma gama de outras atividades. A responsabilidade dessas equipes passou de fazer para viabilizar.

Assim, cada vez mais, vemos equipes autônomas e independentes, capazes de se responsabilizar por mais partes do ciclo de vida de entrega do que jamais fizeram. O foco dessas equipes está em diferentes áreas de produto, em vez de estar em uma tecnologia ou em uma atividade específica – do mesmo modo que estamos passando de serviços orientados do ponto de vista técnico para serviços modelados em torno de fatias verticais das funcionalidades de negócio. O ponto importante a ser compreendido, porém, é que, embora essa mudança seja uma tendência definitiva evidente há

vários anos, ela não é universal, e uma mudança como essa tampouco implica uma transformação rápida.

Não há uma solução única para todos

Começamos este capítulo discutindo como sua decisão sobre o uso de microsserviços deve estar fortemente ligada aos desafios que você enfrenta e às mudanças que deseja promover. Fazer mudanças em sua estrutura organizacional é igualmente importante. Compreender se, e como, sua empresa deve mudar deve ter como base o seu contexto, a sua cultura de trabalho e o seu pessoal. É por isso que apenas copiar o design organizacional de outras empresas pode ser particularmente perigoso.

Já mencionamos superficialmente o modelo do Spotify. Houve um aumento do interesse em saber como o Spotify se organizava a partir do famoso artigo de 2012, “Scaling Agile @ Spotify” (Escalando o Agile no Spotify, <http://bit.ly/2ogAz3d>), de Henrik Kniberg e Anders Ivarsson. Esse é o artigo que popularizou a noção de Squads, Chapters e Guilds – termos que atualmente são muito comuns (ainda que não sejam devidamente compreendidos) em nosso mercado. Em última instância, o artigo levou as pessoas a batizar o modelo de “modelo do Spotify”, embora esse termo jamais tenha sido usado pelo Spotify.

Posteriormente, houve uma corrida das empresas para a adoção dessa estrutura. Contudo, como ocorre com os microsserviços, muitas empresas foram atraídas para o modelo do Spotify sem pensar suficientemente no contexto em que o Spotify atua, em seu modelo de negócios, nos desafios que enfrentam ou na cultura da empresa. O fato é que uma estrutura organizacional que funcionou bem para uma empresa sueca de streaming musical pode não funcionar para um banco de investimentos. Além disso, o artigo original mostrava uma imagem instantânea de como o Spotify funcionava em 2012, e, desde então, a situação mudou. A verdade é que nem mesmo o Spotify utiliza o modelo do Spotify.

O mesmo se aplica a você. Sem dúvida, busque inspiração no que outras empresas fizeram, mas não parta do pressuposto de que aquilo que funcionou para outra empresa também será apropriado para o seu contexto. Como disse Jessica Kerr certa vez acerca do modelo do Spotify, “Copy the questions, not the answers” (Copie as perguntas, e não as respostas, <http://bit.ly/2AKTaXP>). A fotografia instantânea da estrutura organizacional do Spotify refletia mudanças que a empresa havia feito para resolver seus problemas. Copie essa atitude flexível e questionadora sobre como trabalhar e experimente abordagens novas, mas certifique-se de que as mudanças que aplicar estejam baseadas na compreensão que você tem de sua empresa, em suas necessidades, nas pessoas e em sua cultura.

Para dar um exemplo específico, vejo muitas empresas dizendo o seguinte às suas equipes de entrega: “Muito bem, agora todos vocês devem implantar seu software e oferecer suporte 24 horas por dia, sete dias por semana”. Isso pode ser extremamente perturbador e não ajuda em nada. Às vezes, fazer afirmações ousadas pode ser uma ótima maneira de fazer as coisas acontecerem, mas esteja preparado para o caos que isso pode gerar. Se você trabalha em um ambiente no qual os desenvolvedores estão acostumados a trabalhar no horário comercial, não fazem plantão, jamais trabalharam com suporte ou em um ambiente de operações e não saberiam sequer usar um SSH, essa seria uma ótima maneira de deixar seu quadro de funcionários indisposto e perder vários funcionários. Se você acha que esse é o passo certo para a sua empresa, ótimo! No entanto, fale para servir de inspiração, como uma meta que você queira alcançar, e explique os motivos. Em seguida, trabalhe com seu pessoal para levar adiante essa jornada em direção a essa meta.

Se você realmente quiser fazer uma mudança nas equipes para que elas sejam mais responsáveis pelo ciclo de vida completo de seus softwares, entenda que as habilidades dessas equipes devem ser diferentes. Você pode oferecer ajuda e treinamento, acrescentar outras pessoas à equipe (talvez incluindo pessoas da equipe atual

de operações nas equipes de entrega). Não importa as mudanças que você queira implantar, assim como o nosso software, você pode fazê-las gradualmente.

DevOps não quer dizer NoOps!

Há uma confusão que permeia toda a área de DevOps, com algumas pessoas achando que DevOps significa que os desenvolvedores farão todas as operações, e que o pessoal de operações não será mais necessário. Isso está longe de ser verdade. Basicamente, o DevOps é um movimento cultural baseado no conceito de acabar com as barreiras entre o desenvolvimento e as operações. Você ainda pode querer ou não ter especialistas desempenhando essas funções, mas, independentemente do que quiser fazer, deverá promover um alinhamento e uma compreensão comuns entre as pessoas envolvidas na entrega de seu software, não importa quais sejam suas responsabilidades específicas.

Para mais informações sobre esse assunto, recomendo o livro *Team Topologies*¹¹, que explora as estruturas organizacionais do DevOps. Outro recurso excelente sobre esse tópico, embora seja mais amplo quanto ao escopo, é o livro *The Devops Handbook*.¹²

Fazendo uma mudança

Então, se você não deve simplesmente copiar a estrutura de outra empresa, por onde deve começar? Ao trabalhar com empresas que estejam mudando a função das equipes de entrega, gosto de começar listando explicitamente todas as atividades e responsabilidades envolvidas na entrega de um software nessa empresa. Em seguida, mapeie essas atividades à sua estrutura organizacional atual.

Se você já modelou seu caminho até a produção (algo do qual sou grande fã), poderia sobrepor essas fronteiras de responsabilidades em uma visão existente. Como alternativa, algo simples como o que mostra a Figura 2.9 poderia funcionar bem. Basta reunir as pessoas-chaves com todas as funções envolvidas e fazer um brainstorming

de todas as atividades relacionadas ao lançamento de um software em sua empresa.

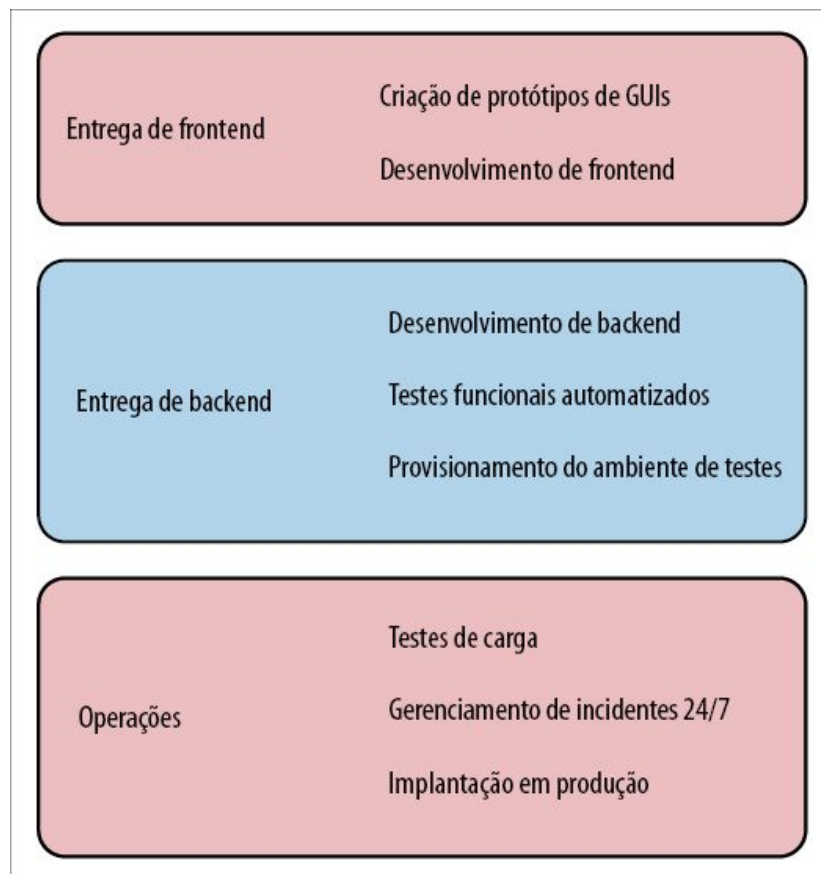


Figura 2.9 – Exibindo um subconjunto das responsabilidades relacionadas à entrega e como elas se mapeiam às equipes atuais.

Ter essa compreensão do estado atual “como é” é muito importante, pois pode contribuir para que todos possam compreender o trabalho envolvido de forma compartilhada. A natureza de uma organização com silos implica que você poderá ter dificuldades em entender o que um silo faz caso esteja em um silo distinto. Acho que isso realmente ajuda as empresas a serem honestas consigo mesmas acerca da rapidez com que as mudanças podem ser feitas. É provável que você também perceba que nem todas as equipes são iguais – algumas talvez já façam muito por conta própria, enquanto outras podem ser totalmente dependentes de outras equipes para tudo, de testes à implantação.

Se você constatar que suas equipes de entrega já estão, elas mesmas, implantando os softwares para testes e testes de usuários, então passar para implantações em produção talvez não seja um passo tão grande. Por outro lado, ainda será necessário considerar o impacto de assumir o suporte de nível 1 (carregar um pager), diagnosticar problemas de produção e assim por diante. Essas habilidades são desenvolvidas por pessoas ao longo de anos de trabalho, e esperar que os desenvolvedores mudem rapidamente, da noite para o dia, é totalmente fora da realidade.

Assim que você tiver uma imagem de sua situação tal como ela é, redesenhe-a com base em sua visão de como tudo deveria ser no futuro, considerando um prazo sensato. Acho que de seis meses a um ano provavelmente é o máximo que você deve explorar em detalhes. Quais responsabilidades mudarão de mãos? Como você fará essa transição ocorrer? O que é necessário para fazer essa mudança? Quais são as novas habilidades que as equipes terão de adquirir? Quais são as prioridades das diversas mudanças que você quer fazer?

Considerando o nosso exemplo anterior, na Figura 2.10 vemos que decidimos combinar as responsabilidades das equipes de frontend e de backend. Também queremos que as equipes sejam capazes de provisionar os próprios ambientes de teste. Contudo, para isso, a equipe de operações precisará oferecer uma plataforma self-service para que a equipe de entrega possa usar. Queremos que a equipe de entrega, em última análise, cuide de todo o suporte para seu software e, desse modo, queremos que as equipes comecem a ficar mais satisfeitas com o trabalho envolvido. Fazer com que sejam responsáveis pelas próprias implantações de teste é um bom passo inicial. Decidimos também que elas cuidarão de todos os incidentes durante o horário de trabalho, o que lhes dará uma oportunidade para se ajustar a esse processo em um ambiente seguro, no qual a equipe de operações atual estará à disposição para orientá-las.

As visões gerais podem realmente ajudar no início das mudanças

que você quer fazer, mas também será necessário dispor de tempo junto às pessoas que estão na base para determinar se essas mudanças são viáveis e, em caso afirmativo, como concretizá-las. Ao fazer uma separação em responsabilidades específicas, você também poderá adotar uma abordagem incremental para essa mudança. Para você, manter inicialmente o foco em acabar com a necessidade de as equipes de operações provisionarem os ambientes de teste será o primeiro passo apropriado.

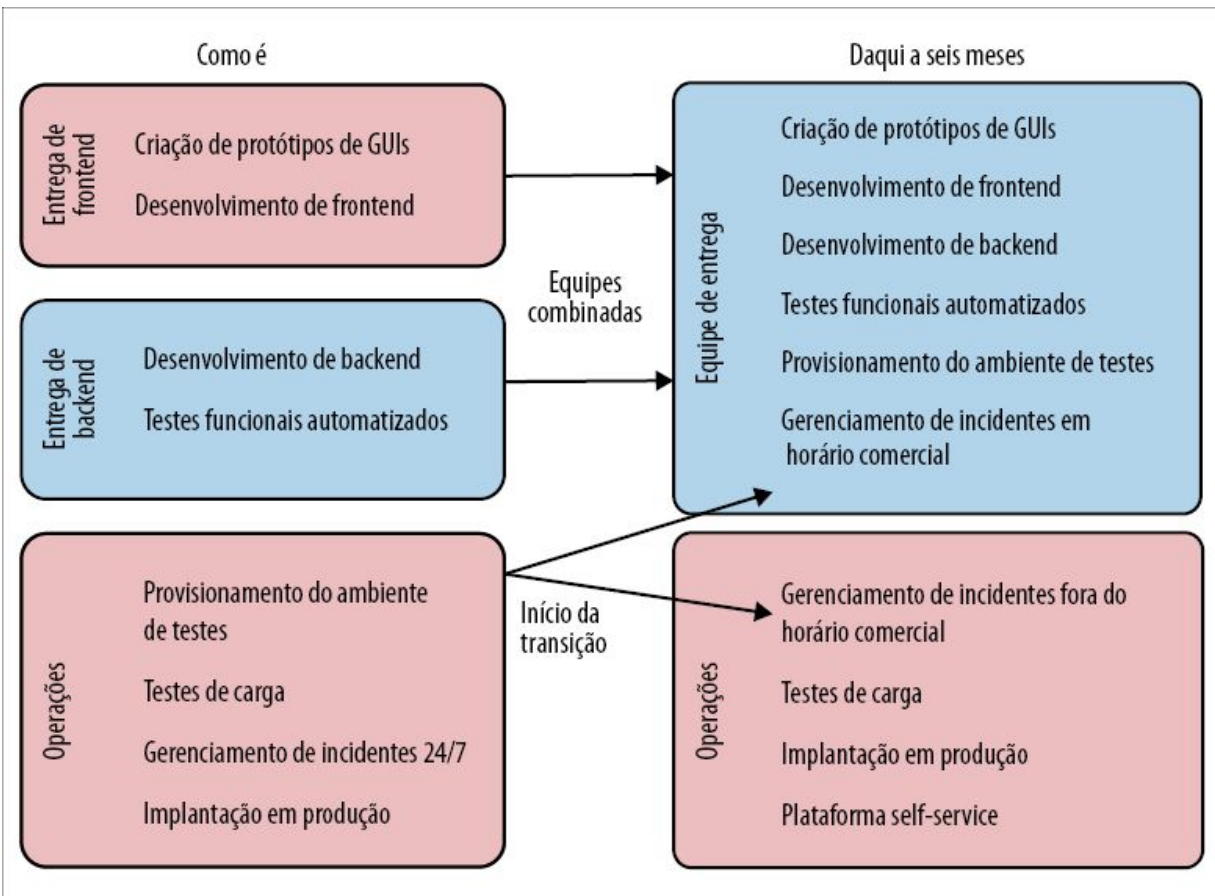


Figura 2.10 – Um exemplo de como poderíamos redistribuir as responsabilidades em nossa empresa.

Outras habilidades

Quando se trata de avaliar as habilidades necessárias às pessoas e ajudá-las a preencher essa lacuna, sou totalmente a favor de fazer as pessoas se autoavaliarem e usar essas informações para ter

uma maior compreensão do suporte de que a equipe precisará para efetuar essa mudança.

Um exemplo concreto dessa abordagem em ação é um projeto com o qual estive envolvido na época em que trabalhei na ThoughtWorks. Havíamos sido contratados para ajudar o jornal *The Guardian* a redefinir a sua presença online (um assunto que será retomado no próximo capítulo). Como parte dessa tarefa, eles precisavam aprender rapidamente uma nova linguagem de programação e conhecer as tecnologias associadas.

No início do projeto, nossas equipes definiram conjuntamente uma lista das habilidades essenciais nas quais seria importante que os desenvolvedores do *The Guardian* trabalhassem. Cada desenvolvedor então se autoavaliou em relação a esses critérios, classificando-os de 1 (“Isso não significa nada para mim!”) a 5 (“Eu poderia escrever um livro sobre esse assunto”). A pontuação de cada desenvolvedor era privada; essas informações foram compartilhadas somente com o orientador de cada pessoa. O objetivo não era que cada desenvolvedor tivesse uma habilidade classificada como 5; era mais para que eles mesmos pudessem definir metas a serem alcançadas.

Como coach, era meu trabalho garantir que, se um dos desenvolvedores que eu estava orientando quisesse melhorar suas habilidades com Oracle, ele tivesse a chance de fazê-lo. Essa tarefa poderia consistir em garantir que eles trabalhassem com casos de uso que fizessem uso dessa tecnologia, recomendar vídeos para que assistissem, considerar a participação em cursos de treinamento ou em conferências etc.

Você pode usar esse processo para criar uma representação visual das áreas nas quais um indivíduo possa querer concentrar seu tempo e seus esforços. Na Figura 2.11, vemos um exemplo desse tipo, o qual mostra que eu realmente quero manter meu tempo e energia concentrados em aumentar minha experiência com Kubernetes e com Lambda, talvez refletindo o fato de que, agora,

vou ter de gerenciar as implantações de meu próprio software.

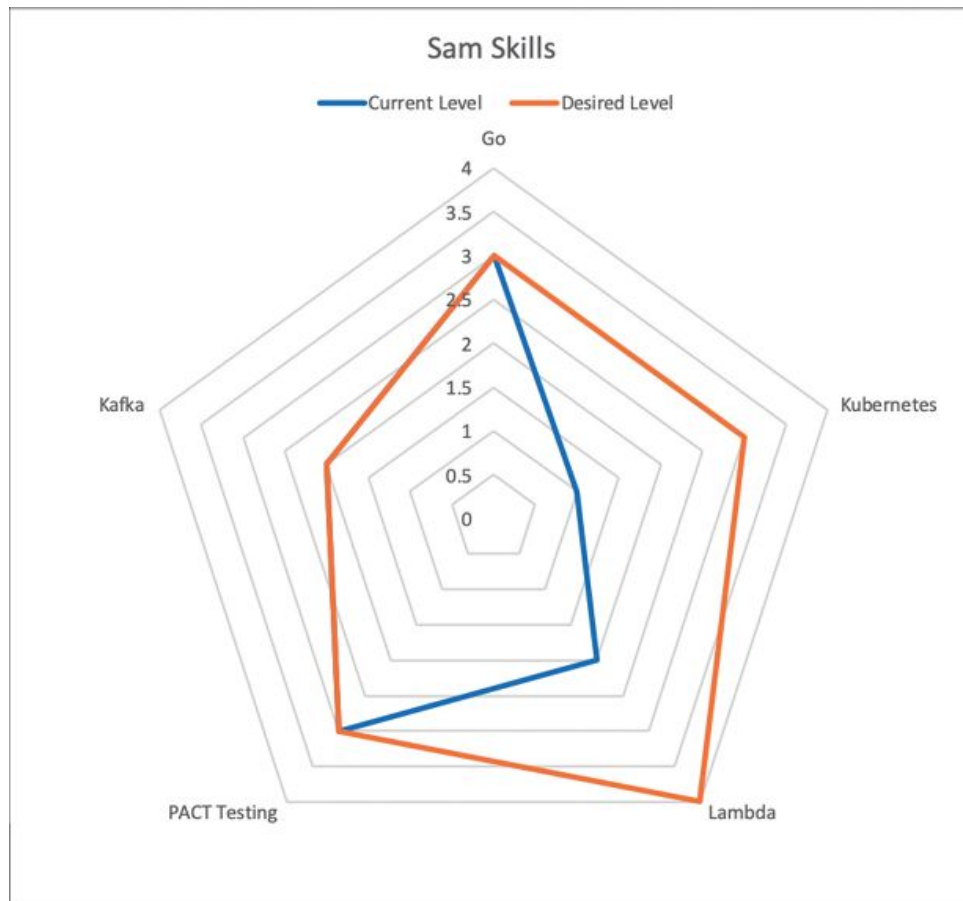


Figura 2.11 – Um exemplo de um diagrama de habilidades, mostrando as áreas nas quais quero me aperfeiçoar.

Do mesmo modo, enfatizar as áreas com as quais você já está satisfeito – nesse exemplo, acho que escrever código em Go não é algo no qual eu deva me concentrar no momento – é igualmente importante.

Manter esses tipos de autoavaliação privados é muito importante. O ponto principal não é alguém se classificar em relação a outra pessoa; é ajudar as pessoas a direcionarem o próprio desenvolvimento. Torne esses dados públicos e você mudará drasticamente os parâmetros desse exercício. Repentinamente, as pessoas se preocuparão com o fato de dar pontuações muito baixas a si mesmas, pois isso poderia ter impacto em suas avaliações de desempenho, por exemplo.

Embora cada pontuação seja privada, você ainda poderá usá-las para ter um quadro geral da equipe como um todo. Utilize as classificações obtidas com as autoavaliações anônimas e crie um mapa de habilidades para a equipe de modo geral. Isso pode ajudar a enfatizar as lacunas que talvez precisem ser preenchidas em um nível mais sistêmico. A Figura 2.12 nos mostra que, embora eu possa estar satisfeito com meu nível de habilidades com PACT, como um todo, a equipe quer melhorar nessa área, enquanto Kafka e Kubernetes são outras áreas que talvez precisem de um foco mais intenso.

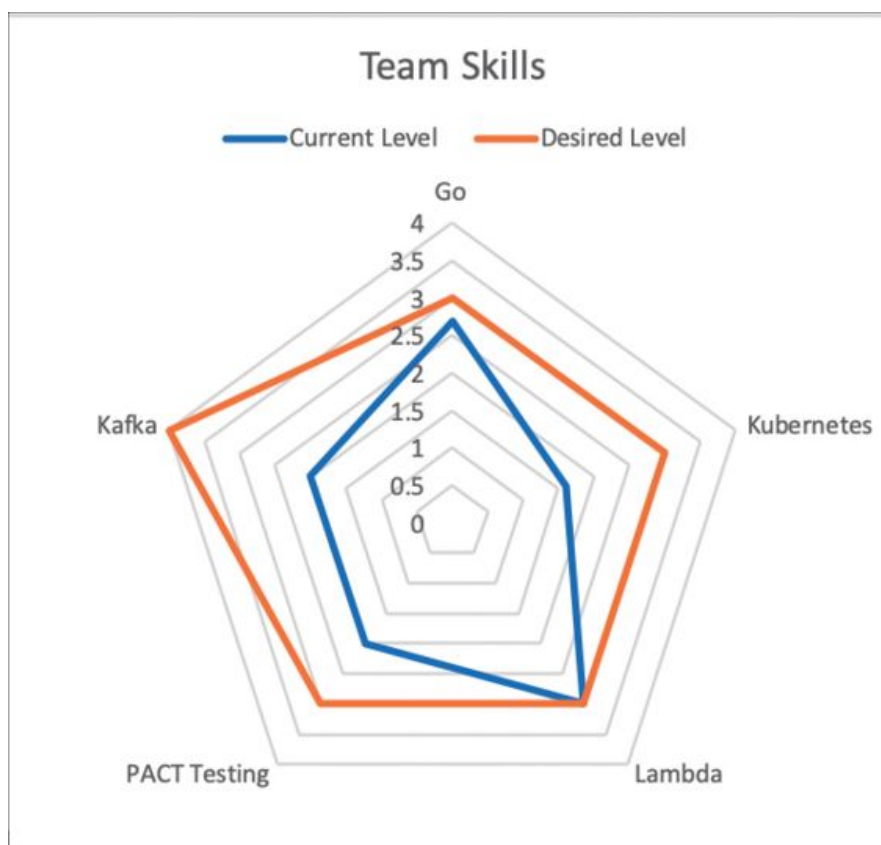


Figura 2.12 – Observando como um todo, a equipe precisa aperfeiçoar suas habilidades com Kafka, Kubernetes e Testes com PACT.

Essa tarefa poderia revelar a necessidade de alguma aprendizagem em grupo e, talvez, justificar um investimento maior, por exemplo, oferecer um curso de treinamento interno. Compartilhar esse quadro

geral com sua equipe também pode ajudar os indivíduos a compreender como eles poderão ajudar a equipe toda a encontrar o equilíbrio necessário.

Mudar o conjunto de habilidades dos membros da equipe atual não é o único caminho a ser seguido, é claro. Em geral, nosso objetivo é ter uma equipe de entrega que, como um todo, assuma mais responsabilidades. Isso não significa necessariamente que cada indivíduo vá fazer mais. A resposta correta poderia ser incluir outras pessoas que tenham as habilidades necessárias na equipe. Em vez de ajudar os desenvolvedores a conhecer melhor o Kafka, você poderia contratar um especialista em Kafka para se juntar à equipe. Isso poderia resolver o problema no curto prazo, e você teria então um especialista na equipe, capaz de ajudar seus colegas a aprender mais sobre esse assunto também.

Poderíamos explorar muito mais este tópico, mas espero ter compartilhado informações suficientes para que você possa começar a trabalhar. O mais importante é que comece a conhecer seus funcionários e a sua cultura, assim como as necessidades de seus usuários. Sem dúvida, busque inspiração nos estudos de caso de outras empresas, mas não se surpreenda se uma mera cópia das respostas de outras pessoas aos seus problemas acabarem não funcionando bem para você.

Como você saberá que a transição está funcionando?

Todos nós cometemos erros. Mesmo que inicie a jornada em direção aos microsserviços com as melhores intenções possíveis, você deve aceitar o fato de que não saberá tudo, e que, em algum ponto do caminho, poderá perceber que nem tudo está funcionando como devia. Estas são as perguntas que devem ser feitas: Você sabe se está funcionando? Você cometeu algum erro?

Com base nos resultados que espera alcançar, tente definir alguns critérios de avaliação que poderão ser monitorados e ajudarão a

responder a essas perguntas. Exploraremos alguns exemplos desses critérios em breve, mas gostaria de aproveitar a oportunidade para dar ênfase ao fato de que não estamos falando somente de métricas quantitativas nesse caso. O feedback qualitativo das pessoas diretamente envolvidas também deve ser levado em consideração.

Essas métricas, quantitativas e qualitativas, devem servir de base para um processo de avaliação contínua. Defina pontos de verificação que permitirão que sua equipe tenha tempo para refletir e saber se está indo na direção correta. A pergunta que você deve fazer a si mesmo nesses pontos de verificação não é apenas “Está funcionando?”, mas “Devemos tentar algo diferente?”.

Vamos analisar uma maneira de organizar essas atividades de verificação, assim como ver alguns exemplos de critérios de avaliação que possam ser monitorados.

Ter pontos de verificação regulares

Como parte de qualquer transição, é importante incluir um tempo em seu processo de entrega para fazer uma pausa e refletir, a fim de analisar as informações disponíveis e determinar se uma mudança de curso será necessária. Para equipes pequenas, isso poderia ser feito informalmente ou, talvez, poderia estar incluído nos exercícios regulares de retrospectiva. Em programas de trabalho mais amplos, essas atividades terão de ser planejadas explicitamente, com uma cadência regular, talvez reunindo os líderes de diversas atividades em sessões mensais para analisar o andamento da situação.

Não importa a frequência com que esses exercícios serão conduzidos e não importa o nível de formalidade (ou de informalidade), sugiro que você se certifique de que incluirá o seguinte:

1. Reafirme o que você espera alcançar com a transição para microsserviços. Se os negócios mudaram de direção, de modo que a direção para a qual você está seguindo não faz mais

sentido, pare!

2. Analise qualquer critério de avaliação quantitativo que tenha sido definido para saber se você está fazendo progressos.
3. Peça feedbacks qualitativos – as pessoas continuam achando que tudo está funcionando?
4. Decida o que você mudará daqui para a frente, se houver algo.

Crítérios de avaliação quantitativos

Os critérios de avaliação selecionados para monitorar o progresso dependerão dos objetivos que você estiver tentando alcançar. Se o seu foco estiver em reduzir o tempo para chegar ao mercado, por exemplo, calcular a duração do ciclo, o número de implantações e as taxas de falha fará sentido. Se estiver tentando escalar a aplicação para lidar com mais carga, contar com o resultado dos testes de desempenho mais recentes seria sensato.

Vale a pena observar que as métricas podem ser perigosas por causa do velho ditado “Você terá aquilo que avaliar”. As métricas podem ser distorcidas – inadvertidamente ou de propósito. Eu me lembro de minha esposa me falando de uma empresa na qual ela trabalhou, em que uma empresa terceirizada era monitorada com base no número de tickets que fechavam, e o pagamento era efetuado de acordo com esses resultados. O que aconteceu? A empresa terceirizada fechava os tickets mesmo que o problema não estivesse resolvido, fazendo as pessoas abrirem novos tickets.

Outras métricas podem ser difíceis de mudar em pouco tempo. Eu ficaria surpreso se você visse muitas melhorias na duração de um ciclo, como parte de uma migração para microsserviços, nos primeiros meses; na verdade, eu esperaria ver inicialmente a situação piorar. Introduzir uma mudança no modo como as tarefas são feitas em geral causa um impacto negativo na produtividade no curto prazo, enquanto a equipe se acostuma com o novo modo de trabalhar. Esse é outro motivo pelo qual dar passos incrementais *pequenos* é tão importante: quanto menor a mudança, menores

serão os possíveis impactos negativos, e mais rápido você poderá solucioná-los quando ocorrerem.

Critérios de avaliação qualitativos

...O software é feito de impressões.

– ASTRID ATKINSON (@SHINYNEW_OZ)

Não importa o que os dados nos mostrem, são as pessoas que fazem o software, e é importante que seu feedback seja incluído na avaliação do sucesso. Elas estão gostando do processo? Elas se sentem empoderadas? Ou se sentem sobrecarregadas? Elas estão tendo o apoio necessário para assumir novas responsabilidades ou adquirir novas habilidades?

Ao comunicar qualquer tipo de tabela de pontuação para a gerência de nível superior nesses tipos de transições¹³, inclua uma avaliação das impressões de sua equipe. Se eles estiverem adorando, ótimo! Se não estiverem, talvez seja necessário fazer algo a respeito. Ignorar o que o seu pessoal está lhe dizendo em favor de contar exclusivamente com métricas quantitativas é uma ótima maneira de se ver envolvido em muitos problemas.

Evitando a falácia do custo irrecuperável

Você deve tomar cuidado com a falácia do custo irrecuperável (sunk cost fallacy); ter um processo de revisão ajuda a manter a honestidade, e espera-se que isso ajude a evitar esse fenômeno. A *falácia do custo irrecuperável* ocorre quando as pessoas se engajam tanto com uma abordagem anterior para fazer algo, que, mesmo diante de evidências que mostrem que a abordagem não está funcionando, elas seguirão em frente. Às vezes, damos justificativas a nós mesmos: “Vai mudar a qualquer momento!”. Em outras ocasiões, talvez tenhamos investido muito capital político para fazer uma mudança em nossa empresa, a ponto de ser impossível retroceder agora. De qualquer modo, é certo que a falácia do custo irrecuperável diz respeito totalmente a um investimento emocional: estamos tão engajados com um modo de pensar antigo que simplesmente não podemos abrir mão dele.

Com base em minha experiência, quanto maior a aposta, e quanto maior o estardalhaço que a acompanha, mais difícil será voltar atrás

se ela fracassar. A falácia do custo irrecuperável também é conhecida como a falácia do Concorde, que recebeu o nome por causa do projeto fracassado, de custo muito elevado, que teve o apoio dos governos britânico e francês, para a construção de um avião supersônico para transporte de passageiros. Apesar de todas as evidências de que o projeto jamais traria retornos financeiros, mais e mais dinheiro era investido no projeto. Independentemente dos sucessos de engenharia que possa ter produzido, o Concorde jamais funcionou como uma aeronave viável para voos comerciais de passageiros.

Se você fizer com que cada passo seja pequeno, será mais fácil evitar as armadilhas da falácia do custo irrecuperável. Será mais fácil mudar de direção. Utilize o método de pontos de verificação discutido antes para refletir sobre o que está acontecendo. Não será necessário desistir ou mudar o curso de ação ao primeiro sinal de problemas, mas ignorar as evidências que você estiver coletando quanto ao sucesso (ou não) da mudança que está tentando realizar será, sem dúvida, uma tolice muito maior do que não coletar nenhuma evidência.

Esteja aberto a novas abordagens

Espero que não seja nenhuma surpresa, considerando que você chegou até este ponto do livro, mas há diversas variáveis envolvidas na divisão de um sistema monolítico, e vários caminhos diferentes que poderíamos tomar. A única certeza é que nem tudo ocorrerá de maneira tranquila, e você deverá estar aberto a reverter as mudanças que fizer, tentar novas abordagens ou, às vezes, simplesmente deixar que a situação se acomode por um tempo a fim de perceber os impactos causados.

Se você tentar adotar uma cultura de melhorias constantes, de estar sempre experimentando algo novo, será muito mais natural mudar de direção quando for necessário. Se você isolar o conceito de mudanças ou de melhoria de processos em fluxos discretos de

trabalho, em vez de incluí-lo em tudo que fizer, correrá o risco de ver as mudanças como atividades transacionais, a serem feitas uma só vez. Assim que o trabalho for feito, acabou-se! Chega de mudanças para nós! Com esse modo de pensar, é assim que você vai se ver daqui a alguns anos, muito atrás de seus concorrentes, diante de outra montanha a ser escalada.

Resumo

Este capítulo abordou diversos assuntos. Vimos por que você pode querer adotar uma arquitetura de microsserviços, e como essa tomada de decisão pode causar impactos no modo de atribuir prioridades ao seu tempo. Consideramos as principais perguntas que as equipes devem fazer a si mesmas ao decidir se os microsserviços são uma opção correta para elas, e vale a pena repeti-las:

- O que vocês esperam alcançar?
- Vocês consideraram alternativas ao uso de microsserviços?
- Como vocês saberão que a transição está funcionando?

Além disso, nunca é demais enfatizar a importância de adotar uma abordagem gradual para a extração dos microsserviços. Erros são inevitáveis, portanto, se aceitar isso como um fato, você deve ter como meta cometer erros pequenos, em vez de grandes. Dividir uma transição para uma arquitetura de microsserviços em passos pequenos e incrementais garante que os erros que cometeremos serão pequenos, e que será mais fácil nos recuperarmos desses erros.

A maioria de nós também trabalha com sistemas que têm clientes no mundo real. Não podemos nos dar ao luxo de gastar meses ou anos em uma reescrita Big Bang de nossa aplicação, deixando a aplicação atual, utilizada pelos nossos clientes, estagnada. O objetivo deve ser a criação gradual de novos microsserviços e fazer com que sejam implantados como parte de sua solução para

produção, a fim de que você comece a aprender com essa experiência e desfrute os benefícios o mais rápido possível.

Sou muito enfático quanto à ideia de que, ao separar uma funcionalidade em um novo serviço, o trabalho não estará concluído até que essa funcionalidade esteja no ambiente de produção e em utilização. Você aprenderá muito no processo de fazer com que seus primeiros serviços sejam realmente *utilizados*. Esse deve ser o seu foco desde cedo.

Tudo isso significa que devemos desenvolver uma série de técnicas que nos permitam criar outros microsserviços e integrá-los ao nosso sistema monolítico cada vez menor (ou assim espero), tornando-os disponíveis no ambiente de produção. Veremos a seguir os padrões que mostram como podemos fazer esse trabalho, ao mesmo tempo que mantemos o sistema ativo, atendemos os clientes e incorporamos novas funcionalidades.

1 N.T.: Referência a uma cultura de valorização cega de culturas tecnologicamente mais avançadas. Veja outras informações sobre essa expressão em https://pt.wikipedia.org/wiki/Culto_%C3%A0_carga.

2 Que, como todos sabem, nem o Spotify usa atualmente.

3 Veja David Woods, “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering” (Quatro conceitos sobre resiliência e as implicações para o futuro da engenharia de resiliência). *Reliability Engineering & System Safety* 141 (2015) 5–9.

4 Veja o padrão de consumidor concorrente (competing consumer) para um exemplo desse tipo no livro *Enterprise Integration Patterns* de Gregor Hohpe e Bobby Woolf, página 502.

5 Veja Frederick P. Brooks, *The Mythical Man-Month*, edição do vigésimo aniversário (Boston: Addison-Wesley, 1995) (N.T.: Edição publicada no Brasil: *O Mítico Homem-Mês* [Alta Books, 2018]).

6 O termo “plataforma em chamadas” em geral é usado para se referir a uma tecnologia considerada no fim da vida. Talvez seja muito complicado ou muito caro dar suporte à tecnologia, ou muito difícil de contratar pessoas com uma experiência relevante. Um exemplo comum de uma tecnologia que a maioria das empresas considera como uma plataforma em chamadas é uma aplicação COBOL em mainframe.

7 N.T.: O termo “brownfield” refere-se a um sistema para o qual já existem partes anteriormente implementadas, em geral códigos legados, enquanto um sistema

“greenfield” implica um desenvolvimento totalmente do zero.

- 8 O modelo de mudança de Kotter está descrito em detalhes em seu livro *Leading Change* (Harvard Business Review Press, 1996) (N.T.: Edição publicada no Brasil: *Liderando mudanças* [Elsevier, 2013]).
- 9 Para mais detalhes sobre a história da iniciativa de mudança para implantação de testes no Google, Mike Bland tem um excelente artigo (<http://bit.ly/2omkxVy>) que vale muito a pena ler. Mike também escreveu uma história detalhada (<http://bit.ly/2ojpWwm>) sobre o Testing on the Toilet (Testando no banheiro).
- 10 Agradeço a Martin Fowler pela inspiração para esses nomes!
- 11 Manuel Pais e Matthew Skelton, Team Topologies (IT Revolution Press, 2019).
- 12 Gene Kim, Jez Humble e Patrick Debois, *The DevOps Handbook* (IT Revolution Press, 2016) (N.T.: Edição publicada no Brasil: *Manual de DevOps* [Alta Books, 2018]).
- 13 Sim, isso aconteceu. Nem tudo é diversão e jogos e Kubernetes...

CAPÍTULO 3

Dividindo o sistema monolítico

No Capítulo 2, exploramos o modo de pensar na migração para uma arquitetura de microsserviços. Mais especificamente, exploramos o fato de, inclusive, essa ser uma boa ideia, e, em caso afirmativo, como devemos proceder quanto à implantação da nova arquitetura e como garantir que estamos indo na direção correta.

Discutimos como um bom serviço deve ser e por que serviços menores podem ser melhores para nós. Porém, como lidar com o fato de talvez já termos várias aplicações que não sigam esses padrões? Como proceder com a decomposição dessas aplicações monolíticas sem embarcar em uma reescrita Big Bang?

Na parte restante deste capítulo, exploraremos uma série de padrões e dicas de migração que ajudarão você a adotar uma arquitetura de microsserviços. Veremos padrões que funcionarão para softwares caixa-preta de terceiros, sistemas legados ou sistemas monolíticos que você planeja continuar mantendo e que vão evoluir. Contudo, para que um rollout gradual funcione, é preciso garantir que possamos continuar trabalhando com o software monolítico atual e que seja possível usá-lo.



Lembre-se de que queremos que nossa migração seja gradual. Queremos migrar para uma arquitetura de microsserviços em passos pequenos para que possamos aprender com o processo e mudar de ideia se for necessário.

Mudar ou não mudar o sistema monolítico?

Um dos primeiros aspectos que você vai ter de considerar como parte de sua migração é se você planeja (ou é capaz de) mudar o

sistema monolítico atual.

Se puder mudar o sistema atual, você terá o máximo de flexibilidade possível no que concerne aos diversos padrões que estarão à sua disposição. Em algumas situações, porém, haverá uma restrição mais rígida, e você não terá essa oportunidade. O sistema atual pode ser um produto de terceiros, do qual você não tem o código-fonte, ou talvez esteja escrito com uma tecnologia para a qual você já não tem mais as habilidades necessárias.

Também pode haver motivos mais simples que poderão fazer você desistir de modificar o sistema atual. É possível que o sistema monolítico atual esteja em um estado muito precário, a ponto de o custo da mudança ser alto demais – como resultado, você preferirá evitar perdas e começar tudo de novo (embora, conforme já expliquei detalhadamente, eu me preocupe com o fato de as pessoas chegarem depressa demais a essa conclusão). Outra possibilidade é que haja várias outras pessoas trabalhando no sistema monolítico e sua preocupação é atrapalhá-las. Alguns padrões, como o padrão branch por abstração (branch by abstraction) que exploraremos em breve, podem atenuar esses problemas, mas você ainda poderá chegar à conclusão de que o impacto causado às outras pessoas será grande demais.

Em uma ocasião memorável, eu estava trabalhando com alguns colegas para ajudar a escalar um sistema que exigia bastante processamento. Os cálculos subjacentes eram feitos por uma biblioteca C que nos foi entregue. Nosso trabalho era coletar os diversos dados de entrada, passá-los para a biblioteca, receber os resultados e armazená-los. A biblioteca propriamente dita estava repleta de problemas. Vazamentos de memória (memory leaks) e um design de API extremamente ineficiente eram apenas duas das principais causas desses problemas. Pedimos o código-fonte da biblioteca durante vários meses para que pudéssemos corrigi-lo, porém não fomos atendidos.

Vários anos depois, eu me encontrei com o sponsor (patrocinador)

do projeto e perguntei por que eles não haviam permitido que modificássemos a biblioteca subjacente. Foi naquele momento que o sponsor finalmente admitiu que haviam perdido o código-fonte, mas se sentiam envergonhados demais para nos contar! Não deixe que isso aconteça com você.

Assim, a expectativa é que estejamos em uma posição na qual possamos trabalhar com a base de código do sistema monolítico atual e modificá-la. No entanto, se isso não for possível, significa que estamos em um beco sem saída? Pelo contrário – há vários padrões que poderão ajudar nesse caso. Discutiremos alguns deles em breve.

Cortar, copiar ou reimplementar?

Mesmo que você tenha acesso ao código do sistema monolítico atual, ao iniciar a migração das funcionalidades para seus novos microsserviços, o que deverá ser feito com esse código nem sempre estará bem claro. Devemos mover o código do modo como está ou devemos reimplementar a funcionalidade?

Se a base de código do sistema monolítico atual estiver suficientemente bem fatorada, talvez você possa economizar um bom tempo se apenas mover o código. O ponto principal, nesse caso, é entender que queremos *copiar* o código do sistema monolítico e, ao menos nessa etapa, não queremos remover essa funcionalidade desse sistema. Por quê? Porque deixar a funcionalidade no sistema monolítico por um tempo nos dará mais opções. Poderemos ter um ponto de rollback ou, quem sabe, uma oportunidade para executar as duas implementações em paralelo. Posteriormente, assim que você estiver satisfeito com o sucesso da migração, será possível remover a funcionalidade do sistema monolítico.

Refatorando o sistema monolítico

Tenho observado que, muitas vezes, a maior barreira para fazer uso

do código existente no sistema monolítico em seus microsserviços é o fato de as bases de código não estarem tradicionalmente organizadas em torno dos conceitos de domínios de negócio. Classificações técnicas são mais predominantes (pense em todos os nomes de pacotes Modelo, Visão, Controlador que você já viu, por exemplo). Pode ser difícil tentar mover funcionalidades do domínio de negócio: a base de código existente não condiz com essa classificação e, desse modo, até mesmo encontrar o código que você está tentando mover pode ser problemático!

Se você optar pelo caminho de reorganizar seu sistema monolítico atual com base nas fronteiras dos domínios de negócio, recomendo bastante o livro *Working Effectively with Legacy Code* de Michael Feathers (Prentice Hall, 2004)¹. Nesse livro, Michael define o conceito de *seam* (ponto de extensão) – isto é, um local em que você pode mudar o comportamento de um programa sem ter de alterar o comportamento que já existe. Basicamente, definimos um ponto de extensão em torno de uma porção de código que queremos mudar, trabalhamos em uma nova implementação desse ponto de extensão e fazemos a troca depois que a mudança estiver pronta. Ele apresenta técnicas para trabalhar de forma segura com pontos de extensão, como um modo de ajudar na limpeza das bases de código.

Embora, de modo geral, o conceito de pontos de extensão de Michael possa ser aplicado em vários escopos, ele é bastante apropriado para os contextos delimitados (*bounded contexts*), que discutimos no Capítulo 1. Desse modo, ainda que *Working Effectively with Legacy Code* possa não se referir diretamente aos conceitos de design orientado a domínio, você poderá usar as técnicas desse livro para organizar o seu código com base nesses princípios.

Um sistema monolítico modular?

Assim que você começar a entender a sua base de código atual, um próximo passo óbvio digno de consideração é usar os pontos de

extensão que você acabou de identificar e começar a extraí-los como módulos separados, transformando seu sistema monolítico em um *sistema monolítico modular*. Você ainda terá uma única unidade de implantação, mas essa unidade implantada será constituída de vários módulos ligados estaticamente. A natureza exata desses módulos dependerá da pilha de tecnologias subjacente – em Java, meu sistema monolítico modular seria composto de vários arquivos JAR; em uma aplicação Ruby, poderia ser uma coleção de gemas (gems) Ruby.

Conforme mencionamos rapidamente no início do livro, ter um sistema monolítico separado em módulos que possam ser desenvolvidos de forma independente pode proporcionar diversas vantagens, ao mesmo tempo que evita vários problemas enfrentados por uma arquitetura de microsserviços, e pode ser uma boa opção para muitas empresas. Já conversei com mais de uma equipe que começou dividindo seu sistema monolítico em um sistema monolítico modular, com vistas a, em algum momento, passar para uma arquitetura de microsserviços, somente para descobrir que o sistema monolítico modular resolvia a maior parte de seus problemas!

Reescritas graduais

Minha tendência geral é sempre tentar aproveitar a base de código existente antes de recorrer a uma reimplementação das funcionalidades, e os conselhos que dei em meu livro anterior, *Building Microservices*, seguiam essa linha. Às vezes, as equipes percebem que têm benefícios suficientes com esse trabalho, a ponto de nem sequer precisarem dos microsserviços!

No entanto, devo reconhecer que, na prática, conheço poucas equipes que adotam a abordagem de refatoração de seu sistema monolítico como precursor para migrar para os microsserviços. O que me parece mais comum é que, assim que as equipes identificam as responsabilidades do microsserviço recém-criado, elas fazem uma nova implementação dessa funcionalidade.

Contudo, não corremos o risco de repetir os problemas associados às reescritas Big Bang se começarmos a reimplementar nossas funcionalidades? O segredo é garantir que você reescreverá somente pequenas funcionalidades de cada vez, e que lançará essas funcionalidades reimplementadas aos seus clientes de forma regular. Se o trabalho de reimplementar o comportamento do serviço exigir alguns dias ou semanas, provavelmente não haverá problemas. Se os prazos passarem a ser de vários meses, eu preferiria rever a abordagem.

Padrões de migração

Já vi muitas técnicas serem usadas como parte de uma migração para microsserviços. Na parte restante deste capítulo, exploraremos esses padrões, veremos quando poderão ser úteis e como podem ser implementados. Lembre-se de que, como ocorre com qualquer padrão, eles não são ideias universalmente “boas”. Para cada padrão, procurei dar informações suficientes para ajudar você a saber se eles fazem sentido em seu contexto.



Certifique-se de que entenderá os prós e contras de cada um desses padrões. Eles não são o modo universalmente “correto” de proceder.

Daremos início analisando as técnicas que permitem fazer a migração e a integração com o seu sistema monolítico; na maior parte, trabalharemos no local em que está o código da aplicação. Para começar, porém, veremos uma das técnicas mais úteis e comuns que existem: a aplicação strangler fig.

Padrão: aplicação strangler fig

Uma técnica que é vista com frequência na reescrita de sistemas se chama aplicação strangler fig (<http://bit.ly/2p5xMKo>). Martin Fowler inicialmente identificou esse padrão, inspirado em certo tipo de figueira que germina nos galhos superiores das árvores. A figueira então desce até o solo para criar raízes, envolvendo aos poucos a

árvore original. Em um primeiro momento, a árvore existente se transforma em uma estrutura de apoio para a nova figueira e, se ela chegar aos estágios finais, você poderá ver a árvore original morrer e apodrecer, restando somente a nova figueira, agora capaz de se sustentar sozinha.

No contexto de software, o paralelo, nesse caso, é fazer com que nosso novo sistema tenha inicialmente o suporte do sistema existente e o encapsule. A ideia é que o velho e o novo possam coexistir, dando tempo ao novo sistema para que cresça e, possivelmente, substitua totalmente o sistema antigo. A principal vantagem desse padrão, conforme veremos em breve, é que ele está alinhado com o nosso objetivo de permitir uma migração gradual para um novo sistema. Além disso, o padrão nos permite fazer uma pausa, e até mesmo interromper totalmente a migração, enquanto podemos continuar usufruindo as vantagens do novo sistema disponibilizado até então.

Como veremos em breve, quando implementamos essa ideia em nosso software, não só nos esforçamos para dar passos graduais em direção à nova arquitetura de aplicação, mas também garantimos que cada passo seja facilmente reversível, reduzindo seus riscos.

Como o padrão funciona

Embora o padrão strangler fig seja de uso comum na migração de um sistema monolítico para outro, nosso objetivo é fazer a migração de um sistema monolítico para uma série de microsserviços. Isso pode envolver uma cópia do código do sistema monolítico (se for possível) ou a reimplementação da funcionalidade em questão. Além disso, se essa funcionalidade exigir persistência de estados, será necessário considerar como esses estados poderão ser migrados para o novo serviço, e como poderão, eventualmente, ser trazidos de volta. Exploraremos os aspectos relacionados aos dados no Capítulo 4.

Implementar um padrão strangler fig depende de três passos, que estão representados na Figura 3.1. Em primeiro lugar, identifique as partes do sistema atual cuja migração você deseja fazer. Você terá de usar de discernimento para saber quais partes do sistema deverá atacar primeiro, usando o tipo de atividade de análise de custo-benefício que discutimos no Capítulo 2. Em seguida, será necessário implementar essa funcionalidade em seu novo microsserviço. Com sua nova implementação pronta, você terá de ser capaz de desviar as chamadas do sistema monolítico para o seu novo microsserviço.

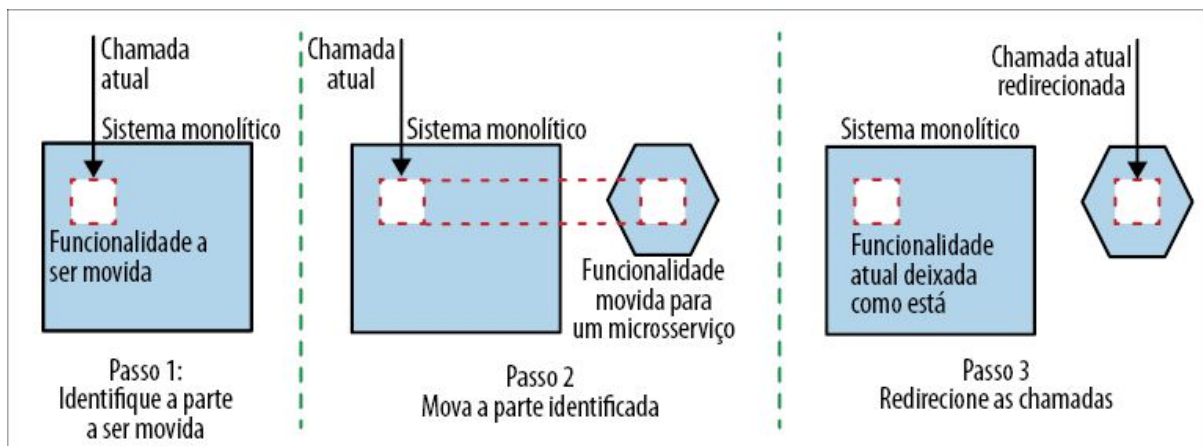


Figura 3.1 – Visão geral do padrão strangler fig.

Vale a pena observar que, até que a chamada para a funcionalidade movida seja redirecionada, a nova funcionalidade não estará tecnicamente ativa – mesmo que seja implantada em um ambiente de produção. Isso significa que você pode dispor de tempo até que a funcionalidade esteja correta, podendo trabalhar em sua implementação durante um tempo. Essas alterações poderiam ser enviadas para um ambiente de produção, e você estaria seguro sabendo que ela ainda não será usada, até estar satisfeito com os aspectos relacionados à implantação e ao gerenciamento do novo serviço. Assim que seu novo serviço implementar a funcionalidade equivalente àquela do sistema monolítico, você poderá considerar o uso de um padrão como o de execução paralela (será explorado em breve) para que tenha certeza de que a nova funcionalidade está

operando conforme desejado.



Separar os conceitos de *implantação* (deployment) e de *lançamento* (release) é importante. Só porque um software é implantado em um dado ambiente não significa que ele será realmente usado pelos clientes. Ao tratar os dois conceitos como distintos, você poderá validar seu software no ambiente de produção definitivo antes que ele seja usado, permitindo reduzir os riscos do rollout do novo software. Padrões como strangler fig, execução em paralelo e versão canário estão entre aqueles que fazem uso do fato de que *implantação* e *lançamento* são atividades distintas.

Um ponto importante dessa abordagem strangler fig não é somente o fato de podermos migrar gradualmente a nova funcionalidade para o novo sistema, mas também possibilitar o rollback dessa mudança com muita facilidade caso seja necessário. Lembre-se de que todos nós cometemos erros – portanto, queremos ter técnicas que nos permitam não só cometer erros com o menor custo possível (por isso, damos vários passos pequenos), mas que também permitam corrigir nossos erros rapidamente.

Se a funcionalidade extraída também for usada por outra funcionalidade no sistema monolítico, será preciso mudar essas chamadas também. Descreveremos algumas técnicas para resolver esse problema, mais adiante neste capítulo.

Quando usar o padrão

O padrão strangler fig permite mover uma funcionalidade para uma nova arquitetura de serviços sem a necessidade de fazer qualquer alteração em seu sistema atual. Isso traz vantagens se houver outras pessoas trabalhando no sistema monolítico atual, pois pode ajudar a reduzir os conflitos. Também será muito conveniente quando o sistema monolítico for um sistema caixa-preta – por exemplo, um software de terceiros ou um serviço SaaS.

Ocasionalmente, você poderá extrair uma porção completa, fim a fim, da funcionalidade, como vemos na Figura 3.2. Isso simplificará bastante a extração, se não considerarmos os problemas relacionados aos dados, que serão abordados mais adiante no livro.

Para fazer uma extração limpa e completa como essa, você poderia estar inclinado a extrair grupos maiores de funcionalidades a fim de simplificar o processo. A decisão pode implicar uma ação de balanceamento complicada – ao extrair fatias maiores de funcionalidades, você se comprometerá com mais trabalho, mas poderá reduzir alguns desafios de integração.

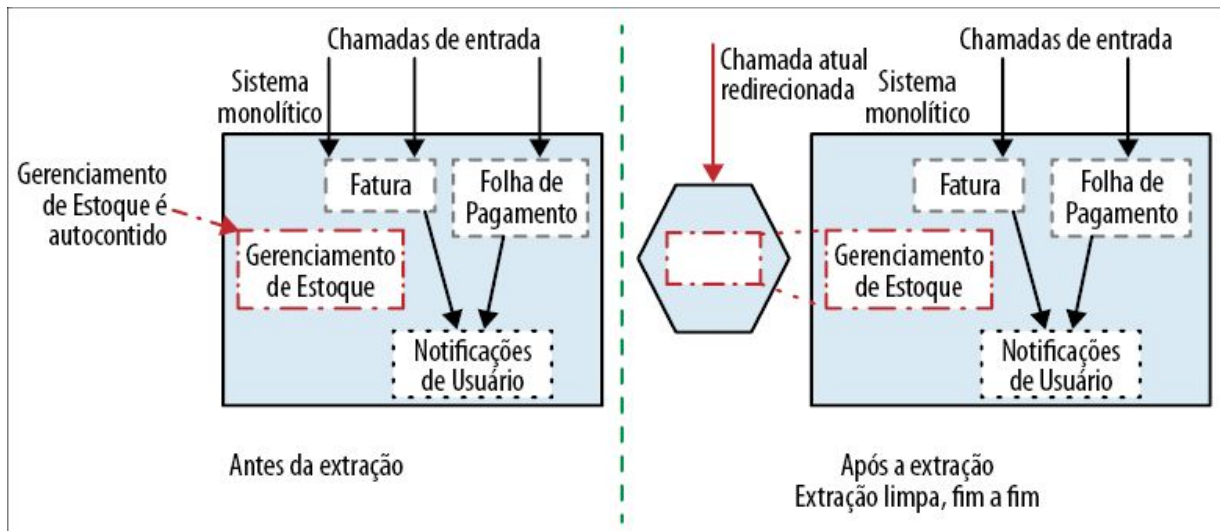


Figura 3.2 – Abstração simples, fim a fim, da funcionalidade de Gerenciamento de Estoque.

Se quiser trabalhar com uma fatia menor, terá de considerar extrações mais “rasas”, como aquelas que vemos na Figura 3.3. Nesse caso, estamos extraindo a funcionalidade de Folha de Pagamento, apesar do fato de ela fazer uso de outra funcionalidade que permanecerá no sistema monolítico – nesse exemplo, é a capacidade de enviar Notificações de Usuário.

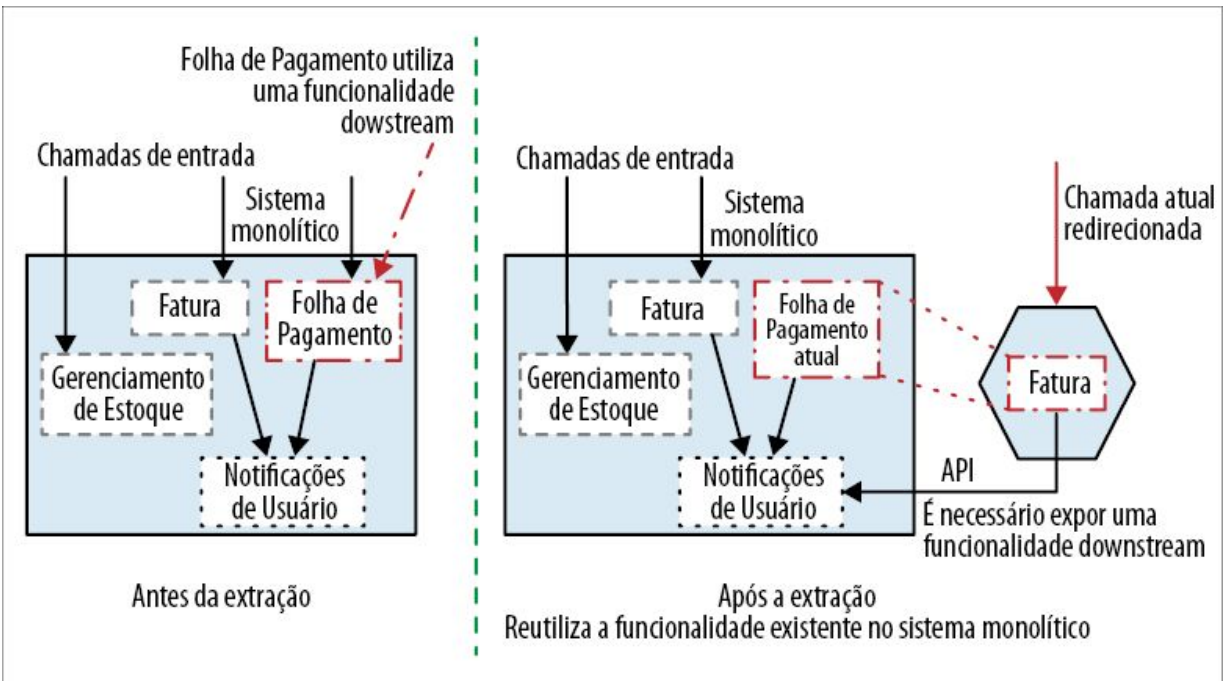


Figura 3.3 – Extraíndo uma funcionalidade que ainda terá de usar o sistema monolítico.

Em vez de reimplementar também a funcionalidade de Notificações de Usuário, expomos essa funcionalidade para o nosso novo microsserviço, de dentro do sistema monolítico – algo que, obviamente, exigiria mudanças no próprio sistema monolítico.

Para que o padrão strangler fig funcione, porém, você deverá ser capaz de mapear claramente a chamada de entrada da funcionalidade de seu interesse para o componente que você deseja mover. Por exemplo, na Figura 3.4, o ideal é que passássemos a capacidade de enviar Notificações de Usuário aos nossos clientes para um novo serviço. No entanto, as notificações são disparadas como resultado de várias chamadas de entrada para o sistema monolítico atual. Assim, não podemos redirecionar claramente as chamadas de fora do próprio sistema. Teríamos então de recorrer a uma técnica como aquela descrita na seção Padrão: branch por abstração.

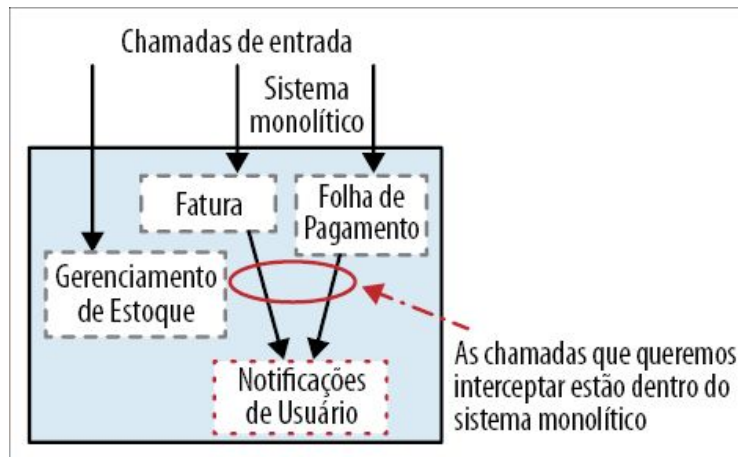


Figura 3.4 – O padrão strangler fig não funciona muito bem se a funcionalidade a ser movida estiver embaixo de muitas camadas no sistema atual.

Você também terá de considerar a natureza das chamadas feitas ao sistema atual. Conforme exploraremos em breve, um protocolo como o HTTP é bastante apropriado para um redirecionamento. O próprio HTTP tem o conceito de redirecionamentos transparentes, e proxies poderão ser usados para compreender claramente a natureza de uma requisição de entrada e fazer o seu desvio de forma adequada. Outros tipos de protocolos, como alguns RPCs, talvez sejam menos propícios a um redirecionamento. Quanto mais trabalho você tiver de fazer na camada de proxy para entender e, possivelmente, transformar a chamada de entrada, menos viável será essa opção.

Apesar dessas restrições, a aplicação strangler fig tem frequentemente provado que é uma técnica de migração muito conveniente. Considerando que é uma abordagem leve e simples para lidar com mudanças graduais, em geral é minha primeira opção ao explorar o procedimento de migração para um sistema.

Exemplo: proxy HTTP reverso

O HTTP tem alguns recursos interessantes, entre eles a extrema facilidade de fazer interceptações e redirecionamentos de modo transparente ao sistema que faz a chamada. Isso significa que um

sistema monolítico com uma interface HTTP será propício à migração com o uso de um padrão strangler fig.

Na Figura 3.5, vemos um sistema monolítico existente expondo uma interface HTTP. Essa aplicação pode ser headless, ou a interface HTTP pode estar realmente sendo chamada por uma UI upstream. De qualquer modo, o objetivo é o mesmo: inserir um proxy HTTP reverso entre as chamadas upstream e o sistema monolítico downstream.

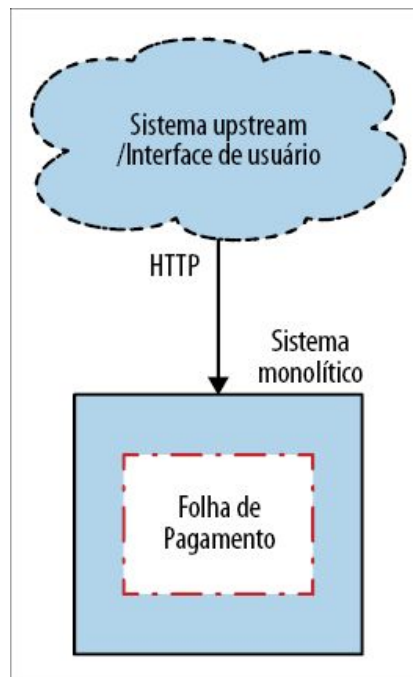


Figura 3.5 – Uma visão geral simples de um sistema monolítico HTTP, antes de um strangler fig ser implementado.

Passo 1: Inserir o proxy

A menos que você já tenha um proxy HTTP apropriado que possa ser reutilizado, sugiro que instale um *antes*, como vemos na Figura 3.6. Nesse primeiro passo, o proxy simplesmente deixará qualquer chamada passar, sem fazer alterações.

Esse passo lhe permitirá avaliar o impacto de inserir um hop (salto) adicional de rede entre as chamadas upstream e o sistema monolítico downstream; configure qualquer monitoração necessária

para o seu novo componente e, basicamente, observe por um tempo. Do ponto de vista da latência, estamos adicionando um hop de rede e um processo no caminho do atendimento de todas as chamadas. Com um proxy e uma rede razoáveis, você poderia esperar um mínimo de impacto na latência (talvez da ordem de alguns milissegundos), mas, se não for o caso, você terá a chance de parar e investigar o problema antes de prosseguir.

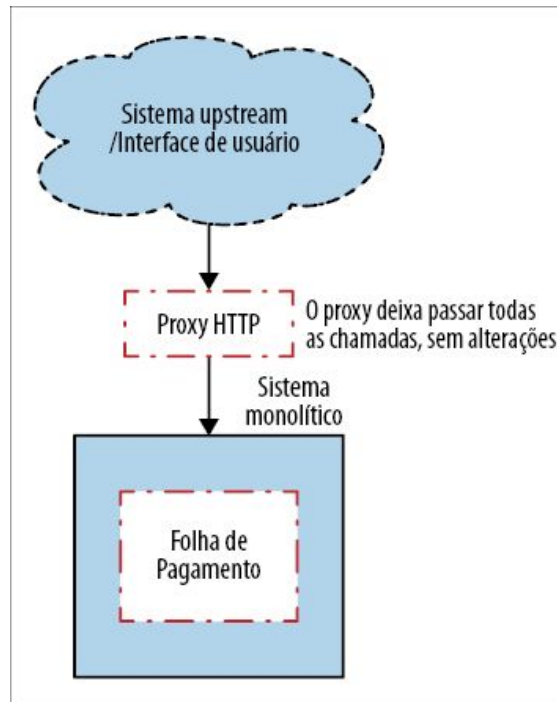


Figura 3.6 – Passo 1: Inserindo um proxy entre o sistema monolítico e o sistema upstream.

Se já houver um proxy instalado na frente de seu sistema monolítico, esse passo poderá ser ignorado – porém, certifique-se de que sabe como esse proxy pode ser reconfigurado para redirecionar posteriormente as chamadas. Sugiro que, no mínimo, você faça experimentos com o redirecionamento a fim de garantir que funcionará conforme desejado, antes de supor que isso poderá ser feito mais tarde. Seria uma surpresa ingrata descobrir que não é possível fazê-lo, pouco antes de ativar o seu novo serviço!

Passo 2: Fazer a migração da funcionalidade

Com nosso proxy instalado, você poderá começar a extrair, em seguida, o seu novo microserviço, como mostra a Figura 3.7.

Este passo pode ser dividido em várias etapas. Em primeiro lugar, crie um serviço básico sem que nenhuma parte da funcionalidade esteja implementada. Seu serviço deverá aceitar as chamadas feitas à funcionalidade correspondente, mas, nessa fase, você poderia simplesmente devolver um 501 Not Implemented (Não implementado). Mesmo nesse passo, eu faria com que o serviço fosse implantado no ambiente de produção. Isso lhe permitirá se sentir mais à vontade com o processo de implantação em produção e testar seu serviço *in situ*. Nesse ponto, seu novo serviço não estará *lançado*, pois você ainda não terá redirecionado as chamadas upstream atuais. Com efeito, estamos separando o passo de implantação do passo de lançamento do software: uma técnica comum de lançamento que retomaremos mais tarde.

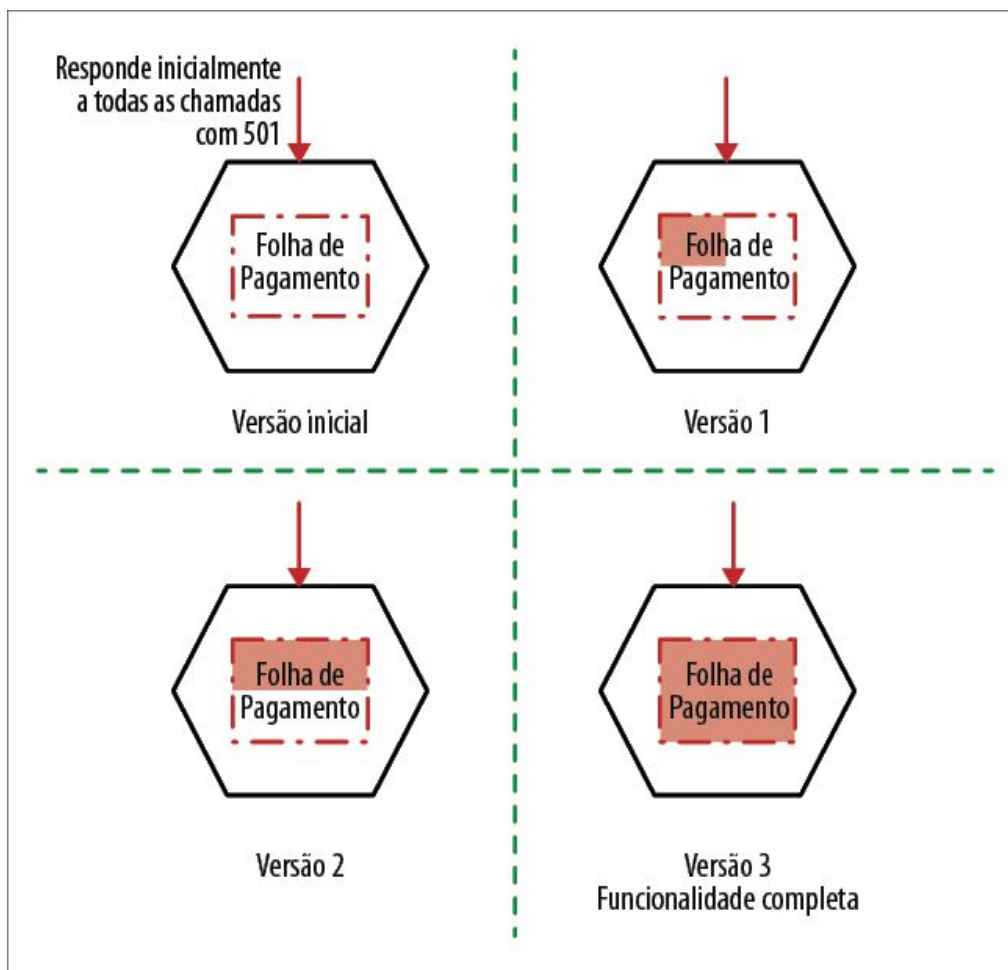


Figura 3.7 – Passo 2: Implementação gradual da funcionalidade a ser movida.

Passo 3: Redirecionar as chamadas

Somente depois de ter acabado de mover toda a funcionalidade, é que você vai reconfigurar o proxy a fim de redirecionar a chamada, conforme mostra a Figura 3.8. Se isso falhar por qualquer que seja o motivo, você poderá desfazer o redirecionamento – na maioria dos proxies, é um processo rápido e fácil, possibilitando um rollback ágil.

Talvez você decida implementar o redirecionamento usando algo como um toggle de recurso, o qual poderá deixar o estado desejado de sua configuração muito mais evidente. O uso de um proxy para redirecionar as chamadas também é uma ótima oportunidade para considerar um rollout gradual da nova funcionalidade com um rollout

canário, ou até mesmo com uma execução em paralelo completa – outro padrão que será discutido neste capítulo.

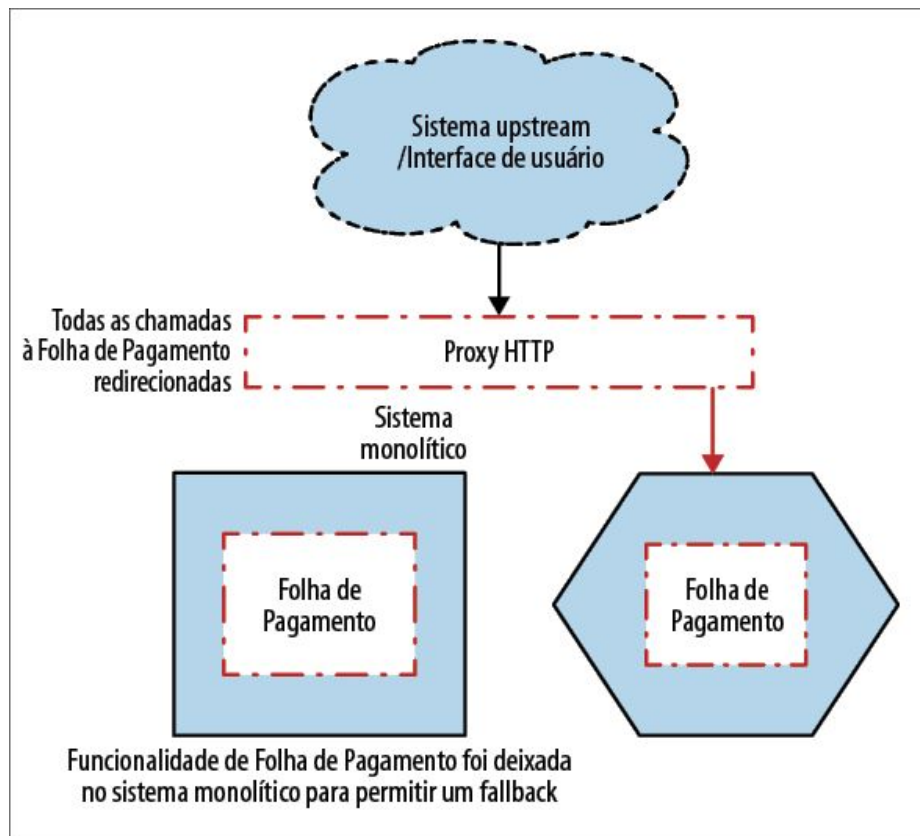


Figura 3.8 – Passo 3: Redirecionando a chamada para a funcionalidade de Folha de Pagamento, concluindo a migração.

Dados?

Até agora, não falamos sobre os dados. Na Figura 3.8, o que acontecerá se o nosso serviço de Folha de Pagamento cuja migração acabamos de fazer precisasse acessar dados que estão atualmente no banco de dados do sistema monolítico? Exploraremos as opções para resolver completamente esse problema no Capítulo 4.

Opções de proxy

O modo como você implementará o proxy dependerá parcialmente do protocolo usado pelo sistema monolítico. Se o sistema monolítico

atual utiliza HTTP, teremos um bom ponto de partida. O HTTP é um protocolo amplamente aceito, de modo que você terá uma infinidade de opções para gerenciar o redirecionamento. Provavelmente, eu optaria por um proxy dedicado como o NGINX, que foi criado exatamente com esses tipos de casos de uso em mente e aceita diversos métodos de redirecionamento testados e comprovados, com boas chances de ter um desempenho muito bom.

Alguns redirecionamentos serão mais simples que outros. Considere os redirecionamentos com base em paths URI, talvez conforme os veríamos com o uso de recursos REST. Na Figura 3.9, passamos o recurso Folha de Pagamento completo para um novo serviço, e é fácil fazer um parse a partir do path URI.

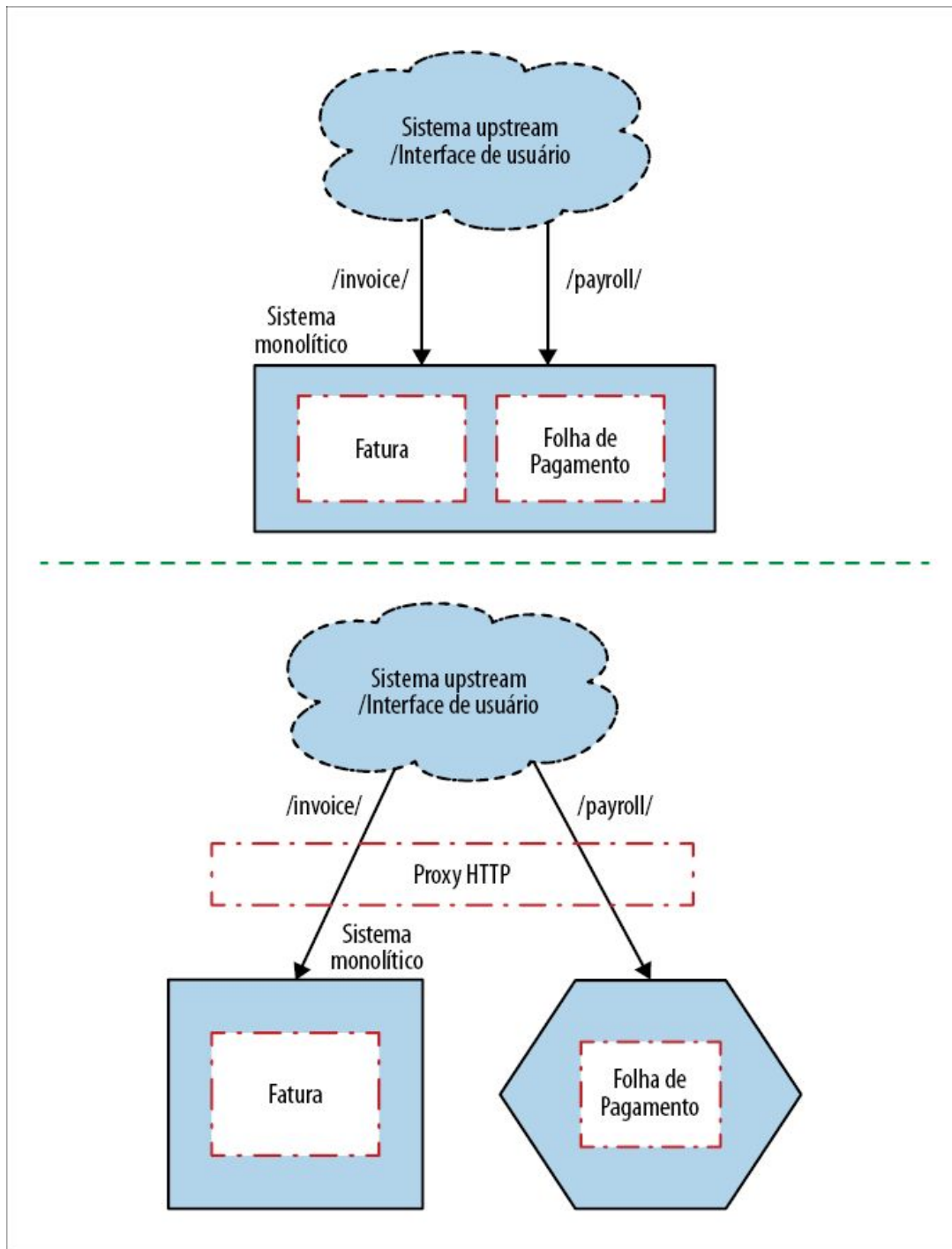


Figura 3.9 – Redirecionamento por recursos.

Contudo, se as informações sobre a natureza da funcionalidade chamada estiverem mais encobertas em algum lugar no corpo da requisição (talvez na forma de um parâmetro) no sistema atual, nossa regra de redirecionamento deverá ser capaz de fazer a

mudança com base em um parâmetro do POST – algo possível, porém mais complicado. Sem dúvida, vale a pena conferir as opções disponíveis no proxy a fim de garantir que ele seja capaz de lidar com esse problema caso você se veja em uma situação como essa.

Se a natureza da interceptação e do redirecionamento for mais complexa, ou em situações em que o sistema monolítico utilize um protocolo com menos suporte, você poderia se sentir tentado a escrever um código por conta própria; no entanto, seja muito cauteloso com essa abordagem. Já escrevi dois proxies de rede manualmente (um em Java e outro em Python) e, embora isso possa dizer muito mais sobre minhas habilidades em escrever código do que qualquer outra coisa, nas duas situações, os proxies eram extremamente ineficientes, acrescentando um atraso significativo no sistema. Atualmente, se eu precisasse de um comportamento personalizado, é provável que eu preferisse considerar o acréscimo desse comportamento em um proxy dedicado – por exemplo, o NGINX permite utilizar um código escrito em Lua para adicionar um comportamento personalizado.

Rollout gradual

Como podemos ver na Figura 3.10, essa técnica permite que mudanças de arquitetura sejam feitas com uma série de passos pequenos, e cada passo pode ser dado lado a lado com outras tarefas sendo realizadas no sistema.

Você poderia considerar que uma mudança para uma nova implementação da funcionalidade de Folha de Pagamento seria grande demais, caso em que poderá considerar porções menores da funcionalidade. Seria possível considerar, por exemplo, a migração de apenas parte da funcionalidade de Folha de Pagamento e fazer devidamente o desvio das chamadas – ou seja, ter parte do comportamento implementado no sistema monolítico e outra parte no microserviço, como mostra a Figura 3.11. Essa solução pode causar problemas caso a funcionalidade no sistema

monolítico e o microsserviço precisem visualizar o mesmo conjunto de dados, pois, provavelmente, exigirá um banco de dados compartilhado, com todos os problemas que isso possa causar.

Não há necessidade de nenhuma redefinição de plataforma Big Bang radical. Essa é uma forma muito mais fácil de dividir o trabalho em etapas que possam ser entregues lado a lado com outros trabalhos. Em vez de dividir suas pendências em “funcionalidades” e “pendências técnicas”, junte todas essas atividades. Torne-se bom em fazer mudanças graduais em sua arquitetura, ao mesmo tempo que entrega novas funcionalidades!

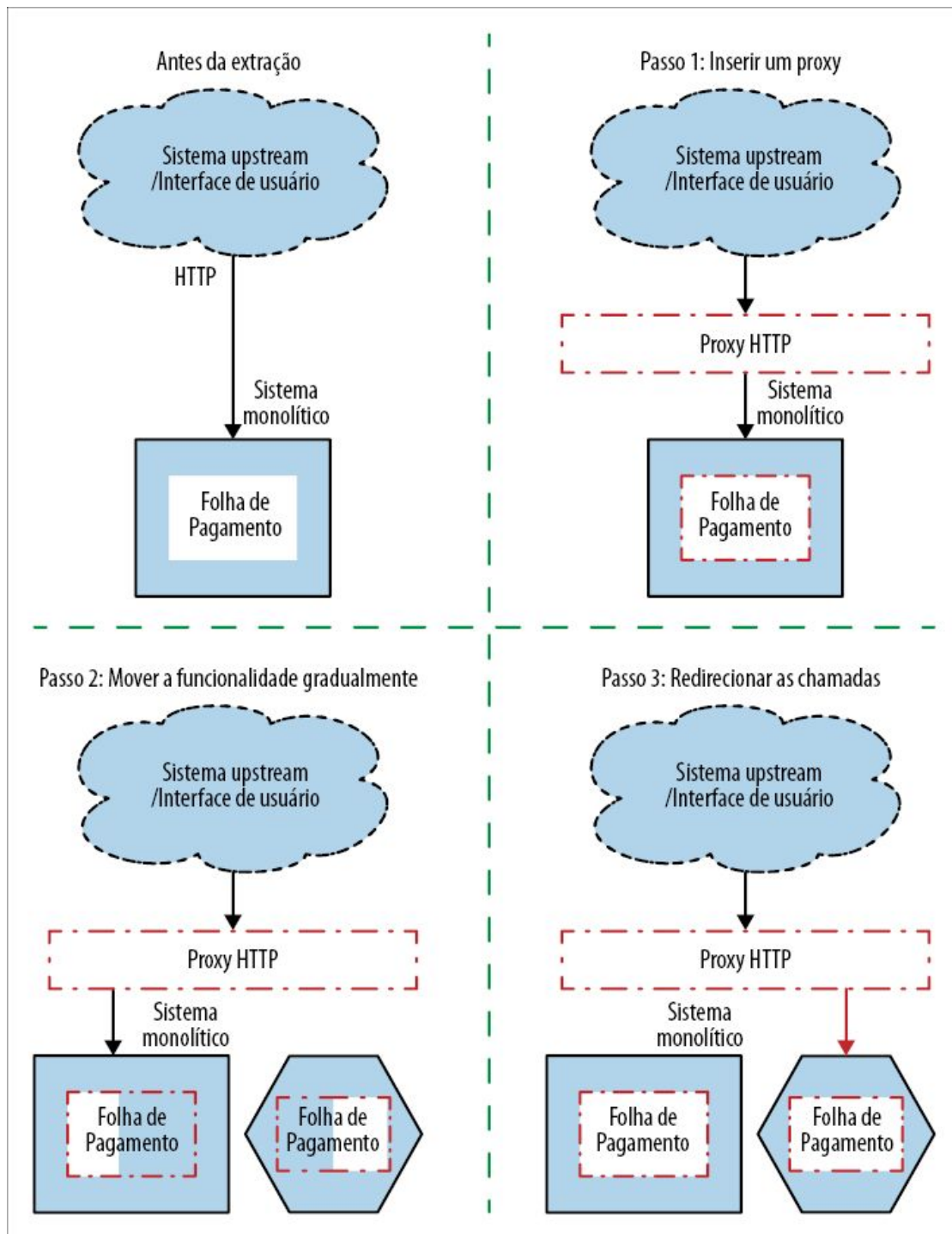


Figura 3.10 – Uma visão geral da implementação de um strangler fig baseado em HTTP.

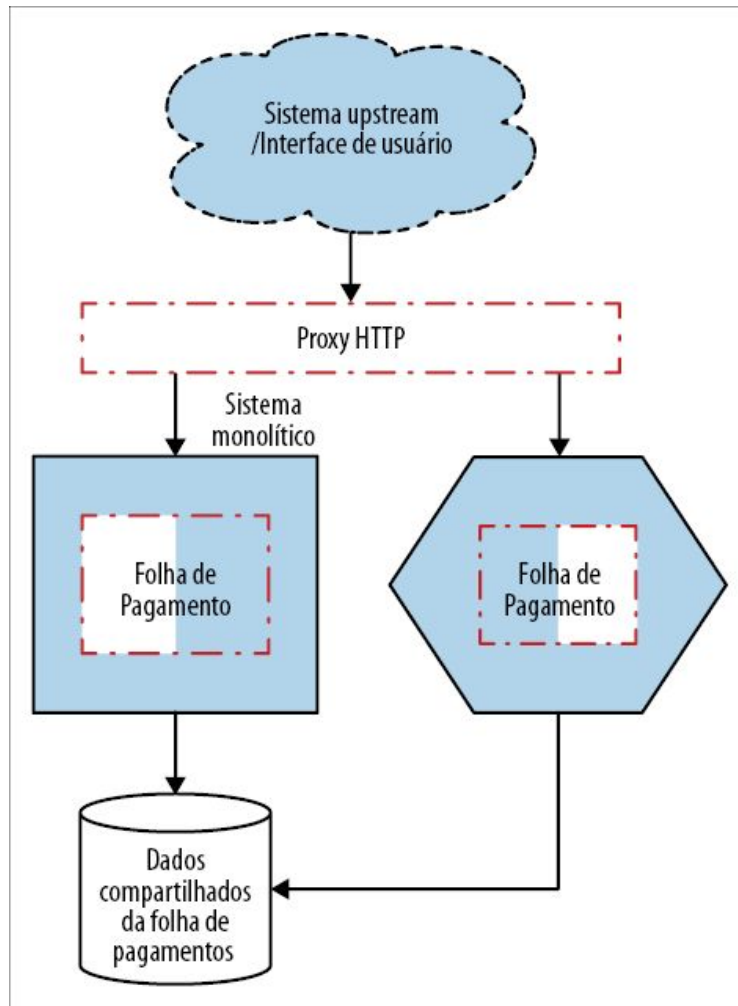


Figura 3.11 – Parte do comportamento implementado no sistema monolítico e outra parte no microsserviço.

Mudança de protocolos

O proxy também pode ser usado para uma mudança de protocolo. Por exemplo, você poderia estar atualmente expondo uma interface HTTP baseada em SOAP, mas seu novo microsserviço aceitará uma interface gRPC. Você poderia então configurar o proxy para transformar as requisições e as respostas de maneira apropriada, como mostra a Figura 3.12.

Tenho minhas preocupações quanto a essa abordagem, principalmente por causa da complexidade e da lógica que começam a se acumular no próprio proxy. Para um único serviço,

não parece ser tão ruim, mas, se você começar a transformar o protocolo para vários serviços, o trabalho feito pelo proxy vai se acumular. Em geral, fazemos otimizações para podermos implantar nossos serviços de modo independente; contudo, se tivermos uma camada de proxy compartilhada, que várias equipes precisem alterar, o processo de fazer e de implantar as mudanças poderá se tornar mais lento. Devemos ter cuidado para não adicionarmos uma nova fonte de conflitos. Há um mantra que costuma ser repetido quando discutimos a arquitetura de microsserviços: “Keep the pipes dumb, the endpoints smart”.² Queremos reduzir o volume de funcionalidades que são passadas para as camadas de middleware compartilhadas, pois isso pode realmente deixar mais lento o desenvolvimento das funcionalidades.

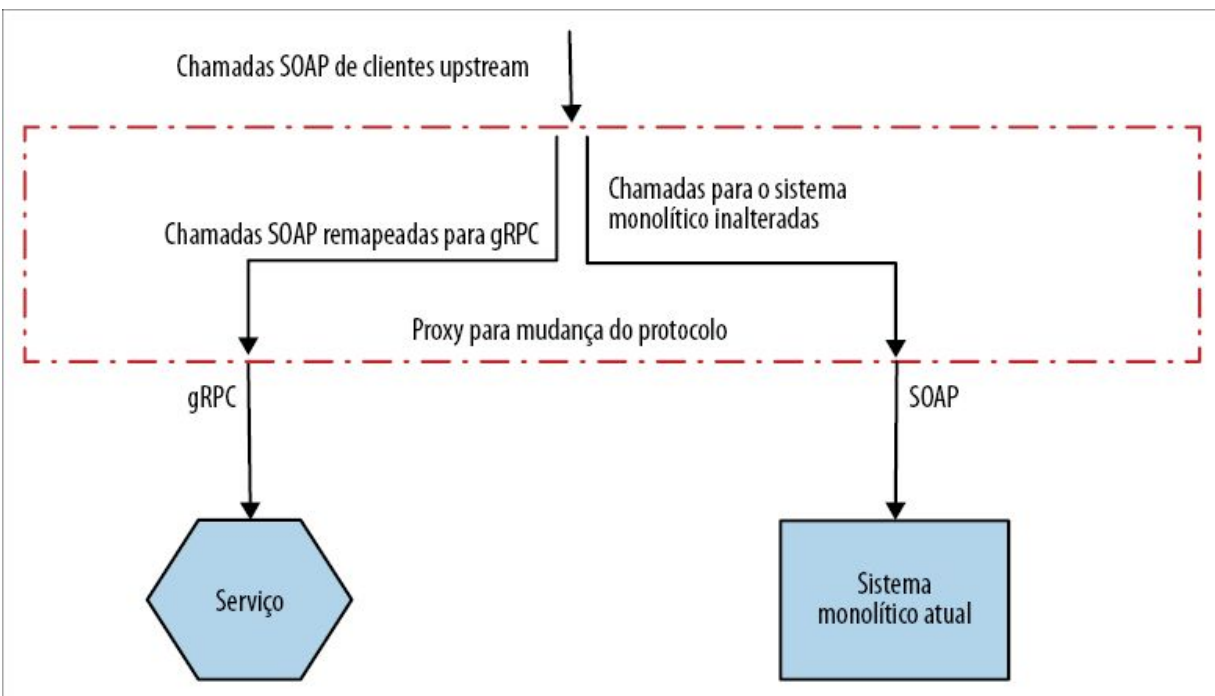


Figura 3.12 – Usando um proxy para mudar o protocolo de comunicação como parte de uma migração strangler fig.

Se quiser mudar o protocolo sendo usado, eu preferiria transferir o mapeamento para dentro do próprio serviço – com o serviço aceitando os dois protocolos de comunicação: o antigo e o novo. No interior do serviço, as chamadas para o protocolo antigo poderiam

ser simplesmente remapeadas para o novo protocolo de comunicação, como vemos na Figura 3.13. Isso evitará a necessidade de gerenciar mudanças nas camadas de proxy usadas por outros serviços e deixará o serviço com total controle sobre as mudanças que poderão ocorrer nessa funcionalidade com o tempo. Você pode encarar os microsserviços como um conjunto de funcionalidades em um endpoint de rede. Essa mesma funcionalidade poderia ser exposta de diversas maneiras para clientes distintos; ao aceitar formatos diferentes de mensagens ou de requisições para esse serviço, estaremos basicamente oferecendo suporte às diferentes necessidades de nossos clientes upstream.

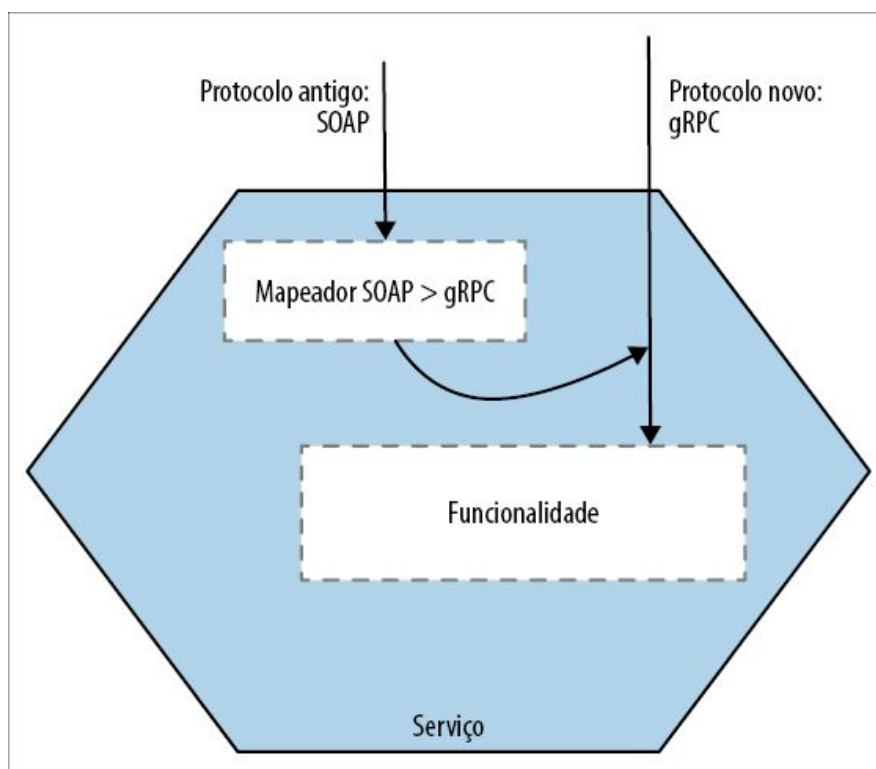


Figura 3.13 – Se quiser mudar o tipo do protocolo, considere fazer com que um serviço exponha seus recursos por meio de vários tipos de protocolo.

Ao passar o mapeamento de requisições e respostas específicas do serviço para dentro dele, a camada de proxy será mais simples e muito mais genérica. Além do mais, se o serviço disponibilizar os

dois tipos de endpoints, você ganhará tempo para que os clientes upstream façam a migração antes de eventualmente aposentar a API antiga.

Service meshes

Na Square, uma abordagem híbrida foi adotada para solucionar esse problema.³ A empresa havia decidido migrar seu mecanismo de RPC proprietário de comunicação entre serviços para o gRPC, um framework RPC de código aberto com bom suporte, com um ecossistema bem amplo. Para simplificar essa tarefa ao máximo, eles queriam reduzir a quantidade de mudanças necessárias em cada serviço. Para isso, um service mesh foi usado.

Com um *service mesh*, exibido na Figura 3.14, cada instância de serviço se comunica com outras instâncias de serviço por meio de seu próprio proxy local e dedicado. Cada instância de proxy pode ser especificamente configurada para a instância do serviço com a qual está associada. Também é possível ter um controle e monitoração centralizados para esses proxies usando um painel de controle. Como não há nenhuma camada de proxy centralizada, você evitará as armadilhas relacionadas a um “smart” pipe⁴ compartilhado – com efeito, cada serviço pode ter o próprio meio de comunicação entre serviços, se for necessário. Vale a pena observar que, por causa do modo como a arquitetura da Square havia evoluído, a empresa acabou tendo de criar seu próprio service mesh, específico às suas necessidades, embora fizessem uso do proxy de código aberto Envoy, em vez de utilizar uma solução consagrada como o Linkerd ou o Istio.

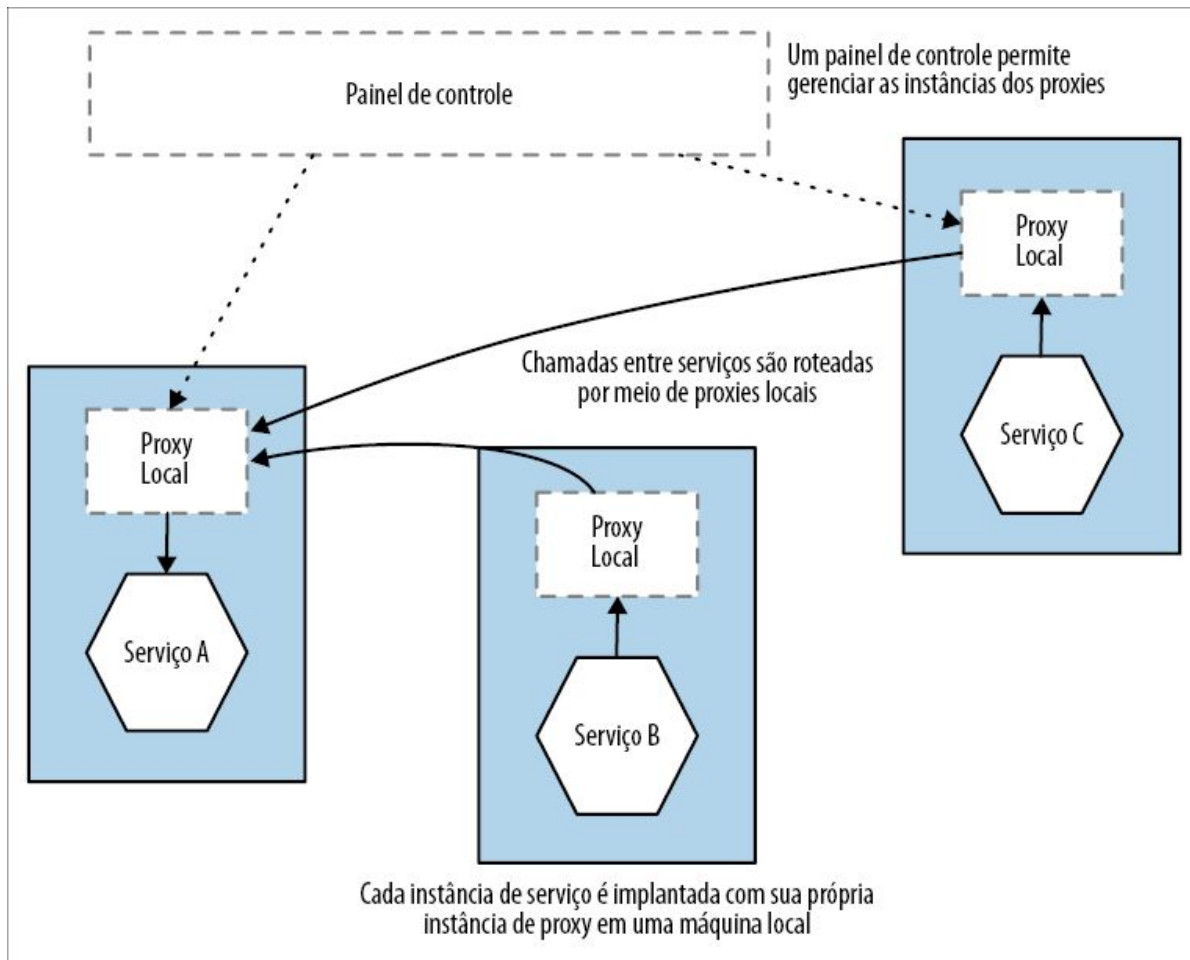


Figura 3.14 – Visão geral de um service mesh.

Os service meshes estão se tornando cada vez mais populares e, conceitualmente, acho a ideia muito boa. Podem ser uma ótima maneira de lidar com problemas comuns de comunicação entre serviços. Minha preocupação é o fato de que, apesar do grande volume de trabalho feito por algumas pessoas muito inteligentes, está demorando um pouco para que as ferramentas nessa área se estabilizem. O Istio parece ser claramente o líder, porém está longe de ser a única opção na área, e novas ferramentas parecem surgir semanalmente. Meu conselho geral tem sido dar um pouco mais de tempo à área de service mesh para que ela se estabilize, se você puder, antes de tomar sua decisão.

Exemplo: FTP

Embora eu tenha falado bastante do uso do padrão strangler fig em sistemas que se baseiam em HTTP, não há nada que impeça você de interceptar e redirecionar outras formas de protocolos de comunicação. A Homegate, uma empresa suíça do setor imobiliário, utilizou uma variação desse padrão para mudar o modo como os clientes carregavam novas listas de imóveis.

Os clientes da Homegate carregavam as listas por meio de FTP, com um sistema monolítico existente cuidando dos arquivos carregados. A empresa estava disposta a migrar para microsserviços, e queria também começar a trabalhar com um novo método de upload no lugar do upload FTP batch (em lote), o qual usaria uma API REST que fosse compatível com um padrão que logo seria ratificado.

A empresa não queria mudar nada do ponto de vista dos clientes – ela queria que as mudanças fossem transparentes. Isso significa que o FTP ainda teria de ser o método pelo qual os clientes interagiriam com o sistema, pelo menos por ora. No final, a empresa interceptava os uploads FTP (detectando mudanças no log do servidor FTP) e direcionava os arquivos recém-carregados para um adaptador, que convertia esses arquivos em requisições para a nova API REST, como mostra a Figura 3.15.

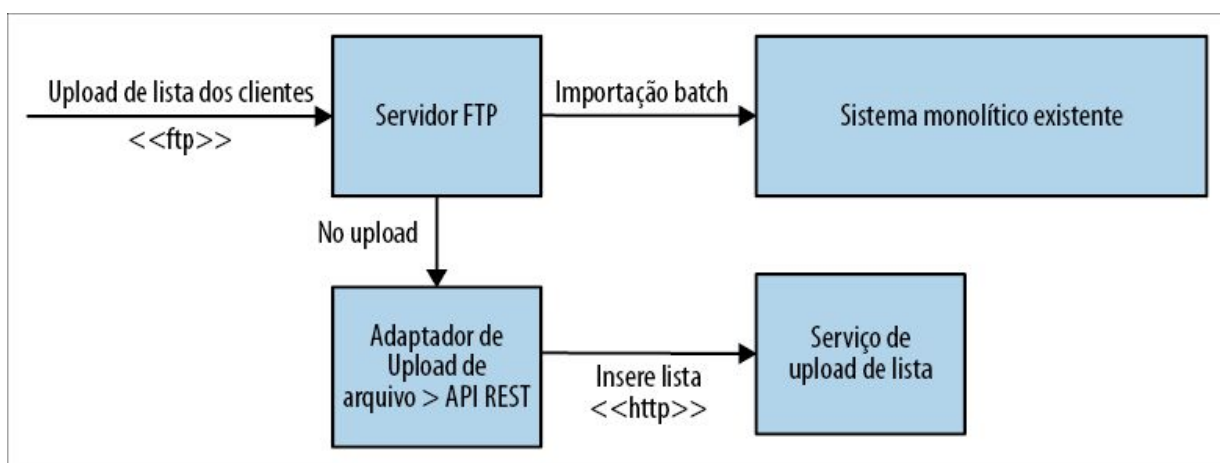


Figura 3.15 – Interceptando um upload FTP e fazendo um desvio para um novo serviço de listas para a Homegate.

Do ponto de vista do cliente, o processo de upload em si não mudou. A vantagem estava no fato de que o novo serviço que lidava com o upload era capaz de publicar os novos dados bem mais depressa, ajudando os clientes a verem os anúncios com muito mais rapidez. Há um plano para expor diretamente a nova API REST aos clientes no futuro. O aspecto interessante é que, durante esse período, os dois métodos de upload de listas estavam ativos. Isso permitiu que a equipe garantisse que ambos os métodos estivessem funcionando corretamente. Este é um ótimo exemplo de um padrão que exploraremos mais adiante na seção Padrão: execução em paralelo.

Exemplo: interceptação de mensagens

Até agora, vimos a interceptação de chamadas síncronas; o que aconteceria, porém, se o seu sistema monolítico estivesse baseado em outra forma de protocolo, talvez recebendo mensagens de um broker? O padrão básico será o mesmo – precisamos de um método para interceptar as chamadas e redirecioná-las para o novo microsserviço. A principal diferença está na natureza do protocolo em si.

Encaminhamento com base no conteúdo

Na Figura 3.16, nosso sistema monolítico recebe várias mensagens, e um subconjunto delas deve ser interceptado.

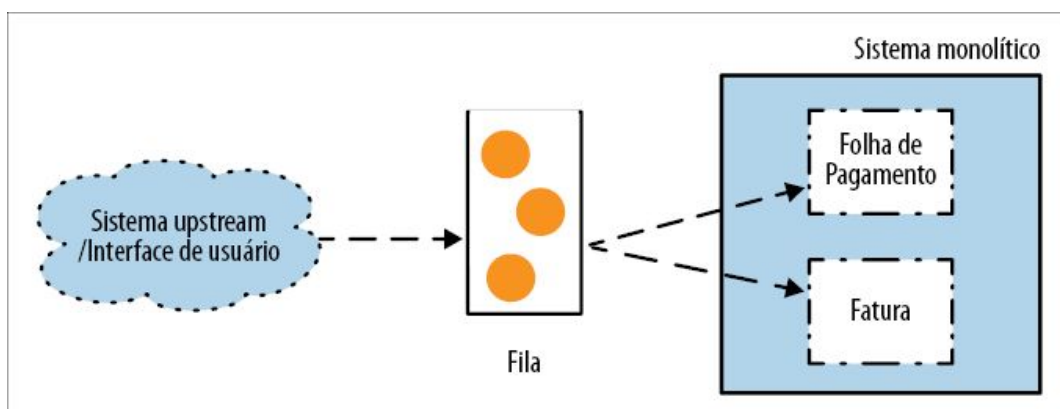


Figura 3.16 – Um sistema monolítico recebendo chamadas por meio

de uma fila.

Uma abordagem simples seria interceptar *todas* as chamadas destinadas ao sistema monolítico downstream e filtrar as mensagens para que sejam enviadas ao local apropriado, conforme mostra a Figura 3.17. É, basicamente, uma implementação do padrão de roteador baseado em conteúdo (content-based router), conforme descrito no livro *Enterprise Integration Patterns*.⁵

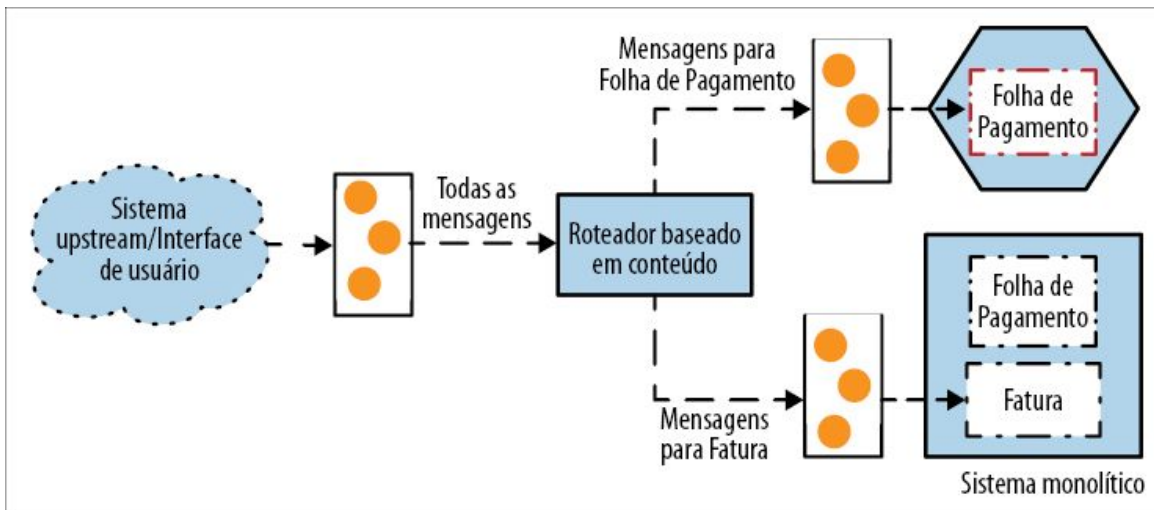


Figura 3.17 – Usando um padrão de roteador baseado em conteúdo para interceptar chamadas de mensagens.

Essa técnica nos permite deixar o sistema monolítico inalterado, porém estamos colocando outra fila no caminho de nossa requisição; isso poderia resultar em uma latência adicional e será outro ponto a ser administrado. Outra preocupação está no nível de “inteligência” que estaríamos colocando em nossa camada de mensagens. No Capítulo 4 de *Building Microservices*, falei dos problemas enfrentados pelos sistemas que colocam muita inteligência nas redes entre os serviços, pois isso pode fazer com que eles se tornem mais difíceis de entender e de modificar. Pedi que você adotasse o mantra “smart endpoints, dumb pipes”⁶ – algo no qual continuo insistindo. Não há dúvidas de que, nesse caso, o roteador baseado em conteúdo implica a implementação de um “smart pipe” – estamos acrescentando complexidade no modo como as chamadas são roteadas entre os sistemas. Em algumas

situações, essa é uma técnica extremamente útil, mas cabe a você encontrar um equilíbrio satisfatório.

Consumo seletivo

Uma alternativa seria alterar o sistema monolítico e fazer com que ele ignore as mensagens enviadas que devam ser recebidas pelo novo serviço, como vemos na Figura 3.18. Nesse caso, tanto o novo serviço como o sistema monolítico compartilham a mesma fila e, localmente, utilizam um tipo de processo de correspondência de padrões para observar as mensagens que sejam de seu interesse. Esse tipo de filtragem é um requisito muito comum em sistemas baseados em mensagens, e pode ser implementado com algo como um Message Selector em JMS ou com uma tecnologia equivalente em outras plataformas.

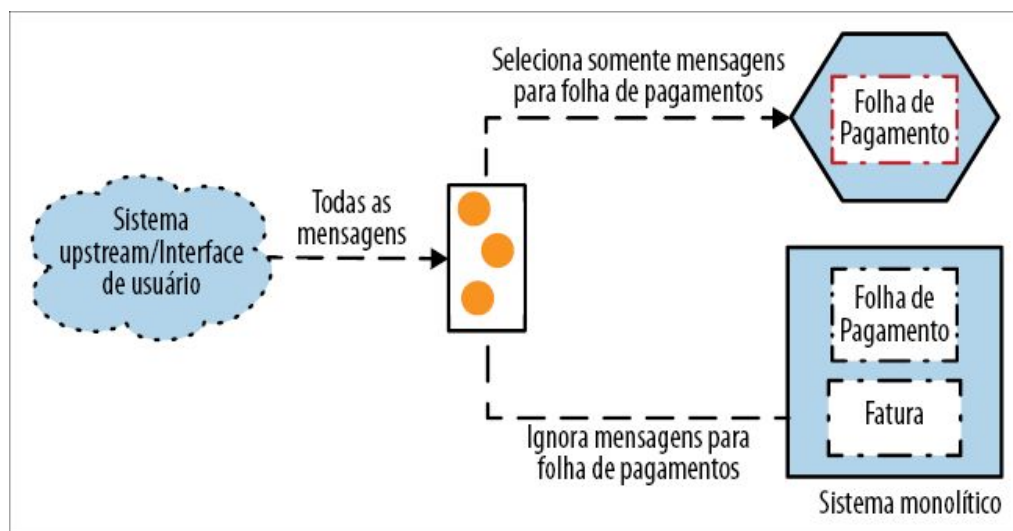


Figura 3.18 – Consumo seletivo das mensagens.

Essa abordagem com filtros reduz a necessidade de criar uma fila adicional, porém apresenta alguns desafios. Em primeiro lugar, sua tecnologia subjacente de troca de mensagens pode permitir ou não que você compartilhe o acesso a uma única fila dessa forma (embora seja um recurso comum, eu não me surpreenderia se não fosse possível). Em segundo lugar, quando quiser redirecionar as chamadas, duas mudanças serão necessárias para que haja uma boa coordenação. Você deverá fazer com que seu sistema

monolítico pare de ler as chamadas destinadas ao novo serviço, e deve fazer com que o novo serviço as receba. Do mesmo modo, reverter a interceptação de chamadas exigirá duas mudanças para fazer um rollback.

Quanto mais tipos de consumidores você tiver para a mesma fila, e quanto mais complexas forem as regras de filtragem, mais problemática poderá ser a situação. É fácil pensar em uma situação na qual dois consumidores comecem a receber a mesma mensagem como consequência de regras sobrepostas, ou até mesmo o contrário – algumas mensagens são totalmente ignoradas. Por esse motivo, eu provavelmente consideraria fazer um consumo seletivo apenas no caso de haver poucos consumidores e/ou se o conjunto de regras de filtragem for simples. Uma abordagem de roteamento com base em conteúdo provavelmente fará mais sentido à medida que o número de tipos de consumidores aumentar, embora você deva tomar cuidado com as possíveis desvantagens já mencionadas, particularmente com o problema dos “smart pipes”.

A complicação adicional dessa solução ou do roteamento com base em conteúdo é que, se estivermos usando um estilo de comunicação assíncrono de requisição-resposta, teremos de garantir que seja possível encaminhar a requisição de volta aos clientes, sem que eles percebam que houve alguma mudança. Há outras opções para o roteamento de chamadas em sistemas orientados a mensagens, muitas das quais poderão ajudar você a implementar migrações com o padrão strangler fig. Recomendo bastante o livro *Enterprise Integration Patterns* como um ótimo recurso nesse caso.

Outros protocolos

Conforme espero que você possa compreender a partir desse exemplo, há várias maneiras de interceptar chamadas para o seu sistema monolítico atual, mesmo que você utilize diferentes tipos de protocolos. E se o seu sistema monolítico funcionar com base em

um upload de arquivos batch? Intercepte o arquivo batch, extraia as chamadas que você quer interceptar e remova-as do arquivo antes de passá-lo para a frente. É verdade que alguns métodos deixam esse processo mais complicado, e será muito mais fácil se algo como HTTP estiver sendo usado; porém, com um pouco de criatividade, o padrão strangler fig poderá ser utilizado em um número surpreendente de situações.

Outros exemplos do padrão strangler fig

O padrão strangler fig é muito conveniente em qualquer contexto no qual queremos mudar a plataforma de um sistema existente de maneira gradual, e seu uso não está limitado às equipes que implementam arquiteturas de microsserviços. O padrão já estava em uso há muito tempo antes de Martin Fowler tê-lo descrito em 2004. Na empresa em que trabalhei antes, a ThoughtWorks, usávamos o padrão frequentemente para ajudar a reconstruir aplicações monolíticas. Em seu blog, Paul Hammant fez a compilação de uma lista não exaustiva de projetos (<http://bit.ly/2paBpyP>) nos quais esse padrão foi usado. A lista inclui um sistema de registros de uma empresa de comércio, uma aplicação de reserva de passagens de uma companhia aérea, um sistema de passagens ferroviárias e um portal de anúncios de classificados.

Mudança de comportamentos concomitantes à migração de funcionalidades

Neste e em outras partes do livro, meu foco está nos padrões que escolhi especificamente porque podem ser usados na migração gradual de um sistema existente para uma arquitetura de microsserviços. Um dos principais motivos para isso é a possibilidade de misturar um trabalho de migração com a entrega constante de funcionalidades. No entanto, ainda há um problema que surge quando queremos modificar ou enriquecer o comportamento do sistema cuja migração está sendo feita.

Suponha, por exemplo, que faremos uso do padrão strangler fig para mover nossa funcionalidade de Folha de Pagamento para fora do sistema monolítico. O padrão strangler fig nos permite fazer esse trabalho em vários passos, e cada passo, teoricamente, permite que um rollback possa ser feito. Se fizéssemos o rollout de um novo serviço de Folha de Pagamento aos nossos clientes e identificássemos um problema, poderíamos desviar as chamadas de volta à funcionalidade de Folha de Pagamento do sistema antigo.

Essa será uma boa solução se a funcionalidade de Folha de Pagamento do sistema monolítico e do microsserviço forem funcionalmente equivalentes; o que aconteceria, porém, se, como parte da migração, mudássemos o modo como a Folha de Pagamento se comporta?

Se o microsserviço de Folha de Pagamento tivesse algumas correções de bug aplicadas no modo como funciona, as quais não tivessem sido portadas para a funcionalidade equivalente no sistema monolítico, um rollback também faria com que esses bugs reaparecessem no sistema. A situação poderia ficar mais complicada se você tivesse adicionado novas funcionalidades no microsserviço de Folha de Pagamento – um rollback exigiria então a remoção de funcionalidades para os seus clientes.

Não há uma solução fácil nesse caso. Se você permitir que sejam feitas mudanças na funcionalidade sendo movida antes de a migração estar concluída, terá de aceitar que qualquer rollback será mais complicado. Será mais fácil se você não permitir nenhuma mudança até que a migração esteja concluída. Quanto mais demorar a migração, mais difícil será garantir um “congelamento de funcionalidade” nessa parte do sistema – se houver uma exigência de que parte de seu sistema mude, é provável que essa exigência não vá desaparecer. Quanto mais tempo demorar para você concluir a migração, mais pressão você sofrerá para “aproveitar a oportunidade e incluir essa funcionalidade”. Quanto menor for cada migração, menos pressão você sofrerá para mudar a funcionalidade

até que a migração esteja concluída.



Ao migrar funcionalidades, procure não fazer nenhuma mudança no comportamento sendo movido – se puder, postergue novas funcionalidades ou correções de bugs até que a migração esteja concluída. Caso contrário, você reduzirá sua capacidade de fazer rollback das mudanças em seu sistema.

Padrão: composição de UI

Com as técnicas que consideramos até agora, passamos a maior parte do trabalho de migração gradual para o servidor – porém, a interface de usuário nos proporciona algumas oportunidades interessantes para combinar uma funcionalidade servida em parte por um sistema monolítico existente e em parte por uma nova arquitetura de microsserviços.

Muitos anos atrás, estive envolvido na tarefa de ajudar na migração do The Guardian (<https://www.guardian.co.uk/>) online, de um sistema de gerenciamento de conteúdo existente para uma nova plataforma personalizada, baseada em Java. Isso iria coincidir com o rollout de uma aparência totalmente nova para o jornal online, a ser vinculada com o relançamento da edição impressa. Como queríamos adotar uma abordagem gradual, a passagem do CMS existente para o novo site atendido pelo novo sistema foi dividida em fases, visando a seções verticais específicas (viagem, notícias, cultura etc.). Mesmo com essas verticais, também buscávamos oportunidades para dividir a migração em partes menores.

Acabamos usando duas técnicas de composição que foram muito úteis. Com base em conversas com outras empresas, nos últimos anos, ficou claro para mim que variações nessas técnicas são uma parte significativa do modo como muitas empresas adotam arquiteturas de microsserviços.

Exemplo: composição de páginas

Com o *The Guardian*, embora tenhamos começado fazendo o rollout

de um único widget (sobre o qual falaremos em breve), o plano sempre havia sido usar, na maior parte, uma migração baseada em páginas, a fim de permitir que uma aparência totalmente nova fosse publicada. Isso foi feito com uma seção vertical do jornal por vez, com a seção Viagens tendo sido a primeira que publicamos. Durante esse período de transição, os visitantes do site viam aparências distintas quando acessavam as novas partes do site. Muito cuidado foi tomado também para garantir que todos os links para as páginas antigas fossem redirecionados para os novos locais (cujos URLs haviam mudado).

Quando o *The Guardian* fez outra mudança na tecnologia, deixando o sistema monolítico Java para trás alguns anos depois, mais uma vez, eles usaram uma técnica semelhante de migração de uma seção vertical por vez. Nessa ocasião, eles fizeram uso do CDN (Content Delivery Network, ou Rede de Fornecimento de Conteúdo) Fastly para implementar as novas regras de roteamento, usando efetivamente o CDN de modo muito parecido com o que você faria com um proxy interno.⁷

O REA Group da Austrália, que apresenta listas de imóveis online, tem equipes distintas responsáveis pelas listas de imóveis comerciais ou residenciais, mas é proprietária dos dois canais. Em uma situação como essa, uma abordagem de composição de páginas faz sentido, pois uma equipe pode ser responsável por toda a experiência fim a fim. A REA, na verdade, emprega estratégias de marca um pouco diferentes para os diferentes canais, o que significa que uma decomposição em páginas faz mais sentido ainda, pois é possível proporcionar experiências bem diferentes para diferentes grupos de clientes.

Exemplo: composição de widgets

No *The Guardian*, a seção Viagens foi a primeira seção identificada para migração para a nova plataforma. O raciocínio, parcialmente, se justificava pelo fato de ela apresentar desafios interessantes

relacionados à classificação, mas também por não ser a parte mais em evidência do site. Basicamente, queríamos colocar algo no ar, aprender com essa experiência, mas também garantir que, se houvesse erros, o problema não afetaria as partes principais do site. Em vez de colocar toda a parte de viagens do site no ar, repleta de reportagens minuciosas sobre destinos glamurosos pelo mundo todo, queríamos um lançamento muito mais discreto para testar o sistema. Implantamos então um único widget exibindo os dez principais destinos de viagens, definidos com o novo sistema. O widget estava associado às páginas de viagem antigas do jornal, como mostra a Figura 3.19. Em nosso caso, fizemos uso de uma técnica chamada ESI (Edge-Side Includes) usando o Apache. Com o ESI, um template é definido em sua página web, e um servidor web fornece esse conteúdo.

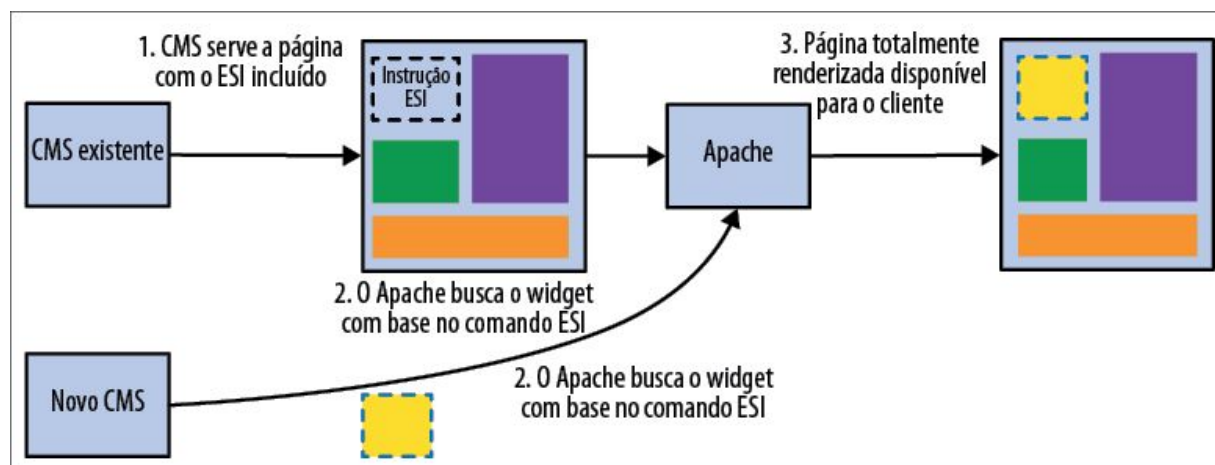


Figura 3.19 – Usando Edge-Side Includes para agregar conteúdo proveniente do novo CMS do The Guardian.

Atualmente, agregar um widget exclusivamente do lado do servidor parece menos comum. Em boa parte, isso se deve ao fato de as tecnologias de navegadores terem se tornado mais sofisticadas, permitindo que haja um nível muito maior de composição no próprio navegador (ou na aplicação nativa – mais sobre esse assunto posteriormente). Isso significa que, para as UIs web com widgets, o próprio navegador muitas vezes faz várias chamadas para carregar diversos widgets utilizando inúmeras técnicas. Essa solução tem

uma vantagem adicional: caso um widget falhe e não seja carregado – talvez porque o serviço que lhe dá suporte esteja indisponível –, os outros widgets ainda poderão ser renderizados, possibilitando uma degradação somente parcial, e não total, do serviço.

Apesar de, no final, termos usado basicamente uma composição de páginas no *The Guardian*, várias outras empresas fazem uso intenso da composição de widgets com serviços de apoio no backend. A Orbitz (que atualmente faz parte da Expedia), por exemplo, criou serviços dedicados apenas a atender a um único widget.⁸ Antes de passar para os microsserviços, o site da Orbitz já estava separado em “módulos” distintos de UI (segundo a nomenclatura da Orbitz). Esses módulos podiam representar um formulário de pesquisa, um formulário de reservas, um mapa etc. Esses módulos de UI eram inicialmente atendidos de forma direta pelo serviço de Orquestração de Conteúdo, conforme vemos na Figura 3.20.

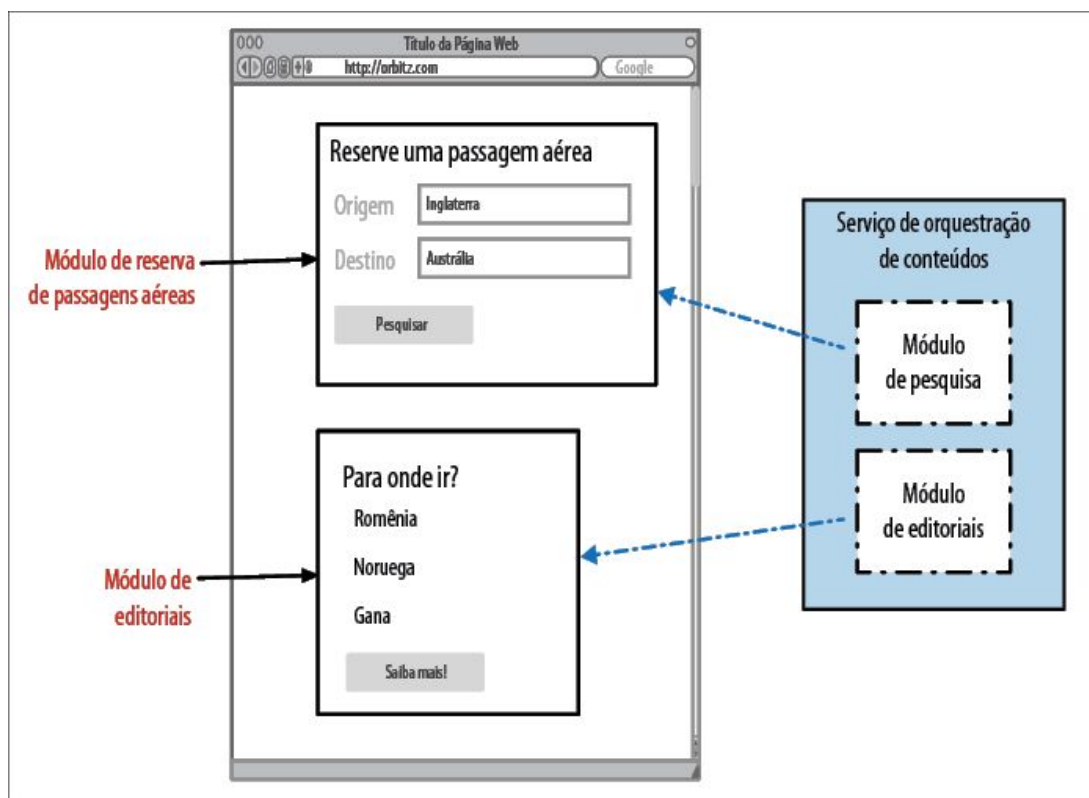


Figura 3.20 – Antes da migração para os microsserviços, o servido

de Orquestração de Conteúdo da Orbitz atendia a todos os módulos.

O serviço de Orquestração de Conteúdo era, na verdade, um grande sistema monolítico. Todas as equipes responsáveis por esses módulos deviam coordenar as mudanças feitas no sistema monolítico, causando atrasos significativos no rollout das mudanças. Este é um exemplo clássico do problema de Desavenças nas Entregas que enfatizei no Capítulo 1 – sempre que as equipes tiverem de coordenar o rollout de uma mudança, o custo da mudança será mais alto. Como parte do processo em direção a ciclos mais rápidos de lançamento de versões, quando a Orbitz decidiu experimentar os microsserviços, o foco da decomposição teve como base esses módulos – a começar pelo módulo de editoriais. O serviço de Orquestração de Conteúdo foi modificado de modo a delegar responsabilidades para novos módulos de serviços downstream, como vemos na Figura 3.21.

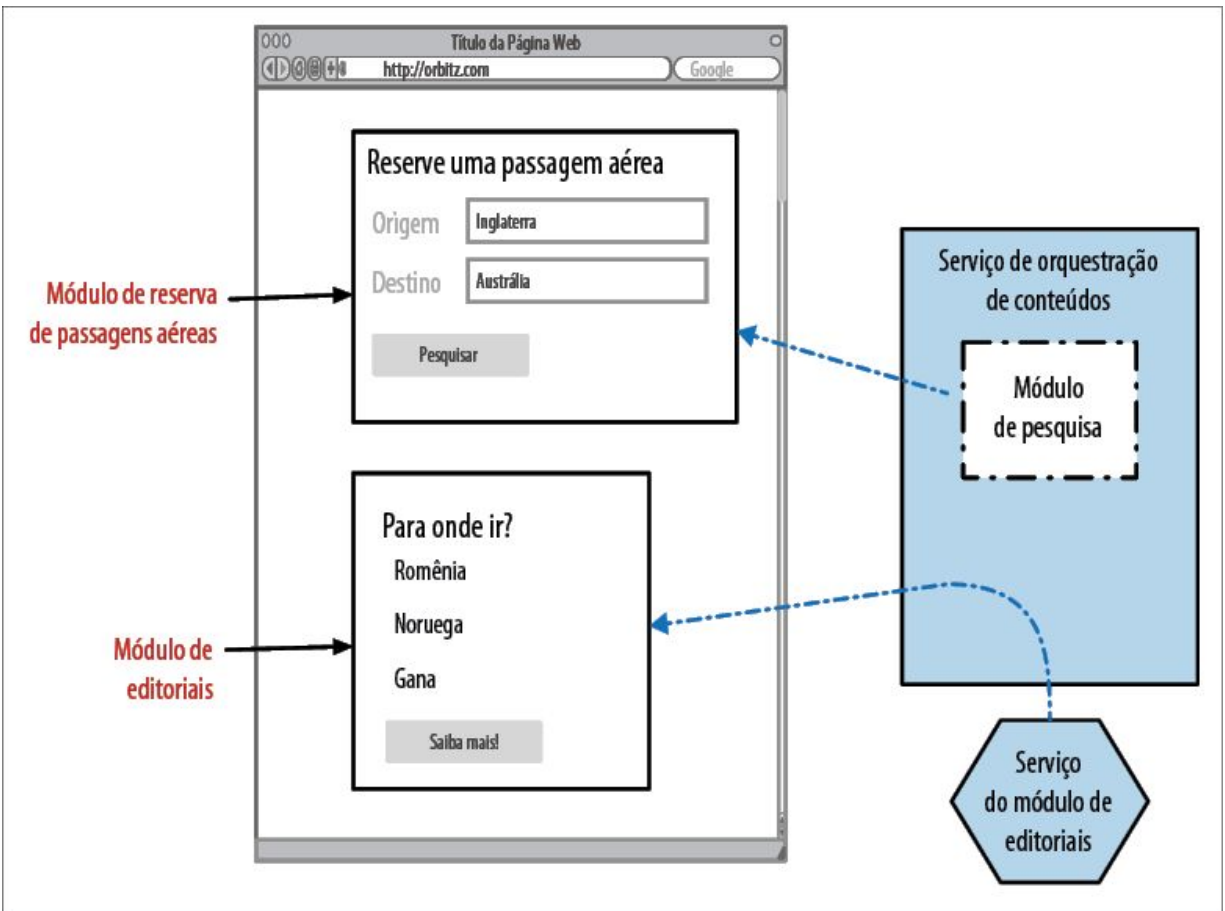


Figura 3.21 – A migração dos módulos foi feita, uma de cada vez, com o serviço de Orquestração de Conteúdo delegando responsabilidades para os novos serviços de apoio.

O fato de que a UI já estava visualmente decomposta segundo essas linhas facilitou bastante realizar a tarefa de maneira gradual. A transição também foi facilitada pelo fato de já haver linhas claras de separação acerca dos responsáveis por cada módulo, facilitando fazer as migrações sem causar interferências em outras equipes.

Vale a pena observar que nem todas as interfaces de usuário são apropriadas para uma decomposição em widgets bem definidos – porém, se for possível, isso facilitará bastante o trabalho de migração gradual para os microserviços.

Aplicativos móveis

Embora tenhamos falado basicamente das UIs web, algumas

dessas técnicas podem funcionar bem para clientes móveis também. Tanto o Android como o iOS, por exemplo, possibilitam separar suas interfaces de usuário em componentes, facilitando trabalhar isoladamente nessas partes da UI, ou recombina-las de maneiras diferentes.

Um dos desafios de implantar mudanças com aplicativos móveis nativos é que tanto a App Store da Apple como o Google Play Store exigem que os aplicativos sejam submetidos e analisados antes que novas versões possam ser disponibilizadas. Embora o tempo para que os aplicativos sejam aceitos pelas lojas em geral tenha reduzido significativamente nos últimos anos, ainda há um período para que novas versões de software lançadas possam ser implantadas. O próprio aplicativo, atualmente, ainda é uma aplicação monolítica: se você quiser mudar uma única parte de um aplicativo móvel nativo, a aplicação toda terá de ser implantada novamente. Você também tem de considerar o fato de que os usuários precisam fazer download do novo aplicativo para ver as novas funcionalidades – algo com o qual, em geral, não é necessário lidar quando uma aplicação web é usada, pois as mudanças são disponibilizadas de forma transparente no navegador dos usuários.

Muitas empresas têm lidado com esse problema permitindo que modificações dinâmicas sejam feitas em um aplicativo móvel nativo existente, sem ter de recorrer à implantação de uma nova versão da aplicação nativa. Ao implantar mudanças do lado do servidor, os dispositivos dos clientes poderão ver imediatamente a nova funcionalidade sem necessariamente ter de implantar uma nova versão do aplicativo móvel nativo. Isso pode ser feito simplesmente usando técnicas como embedded web views, embora algumas empresas utilizem técnicas mais sofisticadas.

A UI do Spotify em todas as plataformas é extremamente orientada a componentes, incluindo seus aplicativos para iOS e Android. Praticamente tudo que você vir é um componente separado, desde um simples cabeçalho de texto até a arte do álbum ou uma playlist.⁹

Alguns desses módulos, por sua vez, são atendidos por um ou mais microsserviços. A configuração e o layout desses componentes de UI são definidos de modo declarativo do lado do servidor; os engenheiros do Spotify são capazes de mudar as visões dos usuários e fazer um rollout rápido da mudança, sem a necessidade de submeter novas versões de seus aplicativos à app store. Isso lhes permite fazer experimentos e testar novos recursos com muito mais rapidez.

Exemplo: Micro Frontends

À medida que a largura de banda e a capacidade dos navegadores web melhoraram, o mesmo ocorreu com a sofisticação do código executado nos navegadores. Muitas interfaces web de usuários atualmente fazem uso de alguma forma de framework de aplicação single-page, o qual se afasta do conceito de uma aplicação composta de diferentes páginas web. Em vez disso, você tem uma interface de usuário mais eficaz, na qual tudo executa em um único painel – são experiências de usuário internas ao navegador, que antes estavam disponíveis somente para aqueles que trabalhavam com SDKs “pesadas” de UI, como o Swing de Java.

Ao disponibilizar uma interface completa em uma única página, obviamente não podemos considerar uma composição baseada em páginas, portanto, será preciso considerar algum tipo de composição baseada em widgets. Tentativas foram feitas para definir formatos comuns de widgets para a web – mais recentemente, a especificação Web Components (Componentes da Web) tem tentado definir um modelo padrão de componentes aceito pelos diferentes navegadores. Contudo, tem demorado bastante para que esse padrão ganhe força, com o suporte dos navegadores (entre outros fatores) sendo um obstáculo significativo.

Pessoas que fazem uso de frameworks para aplicações single-page, como Vue, Angular ou React, não ficaram sentadas esperando que o Web Components resolvesse seus problemas. Muitas pessoas

tentaram atacar o problema de modularizar as UIs desenvolvidas com SDKs que haviam sido inicialmente projetadas para tomar conta de todo o painel do navegador. Isso levou a um deslocamento em direção ao que algumas pessoas chamam de *Micro Frontends*.

À primeira vista, os Micro Frontends dizem respeito apenas a separar uma interface de usuário em diferentes componentes com os quais seja possível trabalhar de modo independente. Quanto a isso, não há nenhuma novidade – softwares orientados a componentes são anteriores ao meu nascimento em vários anos! O mais interessante é que as pessoas estão descobrindo como fazer os navegadores web, os SDKs SPA (Single Page Application) e a separação em componentes funcionarem juntos. Como criar, exatamente, uma única UI a partir de partes de Vue e React sem que suas dependências entrem em conflito, mas ainda permitindo que possam compartilhar informações?

Discutir esse assunto em detalhes está fora do escopo deste livro, parcialmente porque o modo exato como você faria essa solução funcionar variará conforme os frameworks SPA que estiverem em uso. Contudo, se você se vir com uma aplicação single-page que queira separar, você não estará sozinho e há muitas pessoas por aí compartilhando técnicas e bibliotecas para fazer essa abordagem funcionar.

Quando usar o padrão

A composição de UI como uma técnica para permitir uma redefinição de plataforma para os sistemas é extremamente eficaz, pois permite a migração de porções verticais inteiras de funcionalidades. Para que ela funcione, porém, você deverá ser capaz de mudar a interface de usuário atual a fim de possibilitar que novas funcionalidades sejam inseridas de forma segura. Discutiremos as técnicas de composição mais adiante neste livro, mas vale a pena observar que as técnicas que você poderá usar em geral dependerão da natureza da tecnologia utilizada para

implementar a interface de usuário. Um bom site tradicional facilitará a composição de UI, enquanto uma tecnologia de aplicação single-page acrescentará certo nível de complexidade, mas, com frequência, implicará uma variedade incrível de abordagens para implementação.

Padrão: branch por abstração

Para que o padrão strangler fig (<http://bit.ly/2p5xMKo>) funcione, devemos interceptar as chamadas no perímetro de nosso sistema monolítico. No entanto, o que acontecerá se a funcionalidade que queremos extrair estiver demasiadamente entranhada em nosso sistema atual? Retomando um exemplo anterior, considere o desejo de extrair a funcionalidade de Notificação, conforme vimos na Figura 3.4.

Para essa extração, será necessário fazer mudanças no sistema atual. Essas mudanças poderiam ser significativas e causar perturbações a outros desenvolvedores que estivessem trabalhando na base de código ao mesmo tempo. Temos tensões concorrentes nesse caso. Por um lado, queremos fazer nossas alterações em passos graduais. Por outro, desejamos causar menos perturbações às pessoas que estão trabalhando em outras partes da base de código. Essa situação nos levará naturalmente a querer concluir o trabalho com rapidez.

Com frequência, ao modificar as partes de uma base de código existente, as pessoas farão esse trabalho em um branch de código-fonte separado. Isso permitirá que as alterações sejam feitas sem atrapalhar o trabalho de outros desenvolvedores. O desafio é que, assim que as alterações no branch estiverem prontas, será necessário fazer um merge delas, e isso, muitas vezes, pode implicar desafios significativos. Quanto mais tempo o branch existir, maiores serão os problemas. Não descreverei os detalhes relacionados aos problemas dos branches de códigos-fontes de longa duração, exceto para dizer que são contrários aos princípios

da integração contínua. Também poderia acrescentar que os dados coletados com o “The 2017 State of DevOps Report” (Relatório do estado do DevOps em 2017, <http://bit.ly/2nDpJUP>) mostram que adotar um desenvolvimento baseado em tronco (no qual as alterações são feitas diretamente na linha principal e os branches são evitados) e usar branches de curta duração contribuem para que as equipes de TI tenham um desempenho melhor. Vamos apenas dizer que não sou fã de branches de longa duração, e não estou sozinho nessa.

Desse modo, queremos ser capazes de fazer alterações em nossa base de código de forma incremental, mas também queremos perturbar minimamente os desenvolvedores que estiverem trabalhando em outras partes de nossa base de código. Há outro padrão que pode ser usado, o qual nos permite fazer alterações graduais em nosso sistema monolítico sem recorrer a um branching do código-fonte. O padrão branch por abstração consiste em fazer alterações na base de código existente a fim de permitir que as implementações coexistam com segurança, na mesma versão de código, sem causar muitas perturbações.

Como o padrão funciona

O branch por abstração é composto de cinco passos:

1. Crie uma abstração para a funcionalidade a ser substituída.
2. Mude os clientes da funcionalidade atual para que usem a nova abstração.
3. Faça uma nova implementação da abstração com a funcionalidade retrabalhada. Em nosso caso, essa nova implementação chamará o nosso novo microsserviço.
4. Faça a troca da abstração para que a nossa nova implementação seja usada.
5. Faça uma limpeza da abstração e remova a antiga implementação.

Vamos analisar esses passos no contexto de mover nossa funcionalidade de Notificação para um serviço, conforme detalhado na Figura 3.4.

Passo 1: Criar a abstração

A primeira tarefa é criar uma abstração que represente as interações entre o código a ser alterado e quem chama esse código, conforme vemos na Figura 3.22. Se a funcionalidade atual de Notificação estiver razoavelmente bem fatorada, essa tarefa poderia ser simples, bastando aplicar uma refatoração de Extração de Interface (Extract Interface) em nosso IDE. Caso contrário, talvez você precise extrair um ponto de extensão (seam), conforme já mencionamos. Isso pode exigir que você faça pesquisas em sua base de código em busca das chamadas feitas a APIs que enviem emails, SMSs ou qualquer outro método de notificação que possa haver. Encontrar esse código e criar uma abstração que os outros códigos utilizem é um passo obrigatório.

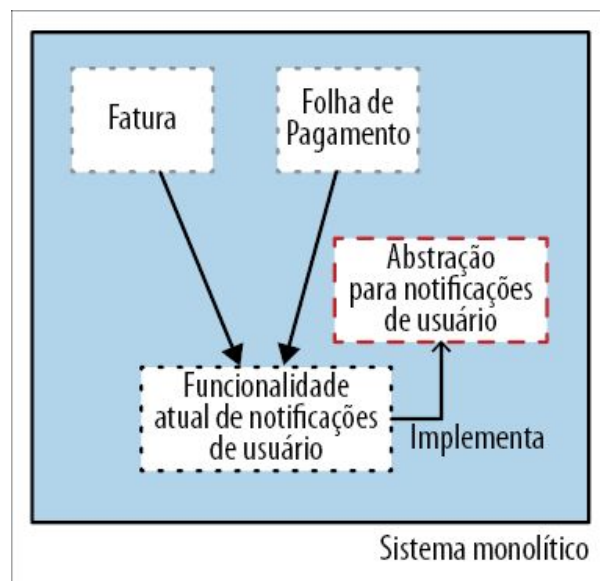


Figura 3.22 – Passo 1: Crie uma abstração.

Passo 2: Use a abstração

Com nossa abstração criada, precisamos agora refatorar os clientes atuais da funcionalidade de Notificação para que usem esse novo

ponto de abstração, como vemos na Figura 3.23. É possível que uma refatoração Extrair Interface (Extract Interface) pudesse fazer esse trabalho para nós automaticamente – mas, com base em minha experiência, é mais comum que esse processo seja incremental, envolvendo uma identificação manual das chamadas de entrada à funcionalidade em questão. O ponto positivo, nesse caso, é que essas alterações são pequenas e graduais; são fáceis de fazer em passos pequenos, sem causar muito impacto no código existente. A essa altura, não deve haver nenhuma mudança funcional no comportamento do sistema.

Passo 3: Crie outra implementação

Com nossa nova abstração pronta, podemos agora começar a trabalhar na implementação da chamada do novo serviço. No sistema monolítico, nossa implementação da funcionalidade de Notificação será, basicamente, apenas um cliente chamando o serviço externo, como mostra a Figura 3.24 – a maior parte da funcionalidade estará no próprio serviço.

O ponto principal a ser compreendido agora é que, embora tenhamos simultaneamente duas implementações da abstração na base de código, somente uma delas estará ativa no sistema no momento. Até estarmos satisfeitos com nossa nova implementação da chamada do serviço a ponto de torná-la ativa, ela estará, na verdade, dormente. Enquanto trabalhamos para implementar toda a funcionalidade equivalente em nosso novo serviço, a nova implementação da abstração poderia devolver erros de Não Implementado. Isso não nos impedirá de escrever testes para a funcionalidade que implementarmos, é claro, e essa é uma das vantagens de fazer a integração desse trabalho o mais cedo possível.

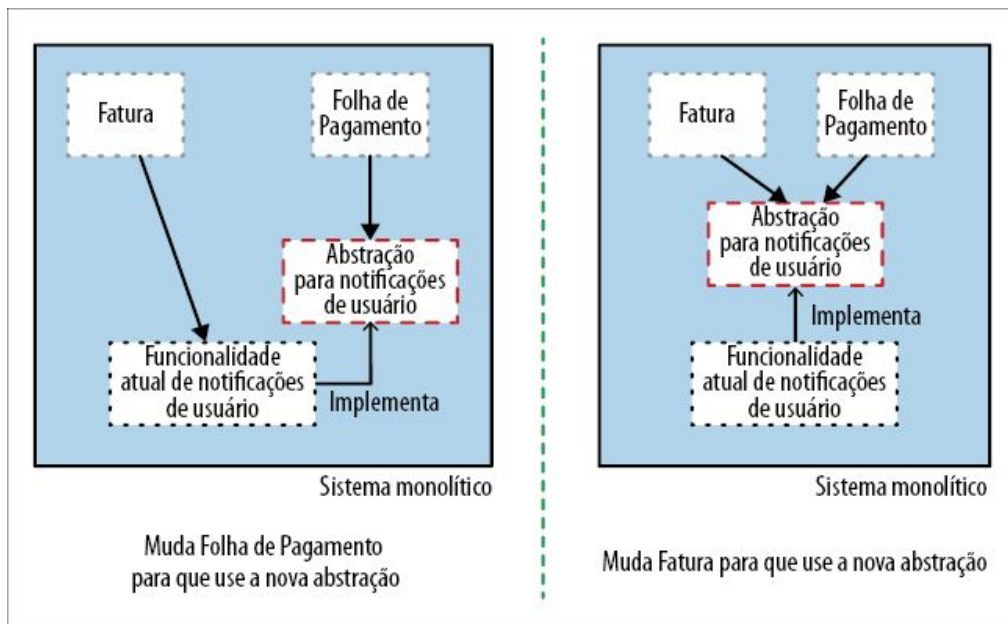


Figura 3.23 – Passo 2: Mude os clientes atuais para que usem a nova abstração.

Durante esse processo, podemos também fazer a implantação de nosso serviço de Notificações de Usuário em andamento no ambiente de produção, como fizemos no caso do padrão strangler fig. O fato de esse serviço ainda não estar pronto não é um problema – nesse momento, como nossa implementação da abstração de Notificações não está ativa, o serviço não será realmente chamado. Contudo, podemos implantá-lo, testá-lo *in situ* e conferir se as partes da funcionalidade que implantamos estão funcionando corretamente.

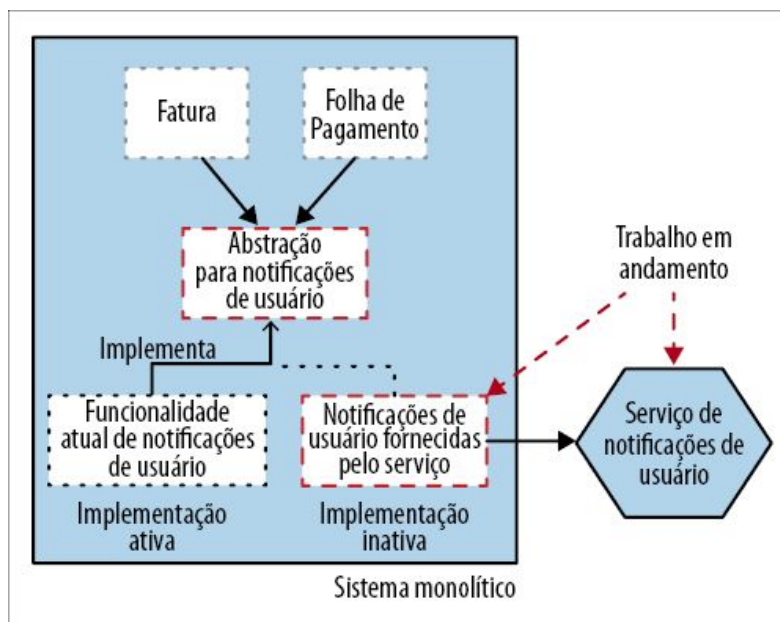


Figura 3.24 – Passo 3: Crie outra implementação para a abstração.

Essa fase poderia durar um período significativo. Jez Humble detalha o uso do padrão branch por abstração (<http://bit.ly/2p95lv7>) para migrar a camada de persistência do banco de dados, usada na aplicação de entrega contínua GoCD (na época, ele se chamava Cruise). A mudança do iBatis para o Hibernate durou vários meses – nesse período, a aplicação continuou sendo lançada aos clientes a cada duas semanas.

Passo 4: Faça a troca da implementação

Assim que nossa nova implementação estiver funcionando corretamente e estivermos satisfeitos com ela, fazemos a troca em nosso ponto de abstração, de modo que a nova implementação se torne ativa e a antiga funcionalidade não seja mais usada, como vemos na Figura 3.25.

O ideal é que, assim como no caso do padrão strangler fig, um método de troca simples seja usado, o qual permita uma fácil alternância. Isso nos permitirá voltar rapidamente para a funcionalidade antiga caso algum problema seja encontrado. Uma solução comum seria usar toggles de recursos (feature toggles). Na Figura 3.26, vemos toggles implementados com um arquivo de

configuração, permitindo mudar a implementação em uso sem ter de modificar o código. Se quiser saber mais sobre toggles de recursos e como implementá-los, Pete Hodgson tem um artigo excelente sobre o assunto.

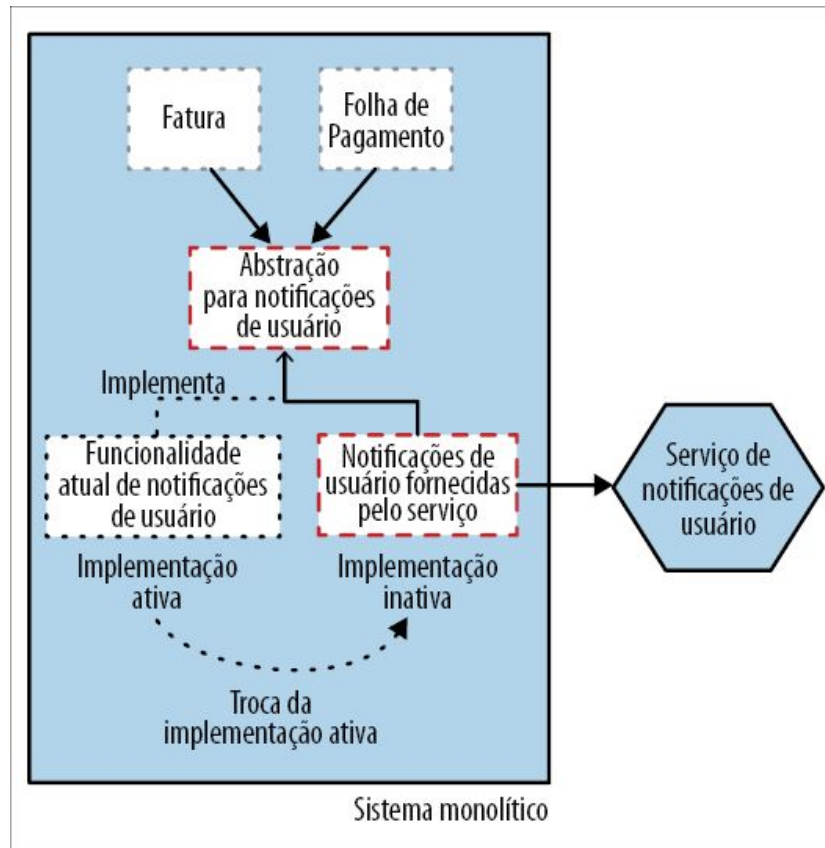


Figura 3.25 – Passo 4: Faça a troca da implementação ativa para que nosso novo microsserviço seja utilizado.

Nessa fase, temos duas implementações da mesma abstração, que *esperamos* que sejam equivalentes quanto à funcionalidade. Podemos usar testes para conferir a equivalência, mas também temos a opção de usar as *duas* implementações no ambiente de produção a fim de fazer verificações adicionais. A ideia será mais bem explorada na seção Padrão: execução em paralelo.

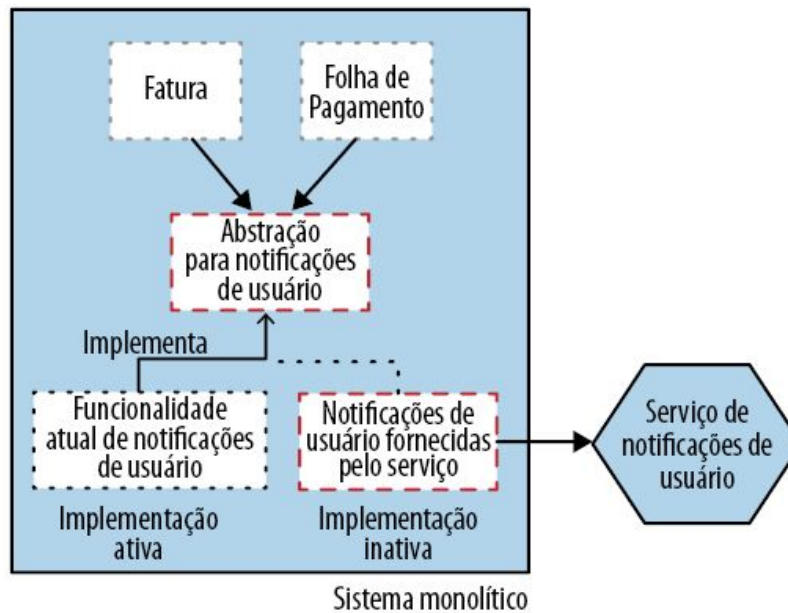
Passo 5: Faça uma limpeza

Com o novo microsserviço agora fornecendo todas as notificações aos usuários, podemos voltar nossa atenção para uma limpeza. A

essa altura, nossa funcionalidade antiga de Notificações de Usuário não está mais em uso, portanto, um passo óbvio seria removê-la, como mostra a Figura 3.27. Nosso sistema monolítico está começando a encolher!

Ao remover a antiga implementação, também faria sentido remover qualquer alternância de flag de recurso que possamos ter implementado. Um dos problemas reais associados ao uso de flags de recursos é manter as flags antigas – não faça isso! Remova as flags que não sejam mais necessárias para simplificar o código.

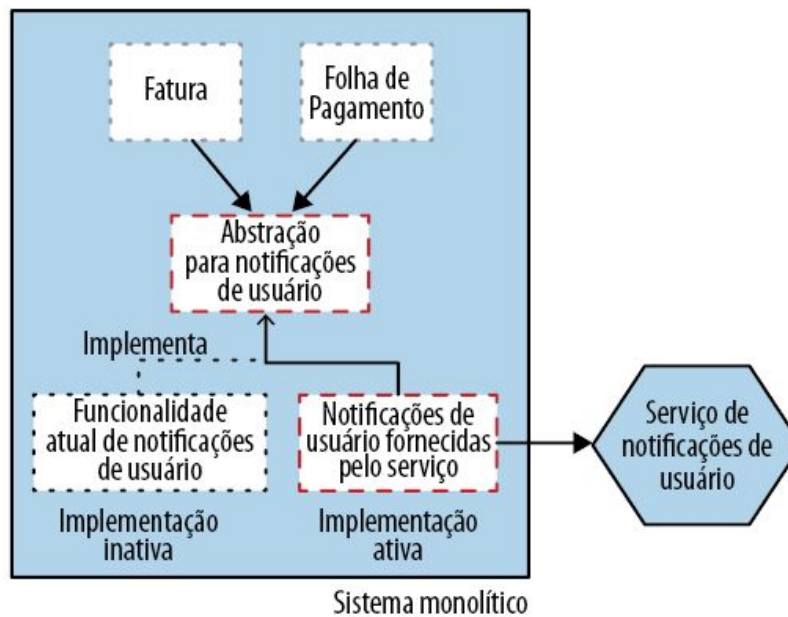
Flag de recurso desativada



```
new_notification_service_enabled = false
```

Arquivo de configuração

Flag de recurso ativada



```
new_notification_service_enabled = true
```

Arquivo de configuração

Figura 3.26 – Passo 5: Usando toggles de recursos para alternar entre as implementações.

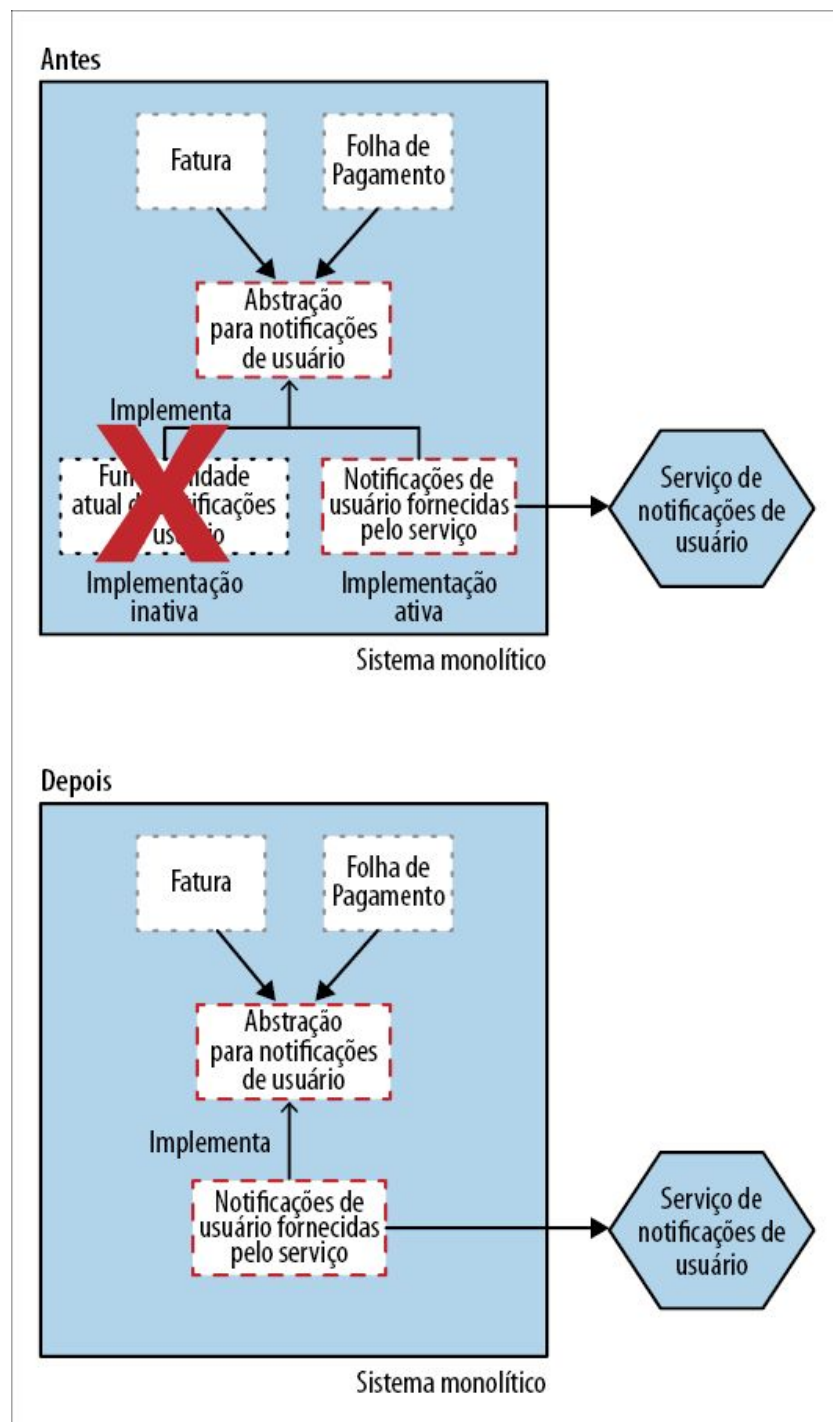


Figura 3.27 – Passo 6: Remova a implementação antiga.

Por fim, com a implementação antiga removida, temos a opção de remover o próprio ponto de abstração, como mostra a Figura 3.28.

Contudo, é possível que a criação da abstração possa ter melhorado a base de código, a ponto de você preferir mantê-la. Se for algo simples como uma interface, mantê-la causará um impacto mínimo na base de código existente.

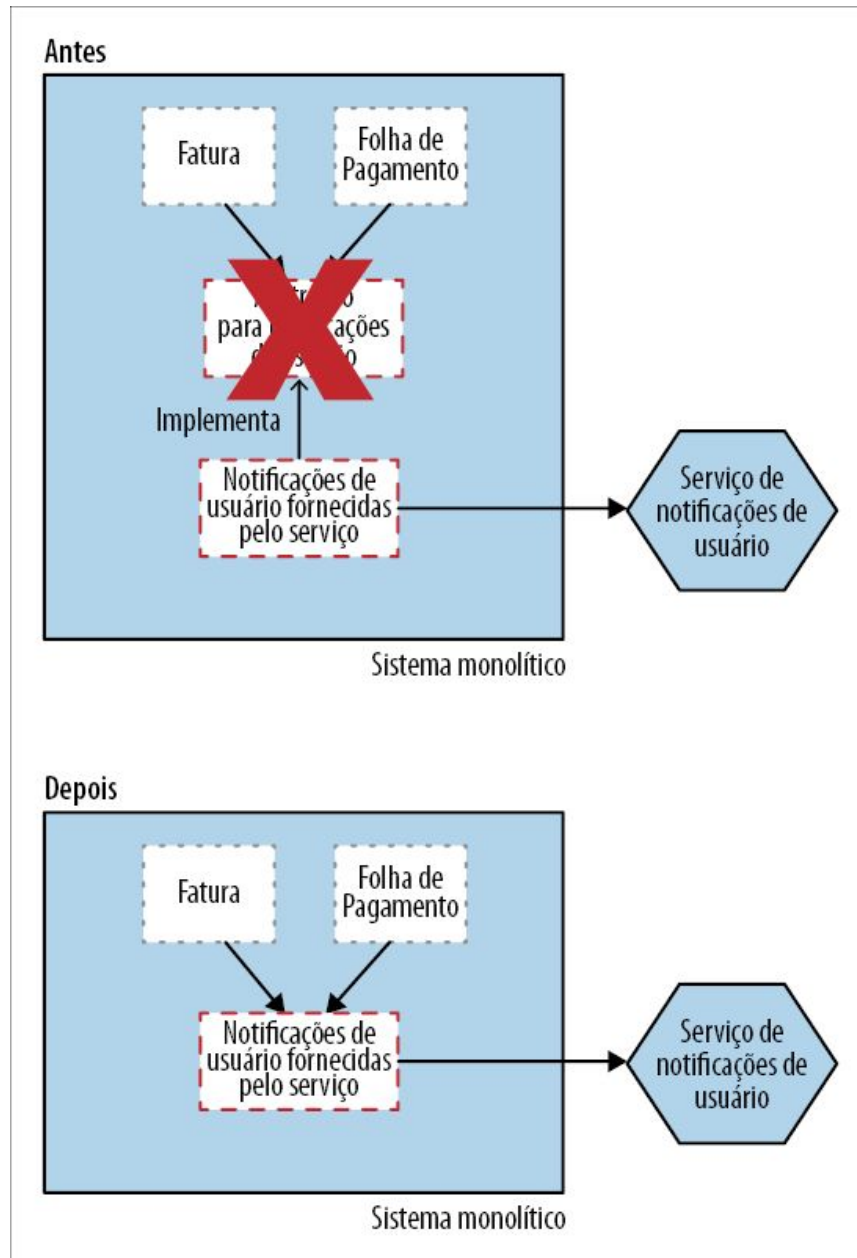


Figura 3.28 – Passo 7: (Opcional) Remova o ponto de abstração.

Como método de fallback

A capacidade de retornar para a implementação anterior caso o

novo serviço não esteja se comportando bem é conveniente; no entanto, seria possível fazer isso automaticamente? Steve Smith detalha uma variação do padrão branch por abstração, chamada branch por abstração com verificação (verify branch by abstraction, <http://bit.ly/2mLVevz>), que implementa também um passo de verificação durante a execução – podemos ver um exemplo disso na Figura 3.29. A ideia é que, se as chamadas à nova implementação falharem, a antiga implementação seja usada em seu lugar.

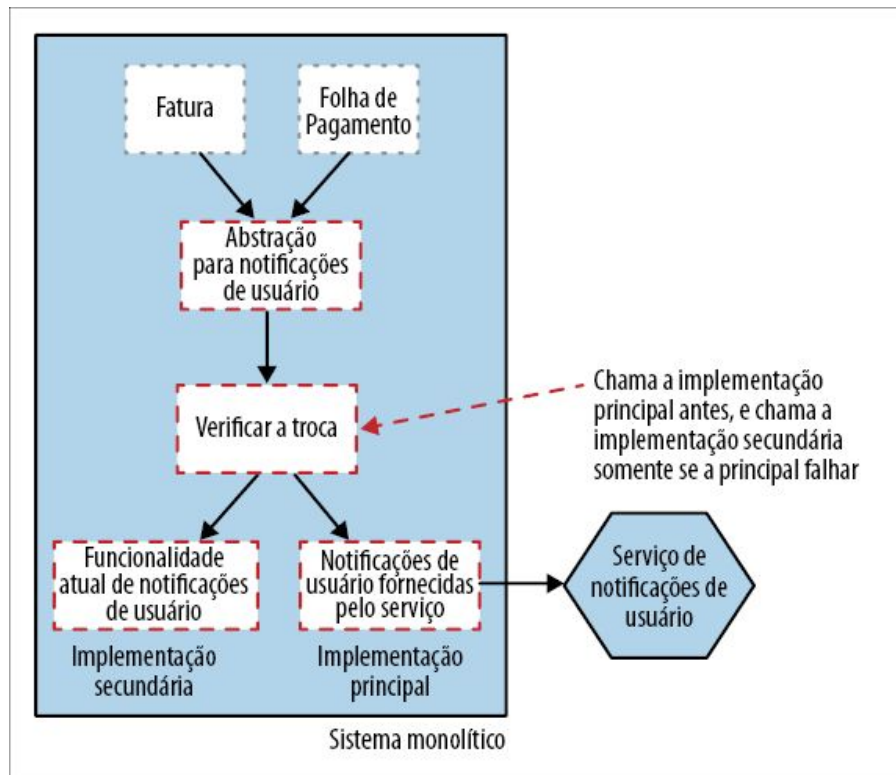


Figura 3.29 – Padrão branch por abstração com verificação.

Sem dúvida, há um acréscimo de complexidade, não só no que concerne ao código, mas também para compreender o sistema. Com efeito, as duas implementações poderão estar ativas em um dado instante, e isso pode dificultar a compreensão do comportamento do sistema. Se as duas implementações tiverem algum tipo de estado (forem stateful), também teremos de considerar a consistência dos dados. Embora a consistência dos dados seja um desafio em qualquer situação em que tenhamos uma

alternância de implementações, o padrão branch por abstração com verificação nos permite alternar entre as implementações a cada requisição, o que significa que você precisará de um conjunto consistente de dados compartilhados, que ambas as implementações possam acessar.

Exploraremos essa ideia com mais detalhes em breve, quando o padrão mais genérico de execução em paralelo for apresentado.

Quando usar o padrão

O branch por abstração é um padrão de propósito geral, útil em qualquer situação na qual mudanças na base de código existente provavelmente demorem para ser feitas, mas você deseja evitar muitas perturbações aos seus colegas. Em minha opinião, essa opção é melhor do que usar branches de código de longa duração, praticamente em qualquer circunstância.

Em uma migração para uma arquitetura de microsserviços, quase sempre procuro usar o padrão strangler fig antes, pois ele é mais simples em vários aspectos. Entretanto, há algumas situações, como no caso das Notificações, em que isso simplesmente não seria possível.

Esse padrão também supõe que você possa modificar o código do sistema atual. Se não puder, por qualquer que seja o motivo, talvez precise buscar outras opções, algumas das quais serão exploradas na parte restante deste capítulo.

Padrão: execução em paralelo

É possível testar sua nova implementação apenas até certo ponto antes de implantá-la. Você fará o melhor que puder para garantir que os testes anteriores ao lançamento de seu novo microsserviço sejam feitos em um ambiente mais parecido possível com o ambiente de produção como parte de um processo rotineiro de testes, mas todos sabemos que nem sempre é possível pensar em todos os cenários que poderão ocorrer em um ambiente de

produção. No entanto, há outras técnicas à nossa disposição.

Tanto o padrão strangler fig como o padrão branch por abstração permitem que uma implementação nova e uma implementação antiga da mesma funcionalidade coexistam no ambiente de produção. Em geral, essas duas técnicas nos permitem executar a implementação antiga do sistema monolítico ou a nova solução baseada em microsserviço. Para atenuar o risco de passar para a nova implementação baseada em serviço, essas técnicas nos permitem retornar rapidamente à implementação antiga.

Ao usar uma execução em paralelo, em vez de chamar a antiga ou a nova implementação, chamamos *ambas*, permitindo que comparemos os resultados a fim de garantir que sejam equivalentes. Apesar de chamar as duas implementações, somente uma será considerada como a fonte da verdade em um dado instante. Geralmente, a implementação antiga será considerada como referência, até que a verificação em andamento comprove que podemos confiar na nova implementação.

Esse padrão tem sido usado de diferentes formas há décadas, embora, em geral, seja utilizado para executar dois sistemas em paralelo. Meu argumento é que esse padrão pode ser igualmente útil em um único sistema, quando comparamos duas implementações da mesma funcionalidade.

Essa técnica pode ser usada para conferir não só se nossa nova implementação fornece as mesmas respostas que a implementação existente, mas também se apresenta parâmetros não funcionais aceitáveis. Por exemplo, nosso novo serviço está respondendo de forma suficientemente rápida? Estamos vendo muitos timeouts ocorrerem?

Exemplo: comparando os cálculos de preços de derivativos de crédito

Muitos anos atrás, estive envolvido em um projeto para mudar a plataforma usada nos cálculos de um tipo de produto financeiro

chamado *derivativos de crédito*. O banco no qual eu trabalhava precisava garantir que os diversos derivativos que oferecia seriam um negócio sensato para eles. Ganharíamos dinheiro nessa transação? Era uma transação arriscada demais? Assim que os derivativos fossem lançados, as condições do mercado também mudariam. Então, eles precisavam avaliar o valor das transações atuais para garantir que não estivessem vulneráveis a grandes perdas à medida que as condições do mercado mudassem.¹⁰

Estávamos substituindo quase totalmente o sistema existente que fazia esses cálculos importantes. Por causa do volume de dinheiro envolvido, e pelo fato de os bônus de algumas pessoas serem, em parte, baseados no valor das negociações feitas, havia muita preocupação em torno das mudanças. Tomamos a decisão de executar os dois conjuntos de cálculos lado a lado e fazer comparações diárias dos resultados. Os cálculos de preços se davam por meio de eventos, que eram fáceis de duplicar, de modo que ambos os sistemas poderiam fazer os cálculos, como vemos na Figura 3.30.

A cada manhã, executávamos um processo batch (em lote) para reconciliação dos resultados, e então precisávamos justificar quaisquer variações nos resultados. Na verdade, nós escrevemos um programa para fazer a reconciliação. Apresentamos os resultados em uma planilha Excel, facilitando discutir as variações com os especialistas do banco.

O fato é houve alguns problemas que tivemos de corrigir, mas também identificamos um número maior de discrepâncias causadas por bugs no sistema existente. Isso significava que alguns dos resultados diferentes, na verdade, estavam corretos, mas tínhamos de mostrar o nosso trabalho (que foi bastante facilitado porque mostramos os resultados no Excel). Eu me lembro de que tive de me sentar com os analistas e explicar-lhes por que nossos resultados estavam corretos, detalhando tudo desde o básico.

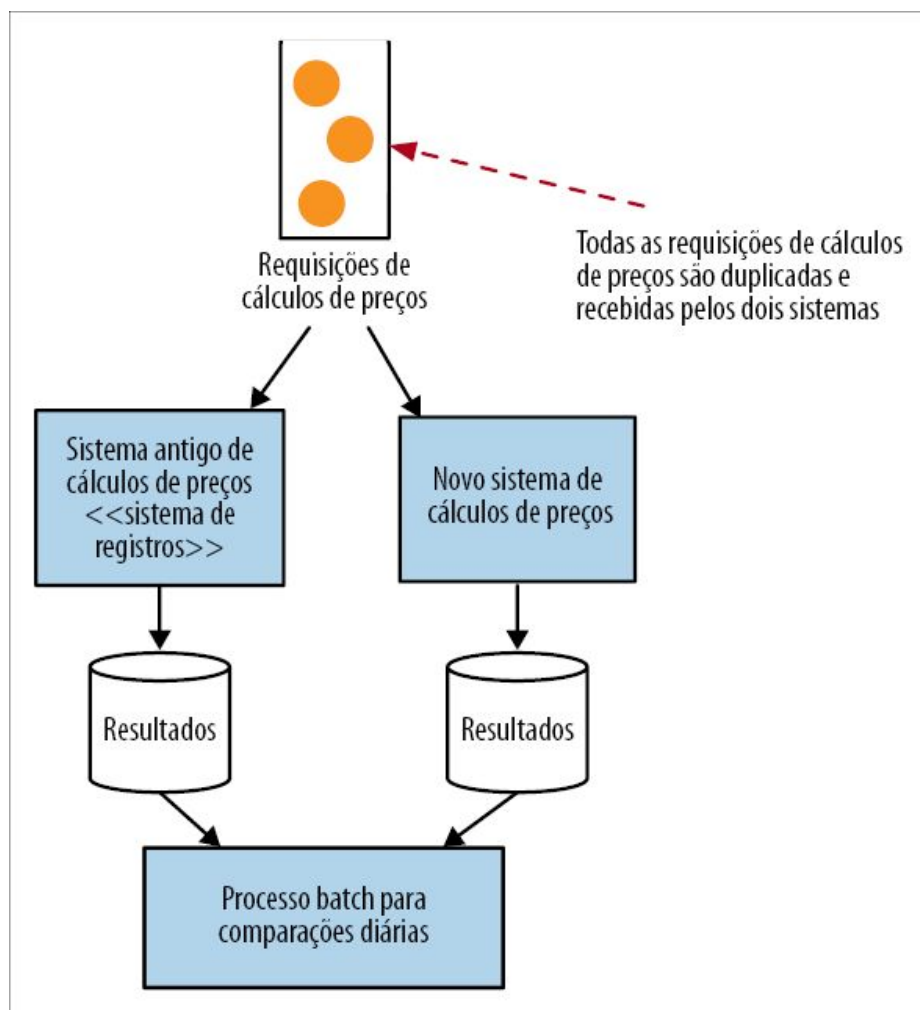


Figura 3.30 – Exemplo de uma execução em paralelo – os dois sistemas de cálculos de preços são chamados, com os resultados comparados offline.

Posteriormente, depois de um mês, passamos a usar nosso sistema como a fonte da verdade para os cálculos, e, depois de um tempo, aposentamos o sistema antigo (nós o mantivemos por mais alguns meses caso houvesse necessidade de alguma auditoria nos cálculos feitos pelo sistema antigo).

Exemplo: geração de listas na Homegate

Conforme discutimos antes na seção Exemplo: FTP, a Homegate executava seus dois sistemas de importação de listas em paralelo, com o novo microserviço que lidava com as importações das listas

sendo comparado com o sistema monolítico existente. Um único upload FTP feito por um cliente fazia com que ambos os sistemas fossem disparados. Assim que confirmaram que o novo microsserviço estava se comportando de forma equivalente, a importação FTP foi desativada no sistema monolítico antigo.

Programação N-versões

Poderíamos argumentar que há uma variação do padrão de execução em paralelo em determinados sistemas de controle nos quais a segurança é crucial, por exemplo, em aeronaves fly-by-wire (sistema de controle por fios elétricos). Em vez de depender de controles mecânicos, as aeronaves contam cada vez mais com sistemas de controle digitais. Quando um piloto utiliza os controles, em vez de puxar cabos para controlar o rudder (leme), uma aeronave fly-by-wire envia dados de entrada para os sistemas de controle, que decidem como devem virar o rudder. Esses sistemas de controle devem interpretar os sinais que recebem e executar a ação apropriada.

Obviamente, um bug nesses sistemas de controle poderia ser extremamente perigoso. Para reduzir o impacto dos defeitos, em algumas situações, várias implementações da mesma funcionalidade são usadas lado a lado. Os sinais são enviados a todas as implementações do mesmo subsistema, as quais enviam, então, as suas respostas. Esses resultados são comparados e o resultado “correto” é selecionado, em geral observando um quórum entre os participantes. Essa é uma técnica conhecida como *programação N-versões* (N-version programming).¹¹

O objetivo dessa abordagem, de modo diferente dos outros padrões que vimos neste capítulo, não é a substituição de nenhuma das implementações. Em vez disso, as implementações alternativas continuarão existindo em paralelo, e espera-se que elas reduzam o impacto de um bug em qualquer dado subsistema.

Técnicas de verificação

Com uma execução em paralelo, queremos verificar a equivalência funcional de duas implementações. Se tomarmos o exemplo anterior dos preços de derivativos de crédito, podemos tratar as duas versões como funções – dadas as mesmas entradas, esperamos as mesmas saídas. No entanto, também podemos (e devemos) validar os aspectos não funcionais. Chamadas que cruzem fronteiras de rede podem introduzir uma latência significativa e podem ser a causa de requisições perdidas como consequência de timeouts, partições e problemas desse tipo. Assim, nosso processo de verificação também deve ser estendido de modo a garantir que as chamadas ao novo microsserviço se completem em um tempo adequado, com uma taxa de falhas aceitável.

Usando Spies

No caso de nosso exemplo anterior com as notificações, poderíamos querer enviar um email para o nosso cliente duas vezes. Nesse caso, um Spy (Espião) poderia ser conveniente. O Spy é um padrão dos testes de unidade e pode tomar o lugar de uma funcionalidade, permitindo que verifiquemos posteriormente se determinadas tarefas foram executadas. O Spy toma o lugar de uma funcionalidade, substituindo-a e servindo como um stub.

Desse modo, em nossa funcionalidade de Notificação, poderíamos usar um Spy para substituir o código de baixo nível usado para enviar o email, como mostra a Figura 3.31. Nosso novo serviço de Notificações usaria então esse Spy durante a fase de execução paralela para que possamos verificar se haveria esse efeito colateral (o envio do email) quando a chamada a `sendNotification` fosse recebida pelo serviço.

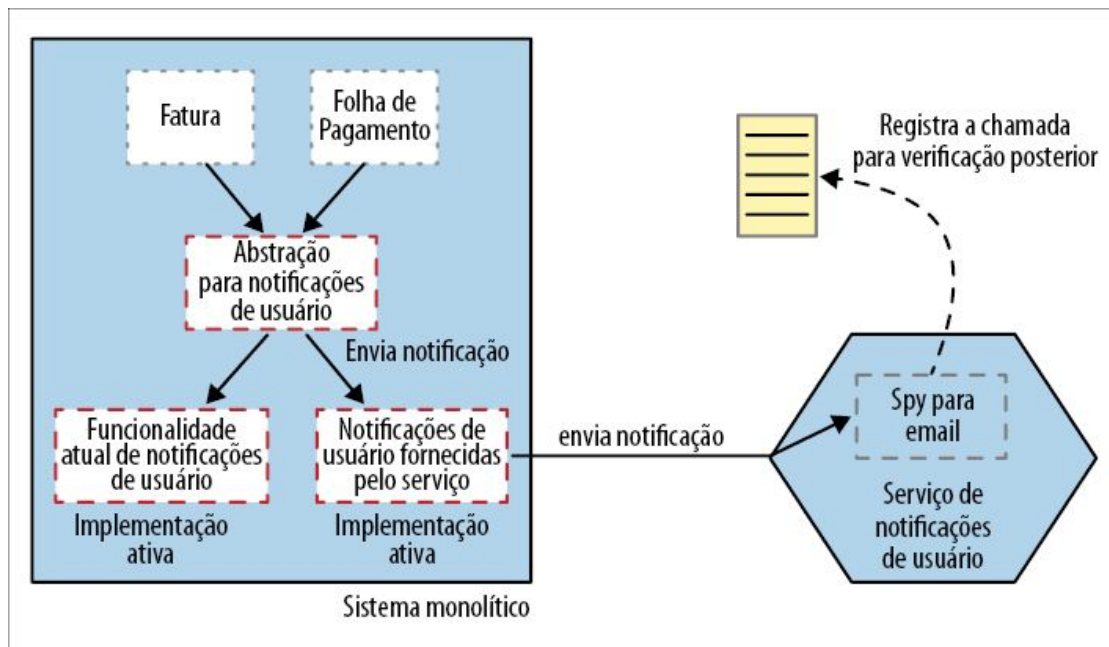


Figura 3.31 – Usando um Spy para verificar se emails seriam enviados durante uma execução em paralelo.

Observe que poderíamos ter decidido usar o Spy dentro do sistema monolítico e evitar que nosso código de RemoteNotification sequer fizesse uma chamada de serviço. No entanto, provavelmente não é isso que você deseja fazer, pois, na verdade, você vai querer levar em consideração o impacto de fazer chamadas remotas – para saber se timeouts, falhas ou a latência em geral para o nosso novo serviço de Notificações estão causando problemas.

Uma complexidade adicional nesse caso é o fato de o nosso Spy executar em um processo separado, o que causaria uma complicação quanto ao momento em que o processo de verificação poderia ser executado. Se quiséssemos que essa atividade ocorresse em tempo real, no escopo da requisição original, é provável que teríamos de expor métodos no serviço de Notificações para permitir que a verificação fosse feita após a chamada inicial ter sido feita para o nosso serviço de Notificações. Isso poderia implicar um volume significativo de trabalho e, em muitos casos, uma verificação em tempo real não será necessária. Um modelo mais provável para verificação de Spies fora do processo seria registrar

as interações para permitir que as verificações sejam feitas separadamente – talvez uma vez ao dia. É claro que, se usamos o Spy para substituir a chamada ao serviço de Notificações, a verificação será mais fácil, mas estaremos fazendo menos testes! Quanto mais funcionalidades você substituir por um Spy, menos funcionalidades serão testadas.

Scientist do GitHub

A biblioteca Scientist do GitHub (<https://github.com/github/scientist>) é uma biblioteca interessante, que ajuda a implementar esse padrão no nível de código. É uma biblioteca Ruby que permite executar implementações lado a lado e capturar informações sobre a nova implementação para ajudar você a saber se ela está funcionando de forma correta. Não a usei pessoalmente, mas posso perceber que ter uma biblioteca como essa realmente poderia ajudar nos testes de sua nova funcionalidade com base em microsserviço quando comparada com o sistema atual – atualmente, há opções para várias linguagens, incluindo Java, .NET, Python, Node.JS e muito mais.

Lançamento no escuro e versão canário

Vale a pena enfatizar que uma execução em paralelo é diferente daquilo que tradicionalmente chamamos de *versão canário* (canary releasing). Uma versão canário envolve direcionar um subconjunto de seus usuários para a nova funcionalidade, mas a maior parte de seus usuários continuará vendo a implementação antiga. A ideia é que, se o novo sistema apresentar problemas, apenas um subconjunto das requisições sofra as consequências. Em uma execução em paralelo, chamamos as *duas* implementações.

Outra técnica relacionada a esse cenário se chama *lançamento no escuro* (dark launching). Em um lançamento no escuro, implantamos a nova funcionalidade e a testamos, porém a nova funcionalidade estará invisível aos usuários. Assim, uma execução em paralelo é

uma forma de implementar um lançamento no escuro, pois a “nova” funcionalidade, com efeito, estará invisível aos seus usuários, até que você faça a mudança para ativar o sistema.

Lançamento no escuro, execuções em paralelo e versão canário são técnicas que podem ser usadas para conferir se nossa nova funcionalidade está correta, e reduzem o impacto caso haja falhas. Todas essas técnicas podem ser incluídas naquilo que chamamos de *entrega progressiva* (progressive delivery) – um termo abrangente, cunhado por James Governor (<http://bit.ly/2lZjrxK>), para descrever métodos que ajudam a controlar o modo como o rollout de um software é feito aos usuários, possibilitando mais nuances e permitindo que você lance um software com mais rapidez, ao mesmo tempo que valida a sua eficácia e reduz o impacto dos problemas caso venham a acontecer.

Quando usar o padrão

Implementar uma execução em paralelo raramente é uma tarefa trivial e, em geral, é reservada aos casos em que a funcionalidade modificada é considerada de alto risco. Analisaremos um exemplo desse padrão sendo utilizado em registros médicos no Capítulo 4. Sem dúvida, eu seria muito seletivo quanto ao uso desse padrão – o custo-benefício do trabalho para implementá-lo deve ser cuidadosamente avaliado. Pessoalmente, utilizei esse padrão apenas uma ou duas vezes, mas, nessas situações, ele foi extremamente útil.

Padrão: colaborador decorador

O que acontece se você quer acionar um comportamento com base em algo que aconteça no sistema monolítico, mas é incapaz de mudar esse sistema? O padrão *colaborador decorador* (decorating collaborator) pode ajudar muito nesse caso. O padrão decorador (decorator), amplamente conhecido, permite vincular uma nova funcionalidade a um código sem que esse código subjacente saiba

de nada. Usaremos um decorador para fazer parecer que nosso sistema monolítico faça chamadas diretamente aos nossos serviços, mesmo que não tenhamos realmente modificado o sistema monolítico subjacente.

Em vez de interceptar essas chamadas antes que alcancem o sistema monolítico, permitimos que elas prossigam do modo usual. Em seguida, com base no resultado dessas chamadas, podemos chamar nossos microsserviços externos por meio de qualquer método de colaboração que tenhamos escolhido. Vamos explicar essa ideia de forma detalhada usando um exemplo com a Music Corp.

Exemplo: programa de fidelidade

A Music Corp está totalmente voltada para seus clientes! Queremos que eles possam ganhar pontos com base nos pedidos feitos, mas nossa funcionalidade atual de pedidos é complexa demais e preferimos que ela não seja modificada no momento. Assim, a funcionalidade de pedidos permanecerá no sistema monolítico atual, porém usaremos um proxy para interceptar essas chamadas e, com base no resultado, decidiremos quantos pontos serão concedidos aos clientes, como mostra a Figura 3.32.

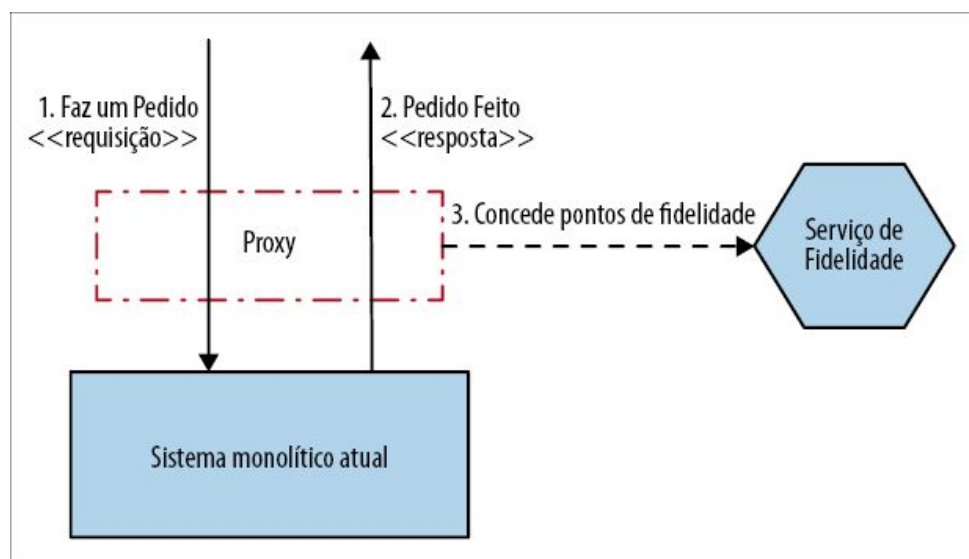


Figura 3.32 – Quando um pedido é feito com sucesso, nosso proxy

chamará o serviço de Fidelidade para conceder pontos ao cliente.

Com o padrão strangler fig, o proxy era razoavelmente simples. Agora, nosso proxy precisa incorporar muito mais “inteligência”. Ele deve fazer suas próprias chamadas para o novo microserviço e deixar passar as respostas de volta para o cliente. Como antes, preste atenção na complexidade incluída no proxy. Quanto mais código você começar a adicionar aí, mais ele se tornará um microserviço por si só, ainda que seja um microserviço técnico, com todos os desafios que discutimos anteriormente.

Outro desafio em potencial é que precisamos de informações suficientes da requisição de entrada para que possamos fazer a chamada para o microserviço. Por exemplo, se quisermos conceder pontos com base no valor do pedido, mas esse valor não estiver claro na requisição Fazer Pedido, e nem na resposta, será necessário buscar informações adicionais – talvez chamando o sistema monolítico para extrair as informações necessárias, como vemos na Figura 3.33.

Considerando que essa chamada poderia gerar uma carga adicional e, sem dúvida, introduz uma dependência circular, talvez seja melhor modificar o sistema monolítico para que forneça as informações necessárias quando a solicitação do pedido estiver concluída. Isso, porém, exigiria alterar o código do sistema monolítico ou, talvez, utilizar uma técnica mais invasiva, por exemplo, a captura de dados modificados (change data capture).

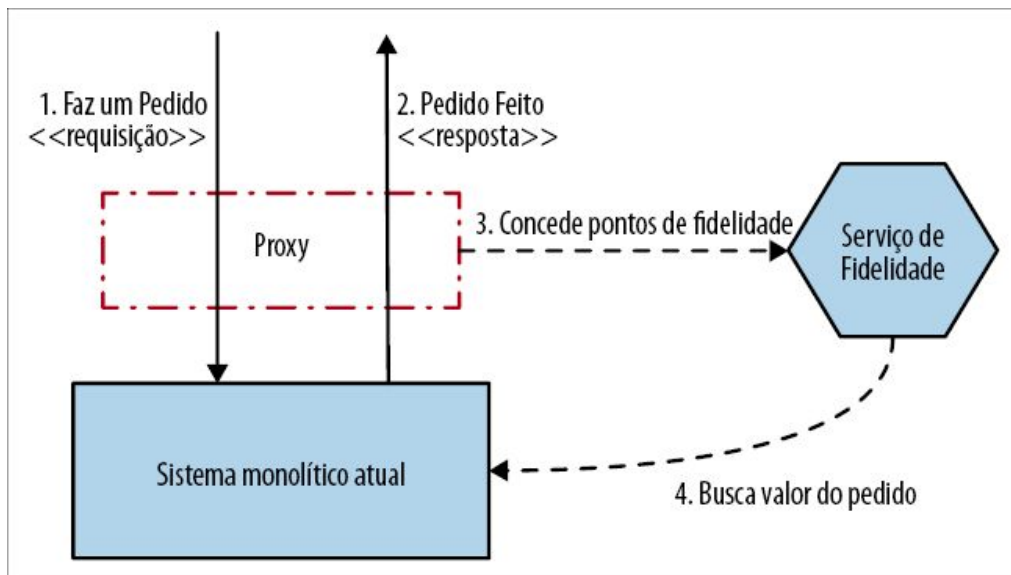


Figura 3.33 – Nosso serviço de Fidelidade precisa carregar detalhes adicionais sobre o pedido para descobrir quantos pontos serão concedidos ao cliente.

Quando usar o padrão

Quando é implementada de forma simples, esta é uma abordagem mais elegante e menos acoplada do que a captura de dados modificados. Esse padrão funciona melhor quando as informações necessárias podem ser extraídas da requisição de entrada ou da resposta enviada de volta ao sistema monolítico. Se mais informações forem necessárias para que as chamadas corretas sejam feitas ao novo serviço, mais complexa e confusa será essa implementação. Intuitivamente, acho que, se a requisição e a resposta para o sistema monolítico não contiverem as informações necessárias, você deverá pensar melhor antes de usar esse padrão.

Padrão: captura de dados modificados

Com a *captura de dados modificados* (change data capture), em vez de tentar interceptar e atuar nas chamadas feitas ao sistema monolítico, reagimos a mudanças feitas em um banco de dados. Para que a captura de dados modificados funcione, o sistema de captura subjacente deve estar acoplado ao banco de dados do

sistema monolítico. Esse é um desafio realmente inevitável no caso desse padrão.

Exemplo: gerando cartões de fidelidade.

Queremos integrar uma funcionalidade para imprimir cartões de fidelidade aos nossos usuários quando eles se cadastrarem. No momento, uma conta de fidelidade é criada quando um cliente se cadastra. Conforme vemos na Figura 3.34, quando o registro é devolvido pelo sistema monolítico, sabemos apenas que o cliente foi cadastrado com sucesso. Para que possamos imprimir um cartão, precisamos de mais detalhes sobre o cliente. Isso faz com que inserir esse comportamento upstream, talvez usando um colaborador decorador, seja mais difícil – no ponto em que a chamada retorna, teríamos de fazer consultas adicionais ao sistema monolítico a fim de extrair as demais informações de que precisamos, e essas informações poderiam ou não ser expostas por meio de uma API.

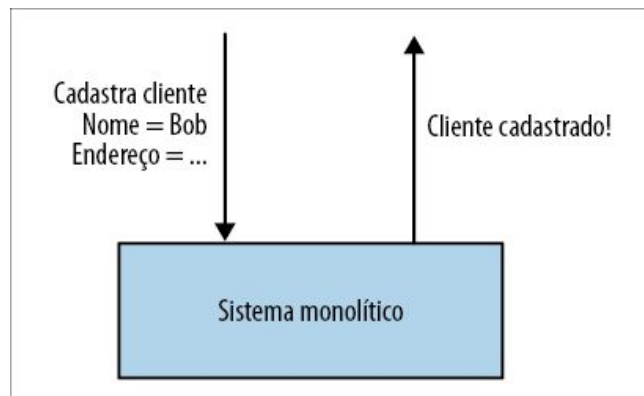


Figura 3.34 – Nosso sistema monolítico não compartilha muitas informações quando um cliente é cadastrado.

Em vez disso, decidimos usar uma captura de dados modificados. Detectamos qualquer inserção na tabela `LoyaltyAccount` e, quando isso ocorre, fazemos uma chamada ao nosso novo serviço de Impressão de Cartão de Fidelidade, como vemos na Figura 3.35. Nessa situação em particular, decidimos disparar um evento `Conta de Fidelidade Criada`. Nosso processo de impressão funciona

melhor em lote, portanto, isso permite criar uma lista das impressões que devem ser feitas em nosso broker de mensagens.

Implementando a captura de dados modificados

Poderíamos usar diversas técnicas para implementar uma captura de dados modificados, todas com relações de custo-benefício distintas no que concerne à complexidade, confiabilidade e ocasião. Vamos analisar duas opções.

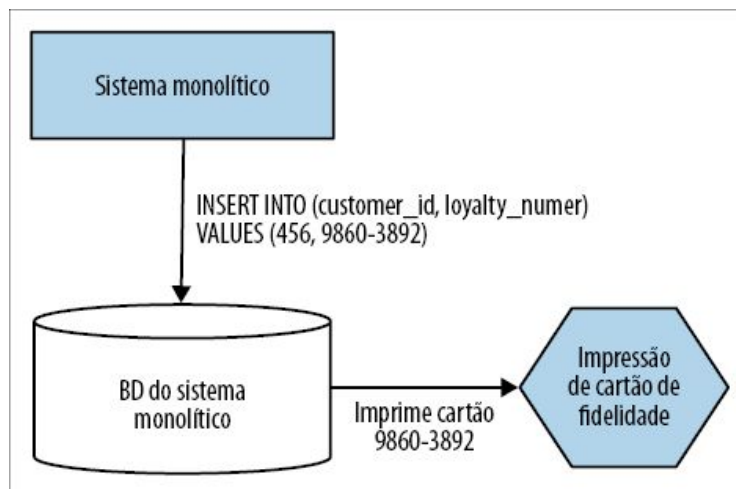


Figura 3.35 – Como uma captura de dados modificados poderia ser usada para chamar nosso novo serviço de impressão.

Triggers de banco de dados

A maioria dos bancos de dados relacionais permite acionar um comportamento específico quando os dados são alterados. Exatamente como esses triggers são definidos e o que podem disparar variam, mas todos os bancos de dados relacionais modernos os aceitam de uma ou outra maneira. No exemplo da Figura 3.36, nosso serviço é chamado sempre que um INSERT é feito na tabela LoyaltyAccount.

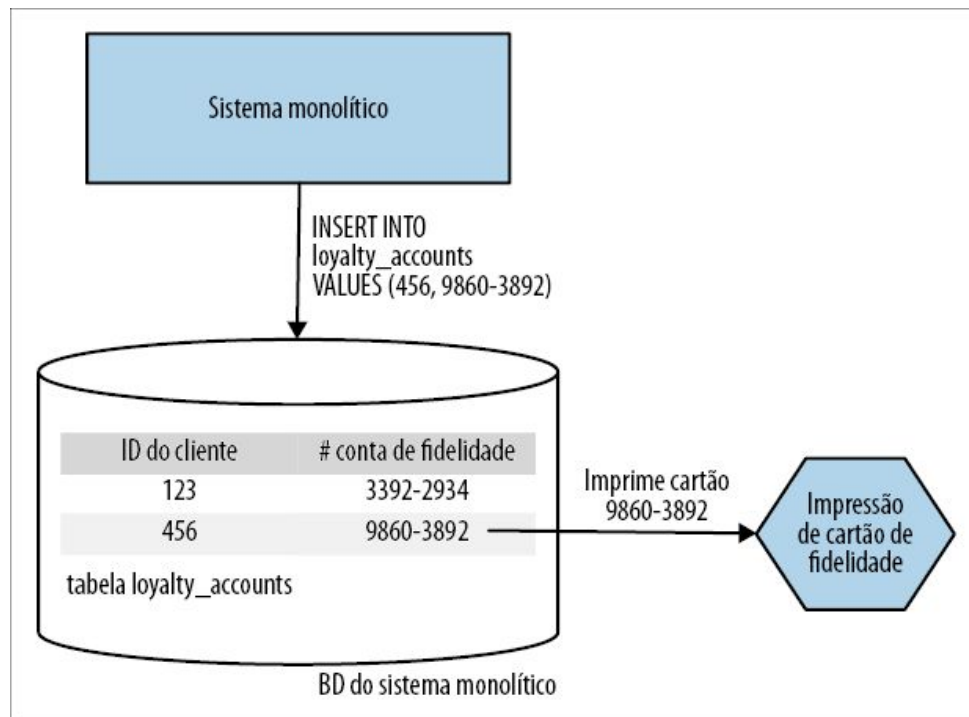


Figura 3.36 – Usando um trigger de banco de dados para chamar nosso microsserviço quando houver inserção de dados.

Os triggers devem ser instalados no próprio banco de dados, assim como qualquer outro procedimento armazenado (stored procedure). Também pode haver limitações quanto ao que esses triggers podem fazer, embora, ao menos no Oracle, não haja problemas com o fato de você chamar web services ou um código Java personalizado com esses triggers.

À primeira vista, parece ser uma tarefa muito simples. Não há necessidade de ter outros softwares executando nem de introduzir qualquer nova tecnologia. No entanto, como no caso dos procedimentos armazenados, os triggers de banco de dados podem ser uma montanha traiçoeira.

Certa vez, um amigo meu, Randy Shoup, disse algo nesta linha: “Ter um ou dois triggers de banco de dados não é ruim. Construir um sistema todo com base neles é uma péssima ideia”. Com frequência, esse é um problema associado aos triggers de banco de dados. Quanto mais triggers você tiver, mais difícil será entender como seu sistema realmente funciona. O problema muitas vezes

está nas ferramentas e no gerenciamento das mudanças dos triggers de banco de dados – utilize triggers demais, e sua aplicação poderá se tornar uma espécie de edifício barroco.

Portanto, se você pretende usá-los, use-os com *muita* moderação.

Polling de logs de transação

Na maioria dos bancos de dados, certamente em todos os principais bancos de dados transacionais, há um log de transações. Em geral, é um arquivo, no qual são gravados os registros de todas as mudanças efetuadas. Em uma captura de dados modificados, as ferramentas mais sofisticadas tendem a fazer uso desse log de transações.

Esses sistemas executam como um processo separado, e sua única interação com o banco de dados existente se dá por meio desse log de transações, como vemos na Figura 3.37. Nesse caso, vale a pena observar que apenas transações cujo commit tenha sido efetuado aparecerão no log de transações (o que é, de certo modo, a sua razão de ser).

Essas ferramentas exigem uma compreensão do formato do log de transações subjacente e, geralmente, esse formato varia entre os diferentes tipos de bancos de dados. Desse modo, exatamente quais ferramentas você terá à disposição nesse cenário dependerá do banco de dados em uso. Há um conjunto enorme de ferramentas nessa área, embora muitas delas sejam usadas para suporte à replicação de dados. Há também diversas soluções criadas para mapear mudanças no log de transações a mensagens a serem enviadas para um broker de mensagens; esse recurso poderia ser muito conveniente se o nosso microsserviço tivesse uma natureza assíncrona.

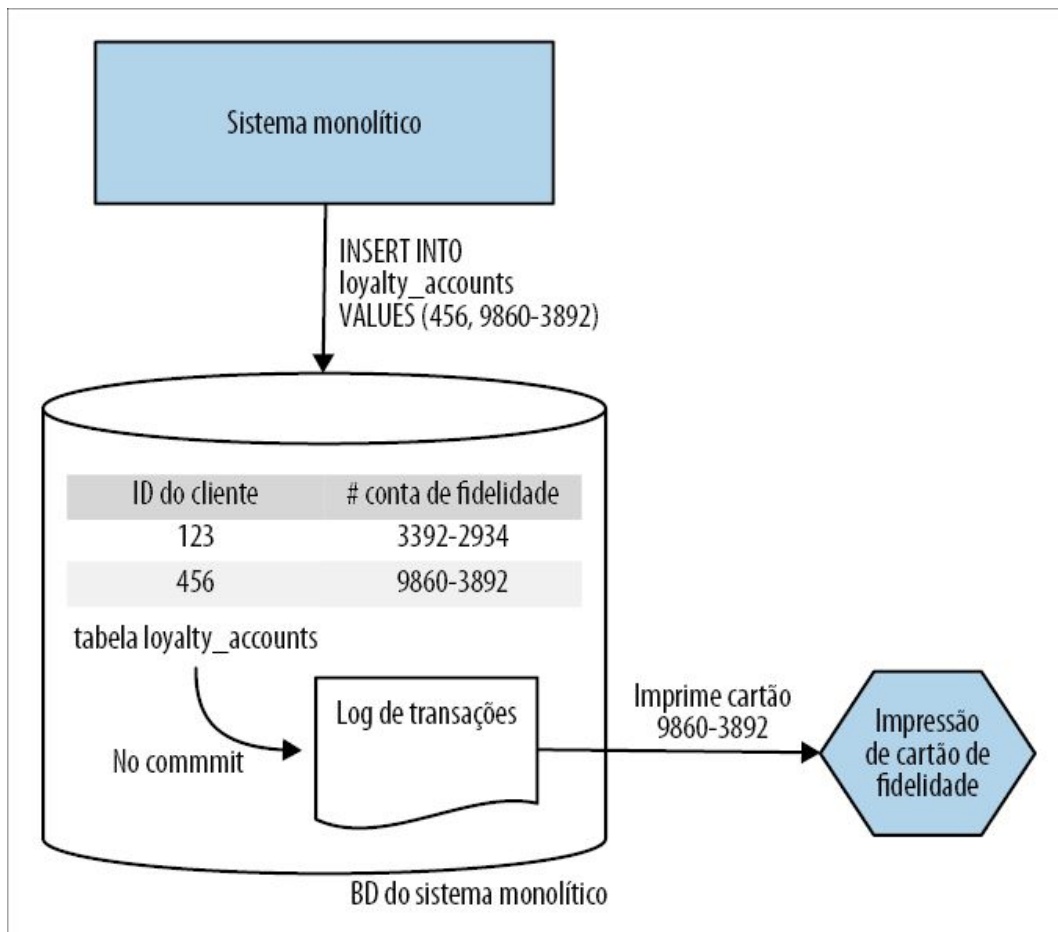


Figura 3.37 – Um sistema de captura de dados modificados fazendo uso do log de transações subjacente.

Apesar das restrições, em vários aspectos, essa é a solução mais organizada para implementar uma captura de dados modificados. O próprio log de transações mostra somente as mudanças nos dados subjacentes, portanto, você não terá de se preocupar com o que mudou. As ferramentas executam fora do banco de dados propriamente dito, e poderão executar a partir de uma réplica do log de transações, portanto, você terá menos preocupações com acoplamentos ou contenções.

Cópia da diferença com ferramentas batch

Provavelmente, a abordagem mais simples consiste em escrever um programa que, regularmente, verifique o banco de dados em questão para saber quais dados mudaram e copie esses dados para

o destino. Essas tarefas em geral são executadas com ferramentas como o cron ou outras ferramentas batch semelhantes de agendamento.

O principal problema é descobrir quais dados realmente mudaram desde a última execução da cópia em batch. O design do esquema pode ou não deixar essa informação óbvia. Alguns bancos de dados permitem que você veja os metadados das tabelas para ver quais partes do banco de dados mudaram, mas isso está longe de ser universal, e podem fornecer timestamps das mudanças somente no nível das tabelas, quando seria preferível que essas informações estivessem disponíveis no nível de linhas. Você poderia começar a acrescentar timestamps por conta própria, mas isso poderia representar um aumento significativo de trabalho, e um sistema de captura de dados modificados resolveria esse problema de modo mais elegante.

Quando usar o padrão

A captura de dados modificados é um padrão de propósito geral muito útil, particularmente se você precisa replicar dados (assunto que será mais bem explorado no Capítulo 4). No caso da migração para microsserviços, a ocasião ideal é aquela em que você precisa reagir a uma mudança de dados em seu sistema monolítico, mas é incapaz de interceptar essa mudança no perímetro do sistema usando um strangler ou um decorador, e não é possível modificar a base de código subjacente.

Em geral, tento manter o uso desse padrão em um nível mínimo por causa dos desafios relacionados a algumas de suas implementações. Os triggers de banco de dados têm suas desvantagens, e as ferramentas maduras para captura de dados modificados que funcionam com base em logs de transação podem acrescentar uma complexidade significativa à sua solução. Apesar disso, se você compreende esses possíveis desafios, essa poderá ser uma ferramenta útil para se ter à disposição.

Resumo

Conforme vimos, diversas técnicas permitem uma decomposição gradual das bases de código existentes, e podem ajudar você a entrar no mundo dos microsserviços com mais facilidade. Com base em minha experiência, a maioria das pessoas acaba usando uma combinação de abordagens distintas; é raro que uma única técnica seja apropriada para qualquer situação. Espero que o que eu tenha conseguido fazer até agora seja apresentar diversas abordagens e ter dado informações suficientes para que você determine quais técnicas poderão funcionar melhor em seu caso.

Falamos superficialmente de um dos principais desafios na migração para uma arquitetura de microsserviços – os dados. Não podemos adiar mais! No Capítulo 4, exploraremos como migrar os dados e separar os bancos de dados.

1 N.T.: Edição publicada no Brasil: *Trabalho Eficaz com Código Legado* (Bookman, 2013).

2 N.T.: Uma possível tradução para esse ditado poderia ser “Mantenha o meio de transporte burro, e os endpoints inteligentes”. Um “dumb pipe” refere-se a um meio de transporte de dados neutro, por exemplo, uma rede, que permita uma troca de dados sem que haja diferentes prioridades conforme o conteúdo, isto é, há uma neutralidade em relação a esse conteúdo. Em contrapartida, um “smart pipe” agrega alguma inteligência no transporte de dados, indo além da simples conectividade.

3 Para uma explicação mais completa, veja a postagem “The Road to an Envoy Service Mesh” (O caminho para um service mesh Envoy, <https://squ.re/2nts1Gc>) de Snow Pettersen no blog de desenvolvedores da Square.

4 N.T.: Veja a nota anterior sobre “smart pipe” neste capítulo.

5 Bobby Woolf e Gregor Hohpe, *Enterprise Integration Patterns* (Addison-Wesley, 2003).

6 N.T.: “Endpoints inteligentes, meios de transporte burros” – veja a nota anterior sobre “dumb pipe/smart pipe” neste capítulo.

7 Foi bom ouvir Graham Tackley do *The Guardian* dizer que o “novo” sistema que eu ajudei inicialmente a implementar durou quase dez anos, até ter sido totalmente substituído pela arquitetura atual. Como leitor do site, pude notar que, na verdade, nunca vi nenhuma mudança ser feita nesse período!

8 Veja Steve Hoffman e Rick Fast, “Enabling Microservices at Orbitz” (Viabilizando

microsserviços na Orbitz, <http://bit.ly/2nGNgnI>), YouTube, 11 de agosto de 2016.

9 Veja John Sundell, “Building Component-Driven UIs at Spotify” (Construindo UIs orientadas a componentes no Spotify, <http://bit.ly/2nDpJUP>), publicado em 25 de agosto de 2016.

10 O fato é que éramos *péssimos* nisso como mercado. Recomendo o livro *The Big Short* (W. W. Norton & Company, 2010) de Martin Lewis, que apresenta uma visão geral excelente do papel desempenhado pelos derivativos de crédito na crise financeira global de 2007 a 2008. Sempre olho para trás e vejo o pequeno papel que desempenhei nesse mercado com uma boa dose de arrependimento. A verdade é que não saber o que você está fazendo e fazê-lo de qualquer maneira pode ter implicações bastante desastrosas.

11 Veja Liming Chen e Algirdas Avizienis, “N-Version Programming: A Fault-Tolerance Approach to Reliability of Software Operation” (Programação N-versões: uma abordagem tolerante a falhas para confiabilidade nas operações de software), publicado no Twenty-Fifth International Symposium on Fault-Tolerant Computing (25º Simpósio Internacional de Computação Tolerante a Falhas) (1995).

CAPÍTULO 4

Decompondo o banco de dados

Conforme já exploramos, há inúmeras maneiras de extrair funcionalidades e colocá-las em microsserviços. No entanto, é preciso encarar o elefante que está na sala, ou seja, o que faremos a respeito dos dados? Os microsserviços funcionam melhor quando trabalhamos ocultando informações, o que, por sua vez, nos leva geralmente em direção a microsserviços que encapsulem totalmente suas próprias armazenagens de dados e os métodos para acessá-los. Isso nos leva a concluir que, ao migrarmos para uma arquitetura de microsserviços, devemos separar o banco de dados de nosso sistema monolítico se quisermos tirar o máximo proveito da transição.

Separar um banco de dados, porém, está longe de ser um empreendimento simples. Devemos considerar os problemas de sincronização de dados durante a transição, a decomposição lógica *versus* a decomposição física dos esquemas, a integridade das transações, as junções, a latência e outros problemas. Neste capítulo, veremos esses problemas e exploraremos padrões que poderão nos ajudar.

Antes de iniciar a separação de qualquer dado, porém, vamos analisar os desafios de gerenciar um único banco de dados compartilhado – e os padrões para enfrentá-los.

Padrão: banco de dados compartilhado

Conforme discutimos no Capítulo 1, podemos pensar no acoplamento como acoplamento de domínios, acoplamento

temporal ou acoplamento de implementação. Dos três tipos, é o acoplamento de implementação que muitas vezes nos deixa mais ocupados quando consideramos os bancos de dados, por haver inúmeras pessoas que compartilham um banco de dados com vários esquemas, conforme vemos na Figura 4.1.

À primeira vista, há uma série de problemas relacionados ao compartilhamento de um único banco de dados entre vários serviços. O principal problema, porém, é que não nos damos a oportunidade de decidir o que será compartilhado e o que será oculto – o que vai contra o nosso movimento em direção à ocultação de informações. Isso significa que pode ser difícil entender quais partes de um esquema poderão ser alteradas com segurança. Saber que um componente externo pode acessar seu banco de dados é um fato, mas não saber qual parte de seu esquema ele utiliza é uma história totalmente diferente. O problema pode ser atenuado com o uso de visões (views), que serão discutidas em breve, mas não é uma solução completa.

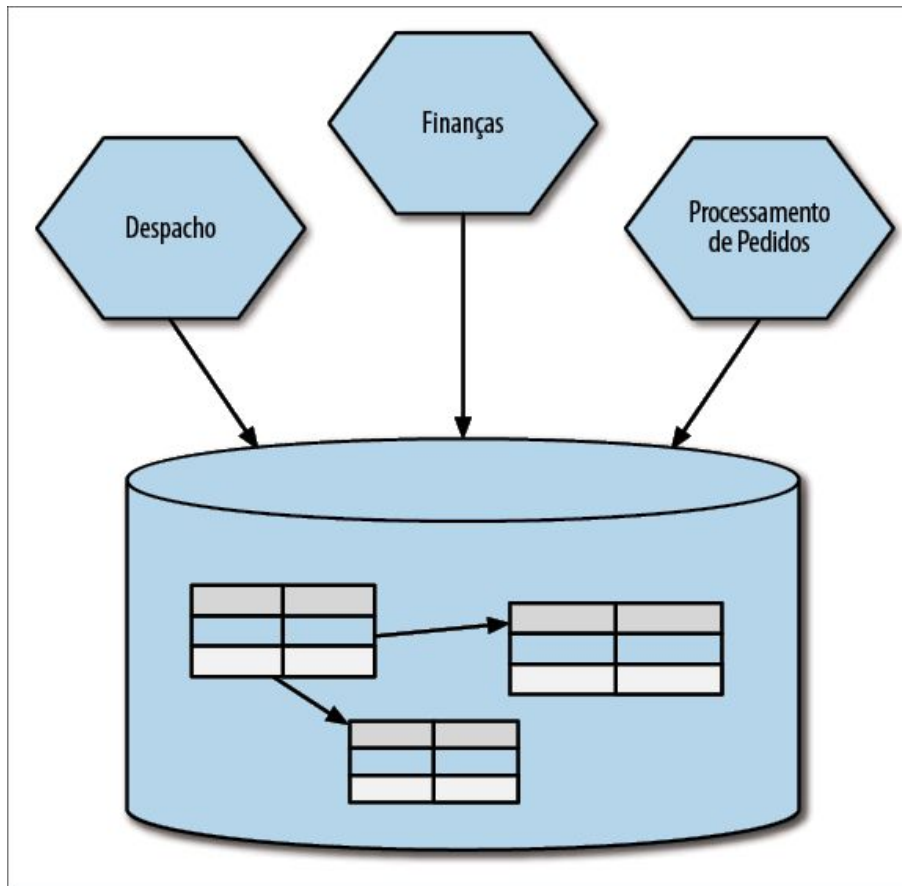


Figura 4.1 – Vários serviços, todos acessando diretamente o mesmo banco de dados.

Outro problema é o fato de não estar claro quem “controla” os dados. Onde está a lógica de negócios que manipula esses dados? Ela está agora espalhada pelos serviços? Por sua vez, isso implica uma falta de coesão na lógica de negócios. Conforme já foi discutido, pensamos em um microsserviço como uma combinação de comportamentos e estados, encapsulando uma ou mais máquinas de estado. Se o comportamento que modifica esse estado estiver agora espalhado pelo sistema, garantir que essa máquina de estados seja devidamente implementada é uma questão complicada.

Como vemos na Figura 4.1, se três serviços puderem modificar diretamente as informações de um pedido, o que acontecerá se esse comportamento for inconsistente entre os serviços? O que acontecerá quando esse comportamento tiver de mudar – terei de

aplicar essas mudanças em todos os serviços? Conforme falamos antes, nosso objetivo é ter um alto nível de coesão nas funcionalidades de negócio, e, com muita frequência, um banco de dados compartilhado implica o oposto.

Padrões para lidar com o problema

Embora possa ser uma tarefa difícil, é quase sempre preferível separar o banco de dados para permitir que cada microsserviço seja dono dos próprios dados. Se isso não for possível, o uso do padrão de visão de banco de dados (database view – veja a seção Padrão: visão de banco de dados) ou a adoção do padrão de serviço de encapsulamento de banco de dados (database wrapping service – veja a seção Padrão: serviço de encapsulamento de banco de dados) poderão ajudar.

Quando usar o padrão

Acho que compartilhar diretamente um banco de dados será apropriado em uma arquitetura de microsserviços somente em duas situações. A primeira é quando consideramos dados de referência estáticos somente de leitura. Exploraremos esse assunto com mais detalhes em breve, mas considere um esquema que armazene informações de códigos das moedas dos países, tabelas para consultas de códigos postais ou CEPs e dados desse tipo. Nesse caso, a estrutura de dados é extremamente estável e o controle de alteração desses dados em geral será uma tarefa de responsabilidade da área de administração.

A outra situação na qual eu acho que vários serviços acessando diretamente o mesmo banco de dados pode ser apropriado é quando um serviço expõe diretamente um banco de dados como um endpoint definido, que foi criado e é administrado com o intuito de lidar com vários consumidores. Apresentaremos essa ideia depois, quando discutirmos o padrão de interface de banco de dados como serviço (veja a seção Padrão: interface de banco de dados como

serviço).

Mas isso não é possível!

Assim, o ideal é querer que nossos serviços tenham os próprios esquemas independentes. Porém, esse não é o nosso ponto de partida com um sistema monolítico existente. Isso significa que devemos sempre separar esses esquemas? Continuo convencido de que, na maioria das situações, essa é a solução apropriada, mas ela nem sempre será factível no início.

Às vezes, conforme exploraremos em breve, o trabalho envolvido exigirá muito tempo, ou implica mudanças em partes particularmente sensíveis do sistema. Em casos como esses, pode ser conveniente usar uma série de padrões para lidar com a situação, os quais vão, no mínimo, evitar que a situação piore e, no melhor caso, poderão ser passos intermediários razoáveis em direção a algo melhor no futuro.

Esquemas e bancos de dados

Sou culpado por ter usado termos como “banco de dados” e “esquema” indistintamente no passado. Às vezes, isso pode causar confusão, pois há certa ambiguidade nesses termos. Tecnicamente, podemos considerar um esquema como um conjunto de tabelas que armazenam dados, separadas do ponto de vista lógico, como mostra a Figura 4.2. Vários esquemas podem estar hospedados em uma única engine de banco de dados. Conforme o seu contexto, quando as pessoas disserem “banco de dados”, elas poderiam estar se referindo ao esquema ou à engine do banco de dados (“O banco de dados caiu!”).

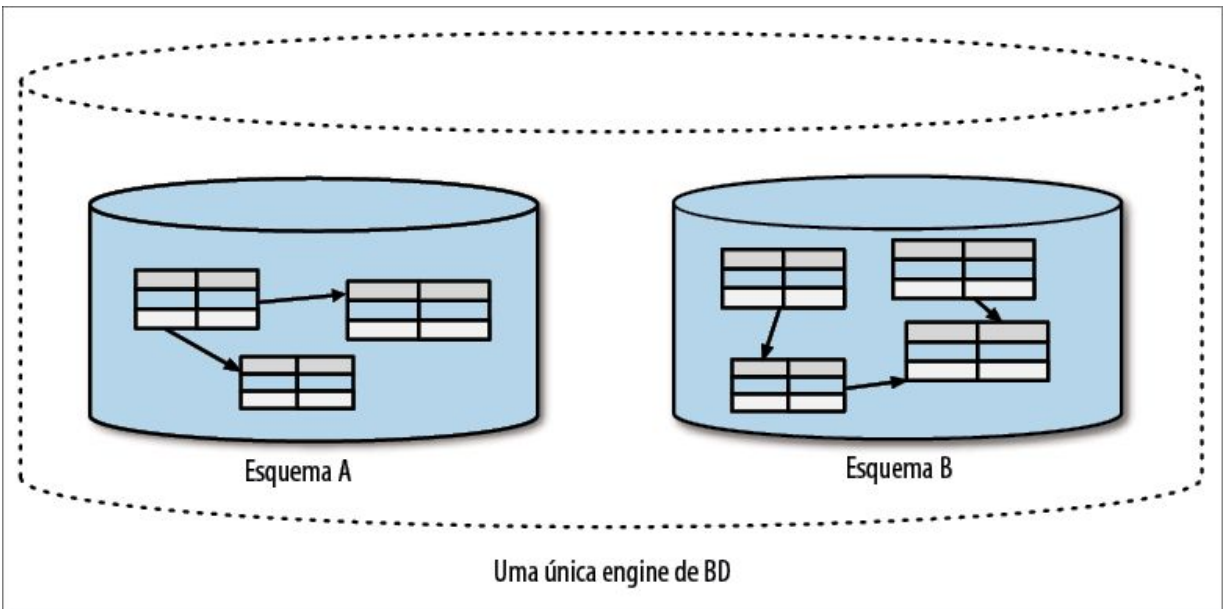


Figura 4.2 – Uma única instância de uma engine de banco de dados pode hospedar vários esquemas, cada um proporcionando um isolamento do ponto de vista lógico.

Como este capítulo, em sua maior parte, terá como foco os conceitos lógicos de banco de dados, e como o termo “banco de dados” é geralmente usado nesse contexto para se referir a um esquema isolado do ponto de vista lógico, mantereire esse uso neste capítulo. Portanto, quando eu disser “banco de dados”, você poderá pensar em um “esquema isolado do ponto de vista lógico”. Para ser mais conciso, omitirei o conceito de engine de banco de dados de nossos diagramas, a menos que seja explicitamente necessário.

Vale a pena mencionar que os diversos bancos de dados NoSQL por aí podem ou não ter o mesmo conceito de separação lógica, particularmente quando lidamos com bancos de dados disponibilizados por provedores de nuvem. Por exemplo, no AWS, o DynamoDB tem apenas o conceito de tabela, utilizando controles de acesso baseados em perfis (role-based access controls) para limitar quem pode ver ou modificar os dados. Isso pode implicar desafios no modo como pensamos na separação lógica, em situações como essas.



Você vai deparar com problemas em seu sistema atual com os quais parecerá impossível lidar no momento. Discuta o problema com os demais membros de sua equipe para que todos possam chegar a um consenso de que esse é um problema que você gostaria de resolver, mesmo que, no momento, não seja possível ver como. Em seguida, certifique-se de que, ao menos, você começará a fazer o que é certo *agora*. Com o tempo, será mais fácil lidar com os problemas que, a princípio, pareciam insuperáveis, quando você tiver novas habilidades e mais experiência.

Padrão: visão de banco de dados

Em uma situação na qual queremos uma única fonte de dados para vários serviços, uma visão (view) pode ser usada para diminuir as preocupações no que concerne ao acoplamento. Com uma visão, um serviço pode ver um esquema que seja uma projeção limitada de um esquema subjacente. Essa projeção pode limitar os dados visíveis ao serviço, ocultando informações às quais ele não deva ter acesso.

Banco de dados como um contrato público

No Capítulo 3, falei de minhas experiências em ajudar a redefinir a plataforma de um sistema de derivativos de crédito para um banco de investimentos que já não existe mais. Nós deparamos com o problema de acoplamento de banco de dados em grande estilo: tínhamos de aumentar o throughput do sistema para prover feedbacks mais rápidos aos corretores que usavam o sistema. Depois de uma análise, percebemos que o gargalo no processamento eram as escritas feitas em nosso banco de dados. Depois de uma investigação rápida, percebemos que era possível melhorar bastante o desempenho das escritas em nosso sistema se reestruturássemos o esquema.

Foi nesse instante que descobrimos que várias aplicações que não estavam sujeitas ao nosso controle tinham acesso de leitura ao nosso banco de dados e, em alguns casos, acesso de leitura/escrita. Infelizmente, descobrimos que todos esses sistemas

externos haviam recebido as mesmas credenciais com nome de usuário e senha, portanto era impossível saber quem eram esses usuários ou o que eles estavam acessando. Havíamos estimado que “mais de 20” aplicações estavam envolvidas, mas era uma informação obtida a partir de uma análise básica das chamadas de entrada de rede.¹



Se cada ator (por exemplo, uma pessoa ou um sistema externo) tiver um conjunto diferente de credenciais, será muito mais fácil restringir o acesso por parte de determinados atores, diminuir o impacto de revogar e trocar as credenciais e entender melhor o que cada ator está fazendo. Gerenciar diferentes conjuntos de credenciais pode ser trabalhoso, sobretudo em um sistema de microsserviços no qual possa haver vários conjuntos de credenciais a serem administrados por serviço. Gosto de usar ferramentas dedicadas de armazenagens de dados confidenciais para resolver esse problema. O Vault da HashiCorp (<https://www.vaultproject.io>) é uma excelente ferramenta nessa área, pois pode gerar credenciais por ator, para componentes como bancos de dados, e elas podem ter uma vida curta e um escopo limitado.

Desse modo, não sabíamos quem eram esses usuários, mas tínhamos de entrar em contato com eles. Em algum momento, alguém teve a ideia de desativar a conta compartilhada que estava sendo usada, e esperar que as pessoas entrassem em contato conosco para reclamar. Obviamente, não era uma ótima solução para um problema que nem sequer deveria existir antes de tudo, mas funcionou – em sua maior parte. Logo percebemos, porém, que a maioria dessas aplicações não estava passando por manutenções ativas, o que significava que não havia chances de que fossem atualizadas para que refletissem um novo design de esquema.² Com efeito, nosso esquema de banco de dados havia se transformado em um contrato público que não podia ser alterado: tínhamos de continuar mantendo aquela estrutura de esquema.

Visões a serem apresentadas

Nossa solução foi resolver primeiro aquelas situações nas quais havia sistemas externos escrevendo em nosso esquema. Felizmente, em nosso caso, eram fáceis de resolver. Para todos os

clientes que queriam ler os dados, porém, criamos um esquema dedicado que hospedava visões que se pareciam com o esquema antigo, e fizemos os clientes apontarem para esse esquema, como vemos na Figura 4.3. Essa solução nos permitiu fazer mudanças em nosso próprio esquema, desde que pudéssemos manter a visão. Vamos apenas dizer que muitos procedimentos armazenados (stored procedures) estavam envolvidos.

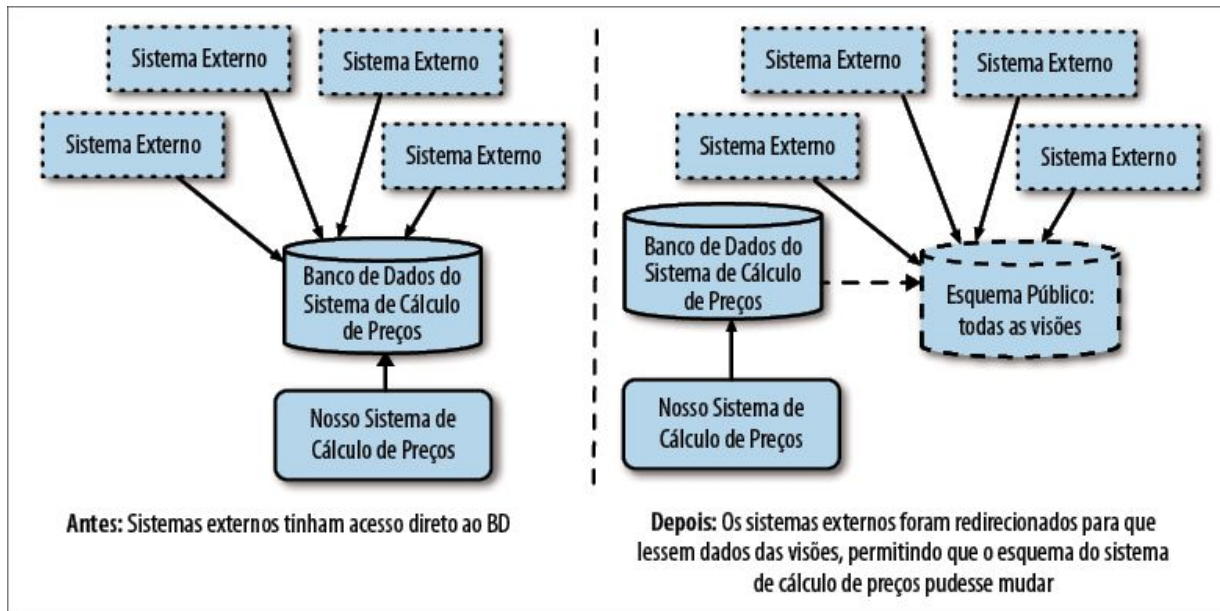


Figura 4.3 – Usando visões para permitir que o esquema subjacente pudesse mudar.

Em nosso exemplo do banco de investimentos, a visão e o esquema subjacente acabaram ficando bem diferentes. Você pode, é claro, usar uma visão de forma muito mais simplificada, talvez para ocultar informações que você não queira deixar visíveis a componentes externos. Como um exemplo simples, na Figura 4.4, nosso serviço de fidelidade era apenas uma lista de cartões de fidelidade em nosso sistema. Atualmente, essa informação é armazenada em nossa tabela de clientes na forma de uma coluna. Assim, definimos uma visão que expõe apenas o mapeamento entre o ID do cliente e o ID do cartão de fidelidade em uma única tabela, sem expor qualquer outra informação da tabela de clientes. Do mesmo modo, qualquer outra tabela que possa estar no banco de dados do

sistema monolítico estará totalmente oculta ao serviço de Fidelidade.

O fato de uma visão poder projetar apenas informações limitadas da fonte de dados subjacente nos permite implementar uma espécie de ocultação de informações. Esse recurso nos dá controle sobre o que é compartilhado e o que será oculto. Contudo, essa abordagem não é uma solução perfeita – ela apresenta limitações.

Conforme a natureza do banco de dados, você poderá ter a opção de criar uma visão materializada (materialized view). Com uma visão materializada, a visão é previamente gerada – em geral, com o uso de cache. Isso significa que uma leitura em uma visão não precisa gerar uma leitura no esquema subjacente, o que pode melhorar o desempenho. O custo-benefício, então, está relacionado ao modo como essa visão previamente gerada é atualizada – pode significar a possibilidade de ler um conjunto de dados “desatualizado” da visão.

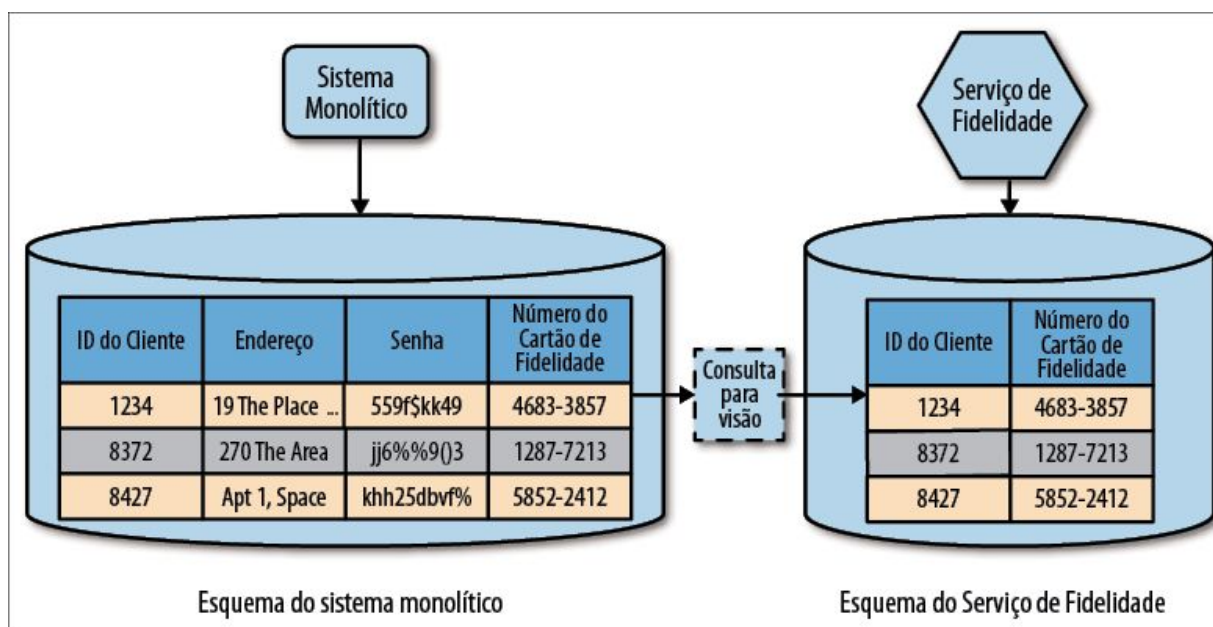


Figura 4.4 – Uma visão do banco de dados projetando um subconjunto de um esquema subjacente.

Limitações

O modo como as visões são implementadas pode variar, mas, em geral, elas são o resultado de uma consulta. Isso significa que a visão propriamente dita é somente para leitura. Esse fato limita prontamente a sua utilidade. Além do mais, embora esse seja um recurso comum em bancos de dados relacionais, e muitos dos bancos de dados NoSQL mais maduros ofereçam suporte para visões (tanto o Cassandra como o Mongo aceitam visões, por exemplo), nem todos têm. Mesmo que sua engine de banco de dados aceite visões, é provável que haja outras limitações, por exemplo, a necessidade de tanto o esquema de origem como a visão estarem na mesma engine de banco de dados. Isso poderia aumentar seu acoplamento físico de implantação, resultando em um possível ponto único de falha.

Responsabilidade

Vale a pena observar que mudanças no esquema subjacente original podem exigir que a visão seja atualizada, portanto, considerações minuciosas se fazem necessárias acerca de quem é o “dono” da visão. Sugiro considerar que qualquer visão de banco de dados publicada seja como qualquer outra interface de serviço e, desse modo, a equipe responsável pelo esquema original deve mantê-la sempre atualizada.

Quando usar o padrão

Em geral, faço uso de uma visão de banco de dados em situações nas quais acho impraticável decompor o esquema monolítico existente. O ideal é que, se for possível, você tente evitar a necessidade de ter uma visão, caso o objetivo seja expor essas informações por meio de uma interface de serviço. Em vez disso, será melhor pressionar em favor de uma decomposição apropriada do esquema. As limitações dessa técnica podem ser significativas. Entretanto, se você achar que o esforço de uma decomposição completa será grande demais, este poderá ser um passo na direção correta.

Padrão: serviço de encapsulamento de banco de dados

Às vezes, quando é muito difícil lidar com algo, ocultar a confusão pode fazer sentido. Com o *padrão de serviço de encapsulamento de banco de dados*, fazemos exatamente isso: ocultamos o banco de dados atrás de um serviço que atua como um wrapper fino, fazendo com que as dependências com o banco de dados passem a ser dependências para um serviço, conforme vemos na Figura 4.5.

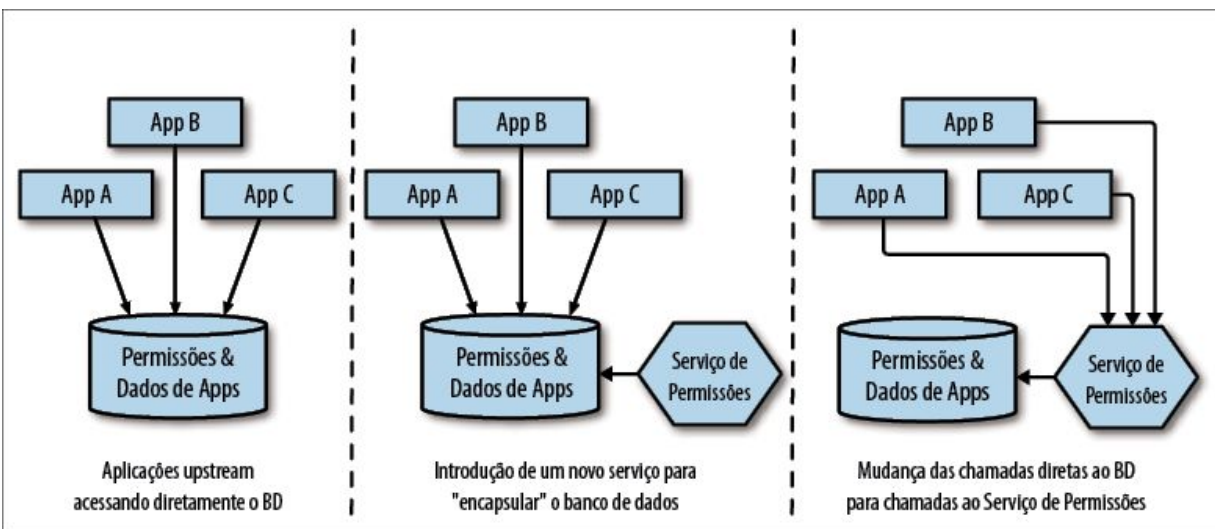


Figura 4.5 – Usando um serviço para encapsular um banco de dados.

Trabalhei em um grande banco australiano alguns anos atrás, em um pequeno projeto para ajudar uma parte da instituição a implementar um melhor caminho até o ambiente de produção. No primeiro dia, marcamos algumas entrevistas com as pessoas-chaves para entender os desafios que o banco estava enfrentando e definir uma visão geral do processo atual. No intervalo entre as reuniões, alguém apareceu e se apresentou como o DBA líder daquela área da empresa. “Por favor” – ele disse – “Faça com que parem de colocar informações no banco de dados!”

Pegamos um café e o DBA descreveu os problemas. Em um período de cerca de 30 anos, um sistema bancário para negócios – uma das joias da empresa – tinha tomado forma. Uma das partes

mais importantes desse sistema consistia em gerenciar o que eles chamavam de “permissões”. Em um sistema bancário para negócios, gerenciar quais indivíduos podem acessar quais contas, e determinar o que eles podem fazer com essas contas, era muito complexo. Para ter uma ideia de como eram essas permissões, considere um contador que tenha permissão para visualizar as contas das empresas A, B e C, mas, para a empresa B, ele pode fazer transferências de até 500 dólares entre as contas, enquanto, para a empresa C, pode fazer transferências ilimitadas entre as contas, mas também pode fazer saques de até 250 dólares. A manutenção e a aplicação dessas permissões eram administradas quase exclusivamente por meio de procedimentos armazenados no banco de dados. Todos os acessos aos dados passavam por essa lógica de permissões.

À medida que o banco havia se expandido, e o volume de lógica e de estados aumentara, o banco de dados havia começado a sofrer com a carga. “Demos todo o dinheiro que era possível dar à Oracle, mas ainda não bastou.” A preocupação era que, considerando o crescimento previsto, e até mesmo contando com melhorias no desempenho do hardware, em algum momento, eles atingiriam um ponto em que as necessidades da empresa seriam maiores do que a capacidade do banco de dados.

À medida que explorávamos melhor o problema, discutimos a ideia de separar partes do esquema a fim de reduzir a carga. O problema consistia no fato de que o emaranhado no meio disso tudo era esse sistema de permissões. Seria um pesadelo tentar desembaraçá-lo, e os riscos associados a erros cometidos nessa área seriam enormes: dê um passo errado, e alguém poderia ser impedido de acessar suas contas ou, pior ainda, alguém que não deveria poderia ter acesso ao seu dinheiro.

Concebemos um plano para tentar resolver a situação. Aceitamos que, no curto prazo, não seríamos capazes de fazer mudanças no sistema de permissões, portanto, era mandatório que, ao menos,

não piorássemos o problema. Assim, precisávamos fazer com que as pessoas parassem de colocar mais dados e comportamentos no sistema de permissões. Assim que isso foi feito, pudemos considerar a remoção das partes do esquema de permissões que eram mais fáceis de extrair, na esperança de reduzir suficientemente a carga a ponto de diminuir as preocupações com a viabilidade no longo prazo. Com isso, poderíamos respirar um pouco mais aliviados e considerar os próximos passos.

Discutimos a introdução de um novo serviço de Permissões, que nos permitiria “ocultar” o esquema problemático. Esse serviço incluiria poucas funcionalidades no início, pois o banco de dados existente já tinha muitas funcionalidades implementadas na forma de procedimentos armazenados. Contudo, o objetivo era incentivar as equipes que escreviam as demais aplicações a pensar no esquema de permissões como pertencente a outra equipe, e incentivá-las a armazenar os próprios dados localmente, como vemos na Figura 4.6.

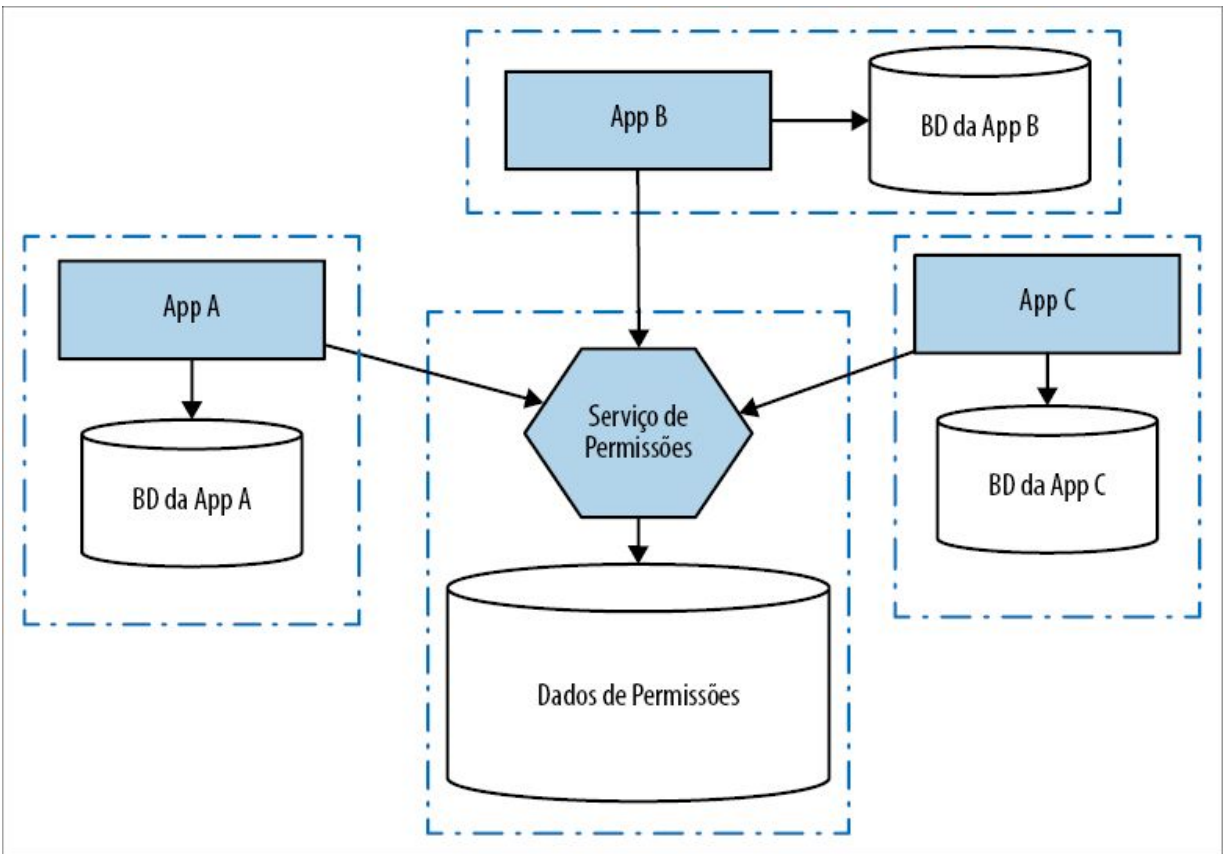


Figura 4.6 – Usando o padrão de serviço de encapsulamento de banco de dados para reduzir a dependência a um banco de dados central.

Assim como em nosso uso das visões de banco de dados, a utilização de um serviço de encapsulamento nos permite controlar o que é compartilhado e o que deve permanecer oculto. Ele apresenta uma interface que pode se manter fixa para os clientes, enquanto mudanças são feitas nos bastidores para melhorar a situação.

Quando usar o padrão

Esse padrão funciona muito bem quando é difícil considerar uma separação do esquema subjacente. Ao colocar um wrapper explícito ao redor do esquema e deixar claro que os dados só podem ser acessados por meio desse esquema, você poderá, no mínimo, colocar um freio no crescimento do banco de dados. Isso determina claramente o que é “seu” e o que é “de outra pessoa”. Acho que

essa abordagem também funciona melhor quando a responsabilidade pelo esquema subjacente e pela camada de serviço é atribuída à mesma equipe. A API do serviço deve ser devidamente encarada como uma interface gerenciada, com uma supervisão apropriada acerca de como essa camada de API pode mudar. Essa abordagem também apresenta vantagens para as aplicações upstream, pois elas podem compreender mais facilmente como o esquema downstream está sendo usado. Isso faz com que atividades como stubbing para testes sejam muito mais fáceis de administrar.

Esse padrão tem vantagens em relação ao uso de uma simples visão de banco de dados. Em primeiro lugar, você não estará limitado a apresentar uma visão que deva estar mapeada a estruturas de tabela existentes; você poderá escrever um código em seu serviço de encapsulamento para que apresente projeções muito mais sofisticadas dos dados subjacentes. O serviço de encapsulamento também pode aceitar escritas (por meio de chamadas de API). É claro que adotar esse padrão exige que os consumidores upstream façam mudanças; eles terão de trocar os acessos diretos ao banco de dados por chamadas de API.

O ideal é que o uso desse padrão seja um passo intermediário para mudanças mais importantes, fazendo com que você ganhe tempo para separar o esquema que está abaixo de sua camada de API. Alguém poderia dizer que estamos apenas fazendo um curativo sobre o problema, em vez de resolver as questões subjacentes. Apesar disso, considerando que queremos fazer melhorias graduais, acho que esse padrão apresenta muitas vantagens.

Padrão: interface de banco de dados como serviço

Às vezes, os clientes só precisam de um banco de dados para consultas. Pode ser porque precisem consultar ou buscar grandes volumes de dados ou, talvez, por haver componentes externos que

já utilizem cadeias de ferramentas que exijam um endpoint SQL para trabalhar (pense em ferramentas como o Tableau que, com frequência, é usado para insights em métricas de negócios). Nessas situações, permitir que os clientes visualizem dados administrados pelo seu serviço em um banco de dados pode fazer sentido; contudo, devemos tomar cuidado para separar o banco de dados que expomos do banco de dados que utilizamos dentro das fronteiras de nosso serviço.

Uma abordagem que já vi funcionar bem consiste em criar um banco de dados dedicado, projetado para ser exposto como um endpoint somente de leitura, e fazer esse banco de dados ser preenchido quando os dados do banco de dados subjacente mudarem. Com efeito, do mesmo modo que um serviço poderia expor um fluxo de eventos como um endpoint e uma API síncrona como outro endpoint, ele poderia igualmente expor um banco de dados aos consumidores externos. Na Figura 4.7, vemos um exemplo do serviço de Pedidos, que expõe um endpoint de leitura/escrita por meio de uma API, e um banco de dados como uma interface somente de leitura. Uma engine de mapeamento percebe mudanças no banco de dados interno e determina quais mudanças devem ser feitas no banco de dados externo.

Padrão de banco de dados para relatórios

Martin Fowler já documentou esse padrão como banco de dados para relatórios (reporting database pattern, <http://bit.ly/2kWW9lr>), então, por que estou usando um nome diferente? À medida que conversava com mais pessoas, percebi que, embora gerar relatórios seja uma aplicação comum para esse padrão, esse não é o único motivo pelo qual as pessoas utilizam essa técnica. Permitir que os clientes definam consultas *ad hoc* é um recurso cujo escopo é muito mais amplo do que somente o uso em fluxos de trabalho batch (em lote) tradicionais. Assim, apesar de esse padrão ser provavelmente usado de modo geral para suporte a esses casos de uso de relatórios, eu queria um nome diferente, que refletisse o fato de o padrão poder ter uma aplicabilidade

mais generalizada.

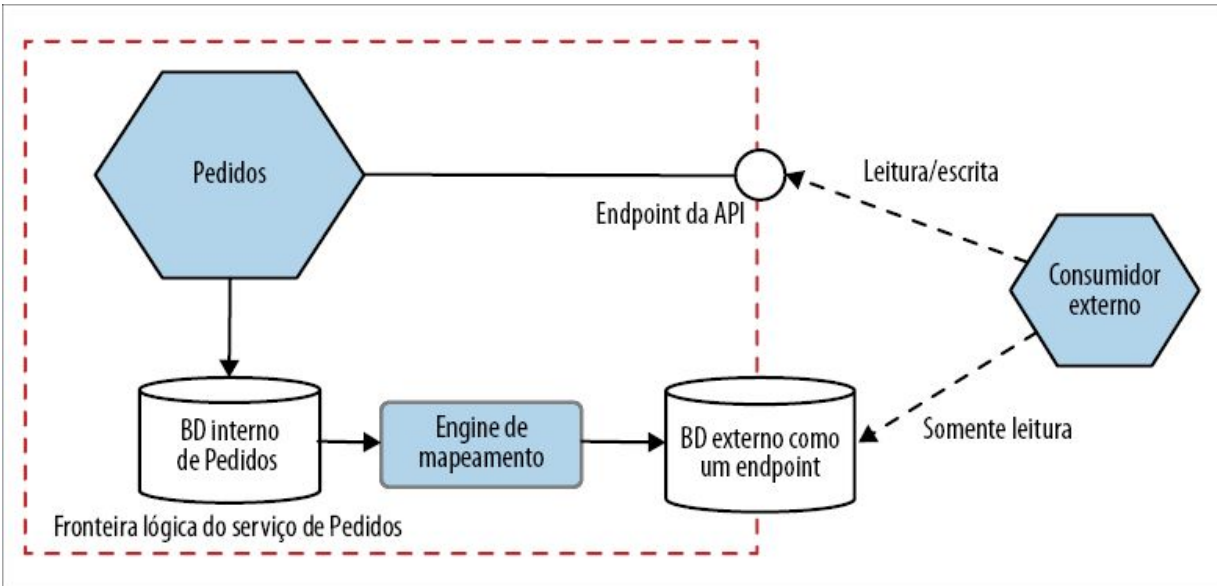


Figura 4.7 – Expondo um banco de dados dedicado como um endpoint, permitindo que o banco de dados interno permaneça oculto.

A engine de mapeamento poderia ignorar totalmente as mudanças, expô-las diretamente ou fazer algo intermediário. O ponto principal é que a engine de mapeamento atue como uma camada de abstração entre os bancos de dados interno e externo. Quando a estrutura de nosso banco de dados interno mudar, a engine de mapeamento deverá fazer uma mudança para garantir que o banco de dados voltado ao público permaneça consistente. Em praticamente todos os casos, a engine de mapeamento apresentará uma defasagem em relação às escritas feitas no banco de dados interno; em geral, a escolha da implementação da engine de mapeamento determinará essa defasagem. Clientes que leem o banco de dados exposto deverão entender que, desse modo, poderão estar vendo dados potencialmente desatualizados, e talvez você ache apropriado que o programa exponha informações sobre quando foi feita a última atualização no banco de dados externo.

Implementando uma engine de mapeamento

O detalhe, neste caso, é descobrir como atualizar os dados, ou seja, como implementar a engine de mapeamento. Já vimos um sistema de captura de dados modificados (change data capture), que seria uma excelente opção nessa situação. De fato, é provavelmente a solução mais robusta, ao mesmo tempo que também fornece a visão mais atualizada. Outra opção seria ter um processo batch que apenas copiasse os dados, embora essa solução pudesse ser problemática, pois é provável que resulte em maiores defasagens entre os bancos de dados interno e externo, e determinar quais dados devem ser copiados pode ser difícil com alguns esquemas. Uma terceira opção poderia ser observar os eventos disparados pelo serviço em questão e usá-los para atualizar o banco de dados externo.

No passado, eu teria usado uma tarefa batch para cuidar desse problema. Atualmente, porém, eu provavelmente utilizaria um sistema dedicado de captura de dados modificados, talvez algo como o Debezium (<https://github.com/debezium/debezium>). Já tive muitos problemas com processos batch que não executavam ou que demoravam demais para executar. Com o mundo se distanciando das tarefas batch e querendo dados mais rapidamente, os processos batch estão cedendo espaço para o tempo real. Ter um sistema de captura de dados modificados instalado para lidar com esse problema faz sentido, sobretudo se você estiver considerando usá-lo para expor eventos para fora dos limites de seu serviço.

Comparação com as visões

Esse padrão é mais sofisticado que uma simples visão do banco de dados. As visões de banco de dados em geral estão vinculadas a uma pilha de tecnologia em particular: se eu quiser apresentar uma visão de um banco de dados Oracle, tanto o banco de dados subjacente como o esquema que hospeda a visão deverão executar no Oracle. Com a abordagem apresentada, o banco de dados exposto pode estar em uma pilha de tecnologia totalmente diferente. Poderíamos usar o Cassandra em nosso serviço, porém apresentar

um banco de dados SQL tradicional como um endpoint para o público.

Esse padrão permite que tenhamos mais flexibilidade do que as visões de banco de dados, mas há um custo adicional. Se as necessidades de seus clientes puderem ser satisfeitas com uma simples visão do banco de dados, é provável que, à primeira vista, haja menos trabalho a ser implementado. Fique ciente, porém, de que essa opção pode impor restrições no modo como a interface poderá evoluir. Você poderia começar com o uso de uma visão de banco de dados e considerar uma mudança para um banco de dados dedicado para disponibilizar informações no futuro.

Quando usar o padrão

Obviamente, como o banco de dados exposto com um endpoint é somente para leitura, essa solução será conveniente apenas para os clientes que precisarem de um acesso somente de leitura. Ela será bem apropriada para os casos de uso de relatórios – situações em que seus clientes poderão ter de fazer junções de grandes volumes de dados armazenados por um dado serviço. Essa ideia poderia ser estendida para, então, importar os dados desse banco de dados para um depósito maior (data warehouse), permitindo que dados de vários serviços sejam consultados. O assunto foi abordado com mais detalhes no Capítulo 5 do livro *Building Microservices*.

Não subestime o trabalho necessário para garantir que essa projeção externa do banco de dados seja devidamente mantida e atualizada. Dependendo de como o seu serviço atual estiver implementado, esse pode ser um empreendimento complexo.

Transferência de responsabilidades

Até agora, não enfrentamos realmente o problema subjacente. Apenas fizemos diversos curativos em um grande banco de dados compartilhado. Antes de começarmos a considerar a intrincada tarefa de extrair dados do gigantesco banco de dados monolítico,

devemos considerar em que local os dados em questão devem realmente estar. À medida que separarmos os serviços do sistema monolítico, parte dos dados deverão vir juntos – enquanto outros deverão permanecer onde estão.

Se adotarmos a ideia de um microserviço que encapsule a lógica associada a um ou mais agregados, também devemos passar o gerenciamento de seus estados e os dados associados para um esquema que seja do próprio microserviço. Por outro lado, se nosso novo microserviço tiver de interagir com um agregado cuja responsabilidade ainda seja do sistema monolítico, teremos de expor esse recurso por meio de uma interface bem definida. Vamos discutir agora essas duas opções.

Padrão: sistema monolítico expando agregados

Na Figura 4.8, nosso novo serviço de Fatura precisa acessar diversas informações que não estão diretamente relacionadas ao gerenciamento das faturas. No mínimo, ele precisará de informações de nossos Funcionários atuais para gerenciar os fluxos de trabalho de aprovações. Atualmente, todos esses dados estão no banco de dados do sistema monolítico. Ao expor informações sobre os Funcionários por meio de um endpoint de serviço (poderia ser uma API ou um stream de eventos) no próprio sistema monolítico, deixamos explícitas as informações que serão necessárias ao serviço de Fatura.

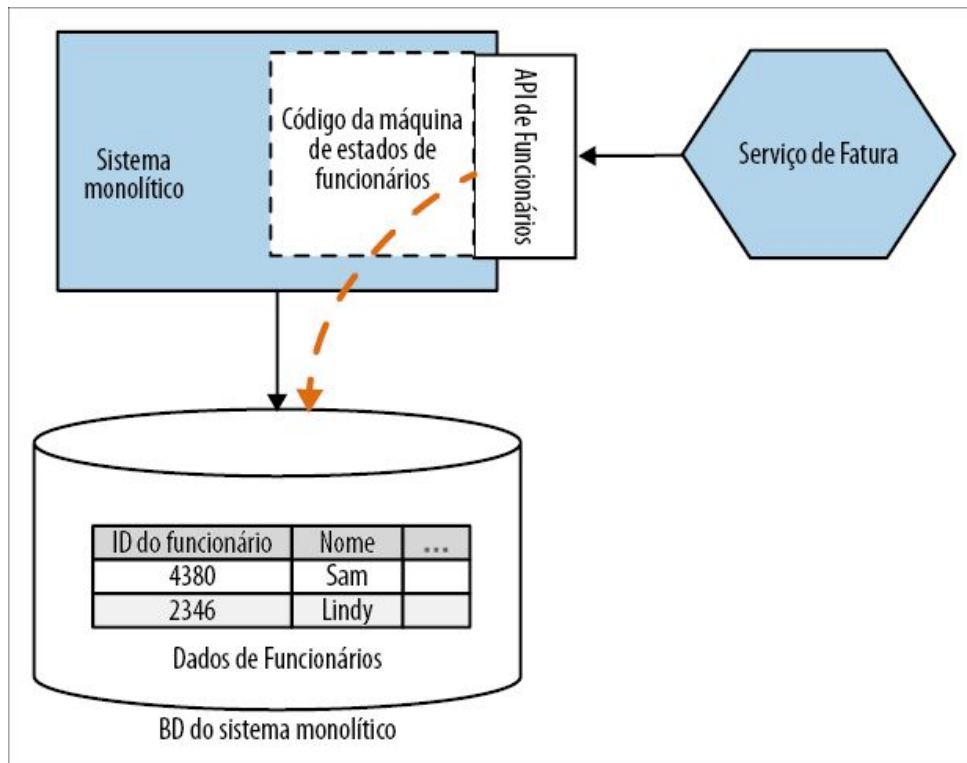


Figura 4.8 – Expondo informações do sistema monolítico por meio de uma interface de serviço apropriada, permitindo que nosso novo microsserviço as acesse.

Queremos pensar em nossos microsserviços como combinações de comportamentos e estados; já discuti a ideia de pensar nos microsserviços como algo que contém uma ou mais máquinas de estado que administram agregados de domínios. Ao expor um agregado a partir do sistema monolítico, queremos pensar da mesma forma. O sistema monolítico continua sendo “responsável” pelo conceito acerca do que é ou não é uma mudança de estado permitida; não queremos tratá-lo somente como um wrapper em torno de um banco de dados.

Além de expor os dados, expomos também operações que permitam que componentes externos consultem o estado atual de um agregado e faça requisições de transições para novos estados. Podemos ainda restringir os estados de um agregado que serão expostos na fronteira de nosso serviço e limitar as operações de transição de estado que poderão ser solicitadas de fora.

Um caminho para outros serviços

Ao definir as necessidades do serviço de Fatura e expor explicitamente as informações necessárias em uma interface bem definida, estaremos no caminho de descobrir futuras fronteiras de serviços em potencial. Neste exemplo, um passo óbvio poderia ser extrair um serviço de Funcionários a seguir, conforme vemos na Figura 4.9. Ao expor uma API para dados relacionados a funcionários, já teremos avançado bastante no caminho de compreender quais poderiam ser as necessidades dos clientes do novo serviço de Funcionários.

É claro que, se realmente extraíssemos os funcionários do sistema monolítico e o sistema monolítico precisasse dos dados desses funcionários, ele teria de ser modificado para usar o novo serviço!

Quando usar o padrão

Se o banco de dados ainda for o “responsável” pelos dados que você quer acessar, esse padrão funcionará bem para permitir que os novos serviços acessem as informações de que precisam. Ao extrair serviços, fazer o novo serviço chamar de volta o sistema monolítico para acessar os dados necessários provavelmente implicará um pouco mais de trabalho do que acessar diretamente o banco de dados do sistema monolítico – mas, no longo prazo, será muito melhor. Eu consideraria usar uma visão do banco de dados a essa abordagem somente se o sistema monolítico em questão não puder ser modificado para expor os novos endpoints. Em casos como esse, uma visão do banco de dados do sistema monolítico poderia funcionar, assim como o padrão de captura de dados modificados discutido antes (veja a seção Padrão: captura de dados modificados), ou criar um padrão de serviço de encapsulamento de banco de dados dedicado (veja a seção Padrão: serviço de encapsulamento de banco de dados) com base no esquema do sistema monolítico, expondo as informações que queremos de Funcionários.

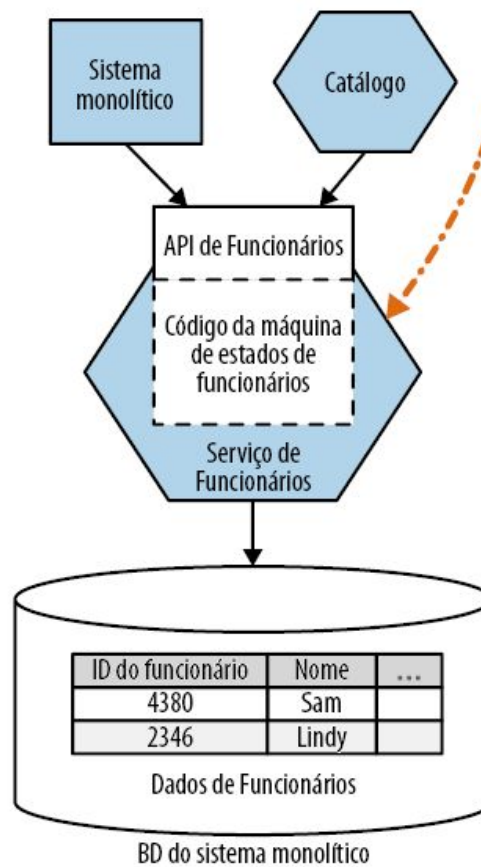
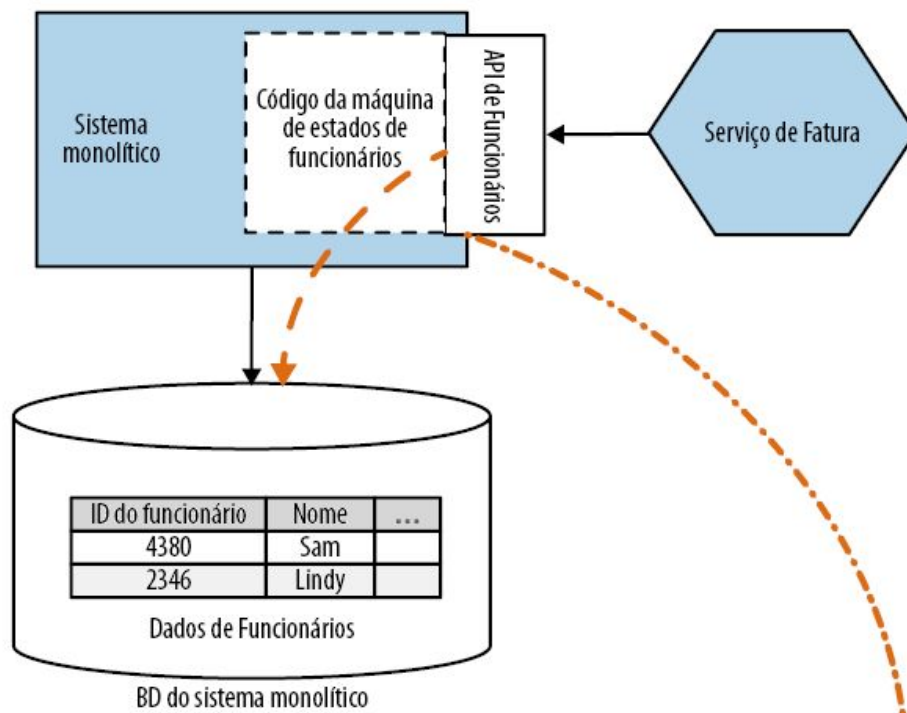


Figura 4.9 – Usando o escopo de um endpoint existente para direcionar a extração de um novo serviço de Funcionários.

Padrão: mudança do responsável pelos dados

Já vimos o que acontece quando o novo serviço de Fatura precisa acessar dados que pertencem a outra funcionalidade, como na seção anterior, quando tivemos de acessar dados de Funcionários. No entanto, o que acontecerá se considerarmos dados que estão atualmente no sistema monolítico, mas que o nosso serviço recém-extraído deveria controlar?

Na Figura 4.10, apresentamos a mudança a ser feita. Devemos retirar os dados relacionados às faturas do sistema monolítico e passá-los para o novo serviço de Fatura, pois é aí que o ciclo de vida dos dados deve ser gerenciado. Então, teríamos de modificar o sistema monolítico para que trate o serviço de Fatura como a fonte da verdade para os dados relacionados a faturas, e alterá-lo de modo que ele recorra ao endpoint do serviço de Fatura para ler os dados ou solicitar mudanças.

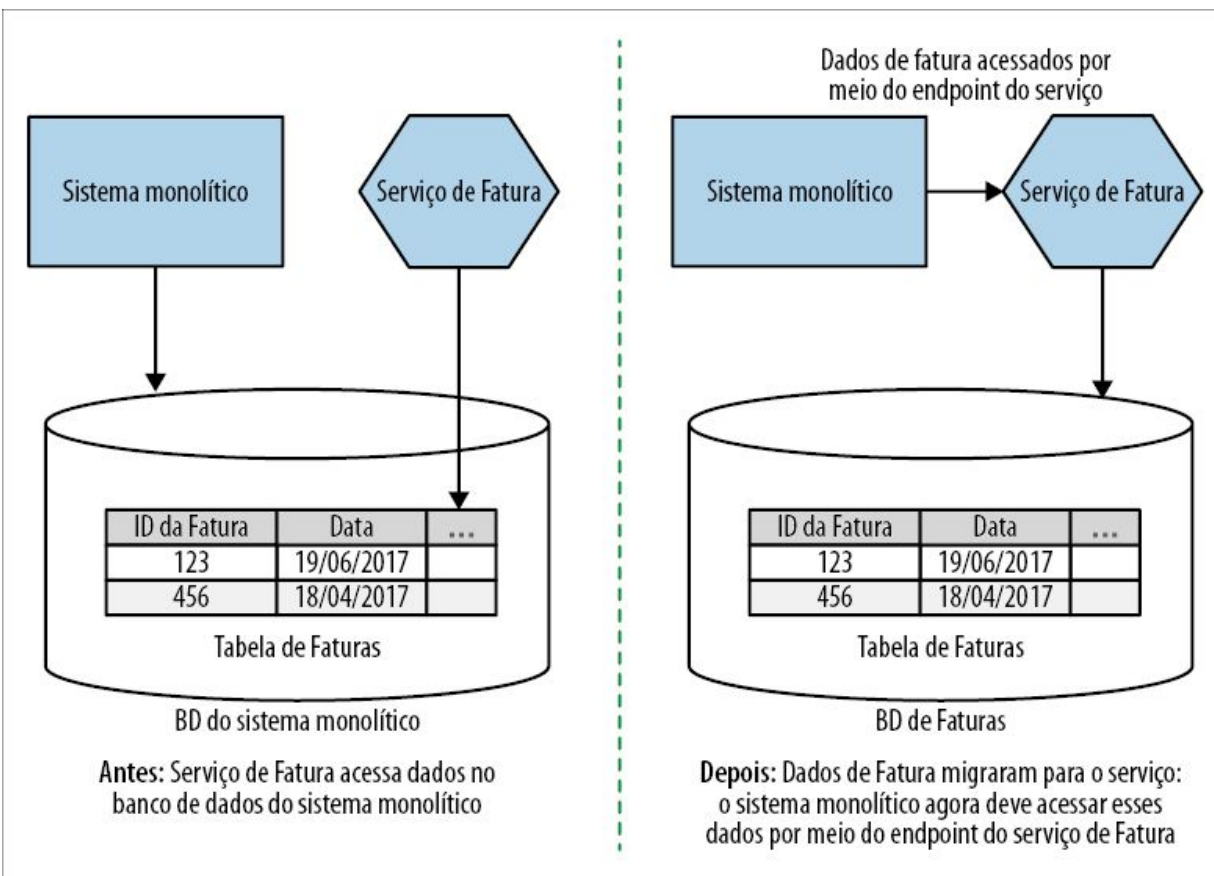


Figura 4.10 – Nosso novo serviço de Fatura assume a responsabilidade pelos dados relacionados a ele.

Contudo, desembaraçar os dados de fatura do banco de dados do sistema monolítico atual pode ser um problema complexo. Talvez tenhamos de considerar o impacto de acabar com restrições de chave estrangeira, quebrar fronteiras de transações e outras questões – retomaremos todos esses assuntos mais adiante neste capítulo. Se o sistema monolítico puder ser modificado de modo que só precise de acesso de leitura aos dados relacionados às Faturas, você poderá considerar a projeção de uma visão a partir do banco de dados do serviço de Fatura, como mostra a Figura 4.11. No entanto, todas as limitações das visões de banco de dados se aplicarão; modificar o sistema monolítico para fazer chamadas diretamente ao novo serviço de Fatura seria uma opção preferível.

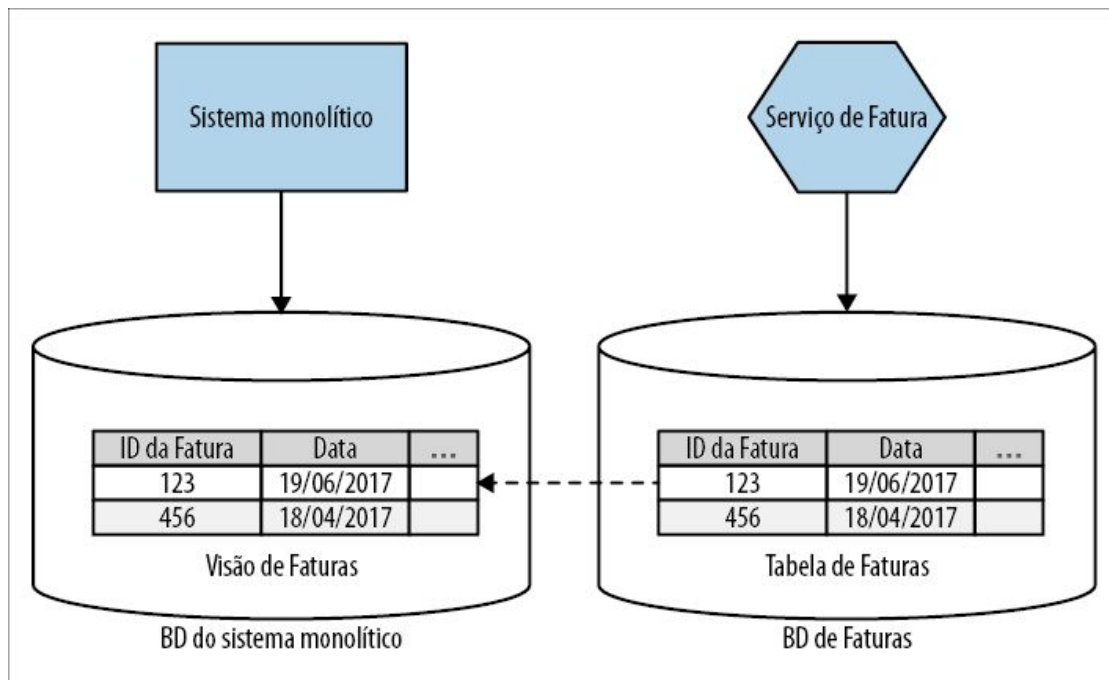


Figura 4.11 – Projetando dados de Fatura de volta ao sistema monolítico na forma de uma visão.

Quando usar o padrão

O uso desse padrão é um pouco mais claro. Se seu serviço recém-extraído encapsula a lógica de negócios que modifica algum dado, o novo serviço deverá controlar esse dado. Os dados deverão ser transferidos do local em que estão para o novo serviço. É claro que o processo de retirar dados do banco de dados atual está longe de ser simples. Com efeito, esse será o foco na parte restante do capítulo.

Sincronização de dados

Conforme discutimos no Capítulo 3, uma das vantagens de um padrão como o strangler fig é que, depois que passarmos a usar o novo serviço, será possível retroceder caso haja alguma falha. O problema ocorre quando o serviço em questão gerencia dados que deverão ser mantidos em sincronia entre o sistema monolítico e o novo serviço.

Na Figura 4.12, vemos um exemplo desse caso. Estamos no

processo de passar para um novo serviço de Fatura. Porém, o novo serviço também administra esses dados, além do código atual equivalente no sistema monolítico. Para continuar sendo possível alternar entre as implementações, devemos garantir que os dois conjuntos de código vejam os mesmos dados, e que esses dados possam ser mantidos de forma consistente.

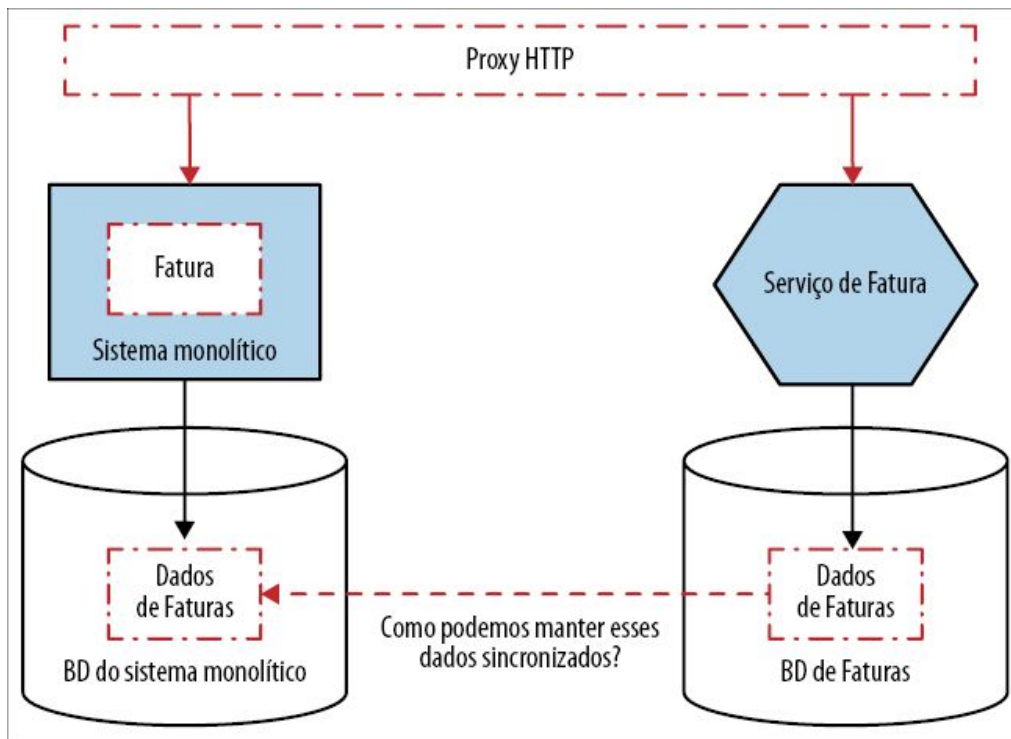


Figura 4.12 – Queremos usar um padrão strangler fig para fazer a migração para um novo serviço de Fatura, mas o serviço administra estados.

Então, quais são as nossas opções? Bem, inicialmente, devemos considerar até que ponto os dados precisam estar consistentes entre as duas visões. Se qualquer um dos conjuntos de código sempre precisar ter uma visão totalmente consistente dos dados de fatura, uma das abordagens mais simples seria garantir que os dados fossem mantidos em um só local. Isso nos levaria provavelmente a fazer com que nosso novo serviço de Fatura lesse os dados diretamente do sistema monolítico durante um tempo, talvez fazendo uso de uma visão, conforme exploramos na seção

Padrão: visão de banco de dados. Assim que estivermos satisfeitos com o sucesso da troca, poderemos, em seguida, fazer a migração dos dados, conforme já discutimos na seção Padrão: mudança do responsável pelos dados. No entanto, nunca é demais enfatizar as preocupações quanto ao uso de um banco de dados compartilhado: você deve considerar essa solução somente como uma medida de curto prazo, como parte de uma extração mais completa; manter um banco de dados compartilhado por muito tempo pode resultar em vários problemas no longo prazo.

Se estivermos fazendo uma mudança Big Bang (algo que eu tentaria evitar), fazendo a migração tanto do código da aplicação como dos dados ao mesmo tempo, poderíamos usar um processo batch para copiar os dados antes da mudança para o novo microserviço. Assim que os dados relacionados às faturas tiverem sido copiados para o novo microserviço, ele poderá começar a atender ao tráfego. No entanto, o que acontecerá se precisarmos fazer um fallback e utilizar a funcionalidade que está atualmente no sistema monolítico? Dados que foram alterados no esquema do microserviço não estarão refletidos no estado do banco de dados monolítico, portanto, poderíamos acabar perdendo estados.

Outra abordagem seria considerar a possibilidade de manter os dois bancos de dados em sincronia por meio de código. Assim, teríamos o sistema monolítico ou o novo serviço de Fatura fazendo escritas nos dois bancos de dados. Essa solução envolve um planejamento minucioso.

Padrão: sincronização de dados na aplicação

Passar dados de um local para outro pode ser um empreendimento complexo, mesmo na melhor das ocasiões, mas poderá ser mais estressante quanto mais importantes forem os dados. Se pensarmos na manutenção de registros médicos, pensar cuidadosamente na migração dos dados será mais importante ainda.

Vários anos atrás, a empresa de consultoria Trifork esteve envolvida em um projeto para ajudar a armazenar uma visão consolidada dos registros médicos de cidadãos dinamarqueses.³ A versão inicial desse sistema armazenava os dados em um banco de dados MySQL, mas, com o tempo, ficou claro que essa solução não seria apropriada para os desafios que o sistema enfrentaria. Uma decisão foi tomada para que um banco de dados alternativo, o Riak, fosse utilizado. A expectativa era que o Riak permitisse que o sistema pudesse escalar melhor a fim de lidar com a carga esperada, mas que também oferecesse características de resiliência.

Um sistema existente armazenava dados em um banco de dados, mas havia limites para o tempo que o sistema poderia ficar offline, e era essencial que os dados não fossem perdidos. Assim, era necessário ter uma solução que permitisse que a empresa passasse os dados para um novo banco de dados, mas que também incluísse métodos para conferir a migração, além de métodos rápidos para um rollback durante o processo.

De acordo com a decisão tomada, a própria aplicação faria a sincronização entre as duas fontes de dados. A ideia era que, inicialmente, o banco de dados MySQL existente permanecesse como a fonte da verdade, mas, durante um tempo, a aplicação garantiria que os dados no MySQL e no Riak fossem mantidos em sincronia. Depois de um tempo, o Riak passaria a ser a fonte da verdade para a aplicação e, depois disso, o MySQL poderia ser desativado. Vamos analisar esse processo com um pouco mais de detalhes.

Passo 1: Sincronizar grandes volumes de dados

O primeiro passo é chegar ao ponto em que haja uma cópia dos dados no novo banco de dados. Para o projeto dos registros médicos, isso envolveu fazer uma migração batch dos dados, do sistema antigo para o novo banco de dados Riak. Enquanto a importação batch ocorria, o sistema existente continuou executando,

portanto, a fonte dos dados para a importação era um snapshot (imagem instantânea) dos dados, obtido a partir do sistema MySQL existente (Figura 4.13). Isso constituiu um desafio, pois, quando a importação batch havia terminado, os dados no sistema de origem poderiam ter sofrido mudanças. Nesse caso, porém, colocar o sistema de origem offline não era viável.

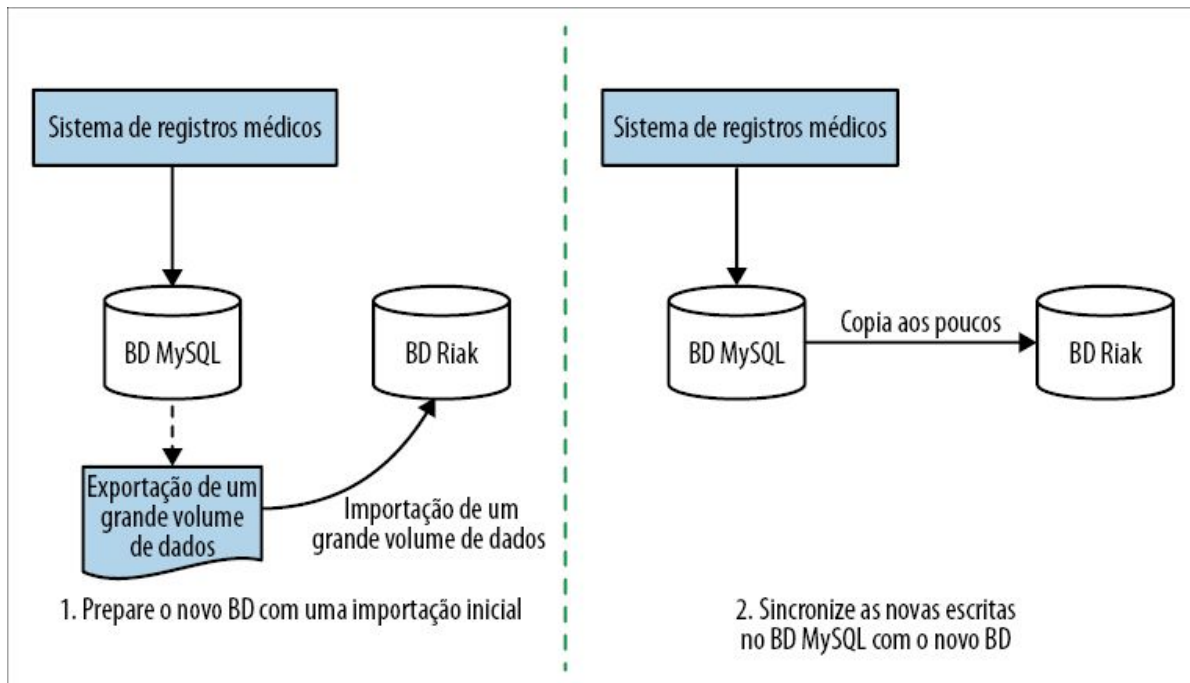


Figura 4.13 – Preparando o novo banco de dados para uma sincronização a ser feita pela aplicação.

Assim que a importação batch foi concluída, um processo de captura de dados modificados (change data capture) foi implementado para que as mudanças feitas após a importação pudessem ser aplicadas. Isso permitiu que o Riak ficasse sincronizado. Assim que essa tarefa foi concluída, era hora de implantar a nova versão da aplicação.

Passo 2: Sincronizar na escrita, ler do esquema antigo

Com os dois bancos de dados agora sincronizados, uma nova versão da aplicação foi implantada, a qual escrevia todos os dados

nos dois bancos de dados, conforme vemos na Figura 4.14. Nessa fase, o objetivo era garantir que a aplicação estivesse escrevendo corretamente nas duas fontes de dados, e certificar-se de que o Riak estivesse se comportando com um nível aceitável de tolerância. Ao continuar lendo todos os dados do MySQL, garantia-se que, mesmo que o Riak falhasse totalmente, os dados poderiam continuar sendo obtidos do banco de dados MySQL existente.

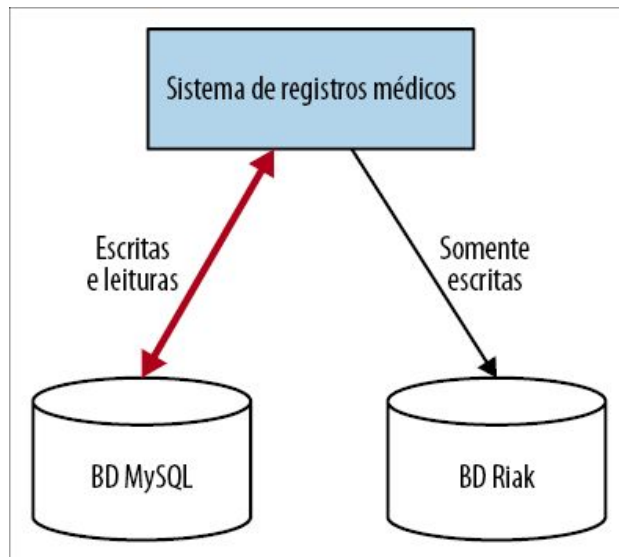


Figura 4.14 – A aplicação mantém os dois bancos de dados sincronizados, mas utiliza somente um para as leituras.

O próximo passo foi dado somente depois que um nível de confiança suficiente em relação ao novo sistema Riak havia sido atingido.

Passo 3: Sincronizar na escrita, ler do novo esquema

Nessa etapa, já houve uma confirmação de que as leituras no Riak estavam funcionando bem. O último passo era garantir que as escritas também estivessem funcionando. Uma mudança simples na aplicação fez com que o Riak passasse a ser a fonte da verdade, conforme vemos na Figura 4.15. Observe que ainda continuamos escrevendo nos *dois* bancos de dados, portanto, se houver algum

problema, temos uma opção de fallback.

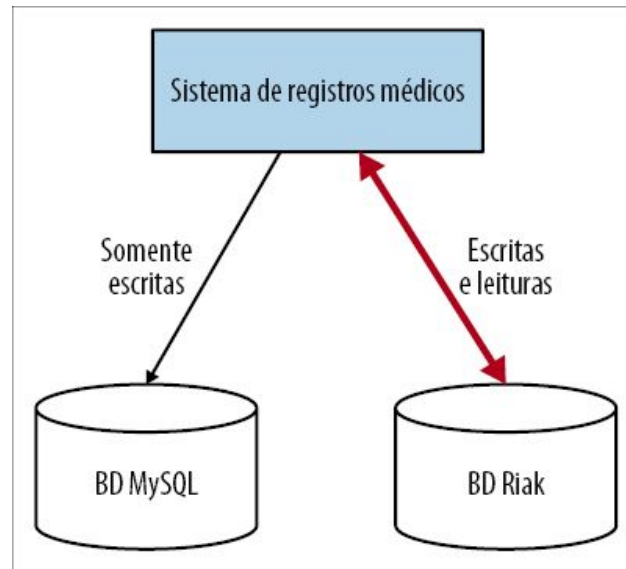


Figura 4.15 – O novo banco de dados agora é a fonte da verdade, mas o banco de dados antigo ainda continua sincronizado.

Assim que o sistema novo estivesse suficientemente estabilizado, o esquema antigo poderia ser removido com segurança.

Quando usar esse padrão

Com o sistema de registros médicos da Dinamarca, havia somente uma aplicação com a qual era preciso lidar. Porém, falamos de situações em que estamos tentando fazer uma separação em microsserviços. Esse padrão pode realmente ajudar? O primeiro ponto a ser considerado é que o padrão pode fazer bastante sentido se você quiser separar o esquema *antes* de separar o código da aplicação. Na Figura 4.16, vemos exatamente uma situação como essa, na qual duplicamos antes os dados relacionados às faturas.

Se for implementado da forma correta, as duas fontes de dados deverão estar sempre sincronizadas, proporcionando vantagens significativas em situações nas quais seja necessário fazer uma troca rápida das fontes de dados em cenários de rollback e em outros casos. O uso desse padrão no exemplo do sistema de registros médicos da Dinamarca parece razoável por causa da

impossibilidade de deixar a aplicação offline por qualquer que fosse o período.

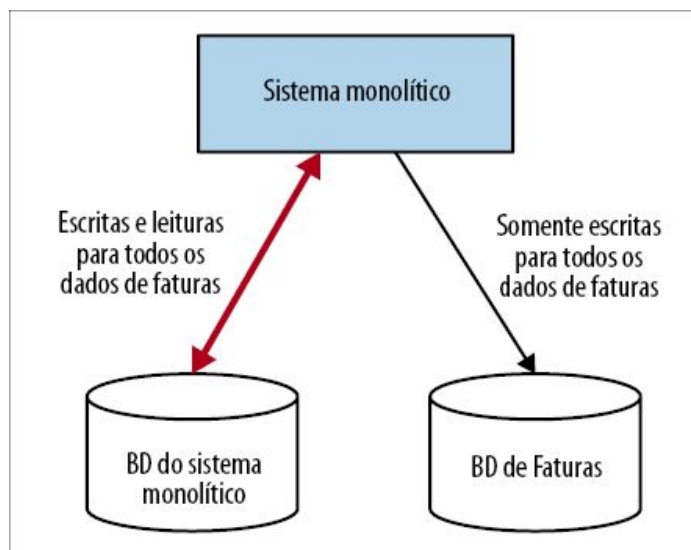


Figura 4.16 – Exemplo de um sistema monolítico que mantém dois esquemas sincronizados.

Você poderia considerar o uso desse padrão quando tanto o seu sistema monolítico como o microsserviço acessarem os dados, mas isso pode se tornar extremamente complicado. Na Figura 4.17, temos uma situação desse tipo.

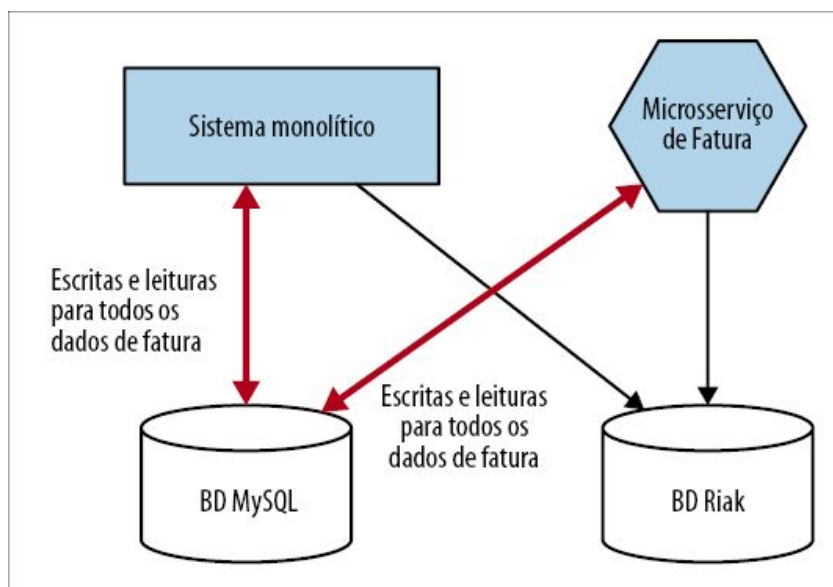


Figura 4.17 – Exemplo de um sistema monolítico e um microsserviço no qual ambos tentam manter os mesmos dois

esquemas em sincronia.

Tanto o sistema monolítico como o microsserviço devem garantir uma sincronização apropriada dos bancos de dados para que esse padrão funcione. Se um deles cometer um erro, você poderá se ver com problemas. Essa complexidade seria bastante atenuada se você pudesse ter a certeza de que, em qualquer dado instante, o serviço de Fatura ou a funcionalidade de Fatura no sistema monolítico estivesse fazendo as escritas – funcionaria bem se estivéssemos usando uma técnica simples de alternância, conforme vimos quando discutimos o padrão strangler fig. Porém, se as requisições puderem ser feitas tanto para a funcionalidade de Fatura no sistema monolítico *como* para a nova funcionalidade de Fatura, quem sabe como parte de um teste canário, talvez você não deva utilizar esse padrão, pois a sincronização resultante seria complicada.

Padrão: tracer write

O padrão tracer write (escrita monitorada), representado na Figura 4.18, sem dúvida é uma variação do padrão de sincronização de dados na aplicação (veja a seção Padrão: sincronização de dados na aplicação). Com um tracer write, movemos a fonte da verdade de nossos dados gradualmente, aceitando que haja duas fontes da verdade durante a migração. Você deve identificar um novo serviço que hospedará os dados relocados. O sistema atual continuará mantendo um registro desses dados localmente, mas, ao fazer mudanças, garantirá também que esses dados sejam gravados no novo serviço por meio de sua interface. O código atual pode ser modificado para que comece a acessar o novo serviço e, assim que todas as funcionalidades estiverem usando o novo serviço como a fonte da verdade, a fonte antiga poderá ser desativada. Considerações minuciosas deverão ser feitas no que concerne ao modo como os dados serão sincronizados entre duas fontes da verdade.

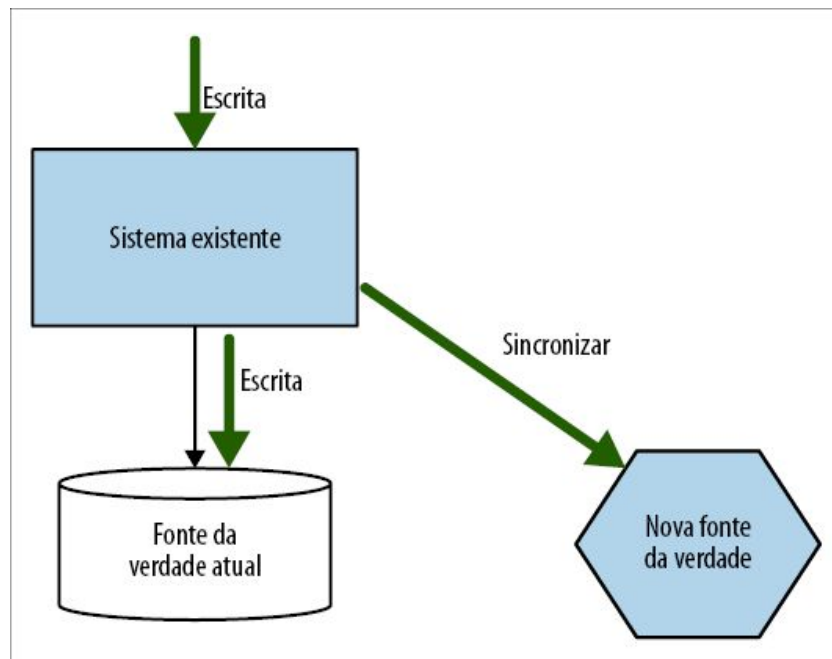


Figura 4.18 – Um tracer write permite uma migração gradual dos dados de um sistema para outro ao aceitar duas fontes da verdade durante a migração.

Querer ter uma única fonte da verdade é um desejo totalmente racional. Isso nos permite garantir a consistência dos dados, controlar o acesso a eles e pode reduzir os custos de manutenção. O problema é que, se insistirmos para que sempre haja apenas uma fonte da verdade para um conjunto de dados, seremos forçados a uma situação na qual modificar o local em que estão esses dados representará uma única grande mudança. Antes do lançamento da versão, o sistema monolítico será a fonte da verdade. Após o lançamento, nosso novo microserviço será a fonte da verdade. O problema é que muita coisa pode dar errado nessa mudança. Um padrão como o tracer write permite uma mudança em fases, reduzindo o impacto de cada lançamento de versão, em troca de sermos mais tolerantes, permitindo que haja mais de uma fonte da verdade.

O motivo pelo qual esse padrão se chama tracer write é que você pode começar com um pequeno conjunto de dados sendo sincronizado, e expandi-lo com o tempo, aumentando

simultaneamente o número de consumidores da nova fonte de dados. Se tomarmos o exemplo apresentado na Figura 4.12, no qual dados relacionados às faturas estavam sendo passados do sistema monolítico para o novo microserviço de Fatura, poderíamos inicialmente sincronizar os dados básicos de fatura e, em seguida, fazer a migração das informações de contato das faturas e, por fim, sincronizar os registros de pagamentos, conforme vemos na Figura 4.19.

Outros serviços que quiserem informações relacionadas às faturas teriam a opção de obtê-las do sistema monolítico ou do novo serviço, dependendo das informações que forem necessárias. Se precisarem de informações que estejam disponíveis somente no sistema monolítico, esses serviços terão de esperar até que os dados e a funcionalidade de suporte tenham sido movidos. Assim que os dados e a funcionalidade estiverem disponíveis no novo microserviço, os consumidores poderão usar a nova fonte da verdade.

O objetivo em nosso exemplo é migrar todos os consumidores para que passem a usar o serviço de Fatura, incluindo o próprio sistema monolítico. Na Figura 4.20, vemos um exemplo de duas etapas da migração. Inicialmente, escrevemos apenas informações básicas de fatura nas duas fontes da verdade. Assim que confirmarmos que essas informações estão sendo devidamente sincronizadas, o sistema monolítico poderá começar a ler dados do novo serviço. À medida que mais dados forem sincronizados, o sistema monolítico poderá usar o novo serviço como fonte da verdade para um volume cada vez maior de dados. Quando todos os dados estiverem sincronizados e o último consumidor da antiga fonte da verdade tiver feito a mudança, poderemos interromper a sincronização dos dados.

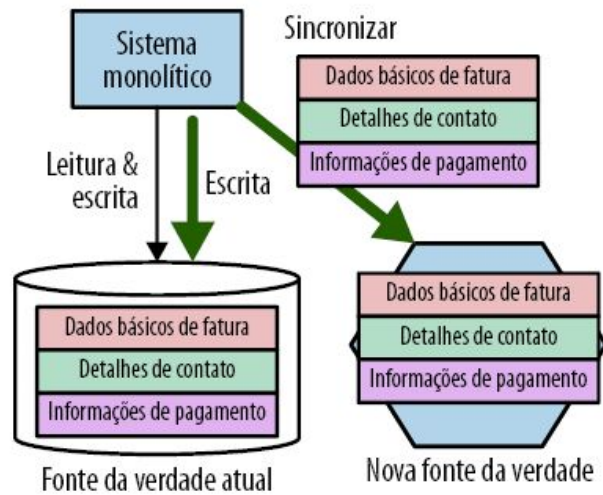
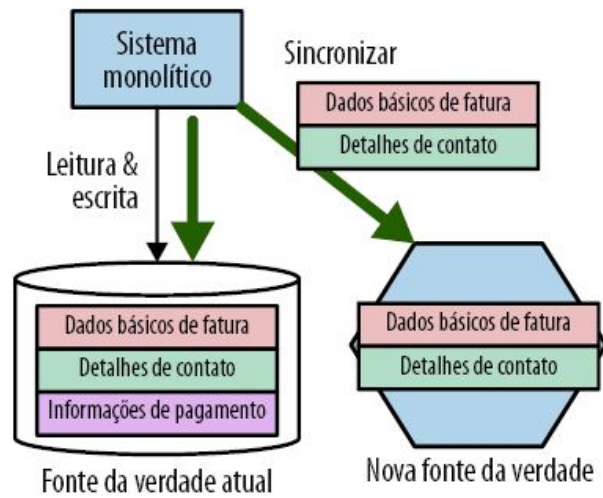
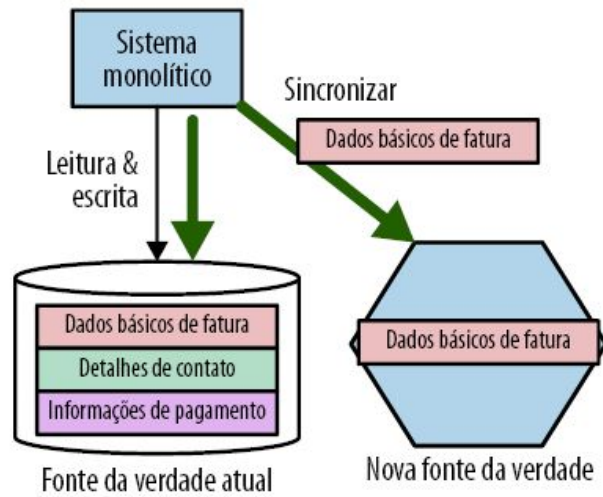


Figura 4.19 – Movendo gradualmente as informações relacionadas às faturas, do sistema monolítico para o serviço de Fatura.

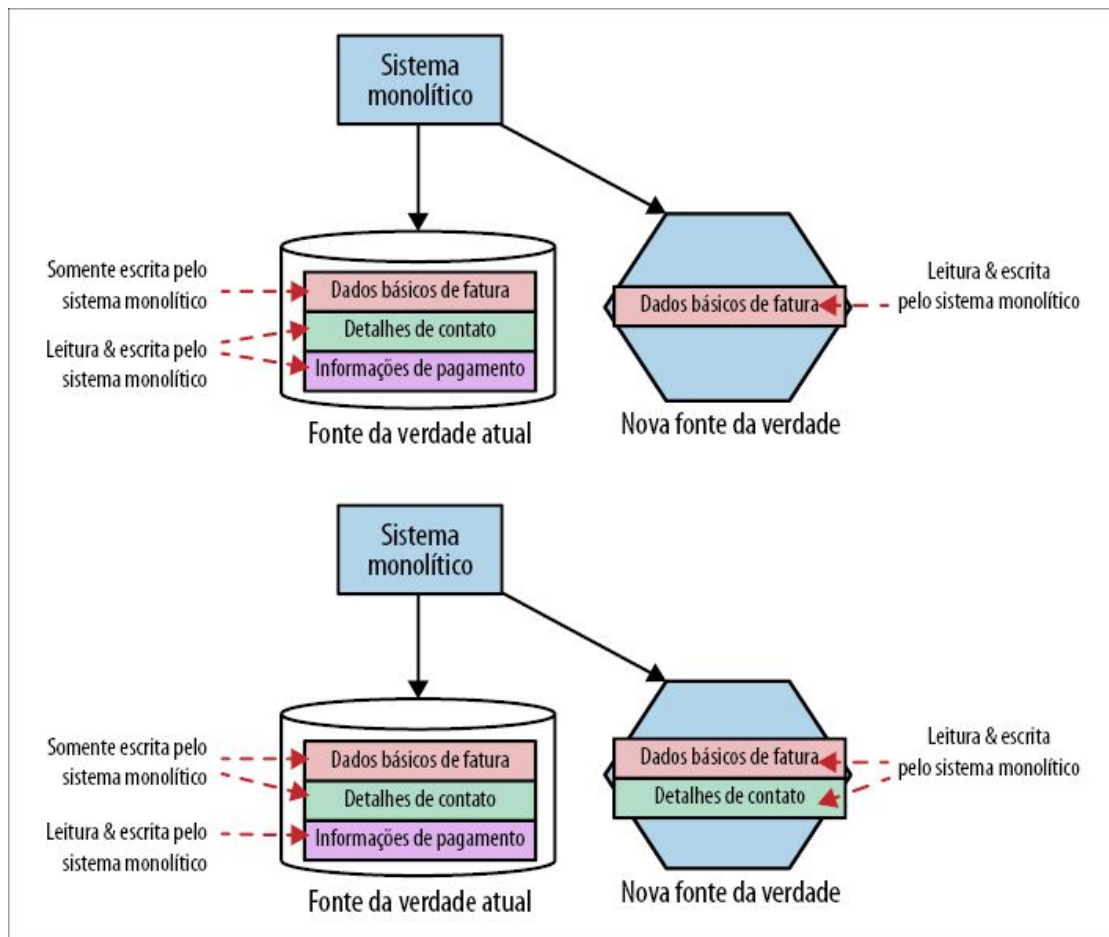


Figura 4.20 – Desativando a antiga fonte da verdade como parte de um tracer write.

Sincronização de dados

O principal problema a ser resolvido com o padrão tracer write é um problema que importuna qualquer situação na qual os dados são duplicados: a inconsistência. Para resolver isso, temos algumas opções:

Escrever em uma só fonte

Todas as escritas são enviadas a uma das fontes da verdade. Os dados são sincronizados com a outra fonte da verdade após a

escrita.

Enviar as escritas para as duas fontes

Todas as requisições de escrita feitas por clientes upstream são enviadas para as duas fontes da verdade. Isso é feito garantindo-se que o cliente faça uma chamada para cada fonte da verdade, ou contando com um intermediário que faça um broadcast da requisição para cada serviço downstream.

Enviar escritas para uma das fontes

Os clientes podem enviar requisições de escrita para uma das fontes da verdade e, nos bastidores, os dados serão sincronizados entre os sistemas, de forma bidirecional.

As duas opções, que consistem em enviar as escritas para as duas fontes da verdade ou enviá-las para apenas uma delas e contar com alguma forma de sincronização em segundo plano, parecem ser soluções viáveis, e o exemplo que exploraremos em breve utiliza ambas as técnicas. No entanto, embora, tecnicamente, seja uma opção, essa situação – na qual as escritas são feitas em uma ou na outra fonte da verdade – deve ser evitada, pois exige uma sincronização bidirecional (algo que pode ser muito difícil de fazer).

Em todos esses casos, haverá certa demora para que os dados se tornem consistentes nas duas fontes da verdade. A duração dessa janela de inconsistência dependerá de diversos fatores. Por exemplo, se você usar um processo batch que execute todas as noites e copie as atualizações de uma fonte para outra, a segunda fonte da verdade poderia conter dados desatualizados em até 24 horas. Se você estiver constantemente enviando atualizações de um sistema para outro, talvez usando um sistema de captura de dados modificados (change data capture), as janelas de inconsistência poderiam ser da ordem de segundos ou menos.

Não importa a duração dessa janela de inconsistência, uma sincronização como essa nos dá o que chamamos de *consistência*

eventual (eventual consistency) – em algum momento, as duas fontes da verdade terão os mesmos dados. Você deve saber qual é o período de inconsistência apropriado para o seu caso, e usar essa informação para direcionar o modo como implementará a sincronização.



É importante que, ao manter duas fontes da verdade dessa forma, você tenha algum tipo de processo de reconciliação para garantir que a sincronização esteja funcionando conforme esperado. Pode ser algo simples, como algumas consultas SQL executadas em cada banco de dados. Contudo, se você não verificar se a sincronização está funcionando conforme esperado, poderá acabar com inconsistências entre os dois sistemas, e só perceberá quando for tarde demais. Executar sua nova fonte da verdade por um tempo, enquanto ela ainda não tiver consumidores, até que você esteja satisfeito com o modo como tudo está funcionando – o que, conforme exploraremos na próxima seção, foi feito pela Square, por exemplo – é uma decisão muito sensata.

Exemplo: pedidos na Square

Quem compartilhou originalmente esse padrão comigo foi Derek Hammer, um desenvolvedor da Square; desde então, já vi outros exemplos do padrão sendo usados por aí.⁴ Ele detalhou o uso desse padrão para ajudar a separar parte do domínio da Square relacionado a pedidos de comida para entrega. No sistema inicial, um único conceito de Pedido era usado para gerenciar vários fluxos de trabalho: um para os clientes que pediam comida, outro para o restaurante que preparava a comida e um terceiro fluxo gerenciava o estado relacionado aos entregadores que pegavam a comida e a levavam até os clientes. As necessidades dos três stakeholders (pessoas-chaves) eram diferentes e, embora todos eles trabalhassem com o mesmo Pedido, o significado de um Pedido era diferente para cada um. Para os clientes, é algo que eles escolheram para que fosse entregue e pelo qual precisavam pagar. Para o restaurante, era algo que devia ser preparado e que alguém deveria buscar. E, para o entregador, era algo que deveria ser levado do restaurante para o cliente em um tempo adequado. Apesar dessas diferentes necessidades, o código e os dados

associados aos pedidos estavam todos juntos.

Ter todos esses fluxos de trabalho reunidos nesse único conceito de Pedido dava origem ao que chamei anteriormente de *desavenças nas entregas* – diferentes desenvolvedores tentando fazer alterações para casos de uso distintos acabavam atrapalhando uns aos outros, pois todos precisavam fazer mudanças na mesma parte da base de código. A Square queria separar um Pedido de modo que mudanças em cada fluxo de trabalho pudessem ser feitas isoladamente, além de permitir escalar de modo diferente e atender a diferentes exigências de robustez.

Criando o serviço

O primeiro passo foi criar um serviço de Atendimentos, conforme vemos na Figura 4.21, que gerenciaria os dados de Pedido associados aos restaurantes e aos entregadores. Esse serviço passaria a ser a nova fonte da verdade a partir de então, para esse subconjunto dos dados de Pedido. Inicialmente, o serviço simplesmente expunha uma funcionalidade para permitir que as entidades relacionadas aos Atendimentos fossem criadas. Assim que o novo serviço foi ativado, a empresa tinha um processo worker em segundo plano que copiava os dados relacionados aos Atendimentos, do sistema existente para o novo serviço de Atendimentos. Esse worker somente fazia uso de uma API exposto pelo serviço de Atendimentos, em vez de fazer uma inserção direta no banco de dados, evitando a necessidade de um acesso direto.

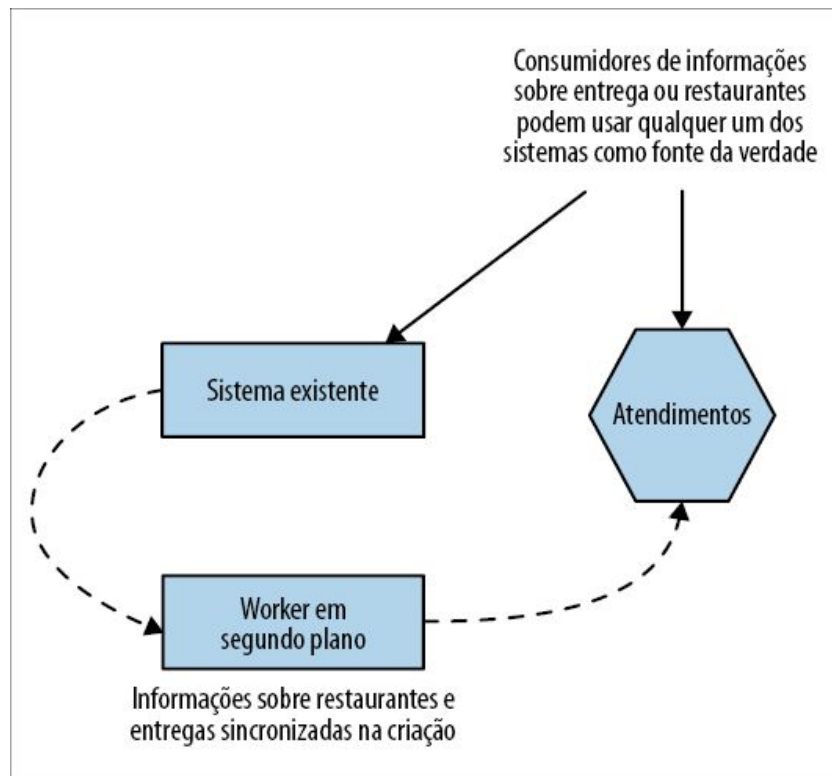


Figura 4.21 – O novo serviço de Atendimentos foi usado para replicar dados do sistema existente, relacionados aos atendimentos.

O worker em segundo plano era controlado por uma flag de recurso (feature flag) que poderia ser ativada ou desativada para interromper o processo de cópia. Isso permitiu que a empresa garantisse que, caso o worker causasse algum problema no ambiente de produção, o processo fosse facilmente desativado. Eles executaram esse sistema em produção por tempo suficiente, até que tivessem certeza de que a sincronização estava funcionando corretamente. Assim que se sentiram satisfeitos em saber que o worker em segundo plano estava funcionando conforme esperado, a flag de recurso foi removida.

Sincronizando os dados

Um dos desafios nesse tipo de sincronização é o fato de ele ser unidirecional. Mudanças feitas no sistema existente fizeram com que os dados relacionados aos atendimentos fossem escritos no novo serviço de Atendimentos por meio de sua API. A Square resolveu o

problema garantindo que todas as atualizações fossem feitas nos dois sistemas, conforme vemos na Figura 4.22. Porém, nem todas as atualizações precisavam ser feitas nos dois sistemas. Como explicou Derek, agora que o serviço de Atendimentos representava somente um subconjunto do conceito de Pedido, apenas mudanças feitas no pedido que fossem do interesse dos clientes de entregas ou dos restaurantes precisavam ser copiadas.

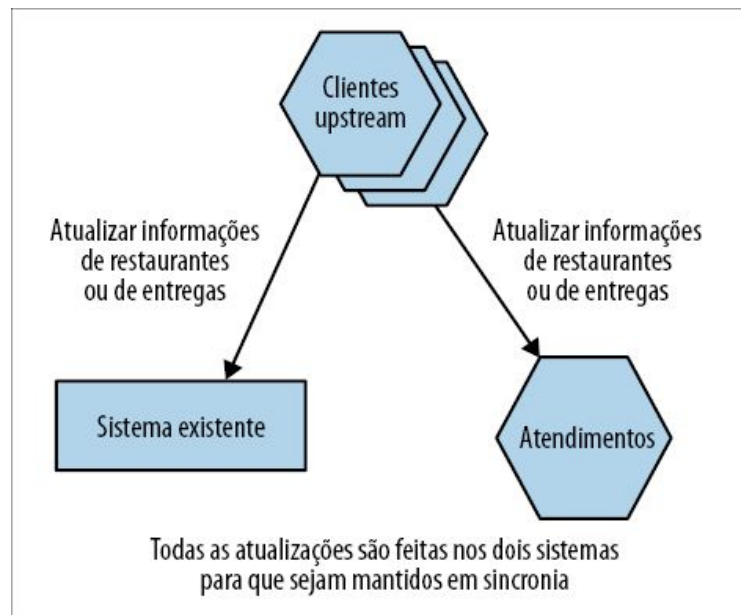


Figura 4.22 – Atualizações subsequentes eram sincronizadas garantindo que todos os clientes fizessem chamadas de API apropriadas para os dois serviços.

Qualquer código que modificasse informações relacionadas a restaurantes ou a entregas teve de ser modificado para que fizesse dois conjuntos de chamadas de API – um para o sistema existente e outro para o microsserviço. Esses clientes upstream também precisavam lidar com quaisquer condições de erro caso a escrita em um dos sistemas tivesse sucesso, mas falhasse no outro. Essas mudanças nos dois sistemas downstream (o sistema de pedidos existente e o novo serviço de Atendimentos) não eram feitas atomicamente. Isso significa que poderia haver uma pequena janela na qual uma mudança estaria visível em um sistema, mas não no outro. Até que as duas mudanças tivessem sido aplicadas,

poderíamos ver uma inconsistência entre os dois sistemas; é uma forma de *consistência eventual*, sobre a qual falamos antes.

No que diz respeito à natureza da consistência eventual das informações de Pedido, nesse caso de uso em particular, não era um problema. Os dados eram sincronizados com rapidez suficiente entre os dois sistemas, a ponto de não causar impacto nos usuários do sistema.

Se a Square estivesse usando um sistema orientado a eventos para gerenciar atualizações de Pedido, em vez de fazer uso de chamadas de API, uma implementação alternativa poderia ter sido considerada. Na Figura 4.23, temos um único fluxo de mensagens que poderia disparar mudanças no estado de um Pedido. Tanto o sistema existente como o novo serviço de Atendimentos receberiam as mesmas mensagens. Os clientes upstream não precisariam saber que há vários consumidores para essas mensagens; é algo que poderia ser tratado com o uso de um broker de estilo pub-sub.

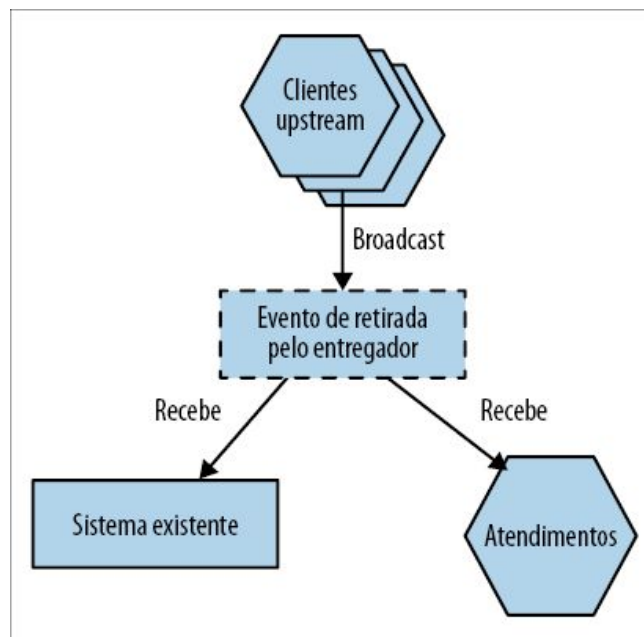


Figura 4.23 – Em uma abordagem alternativa de sincronização, as duas fontes da verdade poderiam se inscrever para receber os mesmos eventos.

Refazer a arquitetura da Square para que fosse orientada a eventos

somente para satisfazer a esse tipo de caso de uso seria muito trabalhoso. No entanto, se você já faz uso de um sistema orientado a eventos, talvez seja mais fácil gerenciar o processo de sincronização. Também vale a pena observar que uma arquitetura como essa continuaria exibindo um comportamento de consistência eventual, pois não é possível garantir que ambos os sistemas – o sistema existente e o serviço de Atendimentos – processariam o mesmo evento ao mesmo tempo.

Migrando os clientes

Com o novo serviço de Atendimentos agora armazenando todas as informações necessárias aos fluxos de trabalho para os restaurantes e os entregadores, o código que gerenciava esses fluxos poderia começar a usar o novo serviço. Durante essa migração, outras funcionalidades poderiam ser adicionadas para atender às necessidades desses clientes; inicialmente, o serviço de Atendimentos só precisava implementar uma API que permitisse a criação de registros para o worker em segundo plano. À medida que a migração de novos clientes ocorresse, suas necessidades poderiam ser avaliadas e novas funcionalidades poderiam ser acrescentadas ao serviço para atendê-las.

Essa migração gradual, tanto dos dados como dos clientes para que usassem a nova fonte da verdade, provou ser extremamente eficaz no caso da Square. Derek disse que chegar ao ponto em que todos os clientes haviam passado para o novo serviço acabou sendo um processo sem grandes contratempos. Foi apenas outra pequena mudança feita durante um lançamento rotineiro de versão (outro motivo pelo qual venho defendendo padrões de migração graduais de maneira tão enfática neste livro!).

Do ponto de vista de um design orientado a domínios, sem dúvida, você poderia argumentar que as funcionalidades associadas aos entregadores, aos consumidores e aos restaurantes representam diferentes contextos delimitados. Com base nesse ponto de vista, Derek sugeriu que o ideal teria sido considerar a separação

adicional desse serviço de Atendimentos em dois serviços – um para Restaurantes e outro para Entregadores. Apesar disso, ainda que houvesse escopo para mais decomposição, essa migração parece ter sido bem-sucedida.

No caso da Square, ela decidiu manter os dados duplicados. Deixar as informações de pedido relacionadas aos restaurantes e às entregas no sistema existente permitiu que a empresa fornecesse visibilidade para essas informações caso o serviço de Atendimentos estivesse indisponível. Isso exigiu que mantivessem a sincronização, é claro. Eu me pergunto se isso será revisto com o tempo. Assim que houver confiança suficiente na disponibilidade do serviço de Atendimentos, remover o worker em segundo plano e a necessidade de os clientes fazerem dois conjuntos de chamadas de atualização poderia ajudar a simplificar a arquitetura.

Quando usar o padrão

A implementação da sincronização provavelmente deve ser a parte que exige mais trabalho. Se você puder evitar a necessidade de uma sincronização bidirecional e, em vez disso, usar alguma das opções mais simples apresentadas, é provável que vá achar esse padrão muito mais fácil de implementar. Se você já faz uso de um sistema orientado a eventos, ou tem um pipeline de captura de dados modificados disponível, provavelmente já deverá ter muitos dos blocos de construção à sua disposição para fazer a sincronização funcionar.

Um planejamento minucioso deve ser feito em relação ao período que você poderá tolerar uma inconsistência entre os dois sistemas. Alguns casos de uso talvez não se importem, mas outros podem querer que a replicação seja quase imediata. Quanto menor a janela de inconsistência aceitável, mais difícil será implementar esse padrão.

Separando o banco de dados

Já discutimos bastante os desafios de usar bancos de dados como um método para integrar vários serviços. Como já deve estar bem claro a essa altura, não sou muito fã dessa solução! Isso significa que precisamos encontrar pontos de separação também em nossos bancos de dados para que possamos dividi-los de forma organizada. Os bancos de dados, porém, são criaturas complicadas. Antes de apresentar alguns exemplos de abordagens, precisamos discutir rapidamente de que modo uma separação lógica e uma implantação física podem estar relacionadas.

Separação física versus separação lógica de um banco de dados

Quando falamos de separar nossos bancos de dados neste contexto, estamos tentando essencialmente fazer uma separação lógica. Uma única engine de banco de dados é perfeitamente capaz de hospedar mais de um esquema separado do ponto de vista lógico, conforme vemos na Figura 4.24.

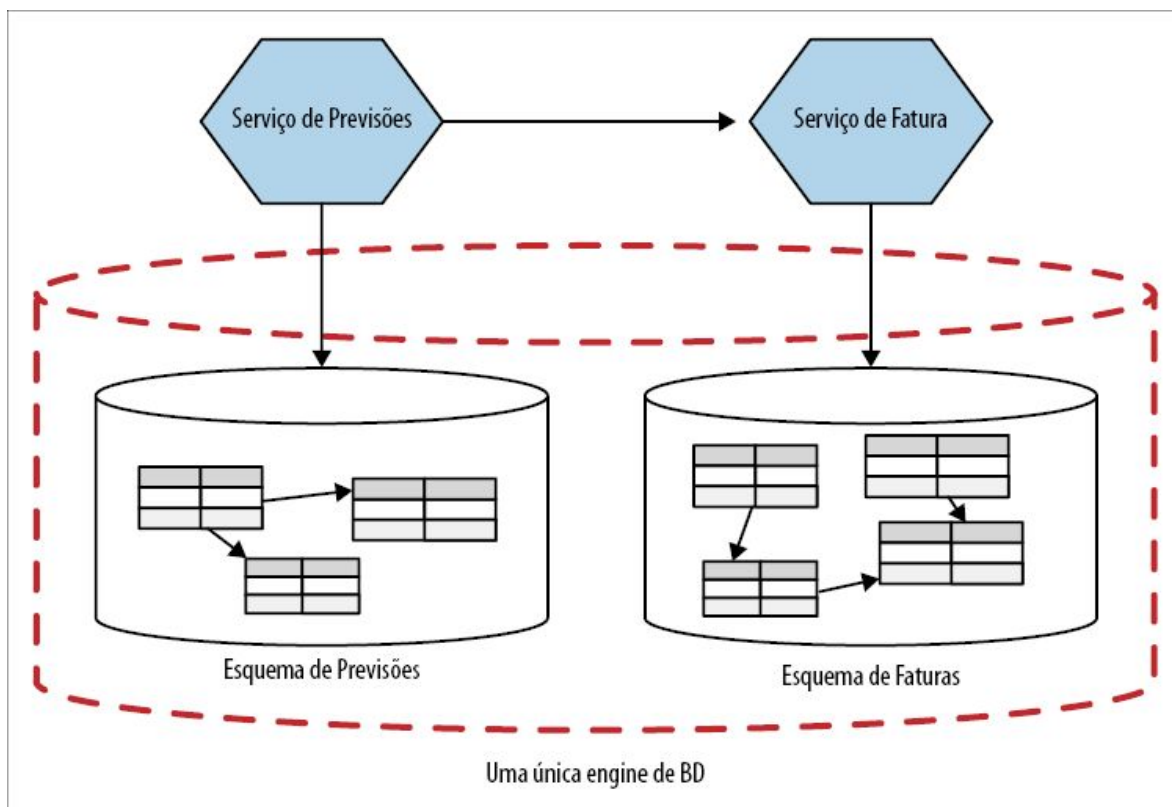


Figura 4.24 – Dois serviços fazendo uso de esquemas separados do ponto de vista lógico, ambos executando na mesma engine física de banco de dados.

Podíamos levar isso adiante e fazer com que cada esquema lógico estivesse também em engines diferentes de banco de dados, proporcionando também uma separação física, como mostra a Figura 4.25.

Por que você iria querer decompor logicamente seus esquemas, mas ainda os manter em uma única engine de banco de dados? Bem, basicamente, uma decomposição lógica e uma decomposição física têm objetivos distintos. Uma *decomposição lógica* permite mudanças independentes de modo mais simples e possibilita ocultar informações, enquanto uma *decomposição física* pode melhorar a robustez do sistema e poderia ajudar a acabar com a contenção de recursos para melhorar o throughput ou diminuir a latência.

Quando Decompomos nossos esquemas de banco de dados do ponto de vista lógico, mas os mantemos na mesma engine física, como vemos na Figura 4.24, temos um possível ponto de falha único. Se a engine de banco de dados falhar, os dois serviços serão afetados. No entanto, o mundo não é tão simples assim. Muitas engines de banco de dados têm sistemas que evitam pontos de falha únicos, como modos multiprimary, sistemas de warm failover e recursos desse tipo. Com efeito, um esforço significativo pode ter sido investido para criar um cluster de banco de dados extremamente resiliente em sua empresa, e poderia ser difícil justificar que haja vários clusters em virtude do tempo, do esforço e dos custos que estariam envolvidos (aquelas terríveis taxas de licença podem se somar e atingir um valor bem alto!).

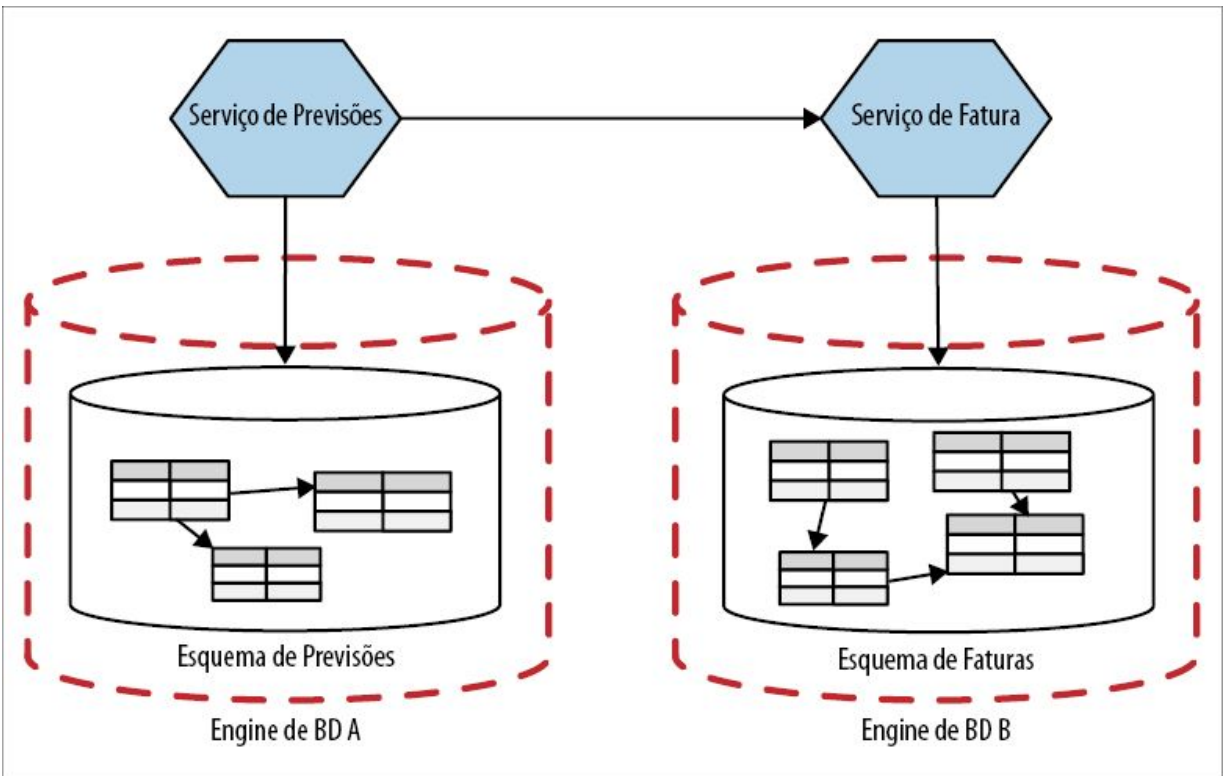


Figura 4.25 – Dois serviços fazendo uso de esquemas separados do ponto de vista lógico, cada um executando em sua própria engine física de banco de dados.

Outra consideração a ser feita é que ter vários esquemas compartilhando a mesma engine de banco de dados pode ser necessário se você quiser expor visões (views) de seu banco de dados. Tanto o banco de dados original como os esquemas que hospedam as visões talvez precisem estar na mesma engine.

É claro que, para que você tenha no mínimo a opção de executar serviços separados em diferentes engines físicas de banco de dados, você já deverá ter decomposto seus esquemas do ponto de vista lógico!

Separar o banco de dados ou o código antes?

Até agora, falamos de padrões que ajudam a trabalhar com bancos de dados compartilhados, na expectativa de passar para modelos com menos acoplamento. Em breve, veremos com mais detalhes os

padrões relacionados à decomposição do banco de dados. Antes disso, porém, precisamos discutir a sequência das tarefas. A extração de um microsserviço não estará “terminada” até que o código da aplicação esteja executando em seu próprio serviço, e os dados controlados por ele tenham sido extraídos em seu próprio banco de dados, isolado do ponto de vista lógico. Contudo, por este livro se tratar, em sua maior parte, de viabilizar mudanças graduais, temos de explorar um pouco a sequência de como essa extração deve ocorrer. Temos algumas opções:

- separar o banco de dados antes, e depois o código;
- separar o código antes, e depois o banco de dados;
- separar ambos ao mesmo tempo.

Cada opção tem seus prós e seus contras. Vamos analisar essas opções agora, junto com alguns padrões que poderão ajudar, conforme a abordagem que você adotar.

Separar o banco de dados antes

Com um esquema separado, existe a possibilidade de aumentarmos o número de chamadas ao banco de dados para que uma única ação seja executada. Enquanto, antes, era possível ter todos os dados que queríamos usando uma única instrução `SELECT`, agora talvez tenhamos de extrair esses dados de dois lugares diferentes e fazer uma junção em memória. Além disso, acabamos com a integridade das transações quando passamos a ter dois esquemas, o que poderia causar impactos significativos em nossas aplicações; discutiremos esses desafios mais adiante neste capítulo, quando abordarmos tópicos como transações distribuídas e sagas, e como elas poderiam nos ajudar. Ao separar os esquemas, mas manter o código da aplicação no sistema monolítico, como vemos na Figura 4.26, criamos a oportunidade de poder reverter as mudanças ou de continuar fazendo ajustes sem causar impactos em nenhum dos clientes de nosso serviço, caso venhamos a perceber que estamos no caminho errado. Assim que estivermos satisfeitos com a

separação do banco de dados, poderemos, então, pensar em separar o código de aplicação em dois serviços.

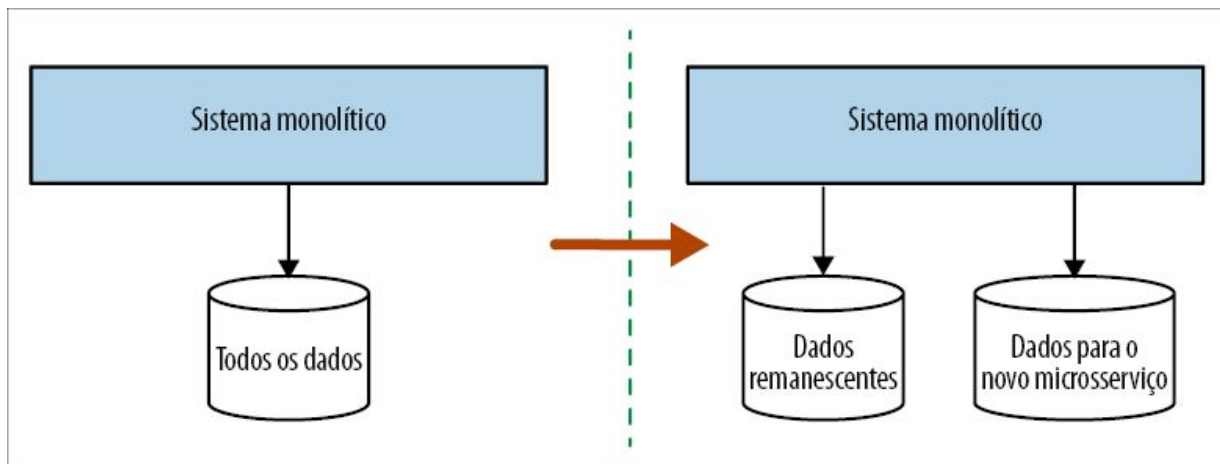


Figura 4.26 – Separar o esquema antes pode permitir que você identifique problemas de desempenho e de integridade de transações mais cedo.

O outro lado da moeda é o fato de ser pouco provável que essa abordagem traga muitos benefícios no curto prazo. Continua sendo necessário fazer a implantação de um código monolítico. Sem dúvida, a dificuldade de ter um banco de dados compartilhado é algo que só será sentida com o passar do tempo; desse modo, estamos investindo tempo e esforços agora para que tenhamos um retorno no longo prazo, sem usufruir vantagens suficientes no curto prazo. Por esse motivo, é provável que eu adotasse essa abordagem se estivesse particularmente preocupado com possíveis problemas de desempenho ou de consistência de dados. Também devemos considerar que, se o sistema monolítico for um sistema caixa-preta – por exemplo, um software comercial –, essa opção não estará disponível para nós.

Uma nota sobre ferramentas

Mudar os bancos de dados é difícil por vários motivos, e um deles é o fato de haver poucas ferramentas disponíveis para que possamos fazer mudanças com facilidade. Com o código, temos ferramentas de refatoração incluídas em nossos IDEs, além da vantagem adicional de que, basicamente, os sistemas que

estamos modificando não têm estados (são stateless). Com um banco de dados, estamos mudando algo que tem estados, porém não há boas ferramentas como aquelas usadas na refatoração.

Muitos anos atrás, essa lacuna nas ferramentas fez com que eu e dois colegas, Nick Ashley e Graham Tackley, criássemos uma ferramenta de código aberto chamada DBDeploy. Atualmente extinta (criar uma ferramenta de código aberto é muito diferente de manter uma!), ela funcionava permitindo que as mudanças fossem capturadas em scripts SQL, que podiam ser executados de modo determinístico nos esquemas. Cada esquema continha uma tabela especial, usada para controlar quais scripts de esquema haviam sido aplicados.

O objetivo do DBDeploy era permitir que você fizesse mudanças graduais em um esquema, com cada mudança podendo ser gerenciada por um sistema de controle de versões, e permitir que as mudanças fossem executadas em vários esquemas em momentos diferentes (pense em esquemas para desenvolvedores, testes e produção).

Atualmente, indico o FlywayDB (<https://flywaydb.org>) ou algo que ofereça recursos semelhantes, mas, qualquer que seja a ferramenta escolhida, peço que você se certifique de que ela permita capturar cada mudança em um script de diferenças, que possa ser gerenciado por um sistema de controle de versões.

Padrão: repositório por contexto delimitado

Uma prática comum é ter uma camada de repositório, com o auxílio de algum tipo de framework como o Hibernate, para vincular seu código ao banco de dados, facilitando mapear objetos ou estruturas de dados de e para o banco de dados. Em vez de ter uma única camada de repositório para todos os nossos problemas de acesso aos dados, é importante separar esses repositórios de acordo com os contextos delimitados, como mostra a Figura 4.27.

Ter o código de mapeamento do banco de dados no mesmo local em que está o código para um dado contexto pode nos ajudar a entender quais partes do banco de dados são usadas por quais

porções de código. O Hibernate, por exemplo, pode deixar isso muito claro se você estiver usando algo como um arquivo de mapeamento por contexto delimitado. Desse modo, podemos ver quais contextos delimitados acessam quais tabelas de nosso esquema. Isso pode nos ajudar a compreender melhor quais tabelas devem ser movidas como parte de qualquer decomposição no futuro.

No entanto, essa não é a história toda. Por exemplo, talvez possamos dizer que o código de finanças utiliza a tabela de contabilidade, e que o código de catálogo utiliza a tabela de itens de pedido, mas não esteja claro que o banco de dados impõe um relacionamento de chave estrangeira da tabela de contabilidade para a tabela de itens de pedido. Para ver essas restrições no nível do banco de dados, as quais poderiam constituir um obstáculo, precisamos de outra ferramenta para visualizar os dados. Um ótimo ponto de partida seria usar uma ferramenta como o SchemaSpy (<http://schemaspy.sourceforge.net>), disponível gratuitamente, que pode gerar representações gráficas dos relacionamentos entre as tabelas.

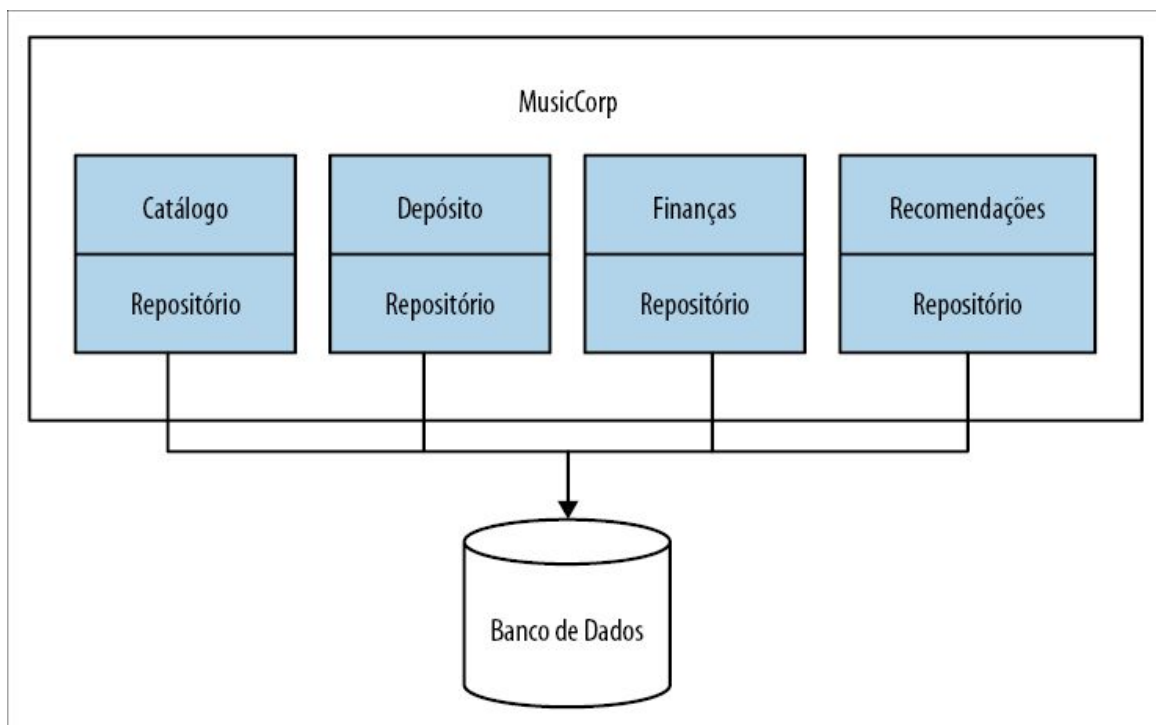


Figura 4.27 – Separando nossas camadas de repositórios.

Tudo isso ajudará você a entender o acoplamento que há entre as tabelas, o qual poderá se estender para além do que, em algum momento, serão as fronteiras do serviço. No entanto, como é possível eliminar essas amarrações? E o que dizer dos casos em que as mesmas tabelas são usadas em vários contextos delimitados? Exploraremos esse assunto com muito mais detalhes posteriormente neste capítulo.

Quando usar o padrão. Esse padrão é realmente eficaz em qualquer situação em que você esteja tentando retrabalhar o sistema monolítico a fim de saber qual é a melhor forma de separá-lo. Separar essas camadas de repositório de acordo com os conceitos de domínio contribuirá bastante para que você determine os locais em que pode haver pontos de separação para os microsserviços, não só em seu banco de dados, mas também no próprio código.

Padrão: banco de dados por contexto delimitado

Assim que você tiver claramente isolado o acesso aos dados do ponto de vista da aplicação, faz sentido continuar com essa abordagem no esquema. No cerne da ideia de implantações independentes para os microsserviços está o fato de que eles devem ser donos de seus próprios dados. Antes de começar a separar o código da aplicação, podemos dar início a essa decomposição separando claramente nossos bancos de dados segundo as linhas dos contextos delimitados que identificamos.

Na ThoughtWorks, estávamos implementando alguns novos métodos para calcular e prever receitas para a empresa. Como parte desse trabalho, havíamos identificado três grandes áreas de funcionalidades que precisavam ser reescritas. Discuti o problema com o líder desse projeto, Peter Gillard-Moss. Peter explicou que as funcionalidades pareciam bem isoladas, mas ele tinha preocupações quanto ao trabalho extra que elas implicariam caso fossem colocadas em microsserviços separados. Na época, sua

equipe era pequena (apenas três pessoas), e havia a impressão de que a equipe não poderia justificar a separação desses novos serviços. No final, eles concordaram com um modelo no qual a nova funcionalidade de receita era efetivamente implantada como um único serviço, contendo três contextos delimitados isolados (cada um deles acabou se transformando em um arquivo JAR separado), como mostra a Figura 4.28.

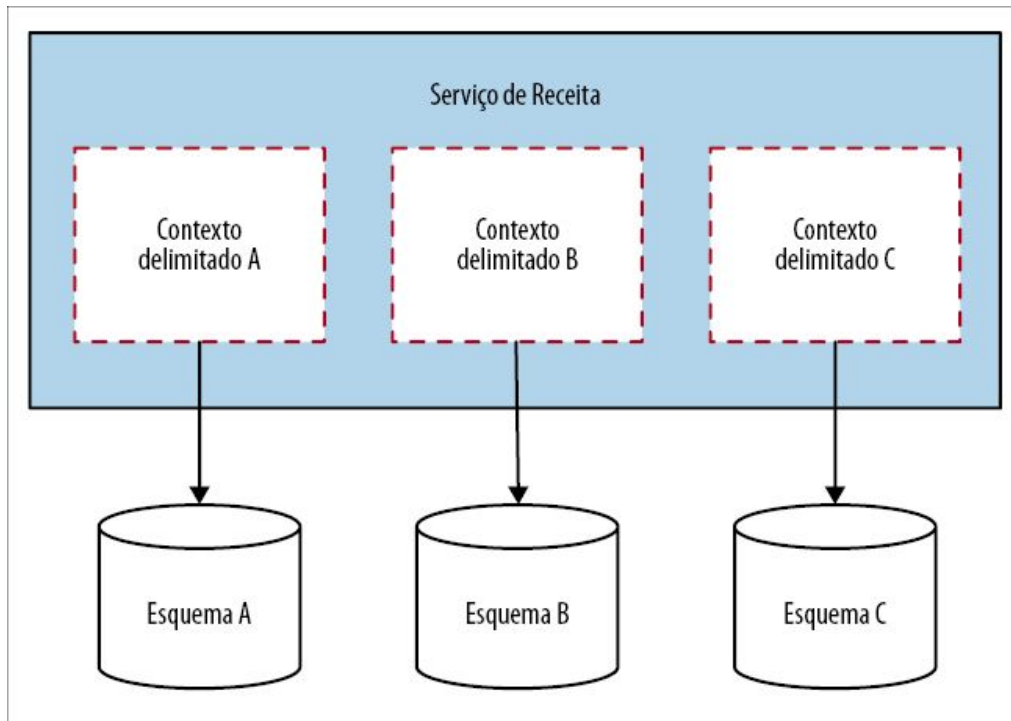


Figura 4.28 – Cada contexto delimitado no serviço de Receita tinha seu próprio esquema de banco de dados separado, permitindo uma separação no futuro.

Cada contexto delimitado tinha seu próprio banco de dado totalmente separado. A ideia era que, se houvesse a necessidade de uma separação em microsserviços mais tarde, essa seria uma tarefa muito mais fácil. No entanto, o fato é que esse trabalho jamais veio a ser necessário. Vários anos depois, esse serviço de receita permanece como está: um sistema monolítico com vários bancos de dados associados – um ótimo exemplo de um sistema monolítico modular.

Quando usar o padrão. À primeira vista, o trabalho extra para manter os bancos de dados separados não fará muito sentido se você deixar tudo como um sistema monolítico. Vejo esse padrão como algo totalmente relacionado a uma aposta no futuro. A solução exige um pouco mais de trabalho em comparação a ter um único banco de dados, mas ela manterá abertas as suas opções quanto a passar para microsserviços mais tarde. Mesmo que você jamais passe a usar microsserviços, ter uma separação clara dos esquemas que dão suporte ao banco de dados pode realmente ajudar, sobretudo se você tiver muitas pessoas trabalhando no sistema monolítico.

Esse é um padrão que quase sempre recomendo às pessoas que estão desenvolvendo sistemas totalmente novos (em oposição a estarem reimplementando um sistema existente). Não sou fã de implementar microsserviços para novos produtos ou startups; sua compreensão sobre o domínio provavelmente não estará suficientemente madura a ponto de poder identificar fronteiras de domínio estáveis. Particularmente no caso das startups, a natureza daquilo que você está construindo pode mudar drasticamente. Esse padrão, porém, pode ser uma boa solução intermediária. Mantenha a separação de esquemas quando achar que você poderá ter serviços separados no futuro. Dessa forma, você terá algumas das vantagens do desacoplamento dessas ideias, ao mesmo tempo que reduzirá a complexidade do sistema.

Separar o código antes

Em geral, percebo que a maioria das equipes separa o código antes, e depois o banco de dados, como mostra a Figura 4.29. Elas se beneficiam com as melhorias de curto prazo do novo serviço (ou, pelo menos, essa é a expectativa), o que lhes dá confiança para concluir a decomposição, separando o banco de dados.

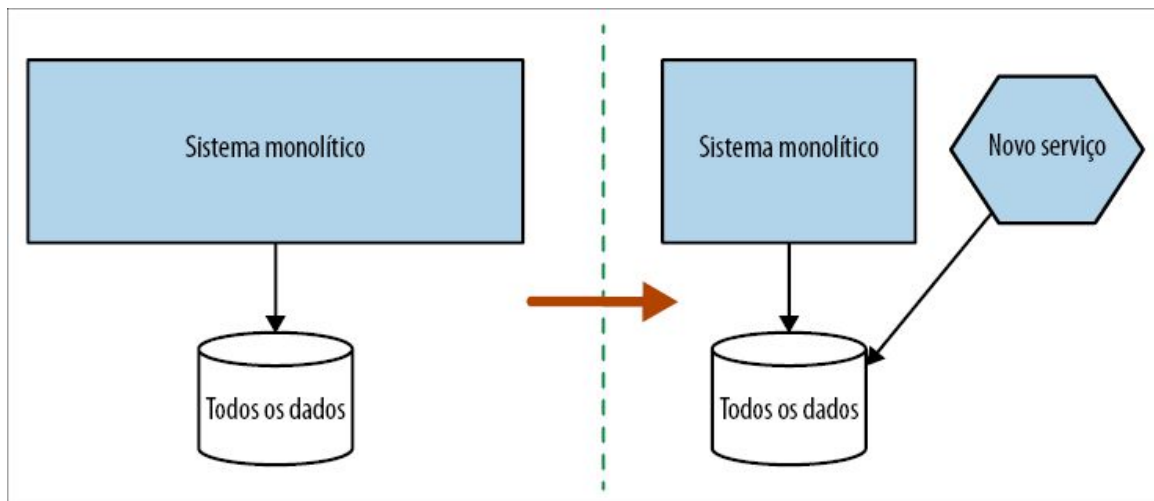


Figura 4.29 – Separar a camada de aplicação antes nos deixa com um único esquema compartilhado.

Ao separar a camada de aplicação, torna-se muito mais fácil entender quais dados são necessários ao novo serviço. Você também terá a vantagem de ter um artefato de código que possa ser implantado de forma independente mais cedo. As preocupações que sempre tive em relação a essa abordagem é que as equipes poderão chegar até esse ponto e então parar, deixando um banco de dados compartilhado em ação continuamente. Se esse é o caminho que você vai tomar, saiba que estará sujeito a problemas no futuro caso não conclua a separação na camada de dados. Já vi equipes que caíram nessa armadilha, mas também posso falar com satisfação de empresas que tomaram a atitude correta nessa situação. A JustSocial é uma das empresas que usou essa abordagem como parte de sua migração para os microsserviços. O outro desafio em potencial é que você poderia estar postergando a descoberta de surpresas desagradáveis resultantes de colocar as operações de junção na camada de aplicação.

Se esse é o caminho que você vai tomar, seja honesto consigo mesmo: você está confiante de que poderá garantir que qualquer dado que seja do microsserviço será separado como parte do próximo passo?

Padrão: sistema monolítico como uma camada de acesso aos

dados

Em vez de acessar os dados diretamente no sistema monolítico, podemos simplesmente passar para um modelo no qual criamos uma API no próprio sistema monolítico. Na Figura 4.30, o serviço de Fatura precisa de informações sobre os funcionários em nosso serviço de Clientes, portanto criamos uma API de Funcionários que permite que o serviço de Fatura os acesse. Susanne Kaiser da JustSocial compartilhou esse padrão comigo, pois a empresa o utilizou com sucesso como parte de sua migração para microsserviços. O padrão proporciona tantas vantagens que estou surpreso com o fato de ele não parecer tão bem conhecido quanto deveria.

Parte do motivo para esse padrão não ser usado de modo mais amplo provavelmente se deve ao fato de as pessoas, de certo modo, pensarem que o sistema monolítico está morto, e que ele não tem mais utilidade. Elas querem se afastar dele. Não consideram torná-lo mais útil! Contudo, as vantagens, nesse caso, são evidentes: não precisamos lidar com a decomposição de dados (ainda), porém conseguimos ocultar as informações, facilitando manter o nosso novo serviço isolado do sistema monolítico. Eu estaria mais inclinado a adotar esse modelo se achasse que os dados do sistema monolítico vão permanecer ali. No entanto, o padrão pode funcionar bem, particularmente se você achar que seu novo serviço será efetivamente sem estados (stateless).

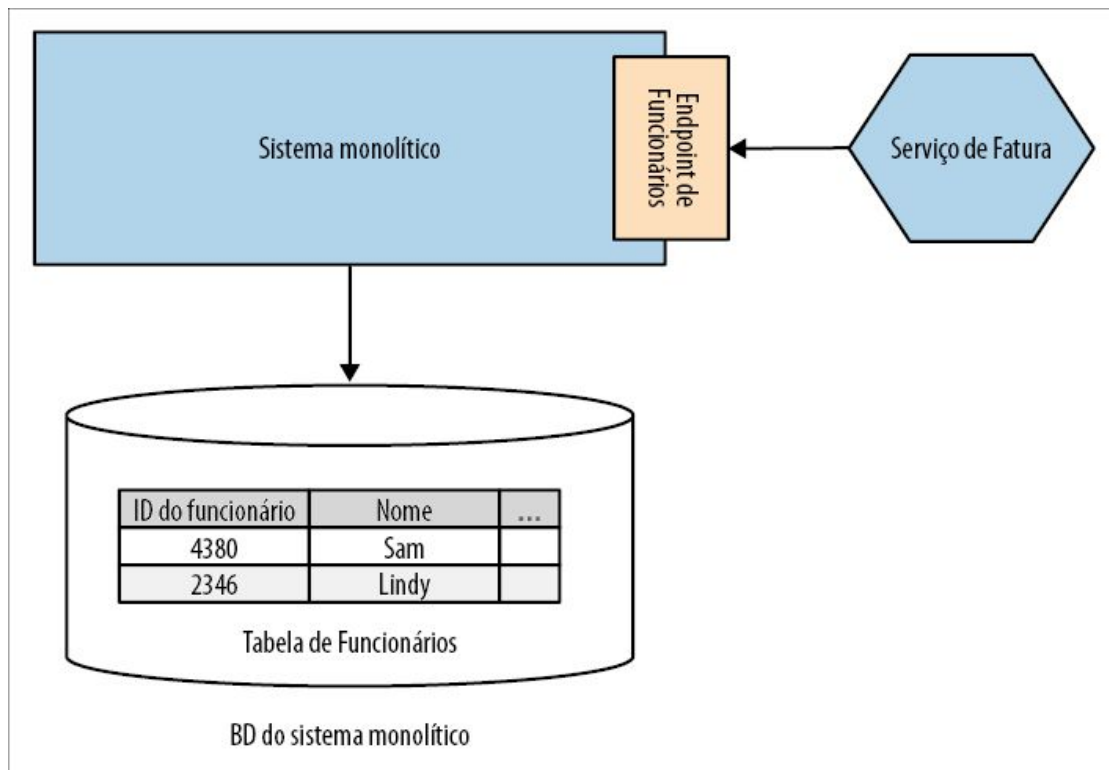


Figura 4.30 – Expor uma API no sistema monolítico permite que o serviço evite uma vinculação direta com os dados.

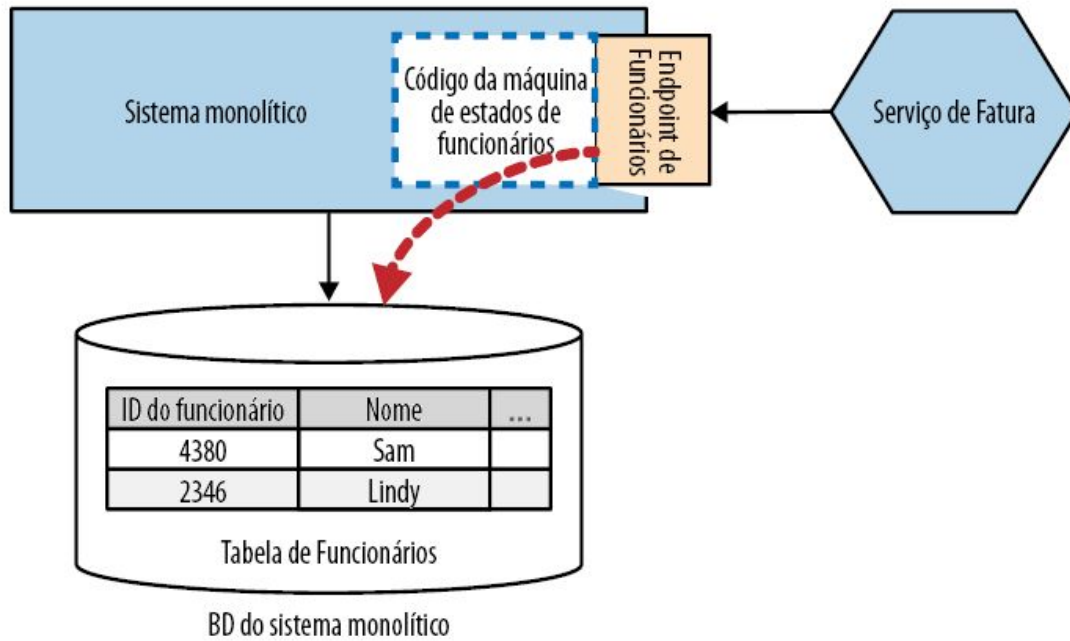
Não é difícil ver esse padrão como uma forma de identificar outros candidatos a serviços. Estendendo a ideia, poderíamos ver a API de Funcionários sendo separada do sistema monolítico a fim de se transformar em um microserviço por si só, como vemos na Figura 4.31.

Quando usar o padrão. Esse padrão funciona melhor se o código que gerencia os dados continuar no sistema monolítico. Conforme já discutimos, uma maneira de pensar em um microserviço quando se trata de dados é pensar no encapsulamento do estado e do código que gerencia as transições desse estado. Portanto, se as transições de estado desses dados continuarem sendo feitas no sistema monolítico, isso implica que o microserviço que quiser acessar esses estados (ou mudá-los) precisará fazê-lo por meio das transições de estado do sistema monolítico.

Se os dados que você estiver tentando acessar no banco de dados do sistema monolítico realmente tiverem de ser de “propriedade” do

microserviço, sinto-me mais inclinado a sugerir que você não use esse padrão e procure separar os dados.

Antes: Dados de funcionários e funcionalidade hospedados no sistema monolítico, acessados por meio de uma API



Depois: Dados e funcionalidade de funcionários extraídos do sistema monolítico e passados para um novo microserviço

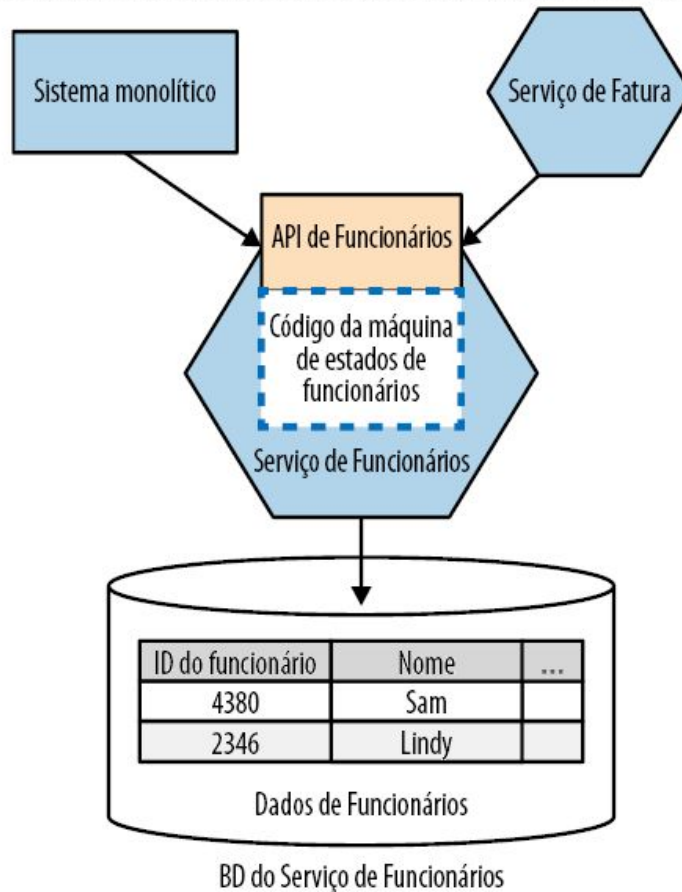


Figura 4.31 – Usando a API de Funcionários para identificar a fronteira de um serviço de Funcionários a ser separado do sistema monolítico.

Padrão: armazenagem com vários esquemas

Conforme já discutimos, não deixar que uma situação ruim piore é uma boa ideia. Se você ainda está fazendo uso direto dos dados de um banco de dados, isso não quer dizer que *novos* dados armazenados por um microserviço também devam ser colocados nesse banco de dados. Na Figura 4.32, vemos um exemplo do nosso serviço de Fatura. Os principais dados de fatura continuam no sistema monolítico, e é aí que eles são acessados (atualmente). Acrescentamos a possibilidade de adicionar comentários às Faturas; isso representa uma funcionalidade totalmente nova, que não está no sistema monolítico. Para dar suporte a ela, precisamos armazenar uma tabela de autores dos comentários, mapeando funcionários a IDs de Fatura. Se colocarmos essa nova tabela no sistema monolítico, estaremos contribuindo para aumentar o banco de dados! Em vez disso, colocaremos esses novos dados em nosso próprio esquema.

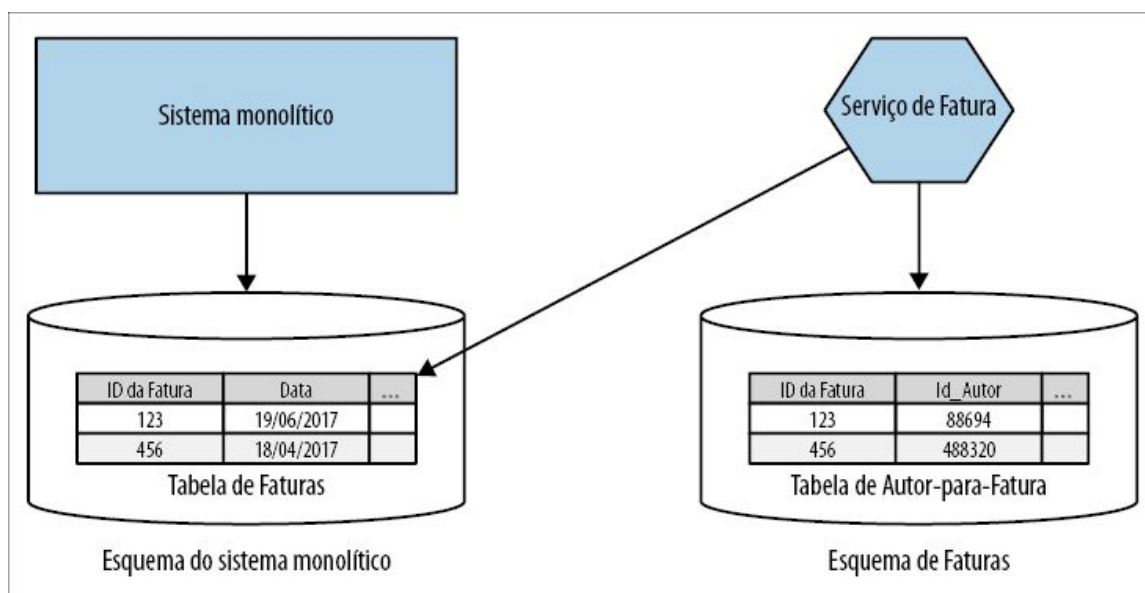


Figura 4.32 – O serviço de Fatura coloca novos dados em seu próprio esquema, mas continua acessando dados antigos

diretamente no sistema monolítico.

Nesse exemplo, devemos considerar o que acontecerá se um relacionamento de chave estrangeira efetivamente ultrapassar a fronteira de um esquema. Mais adiante neste capítulo, exploraremos esse problema com mais detalhes.

Extrair os dados de um banco de dados monolítico exige tempo, e talvez não seja possível fazê-lo em um único passo. Portanto, você deve se sentir muito satisfeito em ter seu microserviço acessando dados no banco de dados do sistema monolítico, ao mesmo tempo que gerencia também a sua própria armazenagem de dados local. À medida que conseguir trazer os dados restantes do sistema monolítico, você poderá fazer a migração de uma tabela de cada vez para o seu novo esquema.

Quando usar o padrão. Esse padrão funcionará bem quando você acrescentar uma funcionalidade totalmente nova em seu microserviço, a qual exija a armazenagem de novos dados. São dados de que, claramente, o sistema monolítico não precisará (a funcionalidade não estará lá), portanto você poderá mantê-los separados desde o princípio. Esse padrão também fará sentido quando você começar a mover dados do sistema monolítico para o seu próprio esquema – um processo que poderá demorar um pouco.

Se os dados que você estiver acessando no esquema do sistema monolítico forem dados que você jamais planejou passar para o seu esquema, recomendo o uso do padrão de sistema monolítico como uma camada de acesso aos dados (veja a seção Padrão: sistema monolítico como uma camada de acesso aos dados), em conjunto com esse padrão.

Separar o banco de dados e o código ao mesmo tempo

Do ponto de vista de etapas, é claro, temos a opção de simplesmente separar tudo em um só grande passo, como vemos na Figura 4.33. Separamos tanto o código como os dados de uma

só vez.

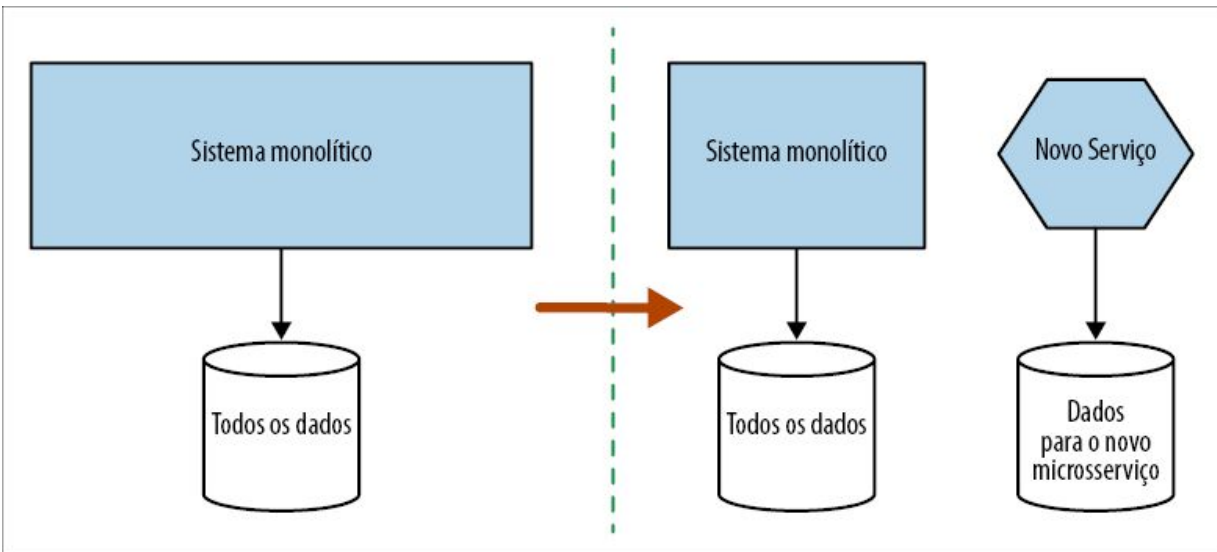


Figura 4.33 – Separando tanto o código como os dados em um só passo.

Minha preocupação nesse caso é que esse é um passo muito maior, e demorará muito para que você possa avaliar o impacto de sua decisão. Recomendo fortemente que evite essa abordagem e, em seu lugar, separe o esquema ou a camada de aplicação antes.

Então, o que devo separar antes?

Eu compreendo: você provavelmente está cansado de toda essa história de “Depende”, não é mesmo? Não posso culpá-lo. O problema é que a situação em que cada pessoa se encontra é diferente; desse modo, estou tentando armar você com contextos e discussões suficientes sobre os diversos prós e contras para que possa tomar a sua própria decisão. No entanto, sei que, às vezes, o que as pessoas querem é um pouco de praticidade, portanto eis o que vou dizer.

Se sou capaz de mudar o sistema monolítico, e estou preocupado com o possível impacto no desempenho ou na consistência dos dados, procurarei separar o esquema antes. Caso contrário, separarei o código e usarei esse processo para ajudar a entender como isso causará impactos na atribuição da responsabilidade pelos

dados. Contudo, é importante que você também pense por si só e leve em consideração quaisquer fatores que possam causar impactos no processo de tomada de decisões em sua situação em particular.

Exemplos de separação de esquemas

Até agora, vimos a separação de esquemas em um nível bem geral, mas há desafios complexos associados à decomposição de bancos de dados, e alguns problemas complicados a serem explorados. Veremos agora alguns padrões de decomposição de dados de nível mais baixo e exploraremos o impacto que podem causar.

Bancos de dados relacionais *versus* NoSQL

Muitas das refatorações de exemplo detalhadas neste capítulo exploram desafios que surgem quando trabalhamos com esquemas relacionais. A natureza desses tipos de bancos de dados dá origem a desafios adicionais no que concerne à separação de esquemas. Muitos de você podem estar usando tipos alternativos de bancos de dados não relacionais. No entanto, vários dos padrões ainda poderão se aplicar. Talvez você tenha menos restrições quanto ao modo como as mudanças podem ser feitas, mas espero que os conselhos continuem sendo úteis.

Padrão: separação de tabelas

Às vezes, você verá dados em uma única tabela que terão de ser separados em duas ou mais fronteiras de serviços, e isso pode ser interessante. Na Figura 4.34, vemos uma única tabela compartilhada, Item, na qual armazenamos informações não só sobre o item sendo vendido, mas também sobre os níveis de estoque.

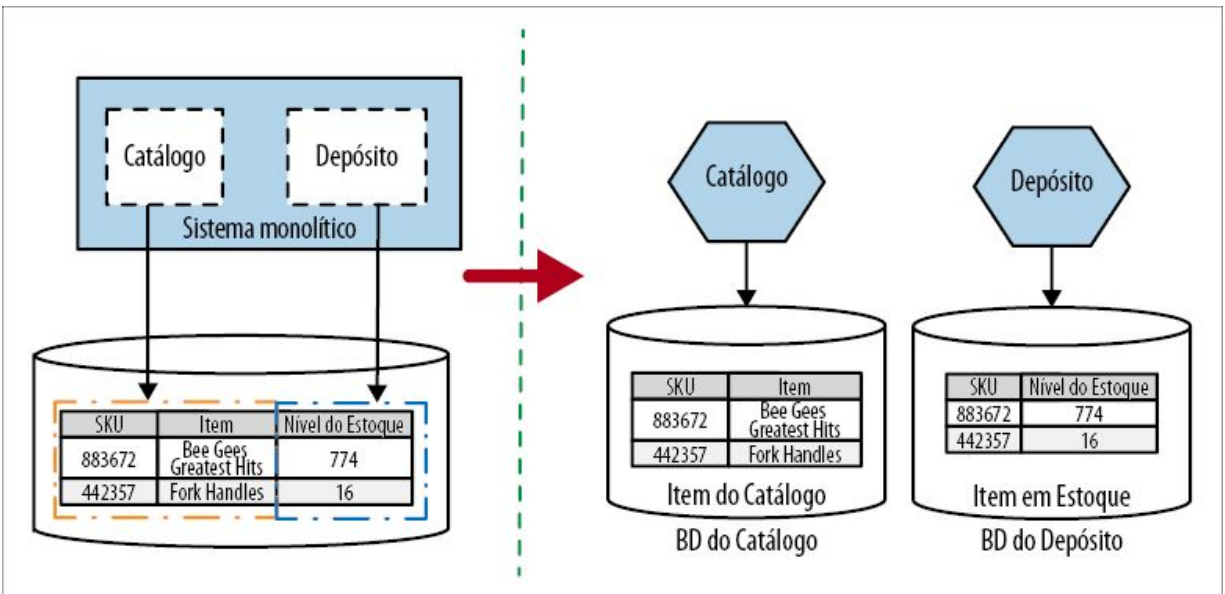


Figura 4.34 – Uma única tabela que liga dois contextos delimitados sendo separada.

Neste exemplo, queremos separar Catálogo e Depósito em novos serviços, mas os dados de ambos estão misturados em uma única tabela. Portanto, precisamos separar esses dados em duas tabelas distintas, como mostra a Figura 4.34. No espírito da migração gradual, faz sentido separar as tabelas no esquema existente antes de separar os esquemas. Se essas tabelas estiverem em um único esquema, faria sentido declarar um relacionamento de chave estrangeira da coluna SKU do Item em Estoque para a tabela Item do Catálogo. No entanto, como, em última análise, planejamos passar essas tabelas para bancos de dados separados, talvez não haja muitas vantagens nisso, pois não teremos um único banco de dados para garantir a consistência dos dados (exploraremos essa ideia com mais detalhes em breve).

Este exemplo é bem simples; foi fácil separar a responsabilidade pelos dados de acordo com a coluna. Todavia, o que acontecerá se várias porções de código atualizarem a mesma coluna? Na Figura 4.35, temos uma tabela de Clientes, que contém uma coluna Status.

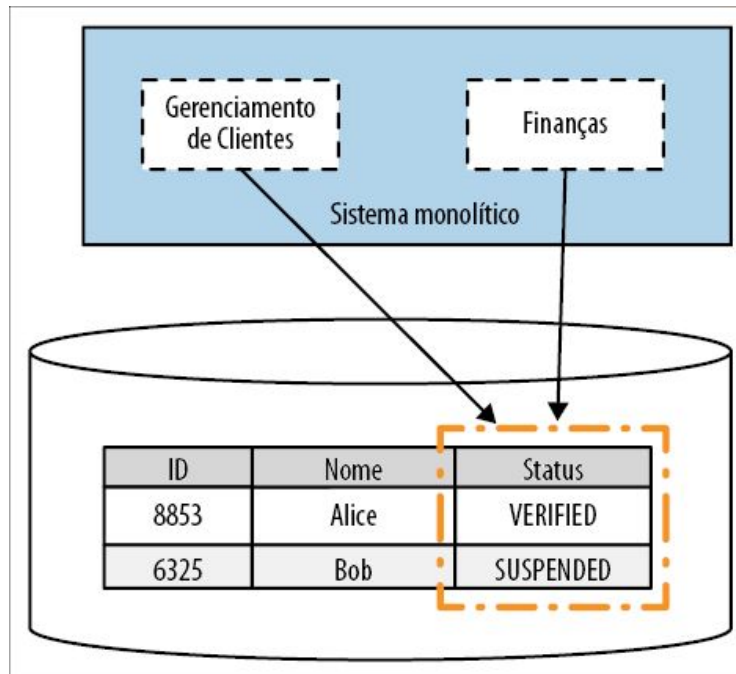


Figura 4.35 – Tanto o código de gerenciamento de clientes como o código de finanças podem alterar o status na tabela de Clientes.

Essa coluna é atualizada durante o processo de cadastramento de clientes para informar que uma dada pessoa teve (ou não) seu email conferido, com o valor passando de NOT_VERIFIED → VERIFIED (Não Verificado para Verificado). Assim que o cliente tiver um status VERIFIED, ele poderá fazer compras. Nosso código de finanças trata da suspensão dos clientes caso suas contas não tenham sido pagas, portanto, ocasionalmente, eles mudarão o status de um cliente para SUSPENDED (Suspensão). Nessa situação, o Status de um cliente ainda parece fazer parte do modelo de domínio do cliente e, desse modo, deveria ser gerenciado pelo serviço de Clientes prestes a ser criado. Lembre-se de que, se for possível, queremos manter as máquinas de estado das entidades de nossos domínios dentro das fronteiras de um único serviço, e atualizar um Status certamente parece fazer parte da máquina de estados de um cliente! Isso significa que, quando a separação do serviço estiver concluída, nosso novo serviço de Finanças precisará fazer uma chamada de serviço para atualizar esse status, conforme vemos na Figura 4.36.

Um grande problema ao separar tabelas dessa maneira é que perdemos a segurança proporcionada pelas transações de banco de dados. Exploraremos esse assunto com mais detalhes em seções mais próximas ao final deste capítulo, quando veremos as Transações e as Sagas.

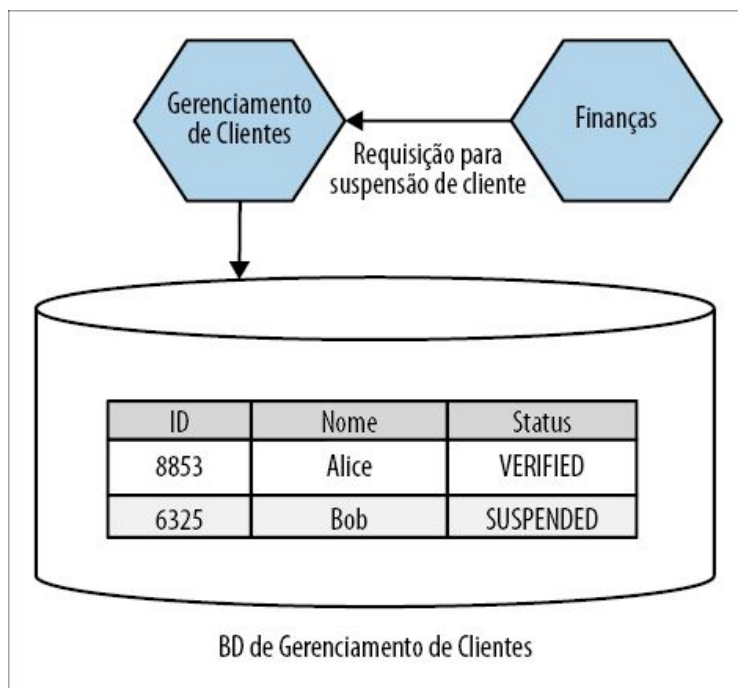


Figura 4.36 – O novo serviço de Finanças deve fazer chamadas de serviço para suspender um cliente.

Quando usar o padrão

À primeira vista, a solução parece ser bem simples. Quando a tabela pertencer a dois ou mais contextos delimitados em seu sistema monolítico atual, será necessário separar a tabela de acordo com essas fronteiras. Se você encontrar colunas específicas nessa tabela que pareçam ser atualizadas por várias partes de sua base de código, será preciso usar de discernimento a fim de determinar quem deverá ser o “responsável” por esses dados. É um conceito de domínio existente que está no escopo? Isso ajudará você a determinar a quem esses dados devem pertencer.

Padrão: transferência de relacionamentos de chave estrangeira para o código

Decidimos extrair o nosso serviço de Catálogo – um código capaz de gerenciar e expor informações sobre artistas, faixas musicais e álbuns. Atualmente, nosso código relacionado ao catálogo no sistema monolítico utiliza a tabela Álbuns para armazenar informações sobre os CDs disponíveis para venda. Esses álbuns acabam sendo referenciados em nossa tabela de Contabilidade, que é a tabela na qual gerenciamos todas as vendas. Podemos ver isso na Figura 4.37. As linhas da tabela de Contabilidade simplesmente registram o valor que recebemos por um CD, junto com um identificador que referencia o item vendido; o identificador em nosso exemplo se chama SKU (Stock Keeping Unit, ou Unidade de Manutenção de Estoque) – uma prática comum em sistemas de varejo.

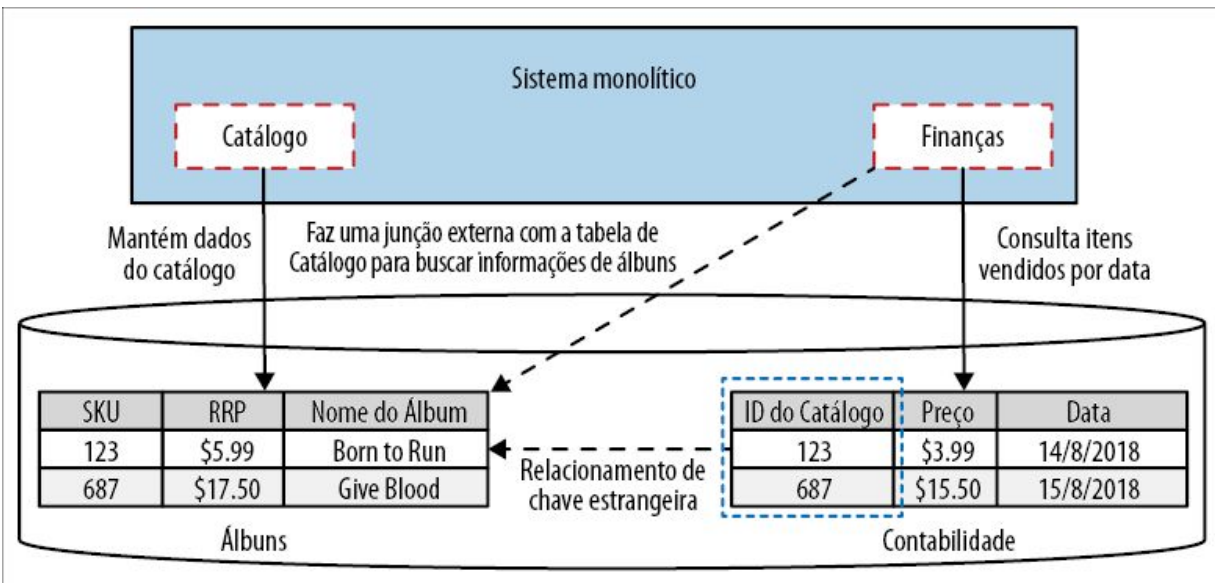


Figura 4.37 – Relacionamento de chave estrangeira.

No final de cada mês, devemos gerar um relatório apresentando os CDs com as maiores vendagens. A tabela de Contabilidade nos ajuda a saber qual SKU vendeu o maior número de cópias, mas as informações sobre esse SKU estão na tabela Álbuns. Queremos deixar o relatório fácil de ler, portanto, em vez de dizer que

“Vendemos 400 cópias do SKU 123 e ganhamos 1.596 dólares”, gostaríamos de acrescentar informações sobre o que foi vendido, dizendo: “Vendemos 400 cópias de *Born to Run* de Bruce Springsteen e ganhamos 1.596 dólares”. Para isso, a consulta no banco de dados disparada pelo nosso código de finanças deve fazer uma junção das informações da tabela de Contabilidade com a tabela Álbuns, como mostra a Figura 4.37.

Definimos um relacionamento de chave estrangeira em nosso esquema, de modo que uma linha da tabela de Contabilidade é identificada como tendo um relacionamento com uma linha da tabela Álbuns. Ao definir um relacionamento como esse, a engine do banco de dados subjacente é capaz de garantir a consistência dos dados – isto é, se uma linha da tabela de Contabilidade referenciar uma linha da tabela Álbuns, saberemos que essa linha existe. Em nosso caso, isso significa que sempre poderemos obter informações sobre o álbum que foi vendido. Esses relacionamentos de chave estrangeira também permitem que a engine do banco de dados faça otimizações de desempenho a fim de garantir que a operação de junção seja tão rápida quanto possível.

Queremos separar o código de Catálogo e o código de Finanças em seus respectivos serviços, e isso significa que os dados devem vir juntos também. As tabelas Álbuns e Contabilidade acabarão em esquemas diferentes, portanto, o que acontecerá com nosso relacionamento de chave estrangeira? Bem, temos dois problemas principais a serem considerados. Em primeiro lugar, quando o novo serviço de Finanças quiser gerar esse relatório no futuro, como ele obterá informações relacionadas ao Catálogo se não puder mais fazer isso por meio de uma junção no banco de dados? O outro problema é: o que fazer com o fato de agora poder haver inconsistência de dados no novo mundo?

Transferindo a junção

Inicialmente, veremos como é possível substituir a junção. Em um

sistema monolítico, para fazer a junção de uma linha da tabela Álbum com as informações de venda da tabela de Contabilidade, faríamos o banco de dados efetuar a junção para nós. Executaríamos uma única consulta SELECT, na qual faríamos uma junção com a tabela Álbuns. Isso exigiria uma única chamada de banco de dados para executar a consulta e extrair os dados necessários.

Em nosso novo mundo com microsserviços, o novo serviço de Finanças é responsável por gerar o relatório dos mais vendidos, mas não tem os dados dos álbuns localmente. Desse modo, ele deverá buscar esses dados junto ao serviço de Catálogo, conforme vemos na Figura 4.38.

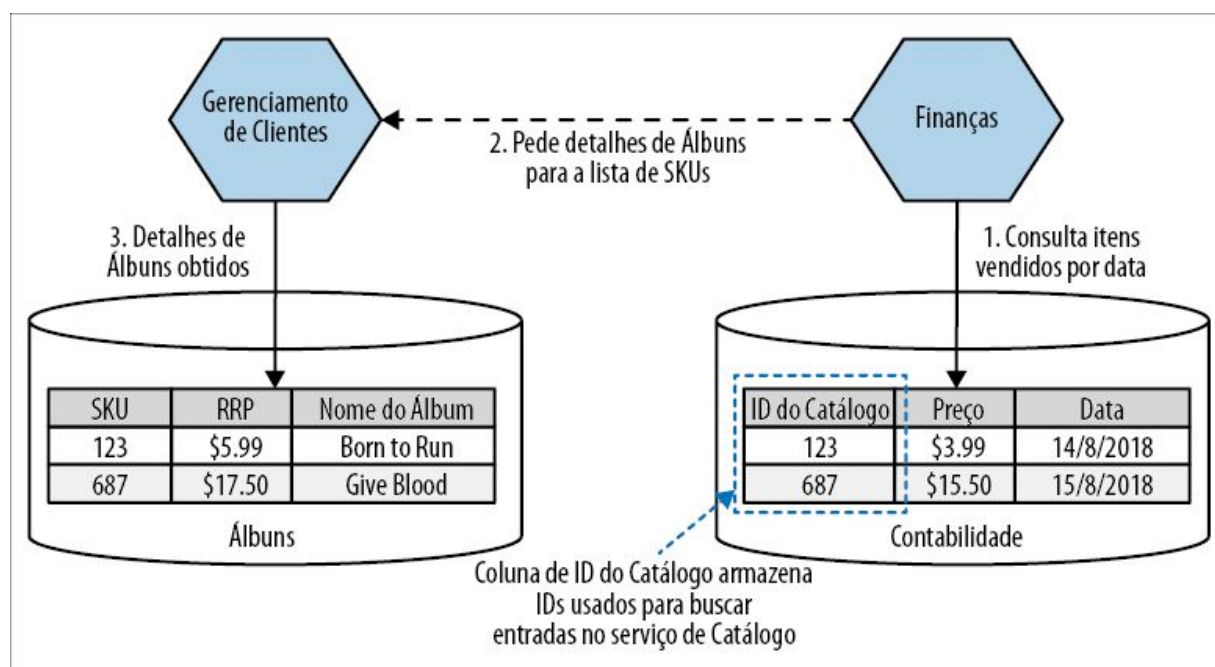


Figura 4.38 – Substituindo uma operação de junção de banco de dados por chamadas de serviço.

Ao gerar o relatório, o serviço de Finanças inicialmente consulta a tabela de Contabilidade, extraíndo a lista dos SKUs dos mais vendidos no último mês. A essa altura, tudo que temos é uma lista de SKUs e o número de cópias vendidas para cada um; é a única informação que temos localmente.

Em seguida, devemos chamar o serviço de Catálogo, solicitando informações sobre cada um desses SKUs. Essa requisição, por sua vez, faria o serviço de Catálogo executar o próprio SELECT local em seu banco de dados.

Do ponto de vista lógico, a operação de junção continua ocorrendo, mas agora acontece dentro do serviço de Finanças, e não no banco de dados. Infelizmente, essa operação está bem longe de ser igualmente eficiente. Passamos de um mundo no qual tínhamos uma única instrução SELECT, para um novo mundo no qual temos uma consulta SELECT na tabela de Contabilidade, seguida de uma chamada ao serviço de Catálogo, o qual, por sua vez, dispara uma instrução SELECT na tabela Álbuns, como mostra a Figura 4.38.

Nessa situação, eu ficaria muito surpreso se a latência total dessa operação não aumentasse. Talvez não seja um problema significativo nesse caso em particular, pois esse relatório é gerado mensalmente e, desse modo, um cache agressivo poderia ser usado. No entanto, se essa fosse uma operação frequente, poderia ser um problema. É possível atenuar o provável impacto desse aumento de latência se permitirmos que os SKUs sejam consultados no serviço de Catálogo em grandes volumes ou, quem sabe, até mesmo fazendo caching localmente das informações necessárias sobre os álbuns.

Em última instância, o fato de esse aumento de latência ser ou não ser um problema é algo que somente você poderá decidir. Você terá de saber qual é a latência aceitável para operações essenciais, e deverá ser capaz de avaliar qual é, atualmente, a latência. Sistemas distribuídos como o Jaeger (<https://www.jaegertracing.io>) podem realmente ajudar nesse caso, pois possibilitam medir a duração exata de operações que englobam vários serviços. Deixar uma operação mais lenta pode ser aceitável se ela ainda for suficientemente rápida, sobretudo se, em contrapartida, houver outras vantagens.

Consistência dos dados

Um aspecto mais intrincado, estando os serviços de Catálogo e de Finanças separados, possuindo esquemas diferentes, é o fato de podermos acabar com uma inconsistência nos dados. Com um único esquema, não seria possível apagar uma linha da tabela Álbuns se houvesse uma referência a essa linha na tabela de Contabilidade. Meu esquema garantia a consistência dos dados. Em nosso novo mundo, essa garantia não existe mais. Quais são as nossas opções nesse caso?

Verifique antes de apagar

Nossa primeira opção poderia ser garantir que, ao apagar um registro da tabela Álbuns, verificássemos junto ao serviço de Finanças para nos certificar de que ele não tenha uma referência ao registro. O problema nesse caso é que garantir que isso seja feito corretamente é complicado. Suponha que queremos apagar o SKU 683. Enviamos uma chamada ao serviço de Finanças dizendo: “Você está usando o SKU 683?”. Ele responde que esse registro não está sendo usado. Então apagamos o registro, mas, enquanto fazemos isso, uma nova referência a 683 é criada no sistema de Finanças. Para impedir que isso aconteça, teríamos de fazer com que novas referências parassem de ser criadas ao registro 683, até que a remoção tivesse ocorrido – uma operação que provavelmente exigiria locks (travas), e todos os desafios que concernem a um sistema distribuído.

Outro problema em verificar se o registro está em uso é o fato de criarmos uma verdadeira dependência reversa com o serviço de Catálogo. Agora teríamos de fazer a verificação junto a qualquer outro serviço que utilizasse nossos registros. Já seria ruim se tivéssemos apenas um serviço usando nossas informações, mas será muito pior se houver mais consumidores.

Sugiro enfaticamente que você não considere essa opção em virtude da dificuldade de garantir que essa operação seja

implementada de forma correta, assim como por causa do alto grau de acoplamento entre serviços que seria introduzido.

Trate a remoção de forma elegante

Uma melhor opção é simplesmente fazer com que o serviço de Finanças lide de forma elegante com o fato de que o serviço de Catálogo possa não ter informações sobre o Álbum. Poderia ser algo simples, como fazer nosso relatório mostrar que “Informações sobre o álbum não estão disponíveis” caso não seja possível consultar um determinado SKU.

Nessa situação, o serviço de Catálogo poderia nos informar caso requisitássemos um SKU que costumava existir. Seria um bom uso de um código de resposta 410 GONE se estivéssemos usando HTTP, por exemplo. Um código de resposta 410 difere do código 404 comumente usado. Um 404 sinaliza que o recurso requisitado não foi encontrado, enquanto um 410 significa que o recurso estava disponível antes, porém não está mais. A distinção pode ser importante, sobretudo para rastrear problemas de inconsistência de dados! Mesmo que um protocolo baseado em HTTP não esteja em uso, considere se você se beneficiaria ou não caso aceitasse esse tipo de resposta.

Se quiséssemos realmente dar um passo além, poderíamos garantir que nosso serviço de Finanças seja informado quando um item do Catálogo for removido, talvez fazendo com que ele se registre para receber eventos. Ao detectar um evento de remoção do Catálogo, poderíamos decidir copiar as informações do Álbum agora removido para o nosso banco de dados local. Essa solução parece ser um exagero nessa situação em particular, mas poderia ser conveniente em outros cenários, particularmente se você quisesse implementar uma máquina de estados distribuída para executar algo como uma remoção em cascata que ultrapassasse as fronteiras entre os serviços.

Não permita a remoção

Uma forma de garantir que não introduziremos muita inconsistência no sistema poderia ser simplesmente não permitir que registros do serviço de Catálogo sejam removidos. Se, no sistema atual, remover um item for o mesmo que garantir que ele não está mais disponível para venda ou algo parecido, poderíamos implementar um recurso de remoção soft. Isso poderia ser feito usando uma coluna de status para marcar que essa linha está indisponível ou, talvez, movendo a linha para uma tabela dedicada de “cemitério”.⁵ O registro do álbum ainda poderia ser requisitado pelo serviço de Finanças nessa situação.

Então, como devemos lidar com a remoção?

Basicamente, criamos um modo de falha que não poderia existir em nosso sistema monolítico. Ao procurar formas de resolver esse problema, devemos considerar as necessidades de nossos usuários, pois diferentes soluções poderiam causar impactos de maneiras distintas. Escolher a solução correta, portanto, exige uma compreensão do contexto específico de cada um.

Pessoalmente, nesta situação em particular, é provável que eu resolvesse o problema de duas maneiras: não permitiria a remoção das informações do álbum no Catálogo e garantiria que o serviço de Finanças pudesse lidar com um registro ausente. Você poderia argumentar que, se um registro não puder ser removido do serviço de Catálogo, a consulta feita pelo serviço de Finança jamais falharia. No entanto, é possível que, como uma consequência de dados corrompidos, o serviço de Catálogo seja restaurado a um estado anterior, o que significa que o registro que estamos procurando não existe mais. Nessa situação, não gostaria que o serviço de Finanças falhasse totalmente. Pode parecer um conjunto improvável de circunstâncias, mas sempre procuro criar sistemas pensando na resiliência e considerar o que acontecerá se uma chamada falhar; lidar com isso de modo elegante no serviço de Finanças parece ser uma solução bem simples.

Quando usar o padrão

Quando começamos a considerar realmente a quebra dos relacionamentos de chave estrangeira, um dos primeiros pontos que devemos garantir é que não estaremos separando dois itens que realmente devam ser apenas um. Se você estiver preocupado com o fato de poder estar separando um agregado, pare e reconsidere. No caso da Contabilidade e de Álbuns nesse exemplo, parece claro que temos dois agregados distintos, com um relacionamento entre eles. Considere, porém, um caso diferente: uma tabela de Pedidos e várias linhas associadas em uma tabela de Itens de Pedido contendo detalhes sobre os itens que foram pedidos. Se separarmos os itens de pedido em um serviço distinto, teríamos de considerar os problemas com a integridade dos dados. De fato, os itens de um pedido fazem parte do próprio pedido. Portanto, devemos vê-los como uma unidade, e, se quisermos mover esses dados para fora do sistema monolítico, eles deverão ser movidos em conjunto.

Às vezes, ao considerar uma fatia maior de nosso esquema monolítico, você será capaz de mover os dois lados de um relacionamento de chave estrangeira, facilitando bastante a sua vida!

Exemplo: dados estáticos compartilhados

Dados de referência estáticos (que raramente mudam, ainda que, em geral, sejam muito importantes) podem dar origem a alguns desafios interessantes, e já vi várias abordagens para lidar com eles. Com muita frequência, esses dados acabam entrando no banco de dados. Talvez eu já tenha visto tantos códigos de países armazenados em bancos de dados (como mostra a Figura 4.39) quantas foram as vezes que escrevi minhas próprias classes String Úteis em projetos Java.

Sempre questionei por que volumes pequenos de dados que raramente mudam, como códigos de países, deveriam estar em

bancos de dados; contudo, independentemente dos motivos subjacentes, esses exemplos de dados estáticos compartilhados armazenados em bancos de dados surgem com muita frequência. Então, o que faremos em nosso exemplo da loja musical, em que várias partes de nosso código precisam dos mesmos dados de referência estáticos? Bem, o fato é que temos *muitas* opções nesse caso.

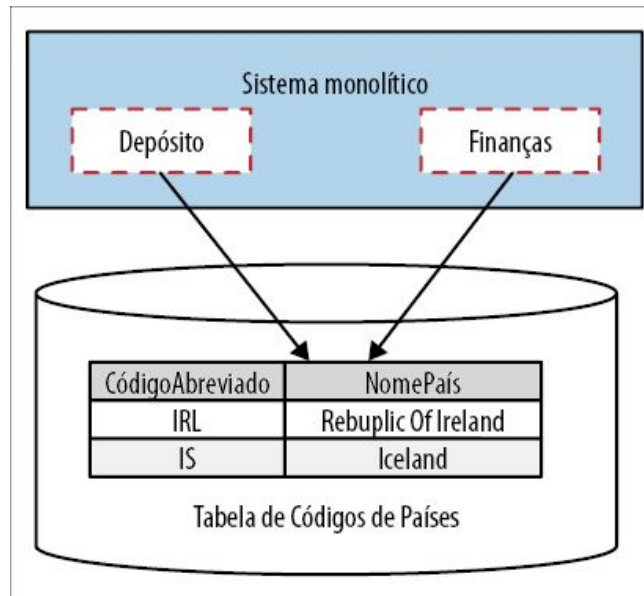


Figura 4.39 – Códigos de países no banco de dados.

Padrão: duplicação de dados de referência estáticos

Por que não fazer com que cada serviço tenha a sua própria cópia dos dados, como vemos na Figura 4.40? É provável que isso fará com que muitos de vocês inspirem com força. Dados duplicados? Você está louco? Ouça-me antes! É menos insano do que você possa imaginar.

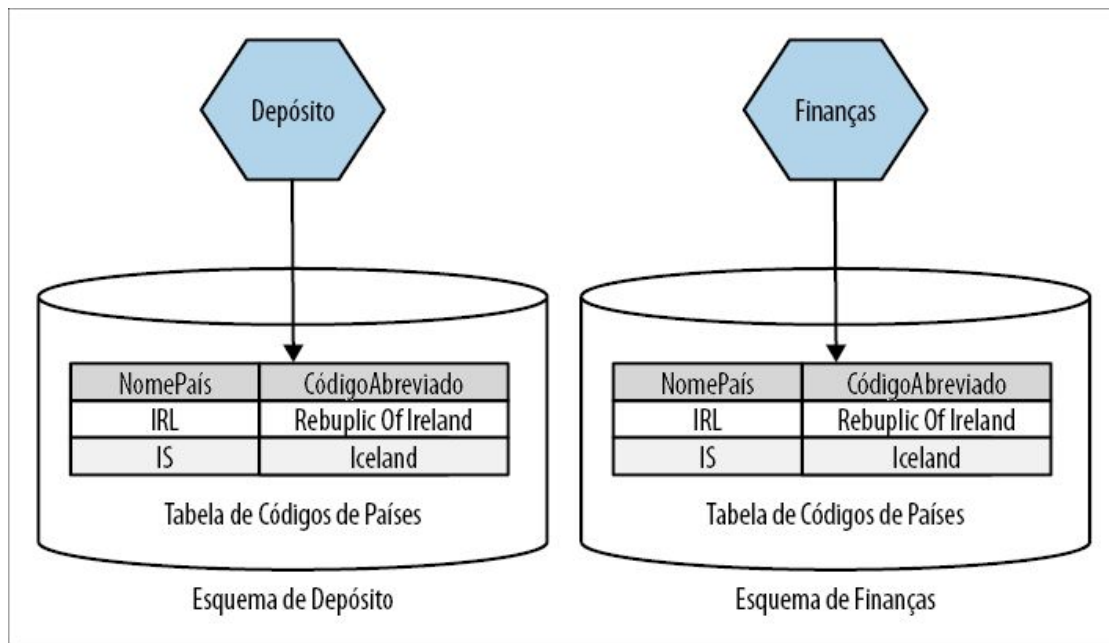


Figura 4.40 – Cada serviço tem seu próprio esquema de Códigos de Países.

Preocupação relacionadas à duplicação de dados tendem a se reduzir a duas questões. A primeira é que, sempre que precisar modificar os dados, terei de fazê-lo em vários lugares. Contudo, nessa situação, com que frequência os dados mudam? A última vez que um país foi criado e oficialmente reconhecido foi em 2011, com a criação do Sudão do Sul (cujo código abreviado é SSD). Portanto, não acho que essa seja uma grande preocupação, não é mesmo? A principal preocupação é o que acontecerá se os dados se tornarem inconsistentes. Por exemplo, o serviço de Finanças sabe que o Sudão do Sul é um país; porém, inexplicavelmente, o serviço de Depósito vive no passado e não sabe nada a esse respeito.

O fato de a inconsistência ser ou não um problema basicamente tem a ver com o modo como os dados são usados. Em nosso exemplo, considere que o Depósito utiliza esses dados de códigos de países para registrar o local em que nossos CDs são fabricados. O fato é que não estocamos nenhum CD fabricado no Sudão do Sul, portanto, o fato de esse dado estar ausente não é um problema. Por outro lado, o serviço de Finanças precisa de informações sobre códigos dos países a fim de registrar informações sobre vendas, e

temos clientes no Sudão do Sul, portanto, precisamos dessa entrada atualizada.

Quando os dados são usados apenas localmente em cada serviço, a inconsistência não será um problema. Pense novamente em nossa definição de contexto delimitado: trata-se de ocultar informações dentro de uma fronteira. Se, por outro lado, os dados fizerem parte da comunicação entre esses serviços, teremos outras preocupações. Se tanto o serviço de Depósito como o serviço de Finanças precisarem ter a mesma visão das informações sobre os países, uma duplicação dessa natureza certamente seria algo com o qual eu deveria me preocupar.

Também poderíamos considerar manter essas cópias sincronizadas usando algum tipo de processo em segundo plano, é claro. Em uma situação como essa, é improvável que vamos garantir que todas as cópias serão consistentes, mas, supondo que nosso processo em segundo plano execute com frequência suficiente (e seja rápido), poderemos reduzir a janela de inconsistência em potencial para os nossos dados, e talvez isso baste.



Como desenvolvedores, com frequência, reagimos mal quando vemos uma duplicação. Nós nos preocupamos com o custo extra de administrar cópias duplicadas de informações, e nos preocuparemos mais ainda com o fato de haver divergências entre esses dados. Às vezes, porém, uma duplicação será a situação menos grave entre duas situações ruins. Aceitar um pouco de duplicação nos dados pode ser uma decisão de custo-benefício razoável se significar que evitaremos a introdução de acoplamentos.

Quando usar o padrão. Esse padrão deve ser usado apenas raramente, e você deve dar preferência a algumas das opções que consideraremos mais adiante. Ocasionalmente, ele será conveniente para grandes volumes de dados, quando não for essencial que todos os serviços vejam exatamente o mesmo conjunto de dados. Algo como arquivos de códigos postais na Inglaterra poderia ser um bom candidato, para os quais você teria atualizações periódicas do mapeamento entre códigos postais e endereços. É um conjunto de dados razoavelmente grande, que

seria provavelmente difícil de administrar em formato de código. Se você quiser fazer uma junção direta com esses dados, este pode ser outro motivo para a escolha dessa opção; contudo, vou ser franco e dizer que não me lembro de jamais ter feito isso pessoalmente!

Padrão: esquema dedicado para dados de referência

Se quiser ter realmente uma única fonte da verdade para os códigos de países, você poderia relocar esses dados para um esquema dedicado, talvez um esquema separado para todos os dados de referência estáticos, conforme podemos ver na Figura 4.41.

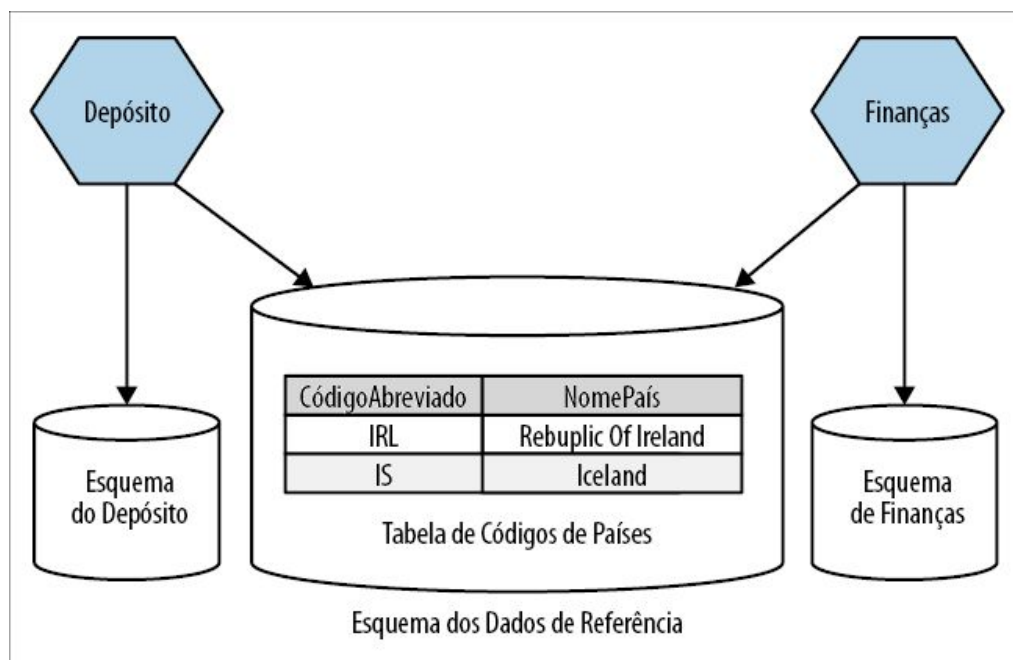


Figura 4.41 – Usando um esquema dedicado compartilhado para dados de referência.

Devemos, porém, considerar todos os desafios de um banco de dados compartilhado. Até certo ponto, as preocupações relacionadas a acoplamentos e mudanças são, de certo modo, reduzidas por causa da natureza dos dados. Esses dados raramente mudam e são estruturados de forma simples; portanto, poderíamos considerar com mais facilidade esse esquema de Dados de Referência como uma interface definida. Nessa situação, eu administraria o esquema de Dados de Referência como uma

entidade independente, sujeita a um controle de versões, e me certificaria de que as pessoas compreendessem que a estrutura do esquema representa uma interface de serviço aos consumidores. Fazer mudanças radicais nesse esquema provavelmente causaria complicações.

Ter dados em um esquema cria a oportunidade para que os serviços ainda usem esses dados como parte de consultas em junção (join queries) com seus próprios dados locais. Para que isso ocorra, porém, é provável que seja necessário garantir que os esquemas estejam na mesma engine subjacente de banco de dados. Isso acrescenta um grau de complexidade no que diz respeito ao modo de mapear o mundo lógico para o mundo físico, além das preocupações com um possível ponto único de falha.

Quando usar o padrão. Esta opção tem vários méritos. Evitamos as preocupações com a duplicação, e é pouco provável que o formato dos dados mude, portanto, algumas de nossas preocupações com acoplamento serão atenuadas. Para grandes volumes de dados, ou se quisemos ter a opção de fazer junções entre esquemas, será uma abordagem válida. Lembre-se, porém, de que qualquer mudança no formato do esquema provavelmente causará um impacto significativo em vários serviços.

Padrão: biblioteca para dados de referência estáticos

Se pensarmos bem, veremos que não há muitas entradas nos dados de códigos de países. Supondo que estamos usando a lista padrão da ISO, veremos apenas 249 países representados.⁶ Seria bem apropriado que esses dados estivessem no código, talvez na forma de um tipo enumerado estático simples. Com efeito, armazenar pequenos volumes de dados de referência estáticos na forma de código é algo que já fiz algumas vezes e que já vi ser feito em arquiteturas de microsserviços.

É claro que preferiríamos não duplicar esses dados se não for necessário, portanto, isso nos leva a considerar a colocação desses dados em uma biblioteca que possa ser estaticamente ligada por

qualquer serviço que queira esses dados. A Stitch Fix, uma empresa norte-americana de varejo ligada à moda, faz uso frequente de bibliotecas compartilhadas como essa para armazenar dados de referência estáticos.

Randy Shoup, que foi VP de engenharia da Stitch Fix, disse que os casos ideais para usar essa técnica foram aqueles em que os dados se apresentavam em um volume pequeno e que raramente ou jamais mudavam (e, se mudassem, você seria avisado da mudança com bastante antecedência). Considere os tamanhos clássicos de vestuários: XS, S, M, L, XL para tamanhos de modo geral, ou medidas de numeração para calças.

Em nosso caso, definimos os mapeamentos de códigos de países em um tipo enumerado `Country`, e o incluímos em uma biblioteca para que nossos serviços a utilizassem, como mostra a Figura 4.42.

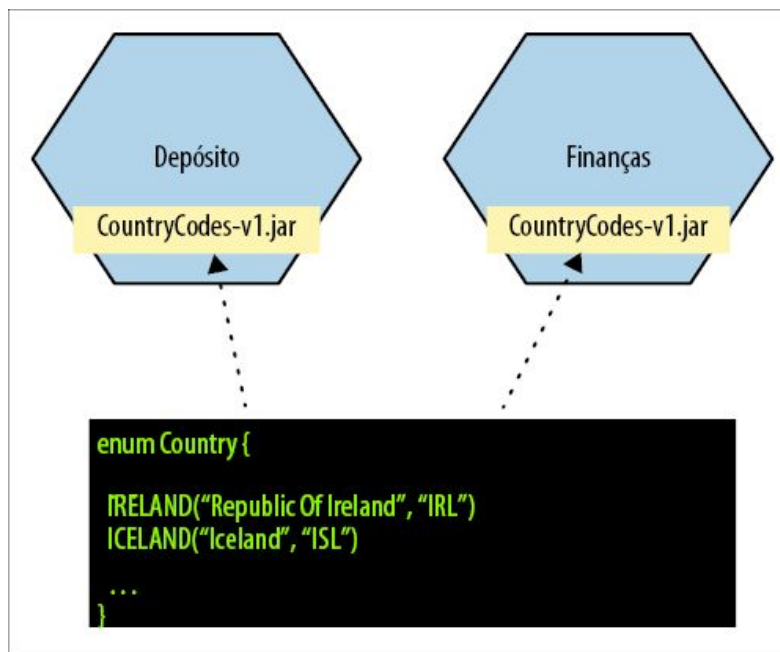


Figura 4.42 – Armazenando dados de referência em uma biblioteca que possa ser compartilhada entre os serviços.

É uma boa solução, mas tem suas desvantagens. Obviamente, se tivermos uma mistura de pilhas de tecnologia, talvez não seja possível ter uma única biblioteca compartilhada. Além disso, você se lembra da regra de ouro dos microsserviços? Devemos garantir que

os microsserviços possam ser implantados de modo independente. Se tivéssemos de atualizar nossa biblioteca de códigos de países e fazer com que todos os serviços pudessem acessar imediatamente os novos dados, seria necessário reimplantar todos os serviços no momento em que a nova biblioteca estivesse disponível. Esse é um caso clássico de lançamento de versão amarrada, e é exatamente o que estamos tentando evitar com as arquiteturas de microsserviços.

Na prática, se for preciso que os mesmos dados estejam disponíveis em todos os lugares, uma simples percepção de que houve uma mudança talvez possa ajudar. Um exemplo dado por Randy foi a necessidade de adicionar uma nova cor a um dos conjuntos de dados da Stitch Fix. Essa mudança deveria ser feita em todos os serviços que faziam uso desse tipo de dado, mas eles tinham tempo suficiente para garantir que todas as equipes tivessem a versão mais recente com antecedência. Se você considerar o exemplo dos códigos de países, novamente, é provável que soubéssemos com bastante antecedência se um novo país tivesse de ser acrescentado. Por exemplo, o Sudão do Sul tornou-se um país independente em julho de 2011, depois de um referendo ocorrido seis meses antes, o que nos deu muito tempo para o rollout de nossa mudança. Novos países raramente são criados em um piscar de olhos!



Se seus microsserviços usam bibliotecas compartilhadas, lembre-se de que você deve aceitar o fato de que poderá ter versões diferentes delas implantadas no ambiente de produção!

Isso significa que, se tivermos de atualizar nossa biblioteca de códigos de países, teremos de aceitar que não é possível garantir que todos os microsserviços terão a mesma versão da biblioteca, como vemos na Figura 4.43. Se isso não for apropriado para você, talvez a próxima opção possa ajudar.

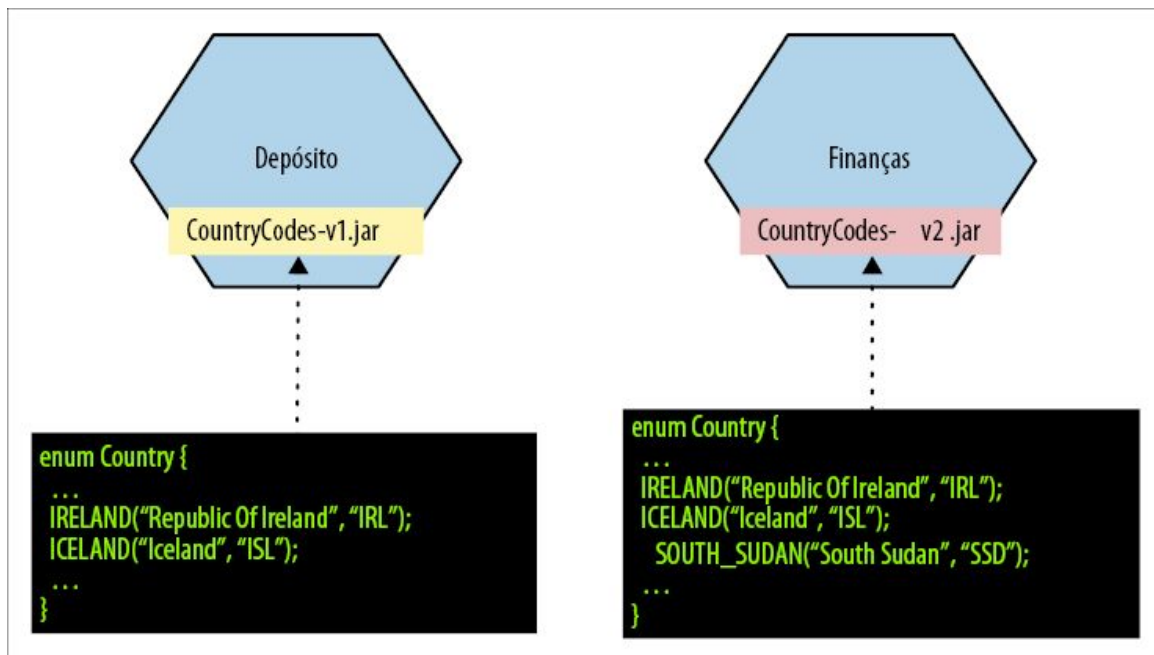


Figura 4.43 – Diferenças entre bibliotecas de dados de referência compartilhadas podem causar problemas.

Em uma variação simples desse padrão, os dados em questão são mantidos em um arquivo de configuração, talvez em um arquivo padrão de propriedades ou, se for necessário, em um formato JSON mais estruturado.

Quando usar o padrão. Para pequenos volumes de dados, em que você não precise se preocupar com o fato de serviços diferentes verem versões diferentes desses dados, essa é uma opção excelente, embora muitas vezes seja menosprezada. Ter visibilidade para saber qual serviço utiliza qual versão dos dados será particularmente conveniente.

Padrão: serviço para dados de referência estáticos

Acho que você já deve suspeitar da direção que estamos tomando. Este é um livro sobre criação de microsserviços, portanto, por que não considerar a criação de um serviço dedicado somente para os códigos de países, como vemos na Figura 4.44?

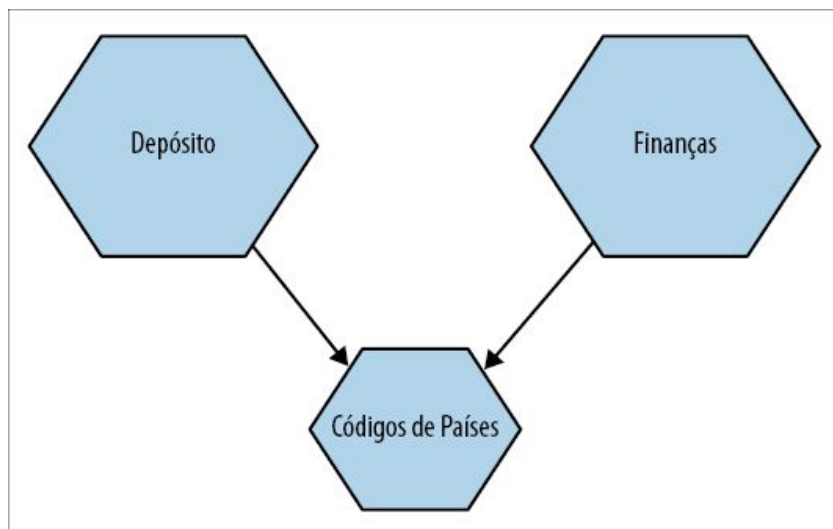


Figura 4.44 – Servindo códigos de países a partir de um serviço dedicado.

Já deparei exatamente com esse cenário com grupos em todo o mundo, e a tendência é que a sala se divida. Algumas pessoas imediatamente pensam: “Isso poderia funcionar!”. Em geral, porém, uma porção maior do grupo começará a balançar suas cabeças e a dizer algo como: “Isso parece loucura!”. Ao explorar melhor a situação, chegamos ao cerne de suas preocupações; a solução parece implicar muito trabalho e um acréscimo de complexidade em potencial em troca de poucas vantagens. A palavra “exagero” surge com bastante frequência!

Vamos então explorar um pouco melhor a situação. Quando converso com as pessoas e tento entender por que algumas delas não têm problemas com essa ideia enquanto outras têm, em geral, os motivos se reduzem a dois pontos. As pessoas que trabalham em um ambiente no qual criar e gerenciar um microserviço é uma tarefa simples têm mais chances de considerar essa opção. Por outro lado, se criar outro serviço, mesmo que seja um serviço simples como este, exigir dias ou até mesmo semanas de trabalho, as pessoas, compreensivelmente, rejeitarão a criação de um serviço desse tipo.

Meu ex-colega de trabalho e igualmente autor da O’Reilly, Kief

Morris⁷, narrou suas experiências com um projeto para um grande banco internacional da Inglaterra, no qual foi necessário quase um ano para que a aprovação da primeira versão de um software fosse obtida. Mais de dez equipes no banco tinham de ser consultadas antes que algo pudesse ser efetivado – isso envolvia de tudo, desde obter a aprovação de designs até o provisionamento de máquinas para a implantação. Experiências como essa estão longe de serem incomuns em empresas maiores, infelizmente.

Em empresas onde implantar um novo software exija muito trabalho manual, aprovações e, talvez, até mesmo a necessidade de adquirir e configurar um novo hardware, o custo inerente de criar serviços é significativo. Em um ambiente como esse, portanto, teríamos de ser extremamente seletivos para criar serviços; esses serviços teriam de proporcionar muitas vantagens para justificar o trabalho extra. Isso poderia fazer com que a criação de um serviço como o serviço de códigos de países não fosse justificável. Por outro lado, se eu puder criar um template de serviço e enviá-lo para o ambiente de produção no prazo de um dia ou menos, e tudo pudesse ser feito por mim mesmo, seria muito mais provável que eu considerasse essa opção como viável.

Melhor ainda, um serviço de Códigos de Países seria bastante apropriado para algo como uma plataforma de Function-as-a-Service (Função como Serviço), por exemplo, o Azure Cloud Functions ou o AWS Lambda. O custo mais baixo das operações das funções é atraente, e eles são muito apropriados para serviços simples como o serviço de Códigos de Países.

Outra preocupação mencionada é que, ao adicionar um serviço para códigos de países, estaríamos acrescentando outra dependência envolvendo redes, o que poderia causar impactos na latência. Não acho que essa abordagem seria pior, e ela poderia ser até mais rápida, do que ter um banco de dados dedicado para essas informações. Por quê? Bem, como já mencionei, há apenas 249 entradas nesse conjunto de dados. Nosso serviço de Códigos de

Países poderia facilmente armazenar esses dados em memória e servi-los diretamente. O serviço provavelmente poderia armazenar esses registros no código, sem a necessidade de usar um banco de dados.

Esses dados poderiam, é claro, ser colocados em cache de modo agressivo do lado do cliente. Afinal de contas, não adicionamos novas entradas com frequência! Poderíamos também considerar o uso de eventos para permitir que os consumidores possam saber caso esses dados mudem, como mostra a Figura 4.45. Quando os dados mudarem, os consumidores interessados poderão ser avisados por meio de eventos e usar isso para atualizar seus caches locais. Acho que um cache tradicional de cliente baseado em TTL provavelmente seria suficiente neste cenário, considerando a baixa frequência das mudanças, mas já usei uma abordagem parecida para um serviço de Dados de Referência de propósito geral muitos anos atrás, com muita eficácia.

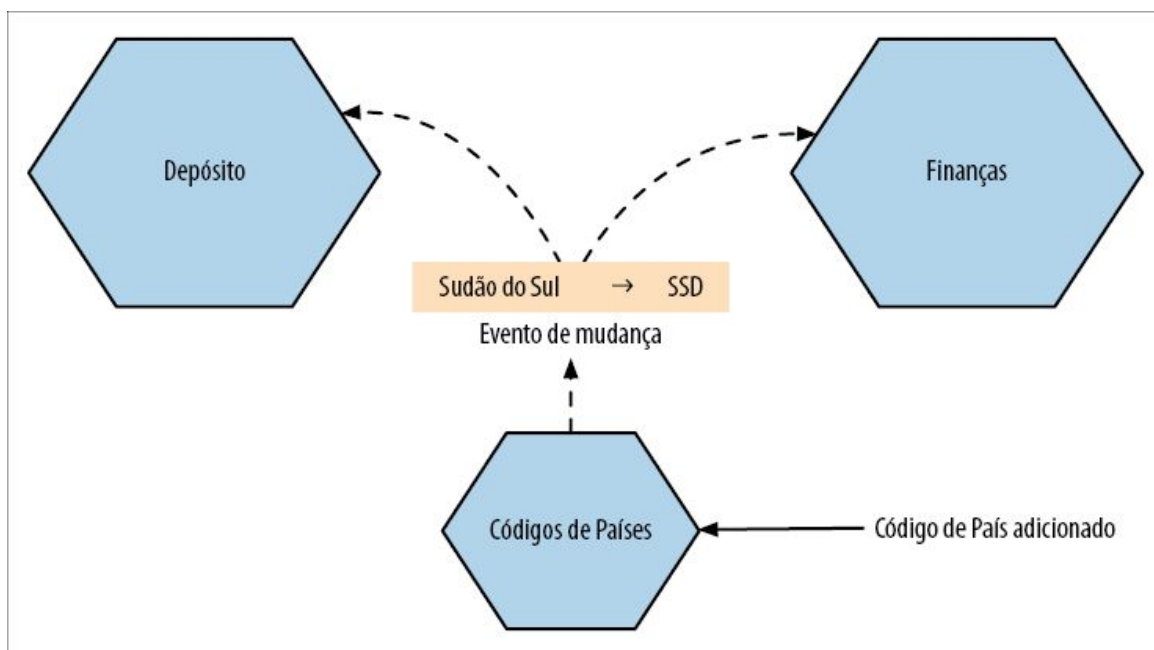


Figura 4.45 – Disparando eventos de atualização para permitir que os consumidores atualizem os caches locais.

Quando usar o padrão. Eu lançaria mão desta opção se estivesse gerenciando o ciclo de vida desses dados no código. Por exemplo,

se eu quisesse expor uma API para atualizar esses dados, seria necessário ter um lugar para colocar esse código, e colocá-lo em um microsserviço faria sentido. A essa altura, temos um microsserviço que inclui a máquina de estados para esse estado. Isso também faz sentido se você quiser gerar eventos quando esses dados mudarem, ou simplesmente quando quiser ter um ponto de contato mais conveniente para a criação de um stub para testes.

O principal problema nesse caso parece se reduzir ao custo de criar outro microsserviço. Ele proporciona vantagens suficientes para justificar o trabalho, ou uma das outras abordagens seria uma opção mais sensata?

O que eu faria?

Tudo bem, mais uma vez, dei muitas opções a você. Então, o que eu faria? Suponho que não posso ficar em cima do muro para sempre, portanto eis o que vou dizer. Se supormos que não será necessário garantir que os códigos dos países sejam consistentes entre todos os serviços o tempo todo, então, provavelmente, eu manteria essas informações em uma biblioteca compartilhada. Para esse tipo de dado, isso parece fazer muito mais sentido do que duplicar os dados em esquemas de serviço locais; os dados são simples por natureza e o volume é pequeno (códigos de países, tamanho de vestimentas e informações desse tipo). Se os dados de referência forem mais complexos ou o volume dos dados for maior, esse poderia ser um sinal para que eu colocasse essas informações no banco de dados local de cada serviço.

Se os dados tiverem de ser consistentes entre os serviços, eu procuraria criar um serviço dedicado (ou, talvez, servir esses dados como parte de um serviço de referência estático de escopo mais amplo). É provável que eu recorresse a um esquema dedicado para esse tipo de dado somente se fosse difícil justificar o trabalho para criar outro serviço.



O que discutimos nos exemplos anteriores são algumas refatorações de

banco de dados que podem ajudar você a separar seus esquemas. Para uma discussão mais detalhada sobre o assunto, você poderá consultar o livro *Refactoring Databases* de Scott J. Ambler e Pramod J. Sadalage (Addison-Wesley).

Transações

Ao separar nossos bancos de dados, já mencionamos alguns problemas que podem surgir. Manter a integridade referencial passa a ser problemático, a latência pode aumentar e atividades como a geração de relatórios podem se tornar mais complexas. Vimos vários padrões para enfrentar alguns desses desafios, mas resta um desafio importante: o que fazer com as transações?

A capacidade de fazer mudanças em nosso banco de dados em uma transação pode deixar nossos sistemas muito mais fáceis de planejar e, desse modo, mais simples para desenvolver e manter. Contamos com nosso banco de dados para garantir a segurança e a consistência de nossos dados, e, assim, podemos nos preocupar com outros assuntos. No entanto, quando separamos os dados em vários bancos de dados, deixamos de usufruir a vantagem de usar uma transação de banco de dados para aplicar mudanças de estado atômicas.

Antes de explorar o modo de enfrentar esse problema, vamos ver rapidamente o que uma transação de banco de dados geralmente oferece.

Transações ACID

Em geral, quando falamos de transações de banco de dados, estamos falando de transações ACID. ACID é um acrônimo que representa as principais propriedades das transações de banco de dados; essas propriedades permitem que tenhamos um sistema que garanta a durabilidade e a consistência na armazenagem dos dados, e com o qual possamos contar. *ACID* quer dizer *Atomicity*, *Consistency*, *Isolation*, and *Durability* (Atomicidade, Consistência, Isolamento e Durabilidade), e essas propriedades nos oferecem o

seguinte:

Atomicidade

Garante que todas as operações feitas na transação estarão completas, ou que todas falharão. Se uma das mudanças que estivermos tentando fazer falhar por algum motivo, a operação como um todo é cancelada, e será como se nenhuma mudança jamais tivesse sido feita.

Consistência

Quando mudanças são feitas em nosso banco de dados, garantimos que ele permanecerá em um estado válido e consistente.

Isolamento

Permite que várias transações sejam feitas ao mesmo tempo, sem interferência. Isso é feito garantindo que qualquer mudança de estado intermediária feita durante uma transação seja invisível às demais transações.

Durabilidade

Garante que, assim que uma transação for concluída, poderemos ter a certeza de que os dados não serão perdidos caso haja alguma falha no sistema.

Vale a pena notar que nem todos os bancos de dados oferecem transações ACID. Todos os sistemas de bancos de dados relacionais que já usei oferecem, assim como muitos dos bancos de dados NoSQL mais recentes, como o Neo4j. O MongoDB, por muitos anos, oferecia suporte para transações ACID somente em um único documento, o que poderia causar problemas se você quisesse fazer uma atualização atômica em mais de um documento.⁸

Este não é um livro para uma exploração profunda e detalhada desses conceitos; sem dúvida, simplifiquei algumas dessas

descrições para ser mais conciso. Para os que gostariam de explorar melhor esses conceitos, recomendo o livro *Designing Data-Intensive Applications*.⁹ Estaremos mais preocupados com a atomicidade nas situações descritas a seguir. Isso não quer dizer que as demais propriedades não sejam igualmente importantes, mas que a atomicidade tende a ser o primeiro problema com que deparamos ao separar as fronteiras das transações.

Continuar sendo ACID, mas sem atomicidade?

Quero deixar claro que ainda podemos usar transações em estilo ACID quando fizermos a separação dos bancos de dados, porém o escopo dessas transações será reduzido, assim como a sua utilidade. Considere a Figura 4.46. Estamos mantendo o controle do processo envolvido na inclusão de um novo cliente em nosso sistema. Alcançamos o final do processo, que envolve modificar o Status do cliente de PENDING para VERIFIED. Como o cadastramento agora está completo, também queremos remover a linha correspondente da tabela PendingEnrollments. Com um único banco de dados, isso é feito no escopo de uma única transação ACID no banco de dados – as duas novas linhas são gravadas, ou nenhuma delas é gravada.

Compare com a Figura 4.47. Estamos fazendo exatamente a mesma mudança, mas, agora, cada mudança é feita em um banco de dados diferente. Isso significa que há duas transações a serem consideradas, cada uma com a possibilidade de ser bem-sucedida ou de falhar, de forma independente.

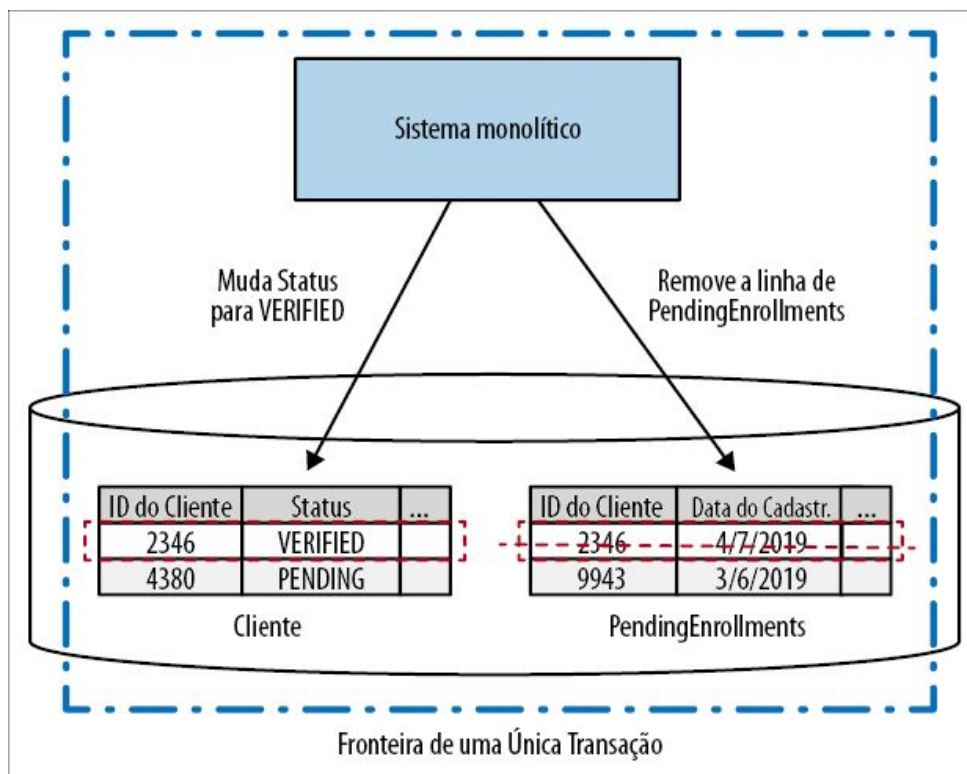


Figura 4.46 – Atualizando duas tabelas no escopo de uma única transação ACID.

Poderíamos ter decidido executar essas duas transações em sequência, é claro, removendo uma linha da tabela PendingEnrollments somente se conseguíssemos modificar a linha na tabela Cliente. Porém, ainda teríamos de pensar no que fazer se a remoção da tabela PendingEnrollments falhasse – isto é, em toda a lógica que teríamos de implementar por conta própria. Contudo, ser capaz de reordenar os passos para lidar com esses casos de uso pode ser uma ideia realmente boa (uma ideia que retomaremos ao explorar as sagas). Basicamente, porém, ao decompor essa operação em duas transações de banco de dados separadas, temos de aceitar que perderemos a garantia da atomicidade da operação como um todo.

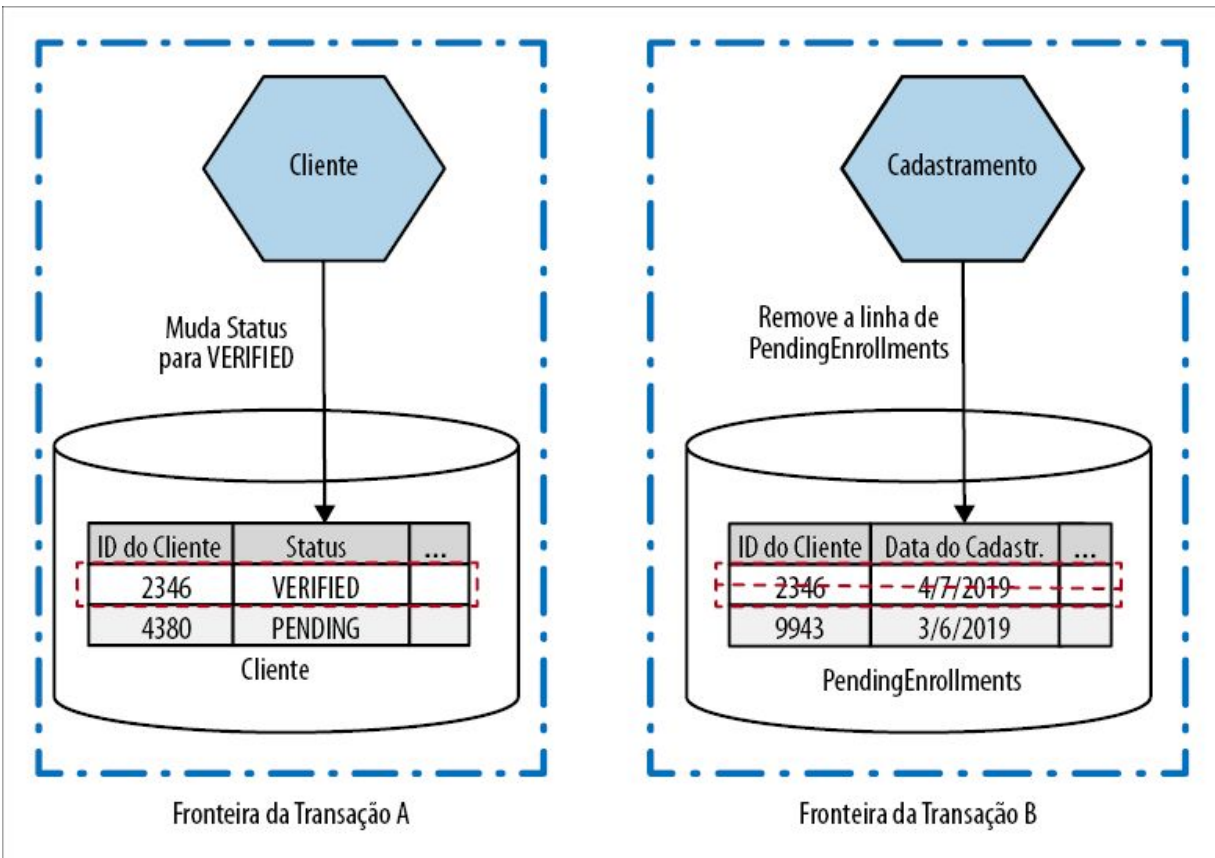


Figura 4.47 – Mudanças feitas tanto em Fatura como em Pedido agora são feitas no escopo de duas transações diferentes.

Essa falta de atomicidade pode começar a causar problemas significativos, sobretudo se estivermos fazendo a migração de sistemas que contavam antes com essa propriedade. É nesse ponto que as pessoas começam a procurar outras soluções para poder pensar em mudanças que devam ser feitas em vários serviços ao mesmo tempo. Em geral, a primeira opção que as pessoas começam a considerar são as transações distribuídas. Vamos ver um dos algoritmos mais comuns para implementar as transações distribuídas, o commit de duas fases (two-phase commit), como forma de explorar os desafios associados às transações distribuídas de modo geral.

Commits de duas fases

O *algoritmo de commit de duas fases* (às vezes abreviado como

2PC) é geralmente usado na tentativa de prover mudanças transacionais em um sistema distribuído, no qual vários processos separados podem precisar ser atualizados como parte da operação como um todo. Gostaria de dizer com antecedência que os 2PCs têm limitações, as quais serão discutidas, mas vale a pena conhecê-los. As transações distribuídas, e os commits de duas fases mais especificamente, em geral são propostas por equipes que estão migrando para arquiteturas de microsserviços como uma maneira de solucionar os desafios que enfrentam. Contudo, como veremos, elas podem não resolver seus problemas, podendo inclusive gerar mais confusão no sistema.

O algoritmo é dividido em duas fases (daí o nome *commit de duas fases*): uma fase de votação e uma fase de commit. Durante a *fase de votação*, um coordenador centralizado entra em contato com todos os workers que farão parte da transação e pede uma confirmação para saber se uma mudança de estado pode ou não ser feita. Na Figura 4.48, vemos duas requisições, uma para modificar o status de um cliente para VERIFIED e outra para remover uma linha de nossa tabela PendingEnrollments. Se todos os workers concordarem que a mudança de estado sobre a qual estão sendo inquiridos possa ocorrer, o algoritmo passará para a próxima fase. Se algum worker disser que a mudança não pode ocorrer, talvez porque a mudança de estado solicitada viole alguma condição local, a operação como um todo será cancelada.

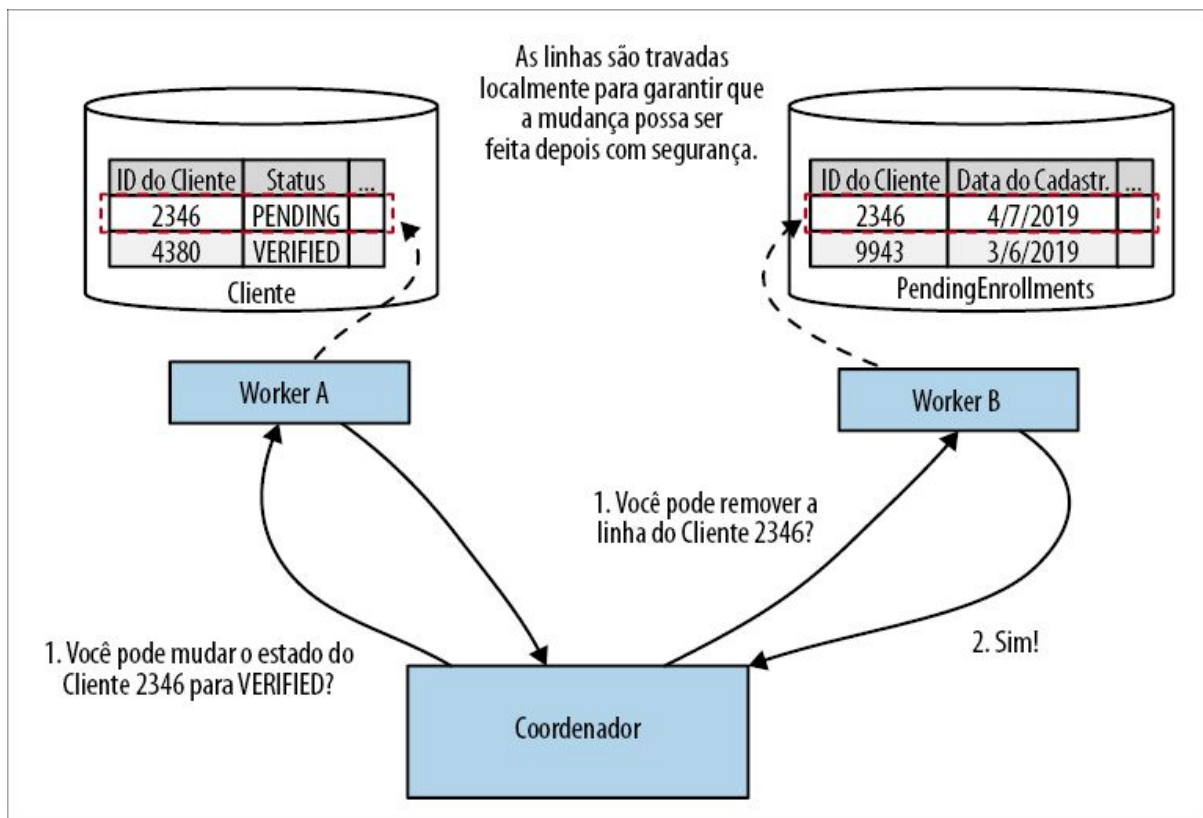


Figura 4.48 – Na primeira fase de um commit de duas fases, os workers votam para decidir se podem fazer uma mudança de estado local.

É importante enfatizar que a mudança não terá efeito imediato, depois que um worker sinalizar que pode fazer a mudança. O worker está apenas garantindo que será capaz de fazer essa mudança em algum momento no futuro. Como o worker pode dar essa garantia? Na Figura 4.48, por exemplo, o Worker A disse que poderá mudar o estado da linha na tabela *Cliente* para atualizar o status desse cliente específico para *VERIFIED*. O que acontecerá se uma operação diferente, logo depois, apagar a linha, ou fizer outra mudança menor que, apesar disso, implique que uma mudança para *VERIFIED* mais tarde seja inválida? Para garantir que essa mudança possa ser feita mais tarde, o Worker A provavelmente terá de travar esse registro a fim de garantir que uma mudança como essa não possa ocorrer.

Se algum worker votar dizendo que não é favorável ao commit, uma

mensagem de rollback deverá ser enviada a todas as partes para garantir que possam fazer uma limpeza local, o que permitirá que os workers liberem qualquer lock que possam estar mantendo. Se todos os workers concordarem em fazer a mudança, passamos para a fase de commit, como vemos na Figura 4.49. Nessa etapa, as mudanças são realmente efetuadas e os locks associados são liberados.

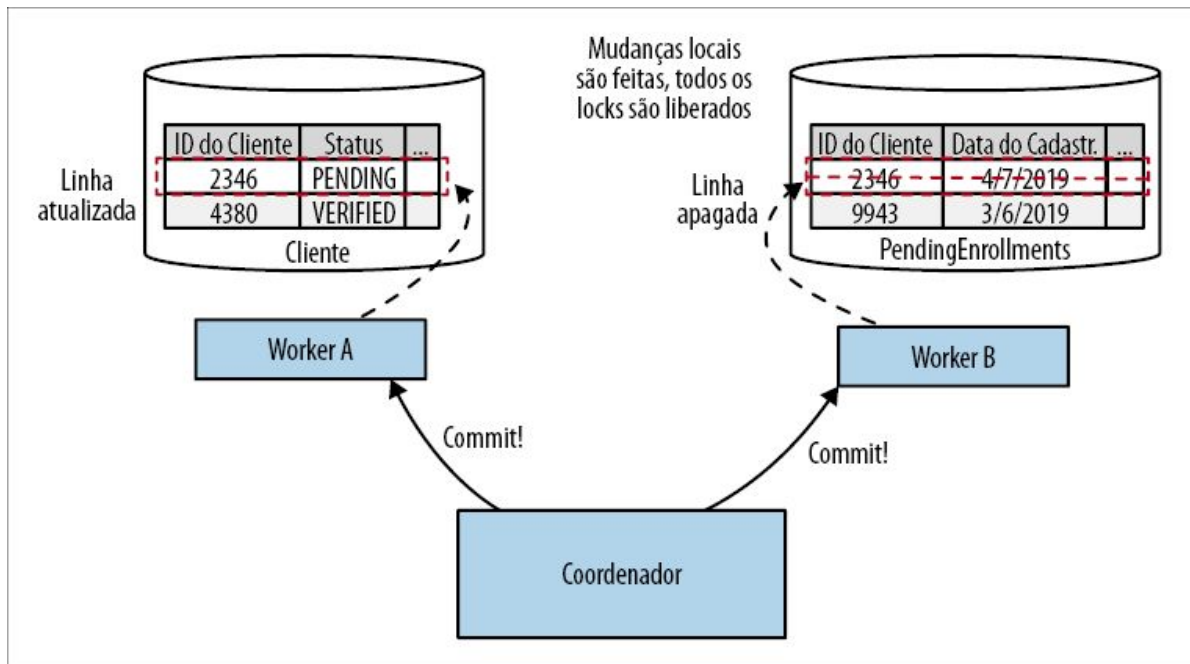


Figura 4.49 – Na fase de commit de um commit de duas fases, as mudanças são realmente aplicadas.

É importante observar que, em um sistema como esse, não podemos, de modo algum, garantir que esses commits ocorrerão exatamente ao mesmo tempo. O coordenador precisa enviar a requisição de commit a todos os participantes, e essa mensagem poderá chegar e ser processada em momentos diferentes. Isso significa que seria possível que você visse a mudança feita no Worker A, mas ainda não visse a mudança no Worker B, caso pudesse visualizar os estados desses workers fora do coordenador de transações. Quanto maior a latência entre o coordenador, e quanto mais lentamente os workers processarem a resposta, maior será essa janela de inconsistência. Voltando à nossa definição de

ACID, o isolamento garante que não veremos estados intermediários durante uma transação. Contudo, em um commit de duas fases, perdemos essa propriedade.

Quando os commits de duas fases atuam, em sua essência, eles estarão geralmente apenas coordenando locks distribuídos. Os workers devem travar os recursos locais a fim de garantir que o commit possa ocorrer na segunda fase. Gerenciar locks e evitar deadlocks em um sistema com um único processo não é divertido. Imagine agora os desafios que surgem ao coordenar locks entre vários participantes. Não é uma perspectiva agradável.

Há inúmeros modos de falha associados aos commits de duas fases, os quais não teremos tempo para explorar. Considere o problema de um worker votando para proceder com a transação, mas, em seguida, não respondendo quando solicitado a fazer o commit. O que deveríamos fazer nesse caso? Alguns desses modos de falha podem ser tratados automaticamente, mas outros podem deixar o sistema em tal estado que seria necessária uma intervenção manual.

Quanto mais participantes houver, e quanto maior a latência no sistema, mais problemas haverá com um commit de duas fases. Eles podem ser um modo rápido de injetar bastante latência em seu sistema, particularmente se o escopo do locking for grande ou a duração da transação for longa. É por esse motivo que os commits de duas fases em geral são usados somente em operações de curta duração. Quanto mais demorada a operação, mais tempo seus recursos ficarão travados!

Transações distribuídas – basta dizer não

Por todos os motivos apresentados até agora, sugiro enfaticamente que você evite o uso de transações distribuídas, como o commit de duas fases, para coordenar mudanças de estado entre seus microsserviços. Então, o que mais podemos fazer?

Bem, a primeira opção poderia ser simplesmente não separar os

dados, para começar. Se você tiver estados que queira administrar de forma realmente atômica e consistente, e não consegue definir um modo de usufruir essas características de forma razoável, sem uma transação de estilo ACID, deixe esse estado em um único banco de dados, e deixe a funcionalidade que administra esse estado em um único serviço (ou em seu sistema monolítico). Se estiver no processo de definir os pontos de separação de seu sistema monolítico, e definir quais decomposições poderão ser mais fáceis (ou mais difíceis), você poderia muito bem decidir que separar os dados que atualmente são administrados por uma transação é uma tarefa difícil demais para ser feita nesse momento. Trabalhe em alguma outra área do sistema e volte para essa parte mais tarde.

Mas o que acontecerá se você realmente precisar separar esses dados, mas não quiser ter todas as dificuldades relacionadas ao gerenciamento de transações distribuídas? Como é possível executar operações em vários serviços, porém evitando um locking? E se a operação demorar minutos, dias ou, quem sabe, até mesmo meses? Em casos como esses, podemos considerar uma abordagem alternativa: as sagas.

Sagas

De modo diferente de um commit de duas fases, uma *saga* é, por design, um algoritmo capaz de coordenar várias mudanças de estado, mas que evita a necessidade de travar recursos por longos períodos. Fazemos isso modelando os passos envolvidos na forma de atividades discretas que podem ser executadas de modo independente. Ela tem como benefício adicional o fato de sermos forçados a modelar explicitamente nossos processos de negócio, o que pode proporcionar vantagens significativas.

A ideia central, inicialmente apresentada por Hector Garcia-Molina e Kenneth Salem¹⁰, refletia os desafios acerca da melhor maneira de lidar com as operações daquilo que eles chamavam de *LLTs* (Long Lived Transactions, ou Transações de Longa Duração). Essas

transações podiam demorar muito (minutos, horas ou talvez até mesmo dias) e, como parte do processo, exigem mudanças em um banco de dados.

Se mapearmos diretamente uma LLT a uma transação usual de banco de dados, uma única transação englobaria todo o ciclo de vida da LLT. Isso poderia resultar em várias linhas ou até mesmo em tabelas inteiras sendo travadas por longos períodos enquanto a LLT fosse executada, causando problemas significativos caso outros processos tentassem ler ou modificar esses recursos travados.

Em vez de fazer isso, os autores do artigo sugerem que devemos dividir essas LLTs em uma sequência de transações, em que cada uma possa ser tratada de modo independente. A ideia é que cada uma dessas “subtransações” tenha uma duração mais curta, e modifique somente parte dos dados afetados por toda a LLT. Como resultado, haverá muito menos contenção no banco de dados subjacente, pois o escopo e a duração dos locks serão bastante reduzidos.

Embora, originalmente, as sagas tenham sido concebidas como um método para ajudar nas LLTs, atuando em um único banco de dados, o modelo também funciona bem para coordenar mudanças envolvendo vários serviços. Podemos separar um único processo de negócios em um conjunto de chamadas que serão feitas para serviços que colaboram entre si, como parte de uma única saga.



Antes de prosseguir, você deve compreender que uma saga *não* nos dá a atomicidade fornecida pelo ACID, com a qual estamos acostumados em uma transação usual de banco de dados. Como dividimos a LLT em transações individuais, não temos a atomicidade no nível da própria saga. Porém, temos atomicidade para cada subtransação da LLT, pois cada uma delas poderá estar relacionada a uma mudança transacional ACID, se for necessário. O que uma saga nos dá são informações suficientes para entender em que estado ela se encontra; cabe a nós lidarmos com as implicações desse fato.

Vamos analisar um fluxo simples de atendimento de pedido, representado na Figura 4.50, que usaremos para explorar melhor as sagas no contexto de uma arquitetura de microsserviços.

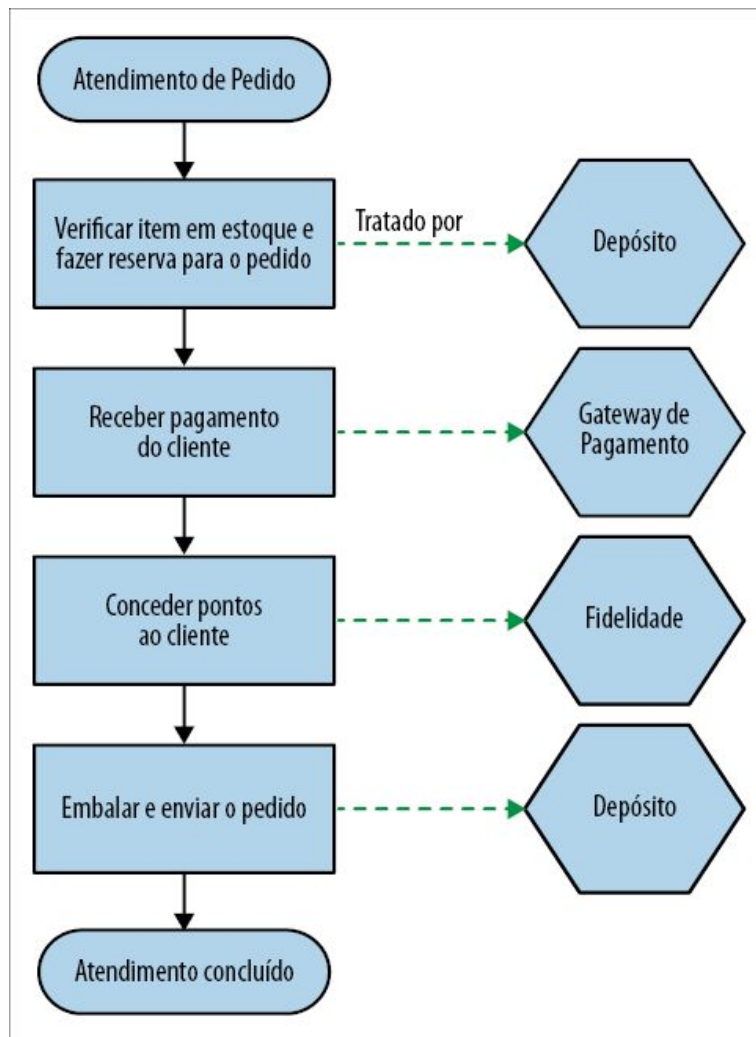


Figura 4.50 – Um exemplo de fluxo de atendimento de pedido, junto com os serviços responsáveis por executar a operação.

Nesse caso, o processo de atendimento do pedido é representado como uma única saga, na qual cada passo do fluxo representa uma operação que pode ser executada por um serviço diferente. Em cada serviço, qualquer mudança de estado pode ser tratada com uma transação ACID local. Por exemplo, quando verificamos e reservamos um item do estoque usando o serviço de Depósito, internamente, esse serviço pode criar uma linha em sua tabela local de Reservas registrando a reserva; essa mudança seria tratada com uma transação usual.

Modos de falha das sagas

Com uma saga dividida em transações individuais, temos de considerar o tratamento das falhas – ou, mais especificamente, da recuperação em caso de falha. O artigo original sobre sagas descreve dois tipos de recuperação: recuperação com retrocesso (backward recovery) e recuperação com avanço (forward recovery).

A *recuperação com retrocesso* envolve reverter a falha e fazer uma limpeza depois – é um rollback. Para que ela funcione, é necessário definir ações de compensação que nos permitam desfazer transações cujo commit já tenha sido feito. Uma *recuperação com avanço* nos permite prosseguir do ponto em que a falha ocorreu e continuar o processamento. Para que ela funcione, deverá ser possível fazer novas tentativas para as transações, o que, por sua vez, implica que nosso sistema está fazendo a persistência de informações suficientes para permitir que essas tentativas ocorram.

Dependendo da natureza do processo de negócios sendo modelado, você pode considerar que qualquer modo de falha vai disparar uma recuperação com retrocesso, uma recuperação com avanço ou, quem sabe, uma combinação de ambas.

Rollbacks de sagas

Em uma transação ACID, um rollback ocorre antes de um commit. Após o rollback, é como se nada houvesse acontecido: a mudança que estávamos tentando fazer não ocorreu. Com a saga, porém, temos várias transações envolvidas, e o commit de algumas delas já pode ter ocorrido antes de termos decidido fazer o rollback de toda a operação. Então, como podemos fazer o rollback de transações depois que seu commit já tiver sido efetuado?

Vamos retomar o nosso exemplo do processamento de um pedido, conforme representado na Figura 4.50. Considere um modo de falha possível. Chegamos ao ponto de tentar embalar o item, somente para perceber que não é possível encontrá-lo no depósito, como mostra a Figura 4.51. Nosso sistema acha que o item existe, mas ele não está na prateleira!

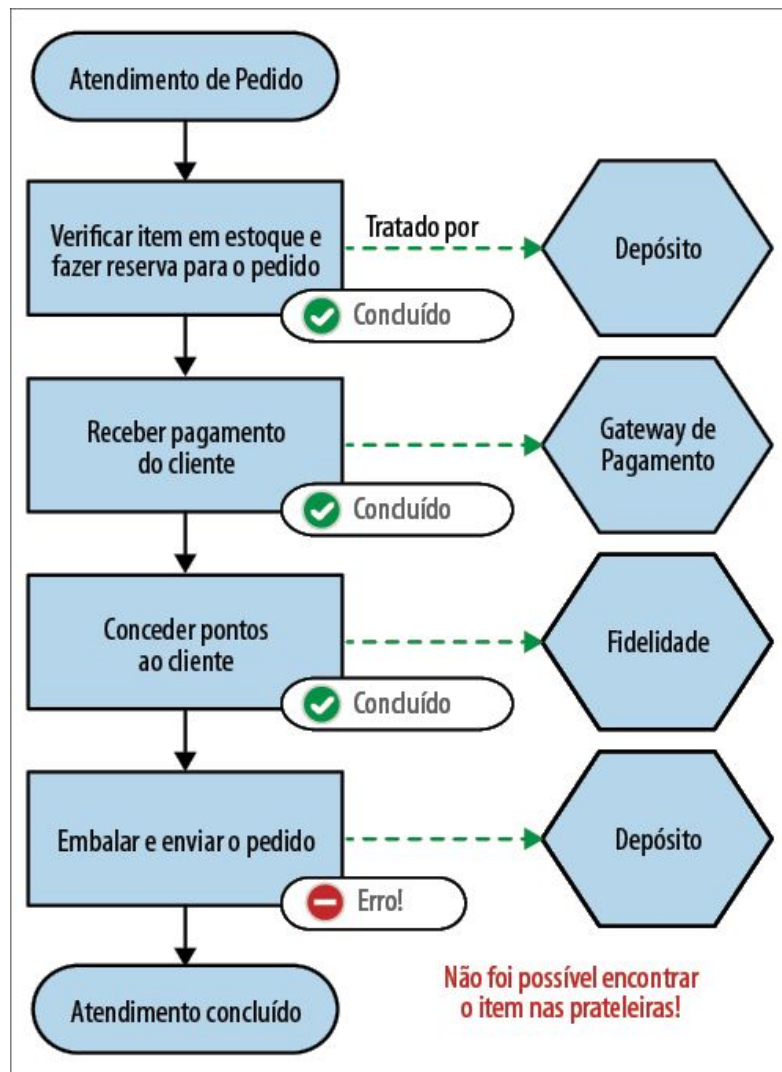


Figura 4.51 – Tentamos embalar o nosso item, mas não foi possível encontrá-lo no depósito.

Vamos agora supor que decidimos que queremos simplesmente fazer um rollback de todo o pedido, em vez de dar ao consumidor a opção de colocar o item em um pedido que ficará aguardando a disponibilidade do produto. O problema é que já recebemos o pagamento e concedemos os pontos de fidelidade ao pedido.

Se todos esses passos tivessem sido feitos em uma única transação do banco de dados, um rollback simples faria a limpeza de tudo. No entanto, cada passo no processo de atendimento do pedido foi tratado por uma chamada de serviço distinta, cada uma atuando em um escopo de transação diferente. Não há um “rollback” simples

para toda a operação.

Em vez disso, se quisermos implementar um rollback, devemos implementar uma transação de compensação. Uma *transação de compensação* é uma operação que desfaz uma transação cujo commit já tenha sido efetuado. Para fazer um rollback de nosso processo de atendimento de pedido, dispararíamos uma transação de compensação para cada passo de nossa saga cujo commit já tivesse sido feito, como mostra a Figura 4.52.

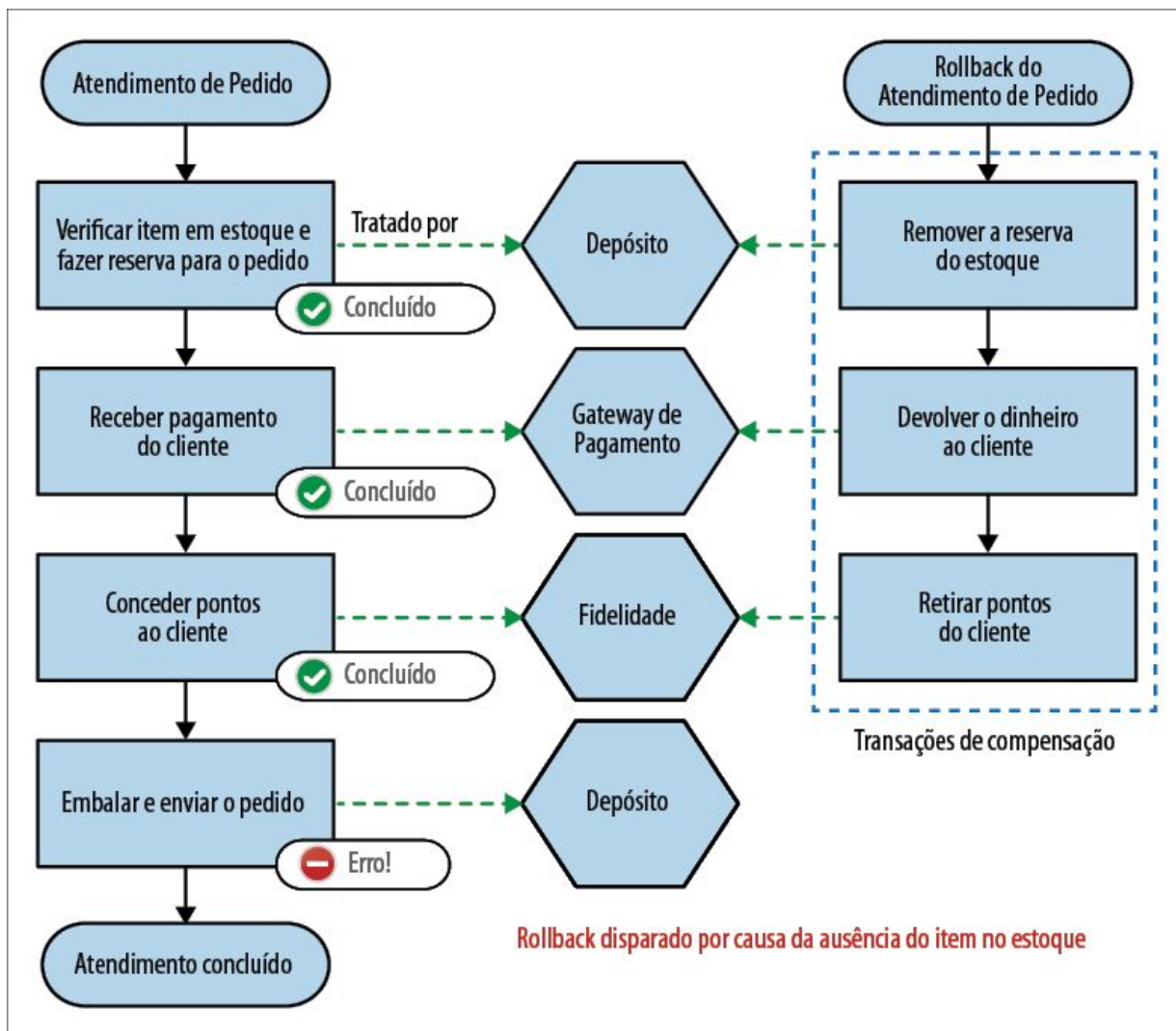


Figura 4.52 – Disparando um rollback para toda a saga.

Vale a pena chamar a atenção para o fato de que essas transações de compensação não podem ter exatamente o mesmo

comportamento de um rollback usual do banco de dados. Um rollback do banco de dados ocorre antes do commit; após o rollback, é como se a transação jamais tivesse ocorrido. Em nosso cenário, essas transações *realmente* ocorreram, obviamente. Estamos criando outra transação para reverter as mudanças feitas pela transação original, mas não podemos voltar no tempo e fazer com que a transação original jamais tivesse ocorrido.

Como nem sempre conseguiremos reverter uma transação de forma completamente limpa, dizemos que essas transações de compensação são *rollbacks semânticos*. Nem sempre será possível limpar tudo, mas fazemos o suficiente no contexto de nossa saga. Como exemplo, um de nossos passos pode ter envolvido o envio de um email para um cliente para lhe informar que seu pedido estava a caminho. Se decidirmos fazer um rollback dessa operação, não será possível “desenviar” um email!¹¹ Em vez disso, sua transação de compensação poderia fazer com que um segundo email fosse enviado ao cliente, informando que houve um problema com o pedido e que este foi cancelado.

É totalmente apropriado que seja feita a persistência das informações relacionadas ao rollback de uma saga no sistema. Com efeito, essas informações podem ser muito importantes. Talvez você queira manter um registro no serviço de Pedido para esse pedido cancelado, junto com informações sobre o que aconteceu, por uma série de motivos.

Reordenando os passos para reduzir os rollbacks

Na Figura 4.52, poderíamos ter deixado nossos possíveis cenários de rollback um pouco mais simples reorganizando a sequência dos passos. Uma mudança simples seria conceder os pontos somente quando o pedido tiver sido realmente despachado, como vemos na Figura 4.53. Dessa maneira, não precisaríamos nos preocupar com o fato de essa etapa precisar de rollback caso tivéssemos algum problema enquanto tentássemos embalar e enviar o pedido. Às vezes, podemos simplificar as operações de rollback somente

ajustando o modo como o processo é conduzido. Ao adiantar os passos que tenham mais chances de falhar e provocar uma falha mais cedo no processo, você evitará ter de disparar transações de compensação mais tarde, pois esses passos nem terão sido executados, para começar.

Essas mudanças, se puderem ser acomodadas, podem deixar sua vida mais fácil, evitando a necessidade até mesmo de criar transações de compensação para alguns passos. Isso pode ser particularmente importante se a implementação de uma transação de compensação for difícil. Você poderá mover o passo para uma fase posterior do processo, para uma etapa na qual ele jamais vá precisar de um rollback.

Combinando situações de falhas com retrocesso com situações de falhas com avanço

É totalmente apropriado ter uma combinação de modos de recuperação de falhas. Algumas falhas podem exigir um rollback; outras podem prosseguir. Para o processamento de pedidos, por exemplo, assim que tivermos recebido o pagamento do cliente e o item estiver embalado, o único passo que restará será despachar o pacote. Se, por qualquer que seja o motivo, não for possível despachá-lo (talvez a nossa empresa de entrega não tenha espaço em seus furgões para retirar o pedido hoje), seria muito estranho fazer um rollback de todo o pedido. Em vez disso, é provável que apenas tentássemos fazer o despacho novamente e, se isso falhar, uma intervenção humana poderia ser necessária para resolver a situação.

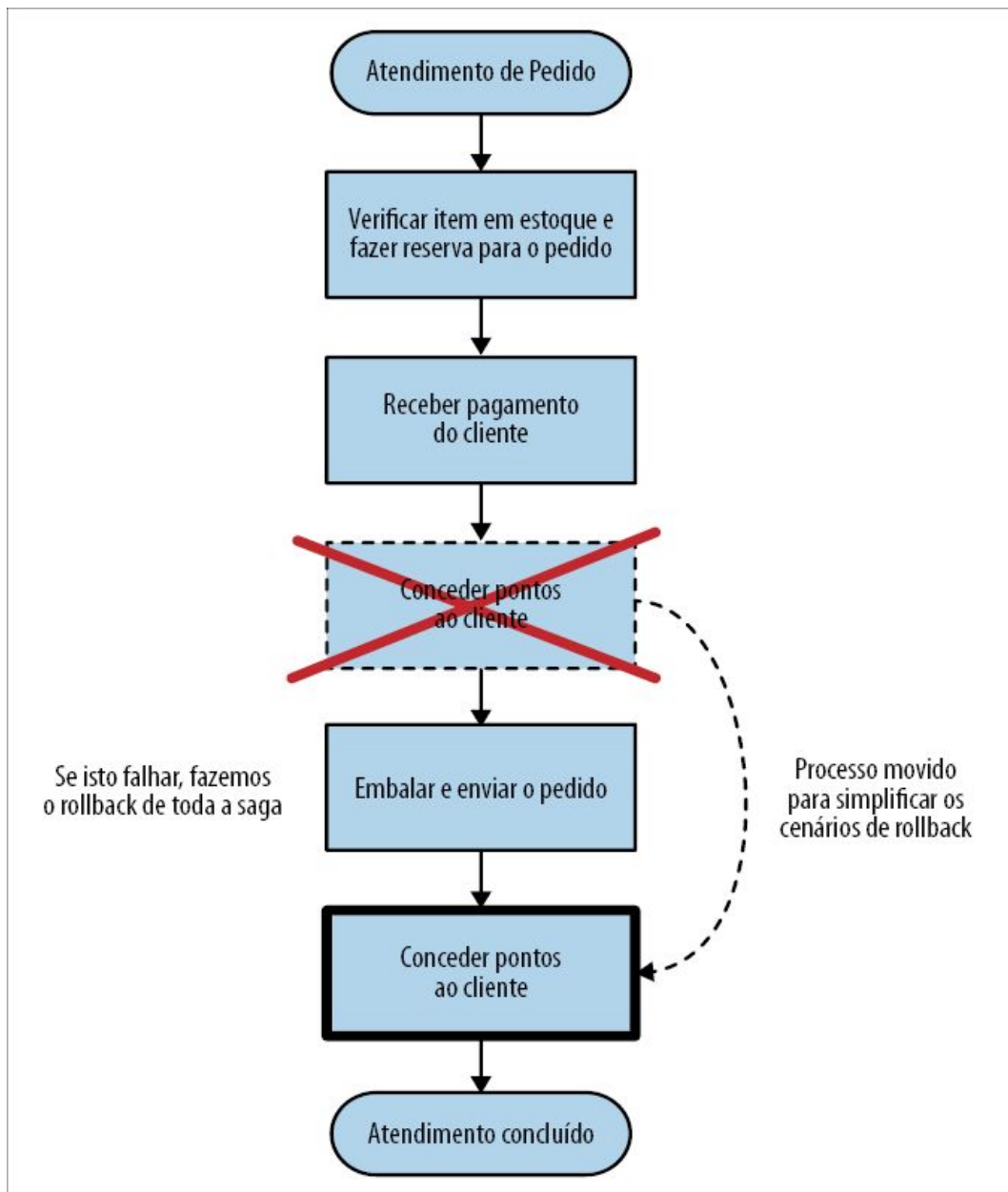


Figura 4.53 – Mover passos para uma fase posterior na saga pode reduzir as transações que precisam de rollback no caso de haver uma falha.

Implementando as sagas

Até agora, vimos o modelo lógico para o funcionamento das sagas, mas precisamos explorar melhor as maneiras de implementá-las. Podemos discutir dois estilos de implementação de sagas. As *sagas orquestradas* seguem mais de perto a solução original e dependem

basicamente de uma coordenação e um controle centralizados. Elas podem ser comparadas com as *sagas coreografadas*, que evitam a necessidade de uma coordenação centralizada em favor de um modelo com menos acoplamento, mas que pode deixar mais complicada a monitoração da execução de uma saga.

Sagas orquestradas

As sagas orquestradas usam um coordenador centralizado (que chamaremos de *orquestrador* a partir de agora) para definir a ordem de execução e disparar qualquer ação de compensação necessária. Podemos pensar nas sagas orquestradas como uma abordagem de comando e controle (command-and-control): o orquestrador central controla o que acontece e quando acontece; com isso, há um bom grau de visibilidade quanto ao que está acontecendo com qualquer saga em particular.

Considerando o processo de atendimento de pedido exibido na Figura 4.50, vamos ver como esse processo de coordenação centralizada funcionaria com um conjunto de serviços colaborativos, como mostra a Figura 4.54.

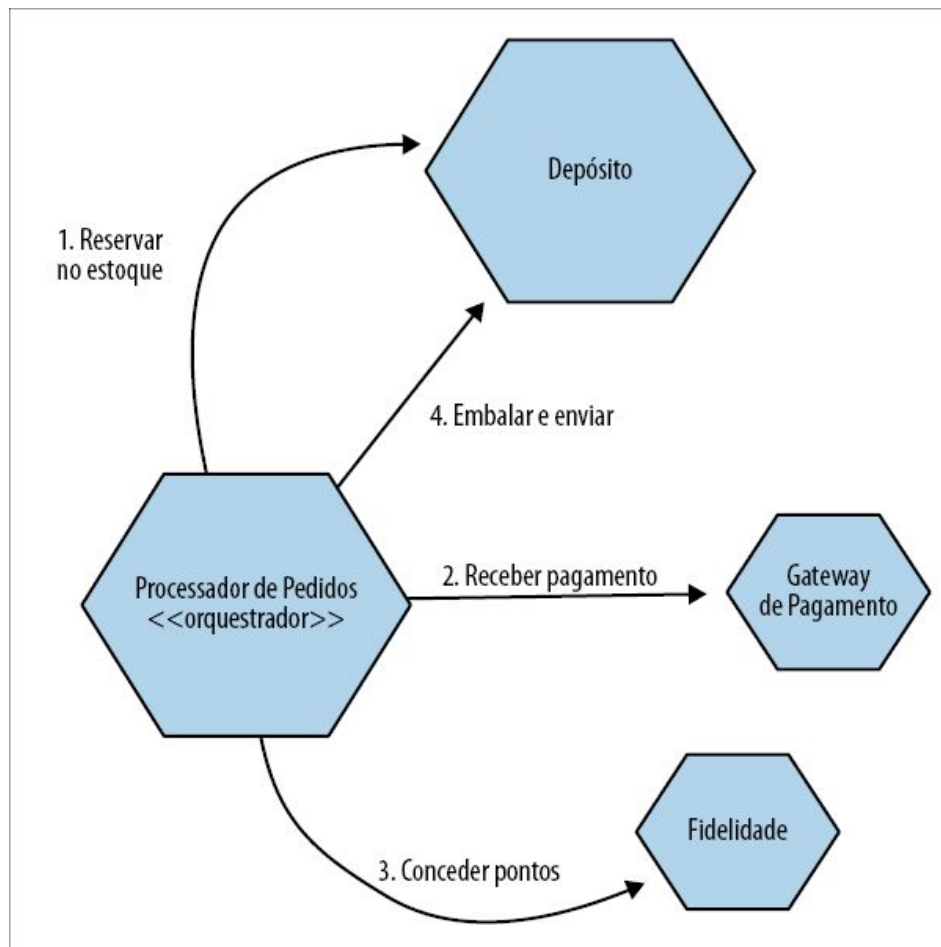


Figura 4.54 – Um exemplo de como uma saga orquestrada pode ser usada para implementar nosso processo de atendimento de pedido.

Nesse exemplo, nosso Processador de Pedidos central, desempenhando a função de orquestrador, coordena o nosso processo de atendimento. Ele sabe quais serviços são necessários para que a operação seja executada e decide quando deve fazer as chamadas para esses serviços. Se as chamadas falharem, ele pode decidir o que será feito como resultado. Esses processadores orquestrados tendem a fazer uso intenso de chamadas de requisição/resposta entre os serviços: o Processador de Pedidos envia uma requisição aos serviços (por exemplo, para um Gateway de Pagamento), e espera uma resposta que lhe permita saber se a requisição foi bem-sucedida e que forneça o resultado da requisição.

Ter um processo de negócios explicitamente modelado no Processador de Pedidos é extremamente vantajoso. Isso nos permite olhar para um só local de nosso sistema e compreender como esse processo deve funcionar. Desse modo, a inclusão de novas pessoas no projeto se torna mais fácil, e mais pessoas poderão compreender melhor as partes essenciais do sistema.

Contudo, há algumas desvantagens que devem ser consideradas. Em primeiro lugar, em virtude de sua natureza, essa é uma abordagem, de certo modo, acoplada. Nosso Processador de Pedidos deve conhecer todos os serviços associados, resultando em um maior grau daquilo que discutimos no Capítulo 1 como acoplamento de domínios. Embora não seja inerentemente ruim, ainda gostaríamos de manter o acoplamento de domínios a um nível mínimo, se for possível. Nesse exemplo, nosso Processador de Pedidos deve ter conhecimento e controlar muitas operações, a ponto de essa forma de acoplamento ser difícil de evitar.

Outro problema, mais sutil, é que a lógica que deveria estar nos serviços pode começar a ser absorvida pelo orquestrador. Se isso começar a acontecer, você poderá ver se serviços se tornarem anêmicos, com pouco comportamento próprio, somente recebendo ordens de orquestradores como o Processador de Pedidos. É importante que você continue considerando os serviços que compõem esses fluxos orquestrados como entidades que tenham o próprio estado local e o próprio comportamento. Eles são responsáveis por suas próprias máquinas de estado locais.



Se a lógica tiver um local em que possa ser centralizada, ele se tornará centralizada!

Uma das maneiras de evitar que haja muita centralização nos fluxos orquestrados pode ser garantir que haja diferentes serviços desempenhando o papel de orquestrador para diferentes fluxos. Você poderia ter um serviço de Processador de Pedidos que cuide do atendimento de um pedido, um serviço de Devoluções para lidar com o processo de devolução e de reembolso, um serviço de

Recepção de Mercadorias que cuide da chegada de novos itens de estoque no depósito e de sua colocação nas prateleiras e assim por diante. Algo como o nosso serviço de Depósito pode ser usado por todos esses orquestradores; um modelo como esse facilita manter a funcionalidade dentro do próprio serviço de Depósito para permitir a reutilização da funcionalidade em todos esses fluxos.

Ferramentas BPM?

Ferramentas BPM (Business Process Modeling, ou Modelagem de Processos de Negócio) estão disponíveis há anos. De modo geral, elas foram criadas para permitir que profissionais que não são desenvolvedores definam fluxos de processos de negócio, muitas vezes usando ferramentas visuais com recursos de arrastar e soltar. A ideia é que os desenvolvedores criem os blocos de construção desses processos e, em seguida, os não desenvolvedores liguem esses blocos de construção, criando fluxos maiores de processo. O uso de ferramentas como essas parece estar bem alinhado com a implementação de sagas orquestradas e, de fato, a orquestração de processos é, basicamente, o caso de uso principal das ferramentas BPM (ou, por outro lado, o uso de ferramentas de BPM leva você à adoção da orquestração).

Com base em minha experiência, passei a desgostar bastante das ferramentas BPM. O principal motivo é que o conceito central – o fato de que os profissionais que não são desenvolvedores definirão o processo de negócios –, de acordo com a minha experiência, quase nunca é verdadeiro. As ferramentas destinadas aos não desenvolvedores acabam sendo usadas *pelos* desenvolvedores, e eles podem ter muitos problemas. Com frequência, elas exigem o uso de GUIs para alterar os fluxos, os fluxos criados podem ser difíceis (ou impossíveis) de gerenciar com um sistema de controle de versões, os próprios fluxos podem não ser criados com testes em mente etc.

Se seus desenvolvedores vão implementar seus processos de negócio, deixe que eles utilizem as ferramentas que conheçam e compreendam, e que sejam apropriadas aos seus fluxos de

trabalho. Em geral, significa simplesmente deixar que utilizem código para implementar isso! Se precisar de visibilidade acerca de como um processo de negócios foi implementado, ou como está funcionando, será muito mais fácil criar uma representação visual de um fluxo de trabalho com um código do que usar uma representação visual de seu fluxo de trabalho para descrever como seu código deve funcionar.

Há esforços para criar ferramentas BPM mais apropriadas aos desenvolvedores. Feedbacks dos desenvolvedores acerca dessas ferramentas parecem ser variados, mas elas têm funcionado bem para algumas pessoas, e é bom ver que há pessoas tentando aperfeiçoar esses frameworks. Se achar que precisa explorar melhor essas ferramentas, consulte o Camunda (<https://camunda.com/>) e o Zeebe (<https://zeebe.io>) – ambos são frameworks de orquestração de código aberto voltados aos desenvolvedores de microsserviços.

Sagas coreografadas

As sagas coreografadas têm como objetivo distribuir a responsabilidade pela operação da saga entre vários serviços que colaboram entre si. Se a orquestração é um comando e controle, as sagas coreografadas representam uma arquitetura trust-but-verify (confiar, mas verificar). Conforme veremos em nosso exemplo na Figura 4.55, as sagas coreografadas em geral fazem uso intenso de eventos para colaboração entre os serviços.

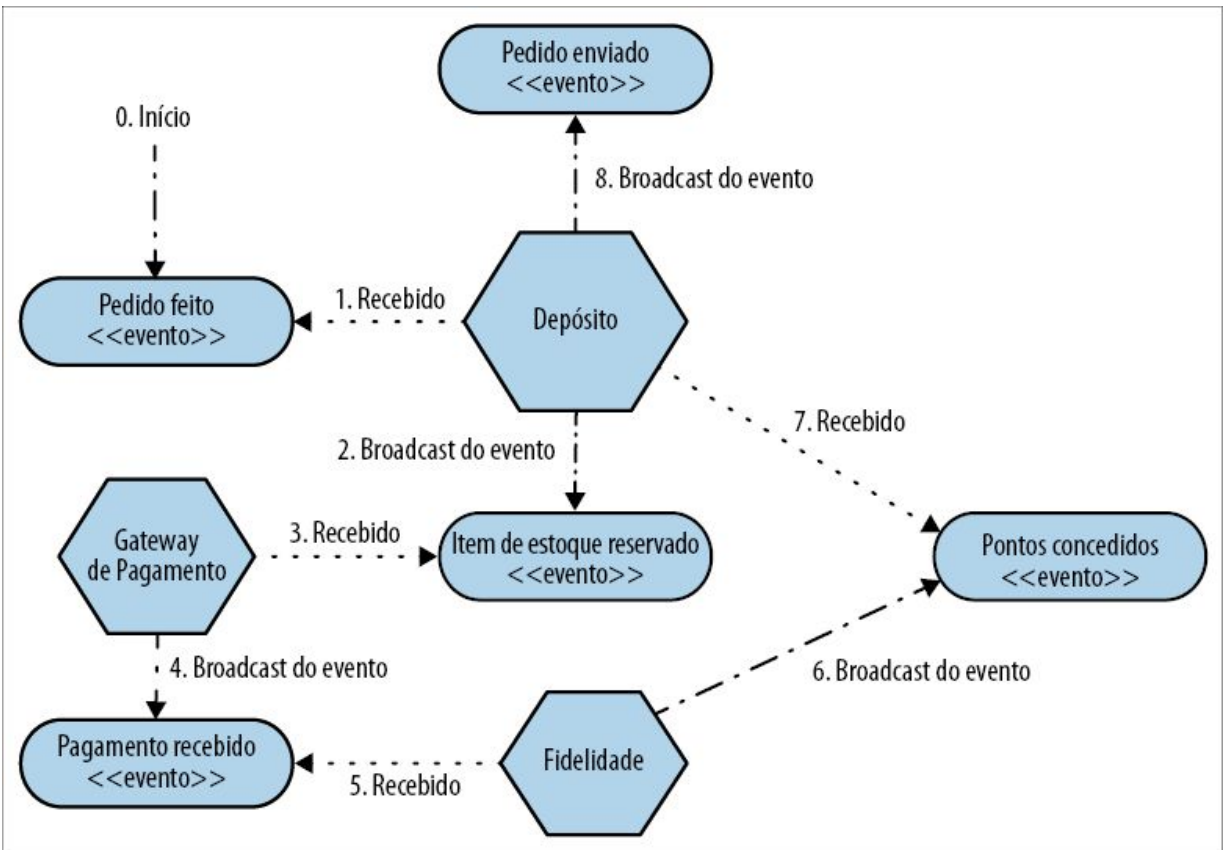


Figura 4.55 – Um exemplo de uma saga coreografada para implementar o atendimento de um pedido.

Há muita coisa acontecendo nesse caso, portanto vale a pena analisar com mais detalhes. Em primeiro lugar, esses serviços estão reagindo a eventos recebidos. Conceitualmente, os eventos são enviados por meio de um broadcast para todo o sistema, e as partes interessadas podem recebê-los. Os eventos não são enviados *para* um serviço; você simplesmente os dispara, e os serviços interessados nesses eventos poderão recebê-los e atuar de acordo com eles. Em nosso exemplo, quando o serviço de Depósito recebe o primeiro evento de Pedido Feito, ele sabe que seu trabalho é reservar o item apropriado do estoque e disparar um evento assim que isso for feito. Se o item do estoque não estiver disponível, o Depósito terá de gerar um evento apropriado (um evento de Estoque Insuficiente, talvez), que poderia resultar no cancelamento do pedido.

Em geral, usaríamos algum tipo de broker de mensagens para gerenciar o broadcast e a entrega de eventos de forma confiável. É possível que vários serviços possam reagir ao mesmo evento, e nesse caso poderíamos usar um tópico. As partes interessadas em um certo tipo de evento poderiam se inscrever para um tópico específico, sem ter de se preocupar com a origem desses eventos, e o broker garantirá a durabilidade do tópico e se certificará de que os eventos relacionados a esse tópico sejam entregues com sucesso aos inscritos. Como exemplo, podemos ter um serviço de Recomendações que também observe os eventos de Pedido Feito, utilizando-o para construir um banco de dados de opções musicais que possam ser do seu gosto.

Na arquitetura anterior, nenhum serviço tem conhecimento de qualquer outro serviço. Eles só precisam saber o que devem fazer quando determinado evento for recebido. Isso faz com que haja, inerentemente, uma arquitetura muito menos acoplada. À medida que a implementação do processo é decomposta e distribuída entre os quatro serviços nesse exemplo, também evitamos as preocupações com a centralização da lógica (se você não tiver um local em que a lógica possa estar centralizada, ela não será centralizada!).

O outro lado da moeda é que agora poderá ser mais difícil entender o que está acontecendo. Com a orquestração, nosso processo era explicitamente modelado em nosso orquestrador. Agora, com essa arquitetura conforme foi apresentada, como você construiria um modelo mental do que o processo deveria fazer? Seria necessário observar o comportamento de cada serviço isoladamente e reconstituir essa imagem em sua mente – está longe de ser uma tarefa fácil, mesmo com um processo de negócio simples como esse.

A falta de uma representação explícita de nosso processo de negócios já é bastante ruim; além disso, não temos um modo de saber em que estado está uma saga e, com isso, não teremos a

oportunidade de associar ações de compensação quando forem necessárias. Podemos passar parte da responsabilidade aos serviços individuais para que executem as ações de compensação, mas, basicamente, precisamos de uma maneira de saber em que estado está uma saga para alguns tipos de recuperação. A falta de um local centralizado para interrogar o status de uma saga é um grande problema. Temos isso com a orquestração, então, como resolveremos o problema nesse caso?

Um dos modos mais fáceis de fazer isso é projetar uma visão no que concerne ao estado de uma saga no sistema existente, consumindo os eventos gerados. Se gerarmos um ID único para a saga, será possível colocá-lo em todos os eventos gerados como parte dessa saga – essa informação é conhecida como *ID de correlação*. Poderíamos, então, ter um serviço cuja tarefa seria simplesmente consumir todos esses eventos e apresentar uma visão do estado em que está cada pedido e, talvez, por meio de código, executar ações para resolver os problemas como parte do processo de atendimento do pedido, caso os outros serviços não consigam fazê-lo por conta própria.

Combinando estilos

Embora possa parecer que as sagas orquestradas e as sagas coreografadas sejam visões diametralmente opostas acerca de como as sagas poderiam ser implementadas, você poderia facilmente considerar uma mistura e uma combinação dos modelos. Talvez haja alguns processos de negócios em seu sistema que sejam mais naturalmente adequados a um ou a outro modelo. Podemos ter também uma única saga que tenha uma combinação dos estilos. No caso de uso do atendimento de pedidos, por exemplo, dentro das fronteiras do serviço de Depósito, quando gerenciamos a embalagem e o despacho de um pacote, podemos usar um fluxo orquestrado, ainda que a requisição original tenha sido feita como parte de uma saga coreografada maior.¹²

Se decidir combinar os estilos, é importante que você ainda tenha

um modo claro de saber o que aconteceu como parte da saga. Sem isso, entender as falhas será complicado, e será difícil se recuperar delas.

Devo usar coreografia ou orquestração?

A implementação de sagas coreografadas pode vir acompanhada de ideias com as quais você e sua equipe talvez não tenham familiaridade. Em geral, elas pressupõem um uso intenso de colaboração orientada a eventos, que não é um assunto compreendido por qualquer pessoa. No entanto, com base em minha experiência, a complexidade adicional relacionada à monitoração do progresso de uma saga quase sempre será superada pelas vantagens associadas em ter uma arquitetura com menos acoplamento.

Deixando de lado minhas preferências pessoais, porém, o conselho geral que dou em relação à orquestração *versus* coreografia é que me sinto muito confortável com o uso de sagas orquestradas quando uma equipe é responsável pela implementação de toda a saga. Em uma situação como essa, quanto mais inerentemente acoplada for a arquitetura, mais fácil será administrá-la dentro das fronteiras de uma equipe. Se você tiver várias equipes envolvidas, prefiro muito mais a saga coreografada decomposta, pois é mais fácil distribuir as responsabilidades pela implementação da saga às equipes, com a arquitetura menos acoplada permitindo que essas equipes trabalhem de modo mais isolado.

Sagas versus transações distribuídas

Como espero ter esclarecido a essa altura, as transações distribuídas implicam alguns desafios significativos, e, afora algumas situações muito específicas, tenho a tendência de evitá-las. Pat Helland, um pioneiro em sistemas distribuídos, sintetiza os desafios básicos da implementação de transações distribuídas para os tipos de aplicações que criamos atualmente:¹³

Na maioria dos sistemas de transação distribuída, a falha em um único nó faz com que o commit da transação seja interrompido. Por sua vez, isso faz com que a aplicação trave. Em sistemas como esses, quanto mais isso acontecer, maior será a probabilidade de um sistema cair. Ao pilotar um avião que exige que todos os seus equipamentos funcionem, o acréscimo de um equipamento reduzirá a disponibilidade do avião.

– PAT HELLAND, *LIFE BEYOND DISTRIBUTED TRANSACTIONS*

Com base em minha experiência, modelar explicitamente os processos de negócio como uma saga evita muitos dos desafios que surgem com as transações distribuídas, ao mesmo tempo que tem a vantagem adicional de deixar os processos que, de outro modo, seriam implicitamente modelados serem muito mais explícitos e evidentes aos desenvolvedores. Fazer com que os processos de negócio essenciais de seu sistema sejam um conceito de primeira classe proporcionará uma série de vantagens.

Uma discussão mais completa sobre a implementação da orquestração e da coreografia, junto com diversos detalhes de implementação, está além do escopo deste livro. O assunto foi abordado no Capítulo 4 do livro *Building Microservices*, mas também recomendo o livro *Enterprise Integration Patterns* para uma exploração mais profunda dos vários aspectos relacionados a esse assunto.¹⁴

Resumo

Decompomos nosso sistema encontrando pontos de separação que possam se transformar em fronteiras de serviços, e essa abordagem pode ser gradual. Ao melhorarmos nossa capacidade de encontrar esses pontos de separação e trabalhar para reduzir o custo de separar os serviços, podemos continuar a expandir nossos sistemas, fazendo com que evoluam, de modo a atender a quaisquer requisitos que possam surgir no futuro. Como podemos ver, parte desse trabalho pode ser difícil, podendo causar problemas

significativos que deverão ser resolvidos. Contudo, o fato de ser uma tarefa que pode ser feita de modo gradual significa que não é preciso ter medo desse trabalho.

Ao separar nossos serviços, introduzimos também alguns problemas novos. No próximo capítulo, veremos os diversos desafios que surgirão à medida que dividirmos o nosso sistema monolítico. Mas não se preocupe, pois apresentarei diversas ideias que ajudarão você a lidar com esses problemas à medida que surgirem.

-
- 1 Se você depende de análise da rede para determinar quem está usando o seu banco de dados, é sinal de que você tem problemas.
 - 2 Havia rumores de que um dos sistemas que usava o nosso banco de dados era uma rede neural Python que ninguém sabia como funcionava, mas que “simplesmente funcionava”.
 - 3 Para uma apresentação detalhada sobre o assunto, você pode assistir a um vídeo de Kresten Krab Thorup, “Riak on Drugs (and the Other Way Around)” (Riak sob efeito de medicamentos [e o inverso]), <http://bit.ly/2m1CvLP>).
 - 4 Sangeeta Handa explicou como a Netflix usou esse padrão como parte de suas migrações de dados na conferência QCon SF, e Daniel Bryant, posteriormente, escreveu um artigo interessante (<http://bit.ly/2m1EwHT>) sobre o assunto.
 - 5 Manter dados históricos em um banco de dados relacional dessa forma pode se tornar complicado, sobretudo se você tiver de reconstruir versões antigas de suas entidades por meio de código. Se você tiver requisitos muito rígidos quanto a isso, talvez valha a pena explorar a geração de eventos como uma alternativa para manter os estados.
 - 6 É a ISO 3166-1 para todos os fãs de ISO por aí!
 - 7 Kief escreveu o livro *Infrastructure as Code: Managing Servers in the Cloud* (Sebastopol: O’Reilly, 2016).
 - 8 Atualmente isso mudou, com o suporte para transações ACID para vários documentos, lançado como parte do Mongo 4.0. Pessoalmente, ainda não usei esse recurso do Mongo; só sei que ele existe!
 - 9 Veja Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol, O’Reilly Media, Inc., 2017).
 - 10 Veja Hector Garcia-Molina e Kenneth Salem, “Sagas”, em *ACM Sigmod Record* 16, nº 3 (1987): 249–259.
 - 11 Realmente, não é possível. Eu tentei!
 - 12 Está fora do escopo deste livro, mas Hector Garcia-Molina e Kenneth Salem

exploraram como várias sagas poderiam ser “aninhadas” para implementar processos mais complexos. Para ler mais sobre esse assunto, veja Hector Garcia-Molina et al, “Modeling Long-Running Activities as Nested Sagas” (Modelando atividades de longa duração como sagas aninhadas), *Data Engineering* 14, nº 1 (março de 1991): 14–18.

13 Veja Pat Helland, “Life Beyond Distributed Transactions” (Vida além das transações distribuídas), *acmqueue* 14, nº 5.

14 As sagas não são explicitamente mencionadas em nenhum dos livros, mas tanto a orquestração como a coreografia são discutidas. Embora não possa falar da experiência dos autores de *Enterprise Integration Patterns*, pessoalmente, eu não conhecia as sagas quando escrevi *Building Microservices*.

CAPÍTULO 5

Dificuldades crescentes

Ao adotar uma arquitetura de microsserviços, você enfrentará desafios pelo caminho. Já vimos alguns desses problemas rapidamente, mas gostaria de explorá-los melhor a fim de dar a você algumas advertências.

Minha expectativa com este capítulo é dar informações suficientes acerca dos tipos de problemas com os quais você poderá deparar. Não posso resolver todos eles neste livro, e muitos dos problemas que apresentarei neste capítulo já foram descritos com mais detalhes no livro *Building Microservices*, escrito exatamente com esses desafios em mente.

Também gostaria de descrever alguns sinais que você deve procurar, os quais ajudarão a identificar quando esses problemas deverão ser resolvidos, além de apresentar uma indicação dos pontos em que esses problemas terão mais chances de aparecer ao longo da jornada.

Mais serviços, mais dificuldades

Quando, exatamente, os problemas ocorrerão em uma arquitetura de microsserviços está relacionado a uma diversidade de fatores. A complexidade das interações entre os serviços, o tamanho da empresa, o número de serviços, as opções de tecnologia, a latência e os requisitos de uptime são apenas um subconjunto das forças que poderão trazer a dor, o sofrimento, a empolgação e o estresse à tona. Isso significa que é difícil dizer quando – e, na verdade, se – você vai realmente deparar com esses problemas.

Em geral, porém, percebi que os tipos de problemas que surgem em uma empresa com dez serviços tendem a ser muito diferentes dos problemas vistos em uma empresa com centenas de serviços. O número de serviços parece ser uma medida tão boa quanto outras para indicar quando determinados problemas terão mais chances de se manifestar. Devo enfatizar que, quando falo do número de serviços, a menos que especifique algo diferente, estarei falando de diferentes serviços lógicos. Quando implantados no ambiente de produção, esses serviços poderão estar na forma de várias instâncias.

Não pense na adoção dos microsserviços como se fosse ligar um interruptor; pense nela como um botão a ser girado. À medida que girar o botão e tiver mais serviços, a expectativa é que você tenha mais oportunidades de se beneficiar com os microsserviços. Contudo, ao girar esse botão, você atingirá diferentes pontos de dificuldade no caminho. Quando isso acontecer, será necessário encontrar meios de resolver esses problemas, os quais poderão exigir novas formas de pensar, novas habilidades, diferentes técnicas ou, talvez, uma nova tecnologia.

A Figura 5.1 mapeia, de modo geral, os pontos de dificuldade que abordaremos na parte restante deste capítulo, com base nos pontos em que, durante o crescimento de seus serviços, esses problemas terão mais chances de surgir. Certamente, não é uma representação científica, e está baseada principalmente na experiência narrada pelas pessoas, mas, como uma visão geral, acredito que seja apropriada.

De 2 a 10 serviços Equipe única ou poucas equipes	De 10 a 50 serviços	Mais de 50 serviços Mais equipes/desenvolvedores
Mudanças que causam incompatibilidade Relatórios	Responsabilidade em escala Experiência dos desenvolvedores Administração de muitas tarefas	Otimização local versus global Serviços órfãos
← Robustez e resiliência → Monitoração e resolução de problemas		

Figura 5.1 – Exibição, em alto nível, dos pontos em que algumas das dificuldades geralmente se manifestam.

Não estou dizendo que você vai, *com certeza*, deparar com todos esses problemas nessas ocasiões, ou se vai sequer vê-los. Há certas variáveis envolvidas, e um diagrama simples como esse não seria capaz de realmente as representar. Um fator em particular, que pode influir no momento em que esses problemas surgirão, é o nível de acoplamento que sua arquitetura terá. Em uma arquitetura mais acoplada, problemas relacionados a robustez, testes, tracing e outros desse tipo poderão se manifestar mais cedo. Tudo que espero fazer é lançar uma luz nas armadilhas em potencial que poderão estar por aí.

No entanto, tenha em mente que você deve usar essas informações como um indicador geral. Certifique-se de que incluirá sistemas de feedback para identificar alguns dos possíveis indicadores que apresentarei neste capítulo.

Agora que já fiz todas as ressalvas relacionadas ao diagrama anterior, vamos analisar cada um desses problemas com um pouco mais de detalhes. Apresentarei algumas pistas acerca dos fatores

que provavelmente poderão trazer esses problemas à tona, uma explicação de como poderão causar impacto, além de algumas diretrizes sobre como enfrentar esses desafios à medida que surgirem.

Responsabilidade em escala

À medida que você tiver cada vez mais desenvolvedores trabalhando em sua arquitetura de microsserviços, atingirá um ponto em que talvez queira reconsiderar o modo de lidar com as responsabilidades.

Martin Fowler já havia feito uma distinção entre os diferentes tipos de responsabilidade (<http://bit.ly/2n5pSAf>) do ponto de vista da responsabilidade por um código genérico, e acho que, de modo geral, ela é apropriada ao contexto da responsabilidade pelos microsserviços também. Nesse caso, estamos olhando para a responsabilidade basicamente do ponto de vista de fazer alterações no código, e não da responsabilidade no que diz respeito a quem cuida das implantações, ao suporte de primeira linha e assim por diante. Antes de falarmos dos tipos de problemas que podem surgir, vamos analisar os conceitos descritos por Martin e colocá-los no contexto da arquitetura de microsserviços:

Responsabilidade forte pelo código

Todos os serviços têm responsáveis. Se alguém que não pertença ao grupo de responsáveis quiser fazer uma alteração, essa pessoa terá de submeter a modificação aos responsáveis, que decidirão se ela será permitida. O uso de um pull request por pessoas que não pertençam ao grupo de responsáveis é um exemplo de como isso poderia ser tratado.

Responsabilidade fraca pelo código

A maioria dos serviços, se não todos, tem alguém como responsável, mas qualquer pessoa ainda pode modificar

diretamente seus módulos sem recorrer à necessidade de algo como os pull requests. Com efeito, o sistema de controle de versões é configurado de modo a permitir que qualquer pessoa possa modificar qualquer código, mas a expectativa é que, se você vai modificar o serviço de outras pessoas, você falará com elas antes.

Responsabilidade coletiva pelo código

Ninguém é dono de nada, e qualquer pessoa pode modificar o que quiser.

Como esse problema pode se manifestar?

À medida que o número de serviços e de desenvolvedores aumentar, você poderá começar a ter problemas com a responsabilidade coletiva. Para que a responsabilidade coletiva funcione, o coletivo precisa estar bem conectado, a ponto de ter uma compreensão compartilhada acerca de como deve ser uma modificação apropriada, e a direção para a qual você quer conduzir um determinado serviço do ponto de vista técnico.

Se não for administrado, já vi um modelo de responsabilidade coletiva pelo código se tornar desastroso em uma arquitetura de microsserviços, ao ser escalado. Uma fintech¹ com a qual conversei narrou histórias sobre uma pequena equipe que passou por um rápido crescimento, passando de 30 a 40 desenvolvedores para mais de 100, mas sem qualquer responsabilidade atribuída às diferentes partes do sistema, ou sem nenhum conceito de “dono” além de “as pessoas sabem o que é certo”.

Como resultado, não havia uma visão clara da arquitetura do sistema à medida que ele evoluía, e um “sistema monolítico distribuído” extremamente emaranhado surgiu. Um dos desenvolvedores dessa empresa se referia à arquitetura como “arquitetura corredor de macarrão” porque era cheia de furos – as pessoas simplesmente “faziam um novo furo” sempre que queriam,

expondo dados ou apenas fazendo várias chamadas ponto a ponto. Na verdade, esses tipos de desafio são mais fáceis de corrigir em um sistema monolítico, porém muito mais difíceis em um sistema distribuído – o custo de reorganizar um sistema monolítico distribuído é muito mais alto.

Quando esse problema poderia ocorrer?

Para muitas equipes que começam pequenas, um modelo de responsabilidade coletiva pelo código faz sentido. Com um pequeno número de desenvolvedores no mesmo local (cerca de 20), sinto-me satisfeito com esse modelo. À medida que o número de desenvolvedores aumentar, ou se esses desenvolvedores estiverem espalhados, será mais difícil manter todos igualmente cientes de informações como o que é um bom commit, ou como os serviços individuais devem evoluir.

Para equipes que passam por um rápido crescimento, um modelo de responsabilidade coletiva será problemático. O problema é que, para um modelo de responsabilidade coletiva funcionar, você precisará de tempo e espaço para criar um consenso, e deverá se manter atualizado à medida que surgirem novidades. De modo geral, isso será mais difícil se houver mais pessoas, e *realmente* difícil se você estiver contratando novos funcionários em um ritmo acelerado (ou transferindo pessoas para o projeto).

Possíveis soluções

Com base em minha experiência, uma responsabilidade forte pelo código é o modelo quase universalmente adotado pelas empresas que implementam arquiteturas de microsserviços em larga escala, que possuam várias equipes e mais de 100 desenvolvedores. Será mais fácil se cada equipe decidir as regras sobre o que constitui uma boa mudança; você pode considerar que cada equipe adotará localmente um modelo de responsabilidade coletiva pelo código. Esse modelo também permite ter equipes orientadas a produtos; se

sua equipe é responsável por alguns serviços, e esses serviços estiverem orientados ao domínio de negócio, suas equipes poderão manter o foco em uma área do domínio de negócio. Isso facilitará manter equipes especializadas no domínio, com foco nos clientes, que muitas vezes incluam os responsáveis pelo produto para orientar seus trabalhos.

Mudanças que causam incompatibilidade

Um microserviço existe como parte de um sistema maior. Ele consome funcionalidades fornecidas por outros microserviços, expõe a sua própria funcionalidade para outros microserviços consumidores ou, possivelmente, tem as duas funções. Em uma arquitetura de microserviços, nós nos esforçamos para permitir implantações independentes; contudo, para que isso ocorra, devemos garantir que, quando fizermos uma alteração em um microserviço, não causaremos falhas em nossos consumidores.

Podemos pensar na funcionalidade que expomos aos demais microserviços como se fosse um *contrato*. Não se trata apenas de dizer: “Estes são os dados que devolverei”. Trata-se também de definir o comportamento esperado para o seu serviço. Independentemente de você ter ou não ter deixado esse contrato explícito com seus consumidores, o fato é que ele existe. Ao fazer uma mudança em seu serviço, você deve garantir que não violará esse contrato; caso contrário, problemas desagradáveis poderão ocorrer no ambiente de produção.

Cedo ou tarde, você terá de lidar com os problemas provocados por mudanças que causem incompatibilidade – seja porque você tomou uma decisão consciente de fazer uma alteração incompatível com versões anteriores, ou talvez porque fez uma modificação inocente que achou que teria impacto somente em seu serviço local, mas descobriu que ela causava problemas em outros serviços de formas que você não havia imaginado.

Como esse problema pode se manifestar?

A pior ocorrência desse problema se dá quando vemos interrupções de serviço no ambiente de produção provocadas por novos microsserviços que são ativados, porém incompatíveis com os serviços existentes. É um sinal de que você não está identificando violações de contrato acidentais com antecedência suficiente. Esses problemas poderão ser catastróficos caso você não tenha um sistema de rollback rápido definido. O único ponto positivo desses modos de falha é que, *normalmente*, eles se manifestam de forma rápida após um lançamento de versão, a menos que a mudança incompatível com versões anteriores tenha sido feita em uma parte do contrato de serviço que seja raramente usada.

Outro sinal é quando você começa a ver pessoas tentando coordenar implantações simultâneas de vários serviços – às vezes, chamamos isso de *lançamento de versão amarrada* (lock-step release). Esse também poderia ser um sinal de que o problema está ocorrendo, pois há uma tentativa de administrar mudanças de contrato entre cliente e servidor. Um lançamento de versão amarrada ocasional não é tão ruim dentro de uma equipe, mas, se for recorrente, uma investigação se fará necessária.

Quando esse problema poderia ocorrer?

Acho que essa é uma dificuldade com a qual as equipes deparam bem cedo, sobretudo se o desenvolvimento estiver distribuído entre mais de uma equipe. Com uma só equipe, as pessoas tendem a ser um pouco mais cientes ao fazer mudanças que causem incompatibilidade, parcialmente porque há uma boa chance de os desenvolvedores estarem trabalhando tanto no serviço sendo modificado como no serviço consumidor. Se você se vir em situações nas quais uma equipe modifica um serviço, que é então consumido por outras equipes, esse problema poderá surgir com mais frequência.

Com o tempo, à medida que se tornarem mais maduras, as equipes

passarão a ser mais criteriosas ao fazer mudanças que, antes de tudo, evitem incompatibilidades, além de implantar métodos que identifiquem precocemente os problemas.

Possíveis soluções

Eu tenho um conjunto de regras para gerenciar violações de contratos. Elas são bem simples:

1. Não viole os contratos.
2. Veja a regra 1.

Tudo bem, estou brincando, mas só um pouco. Fazer alterações que violem os contratos expostos por você não é bom, e é difícil administrá-las. Você realmente deve manter essas violações em um nível mínimo se puder. Considerando o que foi dito, as regras a seguir são mais realistas:

1. Elimine mudanças acidentais que causem incompatibilidade.
2. Pense duas vezes antes de fazer uma mudança que cause incompatibilidade – é possível evitá-la?
3. Se precisar fazer uma mudança que cause incompatibilidade, dê tempo para que seus consumidores possam fazer a migração.

Vamos analisar esses passos com um pouco mais de detalhes.

Elimine mudanças acidentais que causem incompatibilidade

Ter um esquema explícito para o seu microsserviço pode permitir que as violações *estruturais* em seu contrato sejam detectadas. Se você expõe um método de cálculo que costumava receber dois inteiros como parâmetros, mas agora aceita apenas um, obviamente essa é uma mudança que causa incompatibilidade, o que deverá estar evidente pelo novo esquema. Deixar esse esquema explícito aos desenvolvedores pode contribuir para que esse tipo de problema seja detectado mais cedo; se eles tiverem de fazer uma mudança no esquema manualmente, esse será um passo explícito, e espera-se que isso os fará parar um instante e pensar na mudança. Se você tiver um formato de esquema formal, há a opção

de lidar com ele também por meio de programação, é claro, embora não seja algo feito com a frequência que eu gostaria de ver. O protocolock (<http://bit.ly/2kUxvbq>) é um exemplo de uma ferramenta desse tipo, e ela proíbe fazer mudanças incompatíveis nos buffers de seu protocolo.

A opção padrão para muitas pessoas é utilizar formatos de intercâmbio sem esquemas, com JSON sendo o exemplo mais comum. Embora, teoricamente, você possa definir esquemas explícitos para JSON, eles não são usados na prática. Os desenvolvedores tendem inicialmente a lamentar as restrições dos esquemas formais – depois que tiverem de lidar com mudanças que causam incompatibilidades entre serviços, eles mudarão de ideia. Também vale a pena notar que alguns dos formatos de serialização que fazem uso de esquemas conseguem obter melhorias de desempenho na desserialização dos dados por causa dos tipos formais – algo que vale a pena considerar.

As violações estruturais, no entanto, são apenas uma parte da questão. Há também as violações *semânticas* a serem consideradas. Se o nosso método de cálculo continua aceitando dois inteiros, mas a última versão de nosso microserviço multiplica esses dois inteiros, enquanto costumava somá-los antes, isso também constitui uma violação do contrato. Na prática, os testes são uma das melhores maneiras de detectar o problema. Veremos esse assunto em breve.

Não importa o que você faça, um resultado positivo rápido é deixar o mais claro possível aos desenvolvedores se eles fizerem alterações no contrato externo. Isso pode significar evitar tecnologias que serializem dados como se fosse mágica ou que gerem esquemas a partir do código, dando preferência a implementar isso manualmente. Acredite em mim – dificultar a modificação do contrato de um serviço é melhor do que causar falhas constantemente aos consumidores.

Pense duas vezes antes de fazer uma mudança que cause

incompatibilidade

Se for possível, prefira fazer mudanças que causem a expansão de seu contrato. Adicione novos métodos, recursos, tópicos ou qualquer componente que dê suporte à nova funcionalidade, sem remover a funcionalidade antiga. Procure encontrar maneiras de oferecer suporte à antiga funcionalidade, ao mesmo tempo que dá suporte à nova. Isso pode significar que você acabará tendo de dar suporte a um código antigo, mas isso ainda representará menos trabalho do que lidar com uma mudança que cause incompatibilidade. Lembre-se de que, se você decidir violar um contrato, caberá a você lidar com as implicações dessa decisão.

Dê tempo aos consumidores para que façam a migração

Conforme já deixei bem claro desde o princípio, os microsserviços foram concebidos para que sejam *implantados de modo independente*. Ao fazer uma mudança em um microsserviço, você deverá ser capaz de implantar esse microsserviço em um ambiente de produção sem ter de implantar outros itens. Para que isso funcione, é necessário modificar o contrato de seu serviço de modo que os consumidores atuais não sofram impactos – portanto, implica que você deve permitir que os consumidores continuem usando o contrato antigo, mesmo que seu contrato mais recente esteja disponível. Desse modo, você deverá dar um tempo para que todos os consumidores modifiquem seus serviços a fim de migrarem para a nova versão de seu serviço.

Já vi essa tarefa ser feita de duas formas. A primeira consiste em executar duas versões de seu microsserviço, conforme mostra a Figura 5.2: duas versões do serviço de Notificações estão disponíveis ao mesmo tempo, cada uma expondo diferentes endpoints incompatíveis, dentre os quais os consumidores poderão escolher. Os principais desafios com essa abordagem são o fato de ser necessário ter mais infraestrutura para executar os serviços extras, o fato de que, provavelmente, você terá de manter a compatibilidade dos dados entre as versões dos serviços e as

correções de bug talvez tenham de ser feitas em todas as versões em execução, o que, inevitavelmente, exigirá a criação de branches no código-fonte. Esses problemas serão, de certo modo, atenuados, se você tiver as duas versões coexistindo somente por pouco tempo, e essa é a única situação na qual eu consideraria a adoção dessa abordagem.

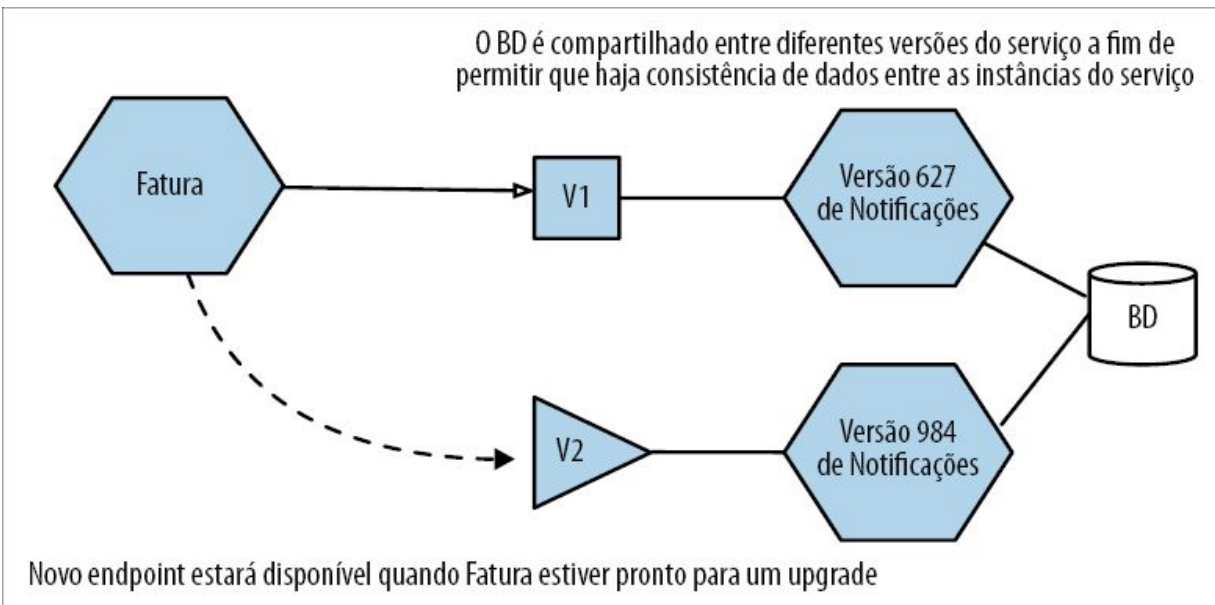


Figura 5.2 – Duas versões do mesmo microserviço coexistindo a fim de oferecer suporte para mudanças que causem incompatibilidade com versões anteriores.

A abordagem de minha preferência é ter uma versão de seu microserviço em execução, mas fazê-la aceitar os *dois* contratos, conforme vemos na Figura 5.3. Essa solução poderia envolver a exposição de duas APIs em portas diferentes, por exemplo. Ela faz com que a complexidade esteja na implementação dos microserviços, porém evita os problemas da abordagem anterior. Já conversei com algumas equipes que dão suporte a três ou mais contratos antigos no mesmo serviço há anos porque os consumidores externos são incapazes de fazer a mudança. Não é muito divertido estar nessa situação, mas, ainda que você se veja diante de consumidores que não farão a mudança, acho que esta continua sendo a melhor opção.

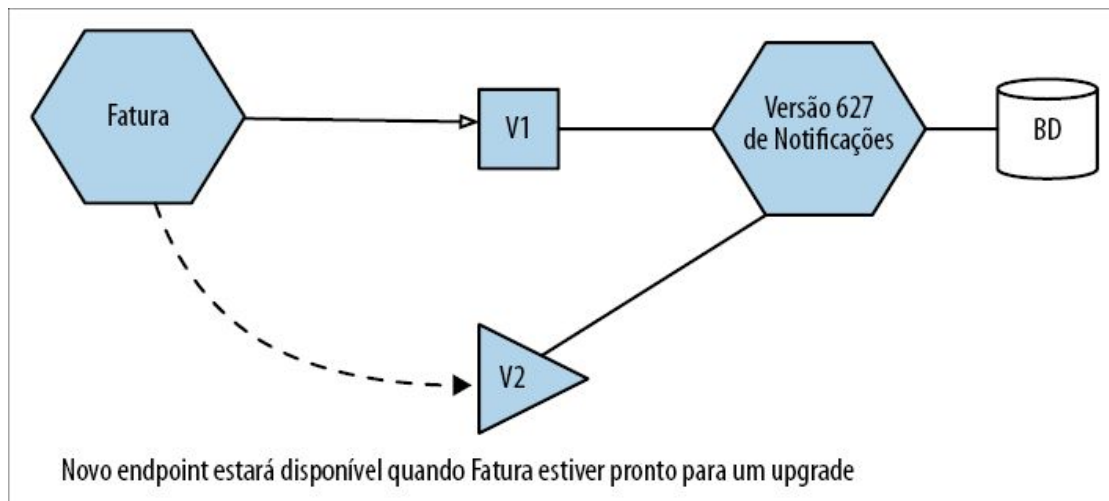


Figura 5.3 – Um serviço expondo dois contratos.

É claro que, se a mesma equipe lida tanto com o consumidor como com o produtor, você poderá fazer um lançamento de versão amarrada e implantar as novas versões, tanto do consumidor como do produtor, ao mesmo tempo. Não é algo que eu gostaria de fazer com frequência, mas, pelo menos, com uma única equipe, talvez seja mais fácil administrar um lançamento coordenado – apenas não faça disso um hábito!

As mudanças dentro de uma equipe são mais fáceis de administrar, pois você controlará os dois lados da equação. À medida que o microserviço que você quiser modificar for mais amplamente utilizado, o custo de gerenciar a mudança será mais alto. Como resultado, você poderá se sentir mais confortável para fazer mudanças que causem incompatibilidades se estiver dentro de uma equipe, mas quebrar a compatibilidade de uma API exposta a empresas de terceiros provavelmente será muito mais complicado.

Independentemente do modo como fizer a mudança, será necessário ter uma boa comunicação com as pessoas que administram os serviços que consomem os seus serviços. É bem provável que você lhes causará inconveniências (no melhor dos casos), portanto, ter um bom relacionamento com elas é uma boa ideia. Trate os consumidores de seu serviço como consumidores – e você deve tratar bem os seus consumidores!

Resolva este problema – rápido!

À medida que as empresas tiverem uma quantidade cada vez maior de microsserviços, em algum momento, elas descobrirão uma forma de eliminar a maior parte das mudanças acidentais que causem incompatibilidade, e criarão um método gerenciado para lidar com mudanças previstas. Se não o fizerem, os impactos serão muito significativos, a ponto de uma arquitetura de microsserviços se tornar inviável. Falando de outra maneira, tenho fortes suspeitas de que empresas pequenas de microsserviços que não resolverem esse problema não durarão o bastante para se transformar em empresas de microsserviços de grande porte.

Relatórios

Em um sistema monolítico, em geral você terá um banco de dados monolítico. Isso significa que os stakeholders (pessoas-chaves) que queiram analisar todos os dados em conjunto, muitas vezes envolvendo grandes operações de junção entre os dados, terão um esquema pronto com base no qual gerarão seus relatórios. Eles podem gerá-los diretamente a partir do banco de dados monolítico, talvez em uma réplica para leitura, conforme mostra a Figura 5.4.

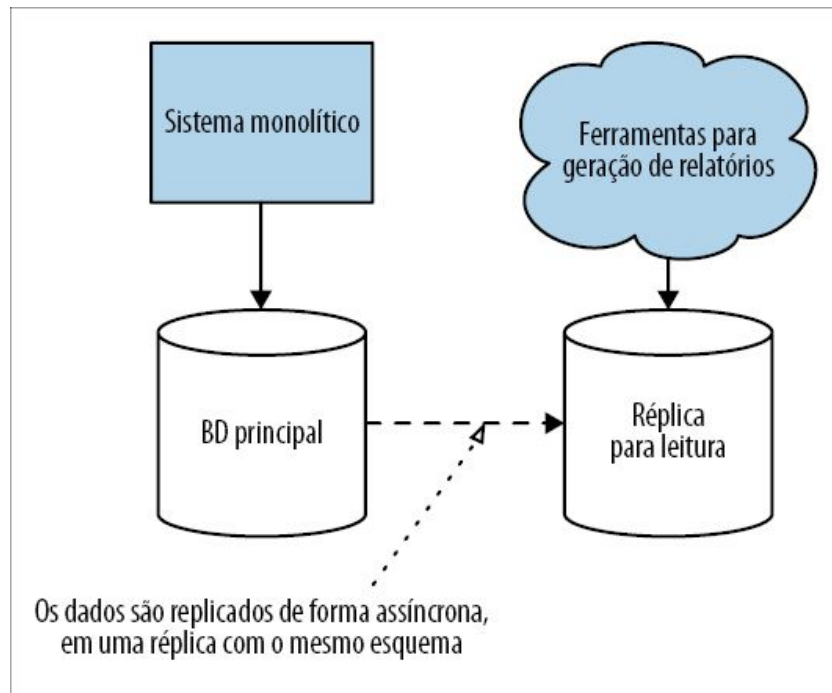


Figura 5.4 – Relatórios gerados diretamente no banco de dados de um sistema monolítico.

Em uma arquitetura de microsserviços, esse esquema monolítico terá sido dividido. Isso não significa que a necessidade de ter relatórios envolvendo todos os nossos dados tenha desaparecido; nós apenas dificultamos o processo, pois, agora, nossos dados estarão espalhados em vários esquemas isolados do ponto de vista lógico.

Quando esse problema poderia ocorrer?

Esse problema tende a se manifestar razoavelmente cedo e, em geral, surge na fase em que você começa a considerar a decomposição de um esquema monolítico. A expectativa é que o problema seja detectado com antecedência, mas já vi mais de um projeto no qual a equipe não havia percebido até a metade do caminho que a direção que a arquitetura estava tomando criaria sérias dificuldades aos stakeholders interessados nos casos de uso de relatórios. Com muita frequência, as necessidades de relatórios downstream não são consideradas com antecedência suficiente,

pois estão fora da esfera do desenvolvimento normal de software e da manutenção do sistema – o que os olhos não veem, o coração não sente.

Você poderá contornar esse problema caso o seu sistema monolítico já utilize uma fonte de dados dedicada para a geração de relatórios, por exemplo, um data warehouse ou um data lake. Então, tudo que você terá de fazer é garantir que seus microsserviços sejam capazes de copiar os dados apropriados para as fontes de dados existentes.

Possíveis soluções

Em muitas situações, os stakeholders interessados em ter acesso a todos os dados em um só local provavelmente também investiram em uma cadeia de ferramentas e processos que esperam ter acesso direto ao banco de dados, em geral fazendo uso de SQL. Isso também implica que a geração de relatórios provavelmente estará vinculada ao design do esquema de seu banco de dados monolítico. Como resultado, a menos que você queira mudar o modo como funcionam, terá de continuar disponibilizando um único banco de dados para a geração dos relatórios – possivelmente um cujo design corresponda ao esquema antigo a fim de limitar o impacto de qualquer mudança.

A abordagem mais simples para resolver esse problema é, inicialmente, considerar a necessidade de ter um único banco de dados para armazenar os dados de relatórios, separando-o dos bancos de dados que seus microsserviços utilizam para armazenar e obter dados, como mostra a Figura 5.5. Isso permite que o conteúdo e o design de seu banco de dados para relatórios estejam desacoplados do design e da evolução dos requisitos de armazenagem de dados de cada serviço. A solução também permite que esse novo banco de dados de relatórios seja modificado com os requisitos específicos dos usuários de relatórios em mente. Assim, tudo que você tem a fazer é descobrir um modo de seus

microsserviços poderem “enviar” dados para o seu novo esquema.

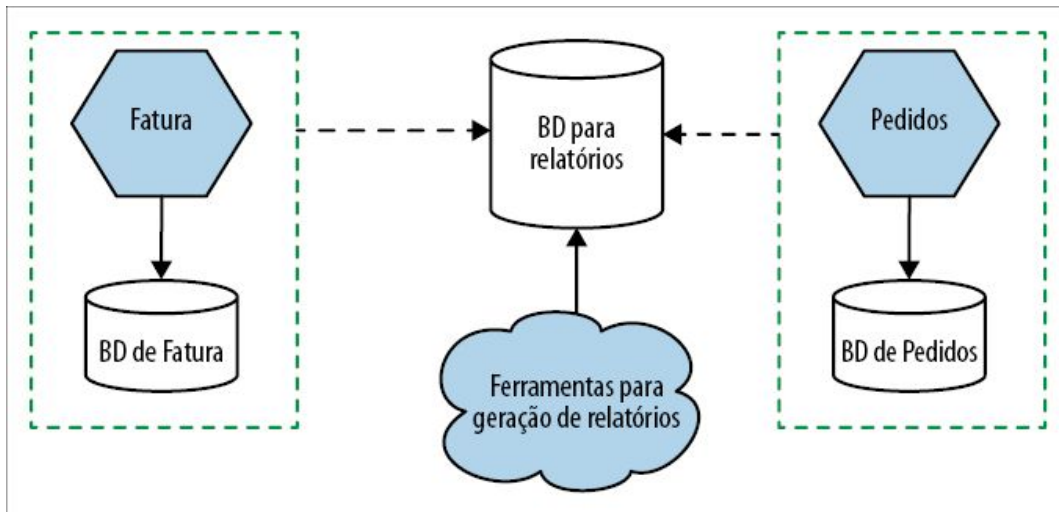


Figura 5.5 – Um banco de dados dedicado para relatórios, com dados enviados a ele por diferentes microsserviços.

Já vimos possíveis soluções para esse problema no Capítulo 4. Um sistema de captura de dados modificados (change data capture) é uma possível solução óbvia para resolvê-lo, mas técnicas como visões (views) também podem ser convenientes, pois você poderá projetar um único esquema para relatórios a partir de visões expostas pelos esquemas dos bancos de dados de vários microsserviços. Também poderíamos considerar o uso de outras técnicas, como fazer com que os dados sejam copiados para o esquema de relatórios por meio de programação, como parte do código dos microsserviços, ou, quem sabe, ter componentes intermediários que possam preencher o banco de dados de relatórios, observando eventos de serviços upstream.

Os desafios e as possíveis soluções relacionadas a esse assunto foram explorados com muito mais detalhes no Capítulo 5 do livro *Building Microservices*.

Monitoração e resolução de problemas

Substituímos nosso sistema monolítico por microsserviços para que cada interrupção de serviço pudesse se parecer mais com um mistério de assassinato.

– HONEST STATUS PAGE (@HONEST_UPDATE), [HTTP://BIT.LY/2MLDXQH](http://bit.ly/2MLDXQH)

Em uma aplicação monolítica padrão, podemos adotar uma abordagem razoavelmente simplista para a monitoração. Há um número pequeno de máquinas com que nos preocupar, e o modo de falha da aplicação é, de certo modo, binário: em geral, a aplicação estará totalmente ativa ou totalmente inativa. Em uma arquitetura de microsserviços, podemos ter falha em apenas uma instância de serviço ou em apenas um tipo de instância – é possível identificá-los de modo apropriado?

Em um sistema monolítico, se sua CPU estiver com 100% de uso constante por um longo período, saberemos que isso é um grande problema. Em uma arquitetura de microsserviços com dezenas ou centenas de processos, podemos dizer o mesmo? Será necessário acordar alguém às três horas da manhã se houver apenas um processo com 100% de consumo constante de CPU?

Identificar em que ponto estão os problemas, e saber se os problemas que você está vendo são realmente situações com as quais você deveria se preocupar, será muito mais complicado se houver mais partes compondo o sistema. O modo de abordar a monitoração e a resolução de problemas terá de mudar quando sua arquitetura de microsserviços crescer. É uma área que exigirá constante atenção e investimentos.

Quando esses problemas poderão ocorrer?

É um pouco mais complicado prever exatamente quando você começará a ter problemas com isso. A resposta simples poderia ser “na primeira vez que algo der errado no ambiente de produção”, mas tentar descobrir o que deu errado, e em que lugar, é algo com o qual os desenvolvedores e os engenheiros de teste terão de lidar antes mesmo de o sistema chegar ao ambiente de produção. Essas

limitações poderiam ser alcançadas se houver apenas dois serviços, ou talvez não surjam até você atingir 20 ou mais serviços.

Como pode ser difícil prever exatamente quando sua abordagem de monitoração começará a deixar você na mão, tudo que posso sugerir é que você dê prioridade à implementação de algumas melhorias básicas com antecedência.

Como esses problemas poderão ocorrer?

De certo modo, isso é razoavelmente fácil de identificar. Você verá problemas no ambiente de produção, os quais não poderão ser explicados ou compreendidos, verá alertas disparados apesar de o sistema estar aparentemente saudável, e será mais difícil responder à simples pergunta: “Está tudo bem?”.

Possíveis soluções

Uma série de métodos – alguns fáceis de implementar, outros, mais complexos – pode ajudar a mudar o modo como você monitora e resolve os problemas em uma arquitetura de microsserviços. Apresentaremos a seguir uma visão geral não exaustiva de alguns pontos essenciais a serem considerados.

Agregação de logs

Com um número pequeno de máquinas, particularmente máquinas de vida longa, quando precisávamos verificar logs, geralmente acessávamos as próprias máquinas e buscávamos as informações. O problema em uma arquitetura de microsserviços é que temos muitos mais processos, em geral executando em mais máquinas, as quais podem ter vida curta (por exemplo, máquinas virtuais e contêineres).

Um *sistema de agregação de logs* permitirá capturar todos os seus logs e encaminhá-los a um local centralizando, no qual será possível fazer buscas e, em alguns casos, os logs poderão ser usados até mesmo para gerar alertas. Há muitas opções, que variam da pilha

ELK (<https://www.elastic.co/elk-stack>) de código aberto (Elastic search, Logstash/Fluent D e Kibana) ao meu preferido, o Humio (<https://humio.com/>), e esses sistemas podem ser extremamente convenientes.



Considere seriamente a implementação de um sistema de agregação de logs como a sua primeira tarefa, antes de implementar uma arquitetura de microsserviços. Ele será extremamente útil, e um bom meio de testar a habilidade de sua empresa de implementar mudanças na área operacional.

A agregação de logs é um dos métodos mais simples para implementar, e é algo que você deve fazer bem cedo. Com efeito, sugiro que seja a sua *primeira* tarefa ao implementar uma arquitetura de microsserviços. Isso se deve, parcialmente, ao fato de esse sistema ser muito útil desde o princípio. Além do mais, se a sua empresa tiver dificuldades para implementar um sistema de agregação de logs apropriado, talvez você queira reavaliar se está pronto para ter microsserviços. O trabalho necessário para implementar um sistema de agregação de logs é razoavelmente simples, e se, como empresa, você não estiver pronto para essa tarefa, os microsserviços provavelmente serão um passo grande demais.

Tracing

Entender em que ponto uma sequência de chamadas entre microsserviços falhou, ou qual serviço causou um pico de latência, pode ser difícil se você somente puder analisar informações de cada serviço isoladamente. Ser capaz de combinar uma série de fluxos e analisá-los como um todo é extremamente útil.

Como ponto de partida, gere IDs de correlação para todas as chamadas que entrarem em seu sistema, como mostra a Figura 5.6. Quando o serviço de Fatura recebe uma chamada, um ID de correlação lhe será atribuído. Quando o serviço fizer uma chamada para o serviço de Notificações, ele passará adiante esse ID de correlação – isso poderia ser feito por meio de um cabeçalho HTTP, com um campo no payload de uma mensagem ou algum outro

método. Em geral, eu faria um gateway de API ou um service mesh gerar o ID de correlação inicial.

Quando o serviço de Notificações tratar a chamada, ele poderá registrar informações no log acerca do que está fazendo, junto com esse mesmo ID de correlação, permitindo a você utilizar um sistema de agregação de logs para consultar todos os logs associados a um dado ID de correlação (supondo que você coloque o ID de correlação em um local padrão no formato do log). Você pode, é claro, executar outras tarefas com os IDs de correlação, por exemplo, gerenciar sagas (conforme discutido no Capítulo 4).

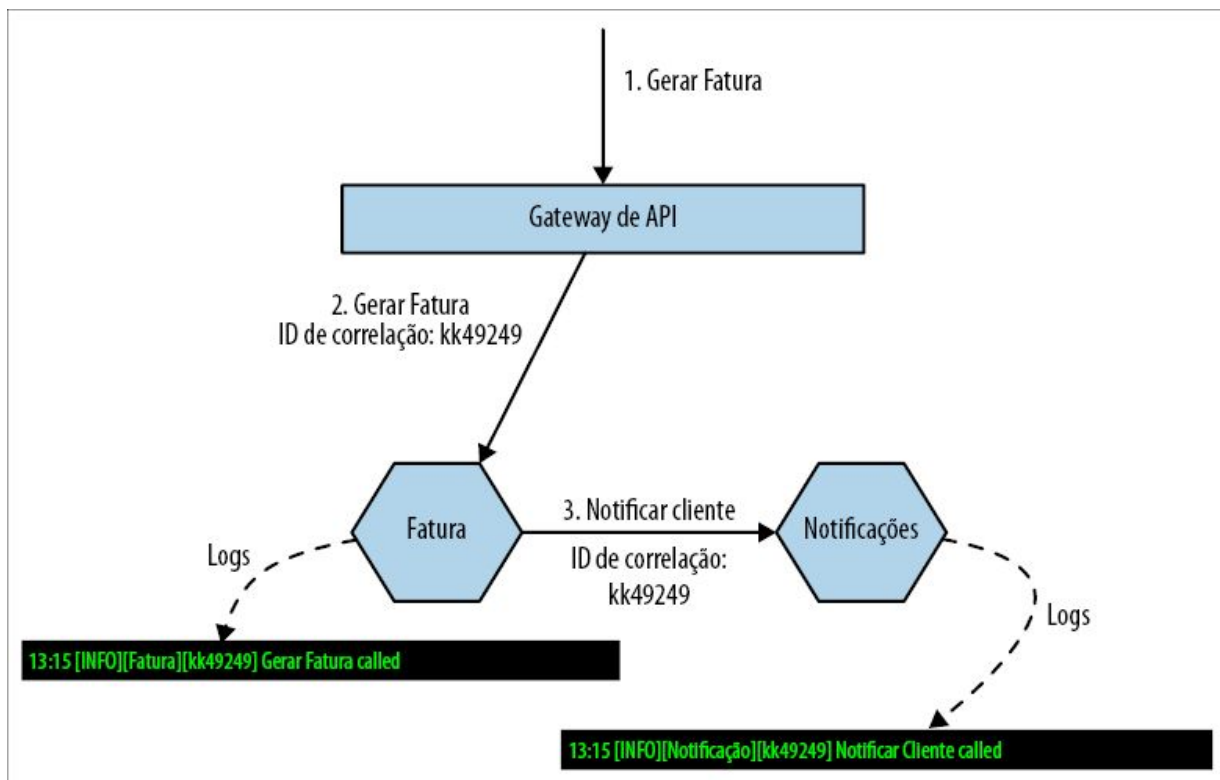


Figura 5.6 – Usando IDs de correlação para garantir que informações sobre uma cadeia específica de chamadas possam ser coletadas.

Levando essa ideia adiante, também podemos usar ferramentas para monitorar o tempo gasto pelas chamadas. Por causa do modo como os sistemas de agregação de logs funcionam, com os logs acumulados e encaminhados para um agente central regularmente,

Quanto mais sensível à latência for a sua aplicação, mais cedo eu pensaria em implementar uma ferramenta de tracing distribuído como o Jaeger. Vale a pena observar que, se você já implementou a geração do ID de correlação e o utiliza em sua arquitetura de microsserviços existente (o que, em geral, é uma ideia que defendo muito antes de você precisar de uma ferramenta de tracing distribuído), é provável que você já saiba quais são os pontos em sua pilha de serviços atual que poderão ser facilmente modificados para que dados sejam enviados a uma ferramenta apropriada. O uso de um service mesh também pode ajudar, pois ele é, no mínimo, capaz de lidar com o tracing de entrada e de saída para você, mesmo que não possa fazer muito no que diz respeito a instrumentar as chamadas nos microsserviços individuais.



capturar informações de traces distribuídos e analisar o desempenho das chamadas individuais.

Testes no ambiente de produção

Testes funcionais automatizados em geral são usados para nos dar feedbacks antes da implantação, com o intuito de saber se o nosso software tem qualidade suficiente para ser implantado. No entanto, uma vez que o software esteja no ambiente de produção, continuamos querendo ter o mesmo feedback! Mesmo que uma dada funcionalidade já tenha sido bem-sucedida no ambiente de produção, uma nova implantação de serviço ou uma mudança no ambiente poderiam causar problemas nessa funcionalidade no futuro.

Ao incluir o comportamento de um usuário fictício em nosso sistema, na forma do que frequentemente chamamos de *transações sintéticas*, podemos definir o comportamento esperado e gerar alertas se não for o caso. Em uma das empresas em que trabalhei, a Atomist, tínhamos um processo de inclusão de novos clientes que era, de certo modo, complexo, e exigia a autorização de nosso software com contas do GitHub e do Slack. Havia muitas partes que logo poderiam ter problemas nesse processo, com fatores como limites para as APIs do GitHub. Um de meus colegas, Sylvain Hellegouarch, gerou um script para o cadastramento de clientes fictícios. Disparávamos regularmente um processo de inscrição para um desses clientes fictícios, e todo o processo era executado, de ponta a ponta. Quando ele falhava, em geral era um sinal de que havia algo errado em nossos sistemas, e era muito melhor identificar o problema com um usuário “fictício” do que com um usuário de verdade!

Um bom ponto de partida para testes em produção poderia ser usar casos de testes fim a fim, retrabalhando-os para que sejam utilizados em um ambiente de produção. Um ponto importante a ser considerado é garantir que esses “testes” não causem impactos imprevistos no ambiente de produção. Na Atomist, criamos contas

no GitHub e no Slack, que controlávamos para uso nas transações sintéticas; desse modo, nenhuma pessoa de verdade estava envolvida ou sofreria impactos, e fazer uma limpeza dessas contas depois era uma tarefa muito simples para os nossos scripts. Por outro lado, já ouvi relatos de uma empresa que acabou acidentalmente fazendo o pedido de duzentas máquinas de lavar roupa para que fossem entregues na matriz porque não haviam levado devidamente em conta o fato de que os pedidos de teste acabariam realmente sendo enviados. Portanto, tome cuidado!

Em direção à observabilidade

Com os processos tradicionais de monitoração e de geração de alertas, pensamos no que poderia dar errado, coletamos informações para saber quando isso acontece e as usamos para disparar os alertas. Portanto, nós nos preparamos basicamente para lidar com causas conhecidas de problemas – espaço em disco que se esgota, uma instância de serviço que não responde ou, talvez, um pico de latência.

À medida que nossos sistemas se tornam mais complicados, será cada vez mais difícil prever todas as maneiras desagradáveis pelas quais nosso sistema poderia nos deixar na mão. Nesse ponto, é importante permitir que façamos perguntas sem respostas definidas sobre o nosso sistema quando esses problemas ocorrerem, com o intuito de nos ajudar, antes de tudo, a estancar a sangria e garantir que o sistema possa continuar funcionando, mas também para permitir que colemos informações suficientes para corrigir o problema no futuro.

Desse modo, devemos ser capazes de coletar muitas informações sobre o que nosso sistema está fazendo, permitindo que, após o fato consumado, façamos perguntas sobre os dados que nem sabíamos que teríamos de fazer. Tracing e logs podem constituir uma fonte importante de dados com base nos quais podemos fazer perguntas e utilizar informações reais, em vez de ficar conjecturando a fim de determinar qual é o problema. O segredo é fazer com que

essas informações sejam fáceis de consultar e de visualizar em seu contexto.

Não parta do pressuposto de que você sabe as respostas com antecedência. Em vez disso, adote a postura de que poderá e ficará surpreso, portanto, torne-se bom em fazer perguntas sobre o seu sistema e certifique-se de que usará conjuntos de ferramentas que permitirão fazer consultas de informações *ad hoc*. Se quiser explorar esse conceito de modo mais minucioso, recomendo o livro *Distributed Systems Observability* como um excelente ponto de partida.²

Experiência do desenvolvedor local

À medida que tiver mais e mais serviços, a experiência do desenvolvedor poderá começar a sofrer consequências. Runtimes que exijam mais recursos, como a JVM, podem limitar o número de microsserviços possíveis de ser executados na máquina de um único desenvolvedor. É provável que eu pudesse executar quatro ou cinco microsserviços baseados em JVM como processos separados em meu notebook, mas seria possível executar dez ou vinte? Provavelmente não. Mesmo com runtimes menos exigentes, há um limite para a quantidade de componentes que podem ser executados localmente, o que, inevitavelmente, dará início a conversas sobre o que fazer quando não for mais possível executar todo o sistema em uma só máquina.

Como esse problema pode se manifestar?

O processo cotidiano de desenvolvimento pode começar a ficar mais lento, com a geração de versão e execução locais demorando mais, como consequência da necessidade de ter mais serviços ativos. Os desenvolvedores começarão a requisitar máquinas mais potentes para lidar com a quantidade de serviços necessária, e, embora isso talvez não seja um problema como uma solução de curto prazo, você vai apenas ganhar um pouco de tempo se o

conjunto de seus serviços continuar a aumentar.

Quando esse problema poderia ocorrer?

Quando, exatamente, o problema se manifestará provavelmente será uma função do número de serviços que um desenvolvedor quiser executar localmente, junto com o uso de recursos por parte desses serviços. Uma equipe que utilize Go, Node ou Python poderia constatar que pode ter mais serviços executando localmente até atingir os limites de recursos – contudo, uma equipe que utilize a JVM poderia deparar antes com esse problema.

Também acho que as equipes que adotam o modelo de responsabilidade coletiva para vários serviços estão mais suscetíveis ao problema. É mais provável que essas equipes precisem da capacidade de alternar entre diferentes serviços durante o seu desenvolvimento. Equipes com responsabilidade forte para alguns serviços manterão o foco, em sua maior parte, apenas em seus próprios serviços, e é mais provável que desenvolvam métodos para stubbing de outros serviços que estejam fora de seu controle.

Possíveis soluções

Se eu quiser fazer desenvolvimentos locais, mas gostaria de reduzir o número de serviços que tenho de executar, uma técnica comum é criar um “stub” para os serviços que não quero executar por conta própria, ou, ainda, ter uma forma de apontar para instâncias que estejam executando em outro lugar. Uma configuração puramente remota para os desenvolvedores permite que você faça desenvolvimentos que dependam de vários serviços hospedados em uma infraestrutura com mais capacidade. No entanto, com isso, temos os desafios associados à necessidade de conectividade (o que pode ser um problema para funcionários que trabalham remotamente ou pessoas que viajam com frequência), possíveis ciclos mais lentos de feedback, com a necessidade de implantar

softwares remotamente antes de poder vê-los funcionando, e a possível explosão de recursos (e dos custos associados) necessários nos ambientes dos desenvolvedores.

O Telepresence (<https://www.telepresence.io>) é um exemplo de ferramenta cujo objetivo é criar um fluxo de trabalho híbrido local/remoto mais simples para os desenvolvedores que sejam usuários do Kubernetes. Você pode desenvolver seu serviço localmente, mas o Telepresence é capaz de fazer o proxy das chamadas para outros serviços em um cluster remoto, permitindo que você tenha o melhor dos dois mundos (pelo menos, é o que se espera). As funções de nuvem do Azure podem ser executadas localmente também, mas podem estar conectadas a recursos remotos de nuvem, permitindo a você criar serviços compostos de funções, com um fluxo de trabalho local rápido para os desenvolvedores, ao mesmo tempo que executam com o apoio de um ambiente de nuvem potencialmente mais amplo.

Perceber como a experiência dos desenvolvedores muda à medida que o número de serviços aumenta é importante – portanto, é necessário ter métodos de feedback definidos. Você terá de investir continuamente para garantir que os desenvolvedores mantenham o máximo possível de produtividade à medida que o número de serviços com os quais eles trabalham aumentam.

Administração de muitas tarefas

Quando houver mais serviços, e mais instâncias desses serviços, você terá mais processos que terão de ser implantados, configurados e administrados. Suas técnicas atuais para lidar com a implantação e a configuração de sua aplicação monolítica talvez não escalem bem quando o número das partes que precisarão ser administradas aumentar.

Um gerenciamento do estado desejado, em particular, torna-se cada vez mais importante. *Gerenciamento do estado desejado* é a capacidade de especificar o número e o local em que estarão as

instâncias dos serviços de que você precisa, e garantir que essa configuração seja mantida com o tempo. Talvez você gerencie isso atualmente com processos manuais em seu sistema monolítico – no entanto, essa solução não escalará bem se você tiver dezenas ou centenas de microsserviços, particularmente se cada um deles tiver um estado desejado diferente.

Como esse problema poderia se manifestar?

Você começará a ver uma parcela de tempo cada vez maior sendo gasta no gerenciamento de implantações e na resolução de problemas que ocorrerem durante essas implantações. Erros sempre serão cometidos se os processos dependerem de atividades manuais – e o impacto de erros inocentes em sistemas distribuídos pode ser difícil de prever.

À medida que adicionar mais serviços e instâncias de serviços, você se verá precisando de mais pessoas para gerenciar as atividades associadas à implantação e à manutenção de seus sistemas em produção. Isso poderia resultar na requisição de mais pessoas para dar suporte à sua equipe de operações ou, talvez, ver uma porcentagem maior do tempo de sua equipe de entrega ser gasta com questões relacionadas à implantação.

Quando esse problema poderia ocorrer?

Isso tem tudo a ver com a escala. Quanto mais microsserviços você tiver, e quanto mais instâncias você tiver desses microsserviços, menos apropriados serão os processos manuais ou as ferramentas mais tradicionais de gerenciamento automatizado de configuração como o Chef ou o Puppet.

Possíveis soluções

Você vai querer ter uma ferramenta que permita ter um alto grau de automação; o ideal é que seja possível que os desenvolvedores façam o provisionamento das implantações por contra própria e que

a ferramenta cuide do gerenciamento automático do estado desejado.

Para os microsserviços, o Kubernetes surgiu como a ferramenta preferida nessa área. Ela exige que você coloque seus serviços em contêineres, mas, assim que fizer isso, você poderá usar o Kubernetes para gerenciar a implantação das instâncias de seus serviços em várias máquinas, garantindo que seja possível escalar a fim de aumentar a robustez e lidar com a carga (supondo que você tenha hardware suficiente).

O Kubernetes básico não é o que eu consideraria como amigável aos desenvolvedores. Muitas pessoas estão trabalhando em abstrações de nível mais alto, mais acessíveis aos desenvolvedores, e espero que esse trabalho continue. No futuro, espero que muitos desenvolvedores que estejam executando softwares no Kubernetes nem sequer o percebam, pois isso passará a ser apenas um detalhe de implementação. Costumo ver muitas empresas de grande porte adotando uma versão do Kubernetes em pacote, como o OpenShift da RedHat, que reúne o Kubernetes com ferramentas que facilitam trabalhar em um ambiente corporativo – talvez lidando com identidades corporativas e controles para gerenciamento de acesso. Algumas dessas versões em pacote também oferecem abstrações simplificadas com as quais os desenvolvedores poderão trabalhar.

Se você for suficientemente afortunado para estar na nuvem pública, será possível usar as várias opções que há para lidar com as implantações de sua arquitetura de microsserviços, incluindo ofertas de Kubernetes gerenciado. Tanto a AWS como o Azure, por exemplo, oferecem diversas opções nessa área. Sou grande fã do FaaS (Function-as-a-Service, ou Função como Serviço) – um subconjunto do que chamamos de *serverless* (sem servidor). Com uma plataforma apropriada, os desenvolvedores se preocuparão somente com o código, e a plataforma subjacente lidará com a maior parte das tarefas operacionais. Ainda que as ofertas atuais de

FaaS tenham suas limitações, a expectativa é que elas reduzam drasticamente o overhead operacional.

Para as equipes com as quais trabalho e que já estejam na nuvem pública, minha tendência não é começar com o Kubernetes ou com plataformas semelhantes para contêineres. Em vez disso, eu adoto uma abordagem serverless antes – tento fazer uso de tecnologias de serverless, como o FaaS, como uma opção padrão, por causa da redução do trabalho operacional. Se seu problema não puder ser resolvido por causa das limitações dos produtos serverless disponíveis, procure outras opções. Obviamente, nem todos os domínios de problemas são iguais, mas acho que, se você já está na nuvem pública, a complexidade de uma plataforma baseada em contêineres, como o Kubernetes, nem sempre será necessária.



Vejo as pessoas recorrerem ao Kubernetes e a plataformas como essa um pouco cedo demais no processo de adoção dos microsserviços, muitas vezes supondo que seja um pré-requisito. Isso está longe de ser verdade – plataformas como o Kubernetes são excelentes para ajudar a gerenciar vários processos, mas você deveria esperar até ter processos suficientes, a ponto de sua abordagem e sua tecnologia atuais começarem a dar mostras de esgotamento. Talvez você descubra que precisará somente de cinco microsserviços, e que poderá lidar satisfatoriamente com eles usando as soluções atuais – se for esse o caso, ótimo! Não adote uma plataforma Kubernetes apenas porque você está vendo todo mundo fazer isso, e o mesmo pode ser dito também acerca dos microsserviços!

Testes fim a fim

Com qualquer tipo de testes funcionais automatizados, você terá de tomar uma decisão delicada no que concerne ao equilíbrio. Quanto mais funcionalidades um teste executar – quanto maior o escopo do teste –, mais confiança você terá em sua aplicação. Por outro lado, quanto maior o escopo do teste, mais tempo ele poderá demorar para executar, e mais difícil será descobrir o que apresenta problemas se houver uma falha.

Testes fim a fim para qualquer tipo de sistema estão na extremidade da escala no que concerne às funcionalidades cobertas, e estamos

acostumados com o fato de serem mais problemáticos para escrever e manter, em comparação com os testes de unidade de escopo mais reduzido. Em geral, porém, vale a pena tê-los, pois queremos ter a confiança que resulta de ter um teste fim a fim usando nossos sistemas do mesmo modo que um usuário o faria.

No entanto, com uma arquitetura de microsserviços, o “escopo” de nossos testes fim a fim se torna *muito* amplo. Temos agora de executar testes envolvendo vários serviços, e todos deverão estar implantados e configurados de forma apropriada para os cenários de teste. Também temos de estar preparados para os falso-negativos que ocorrerão quando problemas de ambiente, por exemplo, instâncias de serviços que morrem ou timeouts de rede em virtude de implantações com falha, fizerem nossos testes falharem. Eu diria que estaremos muito mais vulneráveis a problemas que estão fora de nosso controle quando executamos testes fim a fim em uma arquitetura de microsserviços do que estaríamos com uma arquitetura monolítica padrão.

À medida que o escopo dos testes aumentar, você gastará mais tempo lutando contra os problemas que surgirem, até o ponto em que tentar criar e manter testes fim a fim passará a consumir bastante tempo.

Como esse problema pode se manifestar?

Um sinal é ver que sua suíte de testes fim a fim aumentará, demorando cada vez mais para ser executada. Isso se deve ao fato de haver várias equipes, e elas não terem certeza dos cenários que estão sendo cobertos, adicionando novos testes “somente para garantir”. Você verá mais falhas na suíte de testes fim a fim que não mostrarão problemas em seu código – e os desenvolvedores muitas vezes apenas executarão os testes novamente para ver se passam.

A quantidade de tempo gasta nos testes fim a fim será cada vez maior, até o ponto em que começará a haver pressões para que haja mais engenheiros de testes e, talvez, uma equipe separada

para testes.

Quando esse problema poderia ocorrer?

Esse problema tende a surgir aos poucos, mas vejo que ele é percebido de forma mais acentuada em situações nas quais as diferentes experiências de usuários são tratadas por várias equipes. Quanto mais isolado for o trabalho de cada equipe, mais fácil será para elas gerenciar seus próprios testes localmente. Quanto mais testes de fluxos que envolvam várias equipes houver, mais problemáticos serão os testes fim a fim, com um escopo maior.

Possíveis soluções

Descrevi uma série de opções para ajudar você a mudar o modo de lidar com os testes no livro *Building Microservices*; com efeito, há um capítulo inteiro dedicado ao assunto, mas apresentarei a seguir um resumo rápido para que você tenha um ponto de partida.

Limite o escopo dos testes funcionais automatizados

Se você vai escrever casos de teste que incluam vários serviços, procure garantir que esses testes sejam mantidos dentro da equipe que administra esses serviços – em outras palavras, evite testes de escopos mais amplos, que cruzem as fronteiras entre as equipes. Manter a responsabilidade pelos testes dentro de uma única equipe facilita compreender quais cenários estão sendo devidamente cobertos, garante que os desenvolvedores possam executar e depurar os testes, e a responsabilidade acerca de quem deve garantir que os testes sejam executados e passem estará mais claramente definida.

Utilize contratos orientados aos consumidores

Você poderia considerar o uso de CDCs (Consumer-Driven Contracts, ou Contratos Orientados aos Consumidores) para substituir a necessidade de casos de teste que envolvam mais de um serviço. Com os CDCs, você faz com que o consumidor de seu

microsserviço defina as expectativas acerca de como o seu serviço deverá se comportar na forma de uma especificação executável – um teste. Ao modificar o seu serviço, você deverá garantir que esses testes continuarão sendo bem-sucedidos.

Como esses testes são definidos do ponto de vista do consumidor, teremos uma boa cobertura para identificar violações acidentais de contrato. Também será possível compreender os requisitos de nossos consumidores com base no ponto de vista deles e, acima de tudo, entender como diferentes consumidores podem querer algo diferente de nós.

Você pode implementar os CDCs usando um fluxo de trabalho de desenvolvimento simples, mas o processo poderá ser facilitado com o uso de ferramentas criadas para dar suporte a essa técnica. O melhor exemplo provavelmente é o Pact (<https://pact.io>).

Vale a pena mencionar que já vi algumas equipes terem bastante sucesso com essa abordagem, mas, para outras equipes, tem sido difícil adotá-la. A ideia é bem fundamentada, e sei que ela funciona bem, mas ainda não compreendi totalmente os desafios que algumas pessoas enfrentaram ao adotar essa técnica. Ela continua sendo uma prática subutilizada para solucionar um problema realmente difícil.

Utilize reparação automática de versão e entrega progressiva

Com testes automatizados, em geral tentamos encontrar os problemas antes que eles causem impacto no ambiente de produção, mas poderá ser cada vez mais difícil fazer isso à medida que seu sistema se tornar mais complexo. Desse modo, talvez valha a pena investir esforços para reduzir o impacto dos problemas de produção, caso ocorram.

Conforme mencionamos no Capítulo 3, *entrega progressiva* (progressive delivery) é um termo abrangente para controlar o modo de fazer o rollout gradual de novas versões de seu software para os clientes. A ideia é que você possa avaliar o impacto de sua nova

versão em um grupo menor de clientes, decidindo se e quando deve continuar ou reverter o rollout. Um exemplo de uma técnica de entrega progressiva poderia ser o lançamento de uma versão canário (canary release).

Ao definir critérios aceitáveis acerca de como o seu serviço deve se comportar, será possível controlar a entrega progressiva de modo automático. Como um exemplo simples, você poderia definir um limiar aceitável para latências e taxas de erro no 95º percentil, e prosseguir com o rollout somente se esses critérios forem atendidos. Caso contrário, você poderia fazer um rollback automaticamente para a versão anterior, ganhando tempo para analisar o que aconteceu.

Muitas empresas utilizam essas técnicas de reparação automática de versão (automated release remediation). A Netflix, em particular, falou bastante sobre o uso dessa ideia. Ela desenvolveu o Spinnaker como uma ferramenta de gerenciamento de implantações, em parte para ajudar a controlar a entrega progressiva de seus serviços, mas há várias maneiras diferentes de colocar essas ideias em prática.

Não estou dizendo que você deva considerar o uso da reparação automática de versão *no lugar* dos testes, mas que deve pensar na opção que trará o melhor retorno para os seus esforços. Você poderá acabar com um sistema muito mais robusto se investir esforços na identificação dos problemas caso eles realmente ocorram, em vez de simplesmente se concentrar em impedir que ocorram.

É importante observar que essas técnicas funcionem bem juntas; ainda que você ache que a reparação automática não seja uma opção no momento, pode haver uma série de vantagens em implementar alguma forma de entrega progressiva. Até mesmo controlar a entrega progressiva manualmente pode ser um grande passo em comparação a simplesmente fazer o rollout do novo software para todos.

Aprimore continuamente seus ciclos de feedback sobre qualidade

Compreender como e onde você deve testar é um desafio constante. Você precisará de pessoas que tenham conhecimento do contexto para que vejam holisticamente o processo de desenvolvimento e possam adaptar como e onde você testará a sua aplicação. Isso significa ter pessoas capazes de identificar se é necessário acrescentar novos testes para cobrir áreas do sistema nas quais elas vejam um aumento de problemas no ambiente de produção, mas que também possam remover testes quando já houver uma cobertura, em um esforço para melhorar os ciclos de feedback.

Em suma, trata-se de buscar um equilíbrio entre a necessidade de ter feedbacks rápidos e a segurança. Você deve estar igualmente disposto a identificar, e remover ou substituir, o teste errado, ou adicionar um novo teste.

Otimização global versus local

Supondo que você adote o modelo de equipes com mais responsabilidade pela tomada de decisões locais, talvez sendo responsáveis por todo o ciclo de vida dos microsserviços que gerenciam, você chegará ao ponto em que precisará começar a buscar um equilíbrio entre a tomada de decisões locais e as preocupações mais globais.

Como exemplo de como esse problema poderia se manifestar, considere três equipes que gerenciem os serviços de Fatura, Notificações e Atendimentos. A equipe de Fatura decide usar o Oracle como banco de dados, pois eles o conhecem bem. A equipe de Notificações quer usar o MongoDB porque ele é bastante apropriado ao seu modelo de programação. Enquanto isso, a equipe de Atendimentos quer usar o PostgreSQL, pois eles já o têm. Ao observar cada uma das decisões isoladamente, elas fazem todo sentido, e você é capaz de entender como cada equipe fez a sua

escolha.

Se der um passo para trás e observar o quadro geral, porém, você terá de se perguntar se, como empresa, você quer desenvolver habilidades relacionadas a três bancos de dados com funcionalidades, até certo ponto, semelhantes – e pagar por suas licenças. Não seria melhor adotar somente um banco de dados, aceitando que ele não será perfeito para todos, mas será bom o suficiente para a maioria? Sem a capacidade de ver o que está acontecendo localmente e colocar esses fatos em um contexto global, como você poderia tomar esses tipos de decisões?

Como esse problema pode se manifestar?

O modo mais comum pelo qual já vi esse problema se manifestar é quando alguém de repente percebe que várias equipes resolveram o mesmo problema de maneiras distintas, mas jamais se deram conta de que estavam todas tentando resolver o mesmo problema. Com o tempo, isso pode ser extremamente ineficiente.

Lembro-me de ter conversado com o pessoal da REA, uma empresa do ramo imobiliário na Austrália. Depois de muitos anos desenvolvendo microsserviços, eles chegaram a um ponto no qual perceberam que havia muitas formas de as equipes implantarem os serviços. Isso causava problemas quando as pessoas passavam de uma equipe para outra, pois elas tinham de aprender a nova forma de trabalhar. Também se tornou difícil justificar o trabalho duplicado que cada equipe estava fazendo. Como resultado, eles decidiram investir um pouco de trabalho com vistas a descobrir uma maneira comum de lidar com a questão.

Em geral, você vai perceber essas situações por acaso, talvez depois de ter ouvido um comentário de passagem durante o almoço. Esses problemas poderão ser identificados muito mais cedo se houver algum tipo de grupo técnico envolvendo várias equipes, por exemplo, uma comunidade de práticas.

Quando esse problema poderia ocorrer?

Esse problema tende a surgir com o tempo em empresas com várias equipes, sobretudo em empresas que dão mais liberdade às equipes no que diz respeito à forma de conduzirem seus trabalhos. Não espere ver esse problema tão cedo em sua jornada para os microsserviços. Você provavelmente começará com uma compreensão clara e compartilhada de como o trabalho deve ser feito. Com o tempo, cada equipe estará cada vez mais concentrada em seus problemas locais, e elas otimizarão o modo de resolver seus próprios problemas, de modo que a visão básica compartilhada do “é assim que trabalhamos” começará a divergir.

Muitas vezes, vejo esse problema ser levantado e discutido depois que as empresas passam por um período de expansão. A entrada de muitos desenvolvedores em um curto período dificulta escalar o compartilhamento de informações *ad hoc*. Isso pode resultar em mais silos de informações que precisarão ser conectados.

Se você adota o modelo de responsabilidade coletiva pelos serviços, provavelmente isso ajudará a evitar esses problemas ou, no mínimo, os limitará, pois a responsabilidade coletiva pelos serviços exige um grau de consistência no que diz respeito ao modo como os problemas são resolvidos. Em outras palavras, se você quer ter um modelo de responsabilidade coletiva, será *necessário* resolver esse problema; caso contrário, a responsabilidade coletiva não escalará.

Possíveis soluções

Já mencionamos algumas ideias que poderão ajudar nessa área. No Capítulo 2, exploramos a ideia das decisões irreversíveis e reversíveis, conforme vemos novamente na Figura 5.8. Quanto mais alto o custo da mudança, maior será o impacto, e mais consenso você vai querer ter diante de uma tomada de decisão. Quanto menor o impacto, mais fácil será fazer um rollback, e mais decisões poderão ser deixadas a cargo das equipes locais.

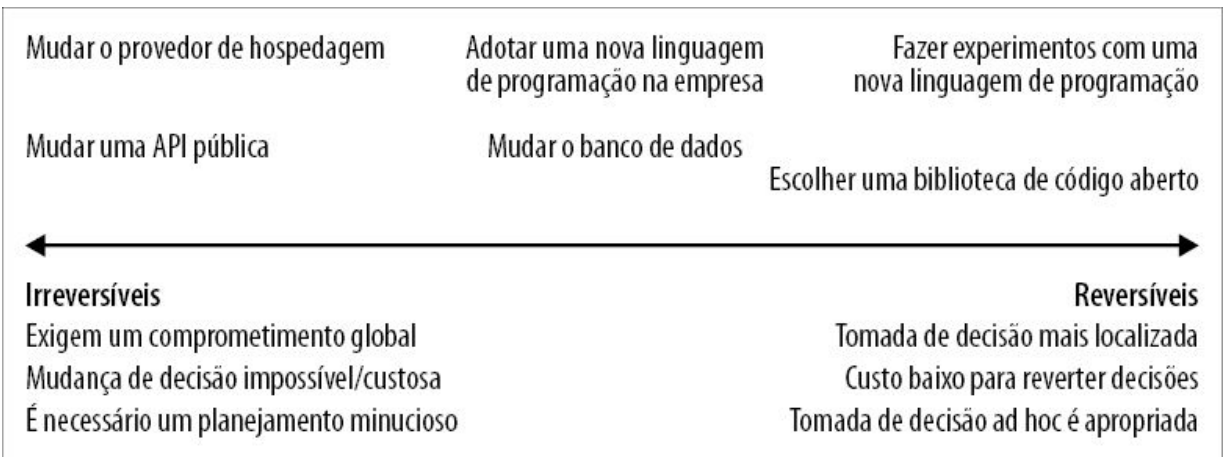


Figura 5.8 – As diferenças entre decisões Irreversíveis e Reversíveis, com exemplos ao longo do espectro.

O truque é ajudar os membros das equipes a perceberem a direção para a qual suas decisões poderão tender em relação às extremidades irreversíveis ou reversíveis nesse espectro. Quanto mais uma decisão tender em direção à extremidade irreversível, mais importante ela será, a ponto de as equipes terem de envolver outras pessoas externas a ela em sua tomada de decisão. Para que isso funcione bem, as equipes devem ter, no mínimo, uma compreensão básica acerca das preocupações relacionadas ao quadro geral, a fim de ver em que ponto elas podem se sobrepor; as equipes também precisarão de um rede na qual possam divulgar esses problemas e ter o envolvimento dos colegas de outras equipes.

Como um método simples, ter pelo menos um líder técnico de cada equipe fazendo parte de um grupo técnico que envolva vários grupos, no qual essas preocupações possam ser discutidas, é uma abordagem razoável. O grupo poderá ser liderado por um CTO, um arquiteto-líder ou outra pessoa que seja responsável pela visão geral técnica da empresa.

Esse grupo multidisciplinar pode funcionar nos dois sentidos. Além de constituir um espaço no qual as equipes poderão apresentar os problemas locais que elas queiram que sejam discutidos em um fórum mais amplo, é também um local para as pessoas poderem

tomar conhecimento dos problemas que dizem respeito a vários grupos. Sem algum tipo de comunicação entre as equipes, como perceberíamos que estávamos resolvendo problemas de modos diferentes localmente, e que, talvez, solucioná-los em âmbito global poderia fazer mais sentido?

Dependendo da natureza de sua empresa, você poderá contar com um processo mais *ad hoc* e informal. Na Monzo, por exemplo, as pessoas podem submeter documentos de formato livre, chamados internamente de “propostas”. Elas são publicadas em um espaço compartilhado, o qual, por sua vez, avisa toda a empresa, via Slack, que uma nova proposta está disponível. As partes interessadas podem então discutir a proposta e ajudar a aprimorá-la. A expectativa é que essas propostas não sejam o artefato final e, com efeito, elas devem estar abertas a mudanças. Esse sistema parece funcionar bem na Monzo, em parte por causa de sua cultura no que concerne à comunicação e ao compartilhamento de responsabilidades.

Basicamente, cada empresa precisa encontrar o equilíbrio correto entre as tomadas de decisão globais e locais. Qual é o nível de responsabilidade que você estará satisfeito em deixar a cargo das equipes? Qual é o nível de controle que você deseja manter de forma centralizada? Quanto mais responsabilidades você deixar a cargo das equipes, mais vantagens você terá com uma maior autonomia; a contrapartida, porém, é que você terá menos consistência quanto ao modo como os problemas são resolvidos. Quanto mais centralizadas as decisões, mais consenso será necessário criar, e esse processo provavelmente será mais demorado. Não posso lhe dizer como você deve alcançar o equilíbrio entre essas duas forças da forma correta; você terá de descobrir isso sozinho. Precisa estar ciente de que esse equilíbrio existe, e deve garantir que coletará as informações corretas a fim de ter certeza de que será capaz de fazer ajustes nesse balanceamento com o tempo.

Robustez e resiliência

Os sistemas distribuídos podem exibir uma série de modos de falha com os quais talvez você não tenha familiaridade caso esteja mais acostumado com sistemas monolíticos. Pacotes de rede podem se perder, chamadas de rede podem sofrer timeout, máquinas podem morrer ou parar de responder. Essas situações podem ser raras em sistemas distribuídos simples, como em uma aplicação monolítica tradicional; porém, à medida que o número de serviços aumentar, essas ocorrências raras se tornarão mais comuns.

Como esse problema pode se manifestar?

Esses problemas, infelizmente, são mais prováveis de surgir em um ambiente de produção. Durante os ciclos tradicionais de desenvolvimento e de testes, recriamos circunstâncias semelhantes às daquelas do ambiente de produção somente por curtos períodos. Há menos chances de essas ocorrências raras surgirem, e, quando o fazem, em geral, elas são menosprezadas.

Quando esse problema poderia ocorrer?

Serei honesto neste caso – se eu pudesse dizer com antecedência quando o seu sistema sofrerá com instabilidades, não estaria escrevendo este livro, pois estaria provavelmente aproveitando meu tempo em uma praia, bebendo mojitos. Tudo que posso dizer é que, à medida que o número de serviços aumentar, e o número de chamadas de serviço aumentar, você estará cada vez mais vulnerável a problemas de resiliência. Quanto mais interconectados estiverem os seus serviços, maior será a probabilidade de você sofrer com problemas como falhas em cascata e backpressure³.

Possíveis soluções

Um bom ponto de partida é fazer a si mesmo duas perguntas sobre cada chamada de serviço que você fizer. A primeira é: eu sei as maneiras pelas quais essa chamada poderia falhar? E a segunda é:

se a chamada falhar, eu sei o que deve ser feito?

Assim que tiver respondido a essas perguntas, você poderá então começar a considerar uma série de soluções. Isolar mais os serviços pode ajudar, talvez introduzindo uma comunicação assíncrona a fim de evitar um acoplamento temporal (um assunto que mencionamos no Capítulo 1). Usar timeouts sensatos também pode evitar a contenção de recursos com serviços downstream lentos e, junto com o uso de padrões como os interruptores de circuito (circuit breakers), você poderá começar a falhar mais rápido a fim de evitar problemas de backpressure.

Executar várias cópias de serviços pode ajudar no caso de instâncias que falham, assim como o uso de uma plataforma que implemente o gerenciamento de estado desejado (ela pode garantir que os serviços sejam reiniciados caso falhem).

Para reforçar um ponto que discutimos no Capítulo 2, a resiliência vai além de simplesmente implementar alguns padrões. Trata-se de toda uma forma de trabalhar – construindo uma empresa que não só esteja pronta para lidar com problemas imprevistos, os quais, inevitavelmente, surgirão, mas que também aprimore as práticas de trabalho conforme forem necessárias. Uma forma concreta de colocar essa ideia em prática é documentar os problemas de produção quando surgirem e manter um registro sobre o que você aprendeu. Com muita frequência, vejo empresas seguindo em frente muito rapidamente, assim que o problema inicial é resolvido ou contornado – apenas para que esses mesmos problemas reapareçam novamente alguns meses depois.

Para ser honesto, abordei o assunto apenas de forma superficial. Para uma análise mais detalhada dessas ideias, recomendo ler o Capítulo 11 do livro *Building Microservices* ou consultar o livro *Release It!* de Michael Nygard (Pragmatic Bookshelf, 2018).

Serviços órfãos

Parece estranho, considerando algumas das incríveis tecnologias

que temos e os sistemas extremamente complexos e massivamente escaláveis que construímos atualmente, mas ainda vemos problemas com algumas das questões mais prosaicas. Um exemplo é que vejo muitas empresas com dificuldades para saber exatamente o que elas têm, onde estão os componentes e quem é o responsável por eles.

À medida que os microsserviços mantêm um foco cada vez maior nos respectivos propósitos, você verá mais e mais serviços executando satisfatoriamente durante semanas, meses ou, quem sabe, até anos, sem que qualquer alteração lhes seja feita. Por um lado, é exatamente isso que queremos; a possibilidade de fazer implantações independentes é um conceito muito atraente, em parte porque permite que o resto do sistema permaneça estável, e manter a estabilidade das partes de nosso sistema que não precisam ser modificadas é uma boa ideia.

Refiro-me a esses serviços como *serviços órfãos*, pois, basicamente, ninguém na empresa é o dono ou o responsável por eles.

Como esse problema pode se manifestar?

Lembro-me de ter ouvido histórias (talvez apócrifas) de velhos servidores descobertos entre quatro paredes em antigos escritórios. Ninguém se lembrava de que estavam lá, mas continuavam executando alegremente, fazendo o que quer que fizessem. Como ninguém se lembrava exatamente do que esses computadores recém-descobertos faziam, as pessoas tinham medo de desligá-los. Os microsserviços podem exibir algumas dessas mesmas características; eles estão por aí e estão funcionando (estamos supondo que estão), mas temos o mesmo problema, isto é, não sabemos o que fazer com eles, e esse medo pode nos dissuadir de modificá-los.

O problema básico é que, se esse serviço parar de funcionar ou se exigir alguma mudança, as pessoas não saberão o que fazer. Já

conversei com mais de uma equipe que contava histórias sobre não saber onde estava o código-fonte do serviço em questão, o que já é um grande problema.

Quando esse problema poderia ocorrer?

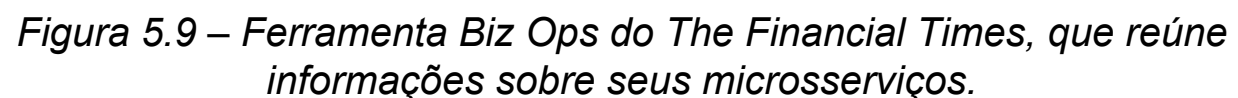
Esse problema em geral ocorre em empresas que vêm usando microsserviços há um bom tempo – tempo suficiente para que já não haja mais uma memória coletiva acerca do que esse serviço fazia. As pessoas envolvidas com esse microsserviço já se esqueceram do que fizeram com ele, ou, talvez, já tenham saído da empresa.

Possíveis soluções

Tenho uma hipótese (não testada) de que as empresas que adotam o modelo de responsabilidade coletiva pelos serviços podem estar menos suscetíveis a esse problema, basicamente porque elas já deverão ter métodos implementados para permitir que os desenvolvedores passem de um serviço para outro e façam alterações. Esses tipos de empresa provavelmente já restringem as opções de linguagem e de tecnologia a fim de reduzir o custo da alternância de contextos entre serviços. Elas também podem ter ferramentas comuns para fazer mudanças em um serviço, testá-lo e implantá-lo. Se essas práticas comuns mudaram desde a última modificação feita no serviço, é claro que isso talvez não ajude.

Já falei com algumas empresas que tiveram esses problemas, e que acabaram criando registros internos simples para ajudar a reunir metadados sobre os serviços. Alguns desses registros simplesmente varriam os repositórios de códigos-fontes, procurando arquivos de metadados para construir uma lista de serviços existentes. Essas informações poderão ser combinadas com dados reais provenientes de sistemas de descoberta de serviços, como consul ou etcd, a fim de compor um quadro mais completo acerca do que está executando, e com quem você poderia conversar a

The Financial Times criou o Biz Ops para ajudar a resolver esse problema. A empresa tem várias centenas de serviços desenvolvidos por equipes no mundo todo. A ferramenta Biz Ops (Figura 5.9) oferece um local único no qual é possível encontrar muitas informações úteis sobre seus microsserviços, além de informações sobre outros serviços de infraestrutura de TI, como redes e servidores de arquivos. Construído com base em um banco de dados de grafos, há muita flexibilidade quanto aos dados coletados e como as informações podem ser modeladas.



A ferramenta Biz Ops, contudo, vai além da maioria das ferramentas que eu já vi. Ela calcula o que eles chamam de System Operability Score (Pontuação sobre a Operacionalidade do Sistema), como mostra a Figura 5.10. A ideia é que há certas tarefas que os serviços e suas equipes devem fazer para garantir que o serviço seja facilmente operado. Essas tarefas podem variar, desde certificar-se de que as equipes tenham fornecido as informações corretas no registro, até garantir que os serviços tenham verificações de sanidade apropriadas. Essas pontuações são calculadas, permitindo que as equipes vejam rapidamente se há pontos que necessitam ser corrigidos.

Ter algo como um registro de serviços pode ajudar, mas o que acontecerá se o seu serviço órfão for anterior ao registro? O ponto principal é alinhar esses serviços “órfãos” recém-descobertos ao modo como os demais serviços são gerenciados, e isso exige que você atribua a responsabilidade a uma equipe existente (se você adota a prática de responsabilidade forte) ou defina tarefas para melhorias no serviço (caso adote a prática de responsabilidade coletiva).

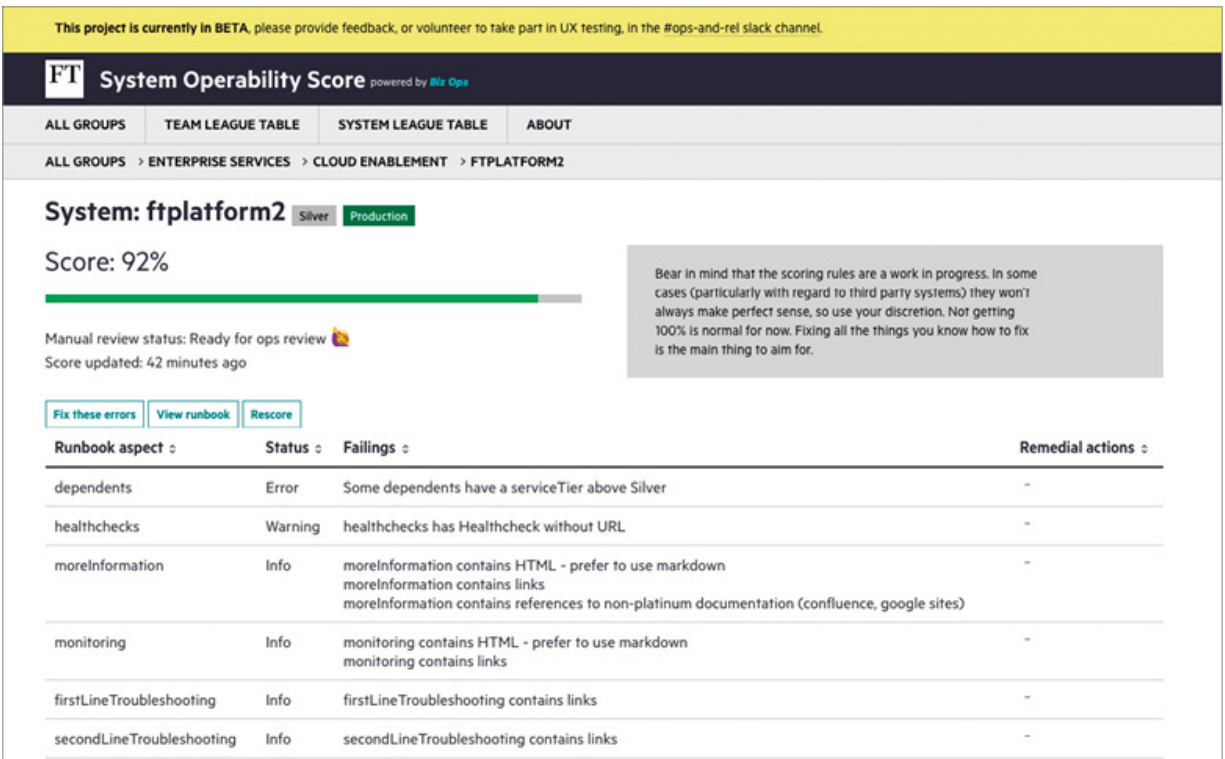


Figura 5.10 – Um exemplo da Service Operability Score (Pontuação para a Operacionalidade de um Serviço) para um microsserviço do The Financial Times.

Resumo

Os assuntos que abordamos neste capítulo não tiveram a intenção de compor uma lista exaustiva de todos os problemas que os microsserviços podem causar, e tampouco listei todas as soluções possíveis. Em vez disso, procurei manter o foco nos problemas mais comuns, com os quais vejo as pessoas lutando.

Espero que eu tenha deixado claro que não há uma regra rígida sobre quando, exatamente, você verá esses problemas. Cada situação é diferente, e diversos fatores entram em cena. O que tentei fazer foi enfatizar que, embora não seja possível prever o futuro, você pode, ao menos, ter sido advertido. A tarefa mais desafiadora é alcançar o equilíbrio correto entre corrigir os problemas antes que ocorram *versus* gastar tempo corrigindo problemas que você jamais terá. Espero que você encerre este

capítulo ciente das advertências às quais deverá prestar atenção.

- 1 N.T.: Uma fintech – termo resultante da união das palavras em inglês *financeira* (financeiro) e *technology* (tecnologia) – refere-se a uma startup que atua principalmente na inovação e otimização de serviços do sistema financeiro (Referência: <https://pt.wikipedia.org/wiki/Fintech>).
- 2 Veja Cindy Sridharan, *Distributed Systems Observability* (Sebastopol: O'Reilly Media, Inc., 2018).
- 3 N.T.: De modo geral, o backpressure é uma força de resistência ao fluxo desejado para o software.

CAPÍTULO 6

Palavras finais

E, assim, chegamos ao final do livro. Ao longo de toda a obra, espero ter conseguido transmitir duas mensagens essenciais. Em primeiro lugar, dê espaço suficiente a si mesmo e colete as informações corretas a fim de tomar decisões racionais. Não simplesmente copie os outros; em vez disso, pense no seu problema e no seu contexto, avalie as opções e siga em frente, ao mesmo tempo em que permanece aberto a mudanças caso sejam necessárias no futuro. Em segundo lugar, lembre-se de que uma adoção gradual dos microsserviços, e de muitas das tecnologias e práticas associadas, é fundamental. Não há duas arquiteturas de microsserviços iguais – embora haja lições a serem aprendidas com o trabalho feito por outras pessoas, você deve investir tempo para encontrar uma abordagem que seja mais apropriada ao seu contexto. Ao dividir a jornada em passos administráveis, você dará a si mesmo a melhor chance possível de ter sucesso, pois poderá fazer adaptações à sua abordagem enquanto prossegue.

Definitivamente, os microsserviços não servem para todos. Espero, porém, que, depois de ter lido este livro, você não só tenha uma melhor noção para saber se eles são a opção correta para você, mas também tenha algumas ideias sobre como iniciar a jornada.

APÊNDICE A

Bibliografia

Bird, Christian, Nachiappan Nagappan, Brendan Murphy, Harald Gall e Premkumar Devanbu. “Don’t Touch My Code! Examining the Effects of Ownership on Software Quality” (Não toque no meu código! Analisando os efeitos da responsabilidade na qualidade do software). <http://bit.ly/2p5RIT1>.

Bland, Mike. “Test Mercenaries” (Mercenários de testes). <http://bit.ly/2omkxVy>.

Bland, Mike. “Testing On The Toilet” (Testando no banheiro). <http://bit.ly/2ojpWwm>.

Brandolini, Alberto. *Introducing EventStorming*. Leanpub, 2019. <http://bit.ly/2n0zCLU>.

Brooks, Frederick P. *O Mítico Homem-Mês*. Alta Books, 2018.

Bryant, Daniel. “Building Resilience in Netflix Production Data Migrations: Sangeeta Handa at QCon SF” (Incluindo resiliência nas migrações de dados no ambiente de produção da Netflix: Sangeeta Handa na QCon SF). <http://bit.ly/2m1EwHT>.

Devops Research & Assessment. *Accelerate: State Of Devops Report 2018* (Relatório do estado do DevOps em 2018). <http://bit.ly/2nPDNLe>.

Evans, Eric. *Domain-Driven Design: atacando as complexidades no coração do software*. Alta Books, 2016.

Feathers, Michael. *Trabalho Eficaz com Código Legado*. Bookman, 2013.

Fowler, Martin. “Strangler Fig Application” (Aplicação strangler fig).

<http://bit.ly/2p5xMKo>.

Fowler, Martin. “Reporting Database” (Banco de dados para relatórios). <http://bit.ly/2kWW9lr>.

Garcia-Molina, Hector e Kenneth Salem. “Sagas”. *ACM Sigmod Record* 16, n. 3 (1987): 249–259.

Garcia-Molina, Hector, Dieter Gawlick, Johannes Klein, Karl Kleissner, Kenneth Salem. “Modeling Long-Running Activities as Nested Sagas” (Modelando atividades de longa duração como sagas aninhadas). *Data Engineering* 14, n. 1 (março de 1991): 14–18.

Helland, Pat. “Life Beyond Distributed Transactions” (Vida além das transações distribuídas). *Acmqueue* 14, n. 5.

Hodgson, Peter. “Feature Toggles (aka Feature Flags)” (Toggles de recursos [também conhecidos como flags de recursos]). <http://bit.ly/2m316zB>.

Hohpe, Gregor e Bobby Woolf. *Enterprise Integration Patterns*. Addison-Wesley, 2003.

Humble, Jez e David Farley. *Entrega Contínua: como entregar software de forma rápida e confiável*. Bookman, 2014.

Humble, Jez. “Make Large-Scale Changes Incrementally with Branch by Abstraction” (Fazendo mudanças em larga escala gradualmente, usando branch por abstração). <http://bit.ly/2p95lv7>.

Kim, Gene, Patrick Debois, Jez Humble e John Willis. *Manual de Devops*. Alta Books, 2018.

Kleppmann, Martin. *Designing Data-Intensive Applications*. O’Reilly, 2017.

Kniberg, Henrik e Anders Ivarsson. “Scaling Agile @ Spotify” (Escalando o Agile no Spotify). Outubro de 2012. <http://bit.ly/2ogAz3d>.

Kotter, John P. *Liderando mudanças*. Elsevier, 2013.

Mitchell, Lorna Jane. *Web Services em PHP*. Novatec, 2013.

Newman, Sam. *Building Microservices*. O'Reilly, 2015.

Nygard, Michael T. *Release It!: Design and Deploy Production-Ready Software*, 2ª edição. Pragmatic Bookshelf, 2018.

Parnas, David. "On the Criteria to be Used in Decomposing Systems into Modules" (Sobre os critérios a serem usados na decomposição de sistemas em módulos). *Information Distributions Aspects of Design Methodology* (Aspectos da distribuição de informações na metodologia de design), Anais do Congresso IFIP de 1971 (1972).

Parnas, David. "The Secret History of Information Hiding" (A história secreta da ocultação de informações). David Parnas. In *Software Pioneers*, editado por M. Broy e E. Denert. Berlin: Springer, 2002.

Pettersen, Snow. "The Road to an Envoy Service Mesh" (O caminho para um service mesh Envoy). <https://squ.re/2nts1Gc>.

Skelton, Matthew e Manuel Pais. *Team Topologies*. IT Revolution Press, 2019.

Smith, Steve. "Application Pattern: Verify Branch By Abstraction" (Padrão de aplicação: branch por abstração com verificação). <http://bit.ly/2mLVevz>.

Sridharan, Cindy. *Distributed Systems Observability*. O'Reilly, 2018. <http://bit.ly/2nPZ73d>.

Thorup, Kresten. "Riak on Drugs (and the Other Way Around)" (Riak sob efeito de medicamentos [e o inverso]). <http://bit.ly/2m1CvLP>.

Vernon, Vaughn. *Domain-Driven Design Distilled*. Addison-Wesley, 2016.

Woods, David, "Four concepts for resilience and the implications for the future of resilience engineering" (Quatro conceitos sobre resiliência e as implicações para o futuro da engenharia de resiliência). *Reliability Engineering & System Safety* 141 (2015): 5–9.

Yourdon, Edward e Larry Constantine. *Projeto Estruturado de Sistemas*. Ed. Campus, 1990.

APÊNDICE B

Índice de padrões

Aplicação strangler fig	Encapsulamento de sua nova arquitetura de microsserviços em torno do sistema monolítico existente. Chamadas que usam a funcionalidade que foi transferida do sistema monolítico para seus microsserviços são desviadas; outras chamadas permanecem inalteradas.
Armazenagem com vários esquemas	Gerencia dados em bancos de dados distintos, geralmente enquanto se faz a migração de um banco de dados compartilhado para um modelo de banco de dados por serviço.
Banco de dados compartilhado	Um único banco de dados é compartilhado entre mais de um serviço.
Biblioteca para dados de referência estáticos	Passa dados de referência estáticos para uma biblioteca ou um arquivo de configuração que possa ser empacotado com cada microsserviço que precisar deles.
Branch por abstração	Coexistência de duas implementações da mesma funcionalidade na mesma base de código simultaneamente, permitindo que uma nova implementação seja gradualmente desenvolvida, até que possa substituir a implementação antiga.
Captura de dados modificados	Transmite mudanças feitas em um banco de dados subjacente para outras partes interessadas.
Colaborador decorador	Dispara funcionalidades em execução em um microsserviço separado, detectando as requisições enviadas ao sistema monolítico e as respostas devolvidas.
Composição de UI	Apresentação de uma única interface de usuário, reunindo várias partes menores.
Duplicação de dados de referência estáticos	Copia dados de referência estáticos para os bancos de dados dos microsserviços.

Esquema dedicado para dados de referência	Um banco de dados dedicado para hospedar todos os dados de referência estáticos. Esse banco de dados pode ser acessado por vários serviços diferentes.
Execução em paralelo	Executa duas implementações da mesma funcionalidade, lado a lado, com o intuito de garantir que a nova funcionalidade se comportará de modo apropriado.
Interface de banco de dados como serviço	Uso de um banco de dados dedicado para fornecer acesso somente de leitura a dados internos ao serviço.
Mudança do responsável pelos dados	Move a fonte da verdade, do sistema monolítico para um microsserviço.
Repositório por contexto delimitado	Separa uma camada única de repositório de acordo com as diferentes partes do domínio, facilitando a decomposição em serviços.
Separação de tabelas	Separação de uma tabela em duas partes antes de fazer a decomposição dos serviços.
Serviço de encapsulamento de banco de dados	Um serviço de fachada é colocado na frente de um banco de dados compartilhado, permitindo que os serviços façam uma migração e deixem de usar diretamente o banco de dados.
Serviço para dados de referência estáticos	Um microsserviço dedicado que oferece acesso a dados de referência estáticos.
Sincronização de dados na aplicação	Sincroniza dados entre duas fontes da verdade em uma única aplicação.
Sistema monolítico como uma camada de acesso aos dados	Acesso a dados gerenciados pelo sistema monolítico por meio de APIs, em vez de acessar diretamente o banco de dados.
Sistema monolítico expondo agregados	Expõe agregados de domínio a partir de um sistema monolítico para permitir que os microsserviços acessem entidades gerenciadas pelo sistema monolítico.
Tracer write	Faz a migração gradual de uma fonte da verdade para outra, tolerando duas fontes da verdade durante a migração.

Transferência de chave estrangeira para o código	Passa o gerenciamento e a garantia oferecida pelos relacionamentos de chave estrangeira de um único banco de dados para a camada de serviço.
Visão de banco de dados	Uma visão é projetada a partir de um banco de dados subjacente, permitindo que partes do banco de dados sejam ocultas.

Sobre o autor

Sam Newman é desenvolvedor, arquiteto, escritor e palestrante, e já trabalhou com várias empresas de diferentes áreas pelo mundo. Atua como profissional independente, com foco principalmente em nuvem, entrega contínua e microsserviços. Antes de publicar este livro, escreveu o best-seller *Building Microservices*, também publicado pela O'Reilly.

Quando não está saltando de uma empreitada para outra, pode ser visto na área rural de East Kent, deixando-se irritar com vários tipos de esporte.

Colofão

O animal na capa de Migrando sistemas monolíticos para microsserviços é a água-viva urticante *Drymonema dalmatinum*. Essa água-viva subtropical vive no Oceano Atlântico Central e no Mar Mediterrâneo e foi inicialmente identificada em 1880 ao largo da costa da Croácia (na época, chamava-se Dalmácia). Raramente foi vista depois da Segunda Guerra Mundial, mas um espécime gigante foi fotografado na costa da Itália em 2014.

Essa água-viva também recebe o apelido de “a grande rosa” por causa de sua coloração castanho-rosada e pelo tamanho impressionante – atinge até cerca de quase um metro de diâmetro. O nome da classe Scyphozoa tem origem em uma palavra grega que significa “taça”, fazendo alusão ao formato do corpo do animal. O subfilo se chama Medusozoa por causa dos longos tentáculos da água-viva, que lembram as cobras que crescem na cabeça do monstro mitológico.

Assim como as demais águas-vivas, a *Drymonema dalmatinum* faz uso de reprodução tanto sexuada como assexuada. Na reprodução sexuada, os machos lançam o esperma e as fêmeas, os ovos, que então se juntam na água. Os ovos fertilizados se transformam em pólipos, que se reproduzem de forma assexuada formando brotos antes de amadurecerem. Acredita-se que a *Drymonema dalmatinum* se alimenta de outras águas-vivas, em geral da água-viva-da-lua.

As águas-vivas são encontradas somente em água salgada, e nunca em água doce.

Muitos dos animais das capas dos livros da O'Reilly estão ameaçados; todos são importantes para o mundo.

A ilustração da capa é de Karen Montgomery, com base em uma imagem em preto e branco do livro *Medusae of the World*.

O'REILLY



Microserviços prontos para a produção

CONSTRUINDO SISTEMAS PADRONIZADOS EM UMA
ORGANIZAÇÃO DE ENGENHARIA DE SOFTWARE



novatec

Susan J. Fowler

Microserviços prontos para a produção

Fowler, Susan J.

9788575227473

224 páginas

[Compre agora e leia](#)

Um dos maiores desafios para as empresas que adotaram a arquitetura de microserviços é a falta de padronização de arquitetura – operacional e organizacional. Depois de dividir uma aplicação monolítica ou construir um ecossistema de microserviços a partir do zero, muitos engenheiros se perguntam o que vem a seguir. Neste livro prático, a autora Susan Fowler apresenta com profundidade um conjunto de padrões de microserviço, aproveitando sua experiência de padronização de mais de mil microserviços do Uber. Você aprenderá a projetar microserviços que são estáveis, confiáveis, escaláveis, tolerantes a falhas, de alto desempenho, monitorados, documentados e preparados para qualquer catástrofe. Explore os padrões de disponibilidade de produção, incluindo: Estabilidade e confiabilidade – desenvolva, implante, introduza e descontinue microserviços; proteja-se contra falhas de dependência. Escalabilidade e desempenho – conheça os componentes essenciais para alcançar mais eficiência do microserviço. Tolerância a falhas e prontidão para catástrofes – garanta a disponibilidade forçando ativamente os microserviços a

falhar em tempo real. Monitoramento – aprenda como monitorar, gravar logs e exibir as principais métricas; estabeleça procedimentos de alerta e de prontidão. Documentação e compreensão – atenuar os efeitos negativos das contrapartidas que acompanham a adoção dos microsserviços, incluindo a dispersão organizacional e a defasagem técnica.

[Compre agora e leia](#)

IMPOSTO DE RENDA NO MERCADO DE AÇÕES

A TRIBUTAÇÃO SOBRE OS GANHOS DE
PESSOAS FÍSICAS NA BOLSA DE VALORES

3ª Edição
Revisada e atualizada



MODALIDADES DE NEGOCIAÇÃO:

À VISTA, A TERMO, FUTURO,
OPÇÕES E DAY TRADE

novatec

Murillo Lo Visco

Imposto de Renda no Mercado de Ações 3ª Edição

Lo Visco, Murillo

9786586057010

312 páginas

[Compre agora e leia](#)

Para muitos investidores da bolsa de valores, a prestação de contas com a Receita Federal revela-se mais árdua do que de fato poderia ser. Mais preocupante do que a dificuldade experimentada no momento da declaração é a falta de atenção dos investidores em relação à própria apuração do imposto que, diferentemente do que muitos pensam, não ocorre na declaração anual, mas sim mensalmente. Considerando essa realidade, o objetivo do livro que você tem em mãos é, numa abordagem prática e com linguagem acessível, promover a aproximação desses dois mundos normalmente considerados complexos: bolsa de valores e tributação. Nesse sentido, o livro tem o propósito de analisar detalhadamente a tributação sobre os ganhos obtidos por pessoas físicas no mercado de ações da bolsa de valores, nas várias modalidades de negociação (à vista, a termo, futuro, opções e day trade), adotando uma abordagem prática, ampla e fundamentada.

[Compre agora e leia](#)

CANDLESTICK

Um método para ampliar lucros na Bolsa de Valores



novatec

Carlos Alberto Debastiani

Candlestick

Debastiani, Carlos Alberto

9788575225943

200 páginas

[Compre agora e leia](#)

A análise dos gráficos de Candlestick é uma técnica amplamente utilizada pelos operadores de bolsas de valores no mundo inteiro. De origem japonesa, este refinado método avalia o comportamento do mercado, sendo muito eficaz na previsão de mudanças em tendências, o que permite desvendar fatores psicológicos por trás dos gráficos, incrementando a lucratividade dos investimentos.

Candlestick – Um método para ampliar lucros na Bolsa de Valores é uma obra bem estruturada e totalmente ilustrada. A preocupação do autor em utilizar uma linguagem clara e acessível a torna leve e de fácil assimilação, mesmo para leigos. Cada padrão de análise abordado possui um modelo com sua figura clássica, facilitando a identificação. Depois das características, das peculiaridades e dos fatores psicológicos do padrão, é apresentado o gráfico de um caso real aplicado a uma ação negociada na Bovespa. Este livro possui, ainda, um índice resumido dos padrões para pesquisa rápida na utilização cotidiana.

[Compre agora e leia](#)



Nas linhas do Arduino

Kenshima, Gedeane

9786586057034

224 páginas

[Compre agora e leia](#)

A plataforma Arduino é muito conhecida por unir eletrônica e programação. Embora programadores que se aventuram em utilizar a plataforma não encontrem dificuldades em programar, o mesmo não se pode afirmar dos profissionais da área de eletrônica ou mesmo de makers. Este livro apresenta desde os conceitos básicos de programação, como estruturas de repetição e decisão, até os mais avançados, como interrupções, sensores e atuadores. Além disso, inclui inúmeros experimentos práticos e exercícios com componentes eletrônicos simples, como LEDs e buzzer. O objetivo desta obra é ensinar a programação Wiring para makers, profissionais e estudantes, com experiência básica em circuitos eletrônicos, tendo contato pela primeira vez com programação aplicada à Robótica e Eletrônica.

[Compre agora e leia](#)



AVALIANDO
EMPRESAS

INVESTINDO EM AÇÕES

A APLICAÇÃO PRÁTICA DA
ANÁLISE FUNDAMENTALISTA NA
AVALIAÇÃO DE EMPRESAS

novatec

CARLOS ALBERTO DEBASTIANI
FELIPE AUGUSTO RUSSO

Avaliando Empresas, Investindo em Ações

Debastiani, Carlos Alberto

9788575225974

224 páginas

[Compre agora e leia](#)

Avaliando Empresas, Investindo em Ações é um livro destinado a investidores que desejam conhecer, em detalhes, os métodos de análise que integram a linha de trabalho da escola fundamentalista, trazendo ao leitor, em linguagem clara e acessível, o conhecimento profundo dos elementos necessários a uma análise criteriosa da saúde financeira das empresas, envolvendo indicadores de balanço e de mercado, análise de liquidez e dos riscos pertinentes a fatores setoriais e conjunturas econômicas nacional e internacional. Por meio de exemplos práticos e ilustrações, os autores exercitam os conceitos teóricos abordados, desde os fundamentos básicos da economia até a formulação de estratégias para investimentos de longo prazo.

[Compre agora e leia](#)